手機課 V1.8

v1.8

ClassTechStar

目录

一、基礎篇（第 0-7 章） **2**

第零章：噫，始之前 **2**

一、此書能助汝何爲?（與不能爲之事） 2

二、吾等之“暗號”釋 2

三、安卓與 iOS，皆論之 2

四、手機容量：何以買 128G，可用卻不及 110G? 2

五、汝之手機乃一座“數字資產庫” 3

第一章：敬禮，吾之私用電腦 **4**

一、手機硬件“五臟六腑”：深析 4

二、操作系統：安卓 VS iOS VS 鴻蒙 10

三、手機接口全解析 11

四、手機產品體系與命名規則 12

五、基帶與天線：手機通信之幕後功臣 13

六、汝之手機，到底在演幾何角色? 14

第一章：敬禮，吾之私用電腦 14

第一章附：鴻蒙操作系統專題 16

第二章：整理汝之數字之室——文件管理 **20**

一、三層存儲之體：汝之數字倉 20

二、識“文件管理器” 20

三、文件系統之結構：深入知之 21

四、數據之備份與恢復：爲汝數據上保險 21

五、文件之安：護汝之隱私 21

六、空間不足乎？來一番大掃除 22

七、常見文件格式全解析 22

八、圖與文件系統之管理：使資產井然有序 24

第三章：使手機接“生命之線”——網絡 **26**

〇、聯網不迷——分清 Wi-Fi 與流量之“路”與“車” 26

一、移動網絡（4G/5G）與流量 27

二、Wi-Fi：連接、分享與安 28

三、藍牙：短距無線通訊之萬能手 29

四、紅外遙控：手機化萬能遙控 30

五、NFC：碰一碰之法 30

六、個人熱點：將手機變 Wi-Fi 路由器 31

第四章：安裝與管理 App——手機之“技能包” **32**

一、官方之店：至安之“應用超市” 32

二、純淨/防護之模：開手機之“防毒牆” 32

三、第三方之來與“電子寄生蟲”：險之極高！ 33

四、裝、更、卸 34

五、權限之管：汝乃 App 之“老闆” 34

六、軟件安裝包之構成要素 34

第五章：使手機更知汝意——基礎設置 **37**

一、聲與震 37

二、顯與亮 37

三、壁紙、主題與桌 37

四、效率之設：使手機爲汝之“外掛腦” 38

五、輔助功能與無障：科技向善之力 39

第六章：守護汝之數字生活——安全與隱私 **43**

一、鎖屏：第一道物理防線 43

二、“查找吾手機”：終極防盜之具 43

三、系統之更：打補丁，堵洞 44

四、識與防電信詐 44

五、御“開盒”與社工攻：斬隱私泄之鏈 45

六、隱私之護：汝之數據乃汝之財 46

七、失/被盜後之“急救”單 46

八、SIM 卡與 eSIM 之安 47

第七章：前沿功能全解析 **48**

一、衛星通：無信亦能求救 48

二、無網通：近距通訊之新方 49

三、車禍之檢：要時救命 49
四、星閃之技：下代之無線連 50

二、軟件篇（第 8-14 章） 53

第八章：通信：短信、電話與社交 53
一、電話 App 之進階功能 53
二、短信與 iMessage / RCS 55
三、微信 / QQ：國民級社交應用之管理 56
四、視頻通話對比 57
五、電子郵件：于手機配置與使用 58

第九章：移動辦公與學習 60
一、文檔之處理：於掌中編輯與創作 60
二、筆記之用：自隨手記至知識體系 60
三、掃描與 OCR：以紙質之世搬入手機 61
四、網盤何以擇：無最好者，惟最適合君者 61
五、一真實之移動辦公場景 62

第十章：移動支付與生活之務 63
一、究竟何以綁定銀行卡？ 63
二、付款碼乃“誰見誰收錢” 63
三、此等安全設置，費五分鐘設好可省甚多之心 63
四、公交地鐵：手機即交通卡 64
五、生活繳費、外賣、打車 64
六、以手機理醫保、社保、政務 64

第十一章：攝影與圖像管理 66
一、君是否亦犯此等錯？ 66
二、數立升出片率之小術 66
三、人像模式與專業模式 67
四、後期修圖：勿過度，恰到好處即足 67
五、照片太多奈何？整理之策 67
六、截圖與錄屏 68
第十一章附：專業影像技術之識 68

第十二章：影音娛樂與閱讀 77

一、聽音樂：擇何 App，何以聽方適 77
二、播客（Podcast）：通勤路上之“耳朵伴侶” 77
三、看視頻：自播放器至流媒體之正解 78
四、電子書閱讀：手機上亦能好好看書 78
五、手機遊戲：玩得爽，亦不傷手機 79

第十三章：工具類 App 推薦 **80**
一、效率工具：管理汝之時與注意力 80
二、健康運動 82
三、旅行出行 83
四、自動化工具啟蒙 85
五、瀏覽器與搜索引擎：爾之信息入口 85
六、實用系統工具：清理、防騷擾與安全防護 88
七、天氣預警、自然災害預警與地震預警 90
八、跨設備協同：遠控電腦、快傳與帳號安全 92

第十四章：密碼管理器專題 **96**
一、何以君需密碼管理器？ 96
二、密碼管理器之工作原理 96
三、加密理簡介 96
四、主流密管器薦 98
五、密強度真實評估 98
六、Passkey：無密登錄之未來 100
七、硬件安全密鑰 102
八、家/團隊密共享 103
九、密管器用最佳實踐 105

三、進階篇（第 15-20 章） **108**

第十五章：電池之理與續航之優化 **108**
一、電池乃何物？循環與健康度 108
二、充電玄學？快充、無線充與黃金之則 108
三、快充協議全解：何以君之充電器充不快？ 109
四、電池新術：硅碳負極電池 112
五、這方乃真正之省電術 112

第十六章：賬號體系與雲端之務 **114**

一、君之手機賬號，究竟管幾何物？ 114
二、三必爲之事，今即爲之 114
三、iCloud：蘋果用戶之“空氣” 115
四、同步與備份，勿混之 116
五、安卓用戶之賬號矩陣 116
六、家庭共享：令家人亦受益 116

第十七章：手機與他設備之聯動 118
一、與電腦協同：手機與電腦不再是二世界 118
二、投屏至電視：全家同觀 118
三、與智能手錶/手環：健康管家長於手腕上 119
四、與智能家居：手機乃家中之“總開關” 119
五、藍牙耳機與音箱：無線之自由 120
六、與車載系統：開車時之安全搭檔 121
七、聯動中之安全與隱私 121

第十八章：數據備份與遷移 122
一、先搞清：君手機內何物失即無矣 122
二、整機備份：一次搞定所有物 122
三、分類備份之策：要緊者單獨另存一份 123
四、換機指南：自舊手機至新手機 124
五、備份之黃金之則：三二一原則 124

第十九章：故障排查與自救 126
一、手機變卡、發熱——最常被問之問題 126
二、應用閃退、無響應——App 不聽話矣 126
三、充不進電或充電慢——恐非電池之鍋 127
四、網絡連不上、Wi-Fi 老是斷 127
五、屏幕問題：觸控不靈、閃屏、留印子 128
六、無法開機、卡於 Logo 界面——先勿慌 128
七、聲音問題：喇叭不響、通話聲音小 128
八、覓官方維修，避彼等坑 129
第二十章：開發者模式與高級權限 129
第二十章附：Termux——Android 設備上之便攜式 Linux 終端環境 136

，不隨關閉窗口而消失。 141

第二十一章：移動網絡與信息安全深探 141

一、VPN：網絡中之“隱形斗篷” 141
二、公共 Wi-Fi 之“甜蜜陷阱”——比君所想更險 141
三、HTTPS 何以護君？——SSL/TLS 握手 142
四、DNS 安全：君之“網絡電話本”安全乎？ 142
五、“無痕瀏覽”真能隱身乎？ 143
六、瀏覽器隱私：勿令瀏覽器賣君 143
七、實用安全清單：自今始護己 144

第二十二章：AI 如何於手機中工作 145

一、AI 相機：君之“後期暗房”已入手機 145
二、語音助手：君之“聲音祕書”何以懂君？ 145
三、端側 AI vs. 雲端 AI 145
四、AI 大模型與手機：君之隨身智能助手 145
五、數個典型應用場景與實操 146
六、提示詞優化：令 AI 聽君指揮之“溝通公式” 147
七、AI 之侷限與挑戰：保持清醒之判斷力 151

第二十三章：編程之思與自動化啓蒙 153

一、自動化到底是個啥？ 153
二、iOS“快捷指令”——蘋果用戶之自動化神器 153
三、安卓自動化——Tasker 和它之朋友們 154
四、自動化思維之日常應用 155
五、踩坑指南：自動化之常見問題 156

第二十三章附：GitHub 入門指南——從代碼託管到協作平臺 157

一、GitHub 究竟爲何物？ 157
二、Git 與 GitHub 之別（關鍵概念!） 157
三、GitHub 之核心概念與功能 158
四、註冊與上手：五分鐘創建君之第一個倉庫 159
五、基本工作流：自本地至雲端 160
六、團隊協作流程：Pull Request 何以運作 160
七、GitHub 之典型應用場景 161
八、予新手之數點建議 161
九、總結：何故每人皆該瞭解 GitHub 162

第二十四章：Root 權限與 BL 鎖深析 **163**

一、深入理解 Bootloader 163
二、解鎖 BL 鎖之詳細流程（以小米爲例） 164
三、Root 之多種方案 165
四、刷機入門 165
五、三星 Knox：BL 解鎖之不可逆代價 165
六、常見問題解答（Q& A） 166

第二十五章：二手手機處理與數字遺產 **167**

一、舊手機之“安樂死”：安全出售或轉贈指南 167
二、數字遺產：君之社交靈魂何以安放？ 168
三、環保之最後一程 172

第二十六章：當下手機技術前沿 **174**

一、AI 手機：乃營銷噱頭，抑真變革？ 174
二、各廠商 AI 戰略對較：誰於認真作，誰於貼牌？ 174
三、衛星通信：自旗艦“噱頭”至中端標配 175
四、折屏生態成熟度：能當主力機矣嗎？ 175
五、6G 與 Wi-Fi 7：下一代連之速度與邊界 176
六、硅碳負極電池：續航終迎來材級突破 176
七、本章速覽：一句話切記此前沿 177

五、實戰篇（第 27-32 章） **179**

第二十七章：父母亦能輕鬆用——適老化手機完全指南 **179**

零、先以心態擺正：此非“老人用不來”，乃“手機沒作好” 179
一、首步：開“簡易模式”——手機變清爽矣 179
二、若不開簡易模式，手調亦可——“大字 + 大圖標”五步走 180
三、防詐——此章請必帶着長者共觀 181
四、遠程協助——汝身於異地亦能助父母修手機 182
五、微信之“適舊改”——與長者曰此幾句遂夠矣 182
六、手機丟矣何以辦？助長者作個“應急預案” 183
七、無線耳機與助聽器兼容——爲忽視之好能 183
八、健康相關能——手機不僅乃通訊工具 183

第二十八章：花對錢買至心儀機——手機選購終極指南 **185**

零、三步選機法：需預算機型 185

一、核心參數何以觀——不用全懂，觀懂此五項遂夠矣 185
二、按預算分檔薦（以 2026 年中市場爲基準） 186
三、按場景選機——汝之需乃何？ 188
四、五個容易踩之坑——買之前對照觀一眼 188
五、何時買最劃算？ 189
六、渠何以選——於哪買靠譜？ 189

第二十九章：數字健康——與手機建立健康關係 191
一、先觀汝之“屏幕時”——數使人清醒 191
二、賜 App 設“用限額”——賜己裝個限速器 191
三、“勿擾之模”之正用法——非關掉告，乃關掉焦慮 192
四、護眼不靠 App 靠習慣——20-20-20 法則 192
五、“低頭輩”之頸椎自救指南 192
六、睡眠與手機——臥室裏最該改之習慣 193
七、信息焦慮症——學會“選擇性失明” 193

第三十章：投屏、演示與車載互聯 195
一、無線投屏三大陣營——勿搞混矣 195
二、有線投屏——更穩、零延遲 195
三、手機變電腦——桌面模式 196
四、車載互聯——開車以手機之正法 196
五、會議室投屏實戰——不復手忙腳亂 197

第三十一章：短視頻拍攝與內容創作入門 198
一、拍前先欲清楚：汝要拍賜誰觀？ 198
二、手機攝基設——開拍前必作之 5 事 198
三、運鏡五招——使畫面“動”起來 199
四、剪輯入門——剪映遂夠矣 199
五、封面之要性——70% 之人僅唯封面決觀不觀 199
六、內容創作之三條心法 200

第三十二章：手機安全急救手冊 201
場景零：地震 / 極端天氣——手機何以助爾保命 201
場景一：手機遺失或被竊——黃金三十分鐘 202
場景二：為人所騙——七十二小時黃金處理之期 202
場景三：手機染毒或木馬——奈何？ 203
場景四：帳號被竊——速復掌控之權 203

場景五：得勒索之訊——勿懼，勿付 204
場景六：AI 換臉、AI 擬聲之詐——眼見不復為實 204
場景七：SIM swap 之攻（SIM 劫持）——爾號為人所「竊」 205

六、展望篇與車聯篇（第 33-34 章） 208

第三十三章：未來展望——手機之下一程 208
一、AR/VR 與空間算：手機外之“次桌面” 208
二、腦機接口：思欲成爲交互媒介 208
三、手機之“隱身”與新身份基石 209
四、倫理思考：吾等，抑手機之主人嗎？ 209
五、持連，亦持獨——汝之數生活哲學 209

第三十四章：車機互聯深指南——手機與汽車之融合 211
一、車機互聯之三大路線 211
二、CarPlay / Android Auto / HiCar 對較 211
三、鴻蒙座艙：手機與車機之無縫流轉 212
四、造車新勢力之車機系統 212
五、手機當車機——能用嗎？ 213
六、實用術 213

兒童與手機專題（第 35 章） 216

第三十五章：兒童與手機——數字時代之教育指南 216
一、幾歲賜孩子手機？ 216
二、家長控制系統全攻略 217
三、青少年模式 218
四、網絡素養教育 218
五、健康使用習慣 219
六、數字遺產與兒童 219

第三十六章：手機與智能家居——使家生智 221
一、智能家居之協議：所擇至要 221
二、三大生態之比 222
三、以機為家居之中樞 222
四、入門套件推薦 223

五、安全注意事項 223

七、AI 編程實戰篇（第 37-47 章） 226

第三十七章：範式之革——自「作碼」至「指 AI 作碼」 226

一、Vibe Coding 之誕生：一言所起之運動 226
二、2026 年之勢：Agentic Engineering 之世 226
三、核心之悟之變：自 Coder 至 Conductor 227
四、方法論何以重於工具 228

第三十八章：AI 編程五大思維之則 230

一、則一：Think Before Coding（先思而後碼） 230
二、則二：Simplicity First（簡為至上） 231
三、則三：Surgical Changes（改之必精） 231
四、則四：Goal-Driven Execution（以之馭行） 232
五、則五：Iterate and Verify（迭代而驗） 232
六、Harness Engineering：Agent = Model + Harness 233

第三十九章：上下文之工——使 AI 解爾之意 235

一、項目記憶之檔：AI 之「長憶」 235
二、規範驅動之開發（SDD） 236
三、上下文之窗之管理 237
四、上下文之工實踐之要 237

第四十章：需求之工——化念為 AI 可行之令 239

一、自晦之念至結構化之需求 239
二、需求拆解分層之藝 239
三、PRD 撰寫模板與實踐 240
四、實戰：從想法到方案之完整推演 241

第四十一章：2026 年 AI 編程工具全景 243

一、三大流派與選擇框架 243
二、IDE 內置 AI 流派 243
三、終端 Agent 流派 244
四、雲端 Builder 流派 245
五、工具屢變，方法論不易 245

第四十二章：大模型全景與 API 實戰 246

一、國際主流大模型：三強格局 246
二、國產主流大模型：百花齊放 248
三、模型選擇決策框架 250
四、API 調用入門：從零接入大模型 251
五、Token Plan 購買指南 254
六、實戰：於專案中接入大模型 API 257

第四十三章：對話工程——與 AI 高效協作之對話技術 260
一、Prompt 分層策略 260
二、調試對話：如何描述錯誤使 AI 快速定位 261
三、迭代策略：小步快跑對大步重構 261
四、多輪對話之上下文管理 262

第四十四章：自 0 到 1 之實戰——完整項目全流程 264
一、項目一：極簡個人工具站（入門級） 264
二、項目二：全棧記賬系統（進階級） 265
三、項目三：AI 增強應用（高級） 266
四、通用實戰流程清單 267

第四十五章：質量保障——使 AI 所寫之代碼可靠 269
一、測試驅動：使 AI 寫測試 269
二、代碼審查技巧 270
三、版本控制：Git 於 AI 編程中之角色 270
四、常見陷阱與避坑指南 271

第四十六章：進階之路——從使用者到架構師 273
一、MCP 協議與工具擴展 273
二、自定義工作流與 Skills 273
三、多 Agent 協作模式 274
四、持續學習路徑 274

第四十七章：速查手冊與資源 276
一、思維模型速查卡 276
二、工具選擇決策樹 276
三、常見問題 FAQ 277
四、推薦學習資源 277
五、結語：工具於手，方法論於心 278

參考文獻 278

八、安卓開發實戰篇（第 48-58 章） 281

第四十八章：認識安卓——從操作系統到開發生態 281

一、安卓之身世：從「小機器人」到「全球最大系統」 281

二、用何語言寫：Java 抑 Kotlin？ 282

三、一個 App 為何以誕生之？ 283

四、安卓技術棧全景：汝之「武器清單」 283

五、安卓 vs iOS：開發者之第一堂「選邊課」 284

第四十九章：搭建開發環境——Android Studio 與第一個項目 286

一、安裝 Android Studio 286

二、創建汝之第一個項目 286

三、跑起來：模擬器與真機 287

四、工程結構拆解：諸文件何所司 288

五、小試牛刀：易「Hello」為汝名 288

六、常見報錯急救：新手前兩週之「急診室」 289

第五十章：Kotlin 速成——安卓開發之首語 290

一、變量與類型：為數據命名 290

二、函數：束邏輯為「按鈕」 291

三、條件與循環：使程序能「思」 291

四、空安全：Kotlin 之「保命符」 292

五、類與對象：造「零件」之模 292

六、lambda 與高階函數：Kotlin 之「語法糖」 293

七、貽零基者之語法速查卡 293

第五十一章：界面開發——Jetpack Compose 入門 295

一、Compose 之思：界面即函數 295

二、基礎組件：文字、按鈕、輸入框、圖片 295

三、布局三件：Column、Row、Box 296

四、狀態：使界面「活」 297

五、主題與深色模式：一配而全局生效 298

六、預覽、動畫與實時編輯：Compose 之「外掛」 298

第五十二章：四大組件與 Activity 生命之期 300

一、四大組件：安卓之「四梁八柱」 300
二、Activity 生命之期：安卓開發之首張「藏寶圖」 300
三、Intent：App 內外通行之「車票」 301
四、多頁與導航 302
五、順而識之：Fragment 為何物（及本書何以不教） 302

第五十三章：數據之儲與網絡之請 304
一、儲存之方何擇：先觀此表 304
二、DataStore：儲少許小配置 304
三、Room：為數據安「真家」 305
四、Retrofit：與服務器言 306
五、JSON 之析：為網絡數據作「譯」 306
六、協程：安卓並發之「通語」 307
八、協程：安卓並發之「通語」 307

第五十四章：項目一——吾之記賬冊（本地版） 309
一、需求之思：先問五問 309
二、數據之層：使「賬」有家 309
三、MVVM：界面與邏輯「分家」 310
四、界面：列表之頁 + 錄入之頁 311
五、編輯與刪除：使數據「能進能出」 312
六、驗收：三層之檢 + 二進階之挑戰 313
八、驗收：三層之檢 + 二進階之挑戰 313

第五十五章：項目二——天氣查詢 App（網絡版） 314
一、申請免費天氣之 API 314
二、網絡之層：Retrofit + JSON 之析 314
三、三種界面之態：載中 / 成 / 敗 315
四、實戰界面之碼：繪天氣以出 316
五、界面與交互之細 316
七、界面與交互之細 316

第五十六章：琢磨與發佈——測試、調試與上架 Google Play 318
一、調試三板斧：Logcat、斷點、UI 檢查器 318
二、測試：為代碼上保險 318
三、簽名打包：為 App 蓋「印」 319
四、上架 Google Play 之全程 319

五、商店素材何由製之：第一印象即繫下載之數 320
五、性能優化與崩潰上報小清單 320

第五十七章：進階之路——權限、通知與現代架構 322
一、運行時權限：先敲門，再進門 322
二、通知：於正確之時說有用之言 323
三、架構進階：依賴注入與模塊化 323
四、2026 年之新玩法：AI 輔助安卓開發 324
五、Compose 性能優化與調試工具 324

第五十八章：速查之冊與資源 326
一、常用代碼速查之卡 326
二、常見問題 FAQ 326
三、推薦學習資源 327
四、Gradle 常用命令速查 327
五、結語：自第一個 App 至第 N 個 App 328
七、結語：自第一個 App 至第 N 個 App 328
參考文獻 328

九、iOS 開發實戰篇（第 59-69 章） 330

第五十九章：識 iOS——蘋果生態與開發生態 330
一、iOS 之身世：一機改一行 330
二、圍牆之園：蘋果生態之「規」與「利」 330
三、以何語言書寫：Objective-C 抑或 Swift？ 331
四、一個 App 何以誕生？ 331
五、iOS 技術棧全景：爾之「武器清單」 332

第六十章：築開發之環境——Xcode 與第一個項目 334
一、備 Mac 一台 334
二、安裝 Xcode 334
三、創爾之第一個項目 335
四、拆工程之結構：每文件何為 335
五、真機調試：裝 App 於爾之 iPhone 336
五、Xcode 效率之技：新手之五「快捷鍵外掛」 336
六、真機調試：裝 App 於爾之 iPhone 336
七、小試其刃加常見報錯之急救 337

九、小試其刃加常見報錯之急救 337

第六十一章：Swift 速成——蘋果開發之第一語言 338

一、變量與類型：為數據起名 338
二、可選值（Optional）：Swift「防崩護身之符」 339
三、函數與閉包：打包邏輯成「鈕」 339
四、結構體與協議：Swift「積木」之體系 340
五、貽零基者之語法速查卡 341

第六十二章：界面之開發——SwiftUI 入門 342

一、SwiftUI 之思想：聲明式，非命令式 342
二、基礎視圖：文字、按鈕、輸入框、圖 343
三、佈局三件套：VStack、HStack、ZStack 343
四、狀態與綁定：使界面「活」起來 344
五、預覽、主題與動畫：SwiftUI 之「外掛」 344
六、常用修飾符速查：一頁在手，界面無憂 345

第六十三章：視圖、狀態與導航 347

一、狀態管理之「數據流全景」 347
二、NavigationStack：頁面跳轉之「棧」 348
三、彈窗與底部標籤：sheet 與 TabView 348
四、List：iOS 之「國民列表」 349
五、程序生命之期：前台與後台 350

第六十四章：數據存貯與網絡之請 351

一、存貯之方何以擇：先觀此表 351
二、UserDefaults：貯些小配置 351
三、SwiftData：為數據安「真家」 352
四、Codable：為 JSON「譯」 353
五、URLSession + async/await：與伺服器言 353
六、網絡之誤亦宜「分級」：勿以凡咎盡擲於用戶 354

第六十五章：項目一——吾之記賬本（本地版） 356

一、思其所需：先問五事 356
二、數據層：使「賬」有其所 356
三、MVVM：界面與邏輯「分家」 357
四、界面：列表頁 + 錄入彈窗 358

五、編、刪與彙總 359
六、驗收：三層之檢 + 二進階之挑戰 359

第六十六章：項目二——天氣查詢 App（網絡版） 361

一、請免費天氣之 API 361
二、網絡層：URLSession + Codable 361
三、界面三態：載入中 / 成 / 敗 362
四、實戰界面之碼：繪天氣而出 363
五、交互之細與所藏之城 363
六、網絡之請何以調試：印之、抓包與擬緩網 364

第六十七章：琢磨與發布——測試、調試與上架 App Store 365

一、調試三斧：print、斷點、預覽 365
二、測試：為碼投保 365
三、署名與打包：為 App 鈐「印」 366
四、上架 App Store 之全程 366
五、商店素材何以為之：第一印象即下載之量 367
六、崩潰之上報與性能優化小單 367

第六十八章：進階之途——權限、通知與現代構架 369

一、權限：先叩門，後入門 369
二、通知：於當時而言有用之言 370
三、構架進階：MVVM 之深與可測之性 370
四、2026 年之新法：AI 輔 iOS 之開發 371
五、Swift 並發之要：@MainActor 與 Task 371

第六十九章：速查之冊與資源 373

一、常用碼速查之卡 373
二、常見之問 FAQ 373
三、Xcode 高頻快捷鍵速查 374
四、薦學習之資 375
六、薦學習之資 375
五、結語：從第一個 App 到第 N 個 App 375
參考文獻 375

附錄 A：新手機開箱“保姆級”流程 378

一、開箱驗機“三部曲”： 378

二、激活與初始設置“重點提醒”： 378
三、裝 App“安全第一”： 378

附錄 B：動手練習小參考 **379**

附錄 C：極簡術語“黑話”對照表 **380**

附錄 D：靠譜資源去哪裏找？ **381**
官方支持乃首選： 381
實用網站與博主： 381
安全自查一句話清單： 381

附錄 E：感謝與說明 **382**
6 版更新要點 382
7 版更新要點 382
8 版更新要點 384
內容增補：實用軟件與騷擾攔截 384
內容增補：天氣預警 / 自然災害預警 / 地震預警 384
內容增補：跨設備協同與效率工具 385

附錄 F：摺疊屏選購與使用專題 **386**
一、摺疊屏今已發展至何程度？ 386
二、何人適買摺疊屏？ 386
三、選購要點：觀此 6 個指標 387
四、摺疊屏獨有之使用技巧 388
五、保養要點：讓摺疊屏活得久一點 388
六、摺疊屏的“坑”和心理預期 389
七、主流品牌何以選（2026 年） 389

結語：持好奇，持清醒 **391**

附錄 G：全書速查索引 **392**

附錄 H：版本路線圖 **396**

一、基礎篇（第 0-7 章）

第零章：噫，始之前

正式探索前，且先論基本規則，以保後述汝皆知。勿急，磨刀不誤砍柴工也。

一、此書能助汝何爲？（與不能爲之事）

其能助汝系統性通曉手機何以用，明功能背後之理，且無論汝乃安卓黨抑或蘋果派，多數思路皆通用。其將引汝動手練習，使汝自“會用”升至“懂用”。

其能助汝系統性通曉手機何以用，明功能背後之理，且無論汝乃安卓黨抑或蘋果派，多數思路皆通用。其將引汝動手練習，使汝自“會用”升至“懂用”。

二、吾等之“暗號”釋

爲說得更明，書中將用些許小標記，此等標記猶如吾等間之默契暗號：

- [按鈕名]：表屏幕上具體之按鈕或 App 之名，譬如 [設置]、[確定]。
- >：表操作之序，譬如” 點 [設置] > [聲音]”，即先點設置，再點聲音。
- **加粗**：至要！或爲關鍵概念或安全警告，請務必留意。
- *斜體*：些許補充之小想或比喻，助汝更善理解。

三、安卓與 iOS，皆論之

汝猜何如，有些事物稱謂不同，然所爲之事一也。譬如：應用（安卓）與 App（iOS），設置（皆稱設置），文件管理（安卓）與文件（iOS）。知其爲同一事即可，勿爲名詞所惑。

此外，今安卓陣營中尚有華爲之鴻蒙系統，其有自身特色，後吾等將專論之。總之，不論汝用何手機，此書皆能助汝。

四、手機容量：何以買 128G，可用卻不及 110G？

容量單位衆所知：1TB = 1024GB，1GB = 1024MB，1MB = 1024KB……然此處有一坑：硬盤廠商通常按 1000 進制算，而系統按 1024 進制算，故汝所購之“128G”硬盤，實際容量將“縮水”。

然汝手機容量縮水之原因主要有三：

1. **系統自佔其地**：操作系統猶如房屋之承重牆，必存。今之系統動輒佔用 10-20GB，此甚正常。
2. **廠商預裝之應用**：新手機中自帶之 App 亦佔空間，有能卸者，有不能者。
3. **分區與格式化損耗**：技術之故，猶如書包之襯布亦佔體積。

買手機之小建議

買手機之小建議

來動手：觀汝手機之”身份證”

來動手：觀汝手機之“身份證”

五、汝之手機乃一座“數字資產庫”

始具體知識前，吾等先立一核心認知：**手機早已不唯通信工具，乃吾等最重要之數字資產庫**。聯繫人、聊天記錄、家庭照片、工作文檔、銀行 App、付款碼、健康數據、電子證件……此等數據相加，價值往往逾手機本身。

此何以至要？

此何以至要？**更恐爲多年積累之不可再生數據**——孩子之成長照片、過世親人之語音、未備份之重要合同。科學地整理與防護習慣，能自三維護汝：

- **提升運行效率**：規範之文件分類使系統查找更速，手機長期不卡頓。
- **延使用壽命**：合理之存儲管理減芯片讀寫之負，電池與存儲皆更耐用。
- **保數據安全**：分層管理 + 定期備份 + 權限收斂，使”數據災難”遠離汝。

吾等之核心理念：“預防爲主，防治結合”

猶養生不得待病方就醫，**手機管理亦須自“日常習慣”抓起**。本書反覆強調者非某一高級技巧，乃此等樸素之原則：

- **預防爲主**：裝 App 唯走官方商店、照片發前先脫敏、密碼不重複——此等”小動作”成本極低，然能擋絕大多數風險。
- **定期體檢**：每月一整理下載文件夾、每季一權限審計、每半年一全量備份——固定節奏比臨時救火有效百倍。
- **留有退路**：啓雲同步、記好賬號密碼、書一份”數字遺產清單”——確保意外發生時汝與家人皆能從容應對。

第一章：敬禮，吾之私用電腦

汝之手機，實則汝囊中最私人之超級計算機。其較昔日送人類登月之計算機強無數倍，卻靜臥汝之掌心。今日，且來善識此“沉默之巨人”。

一、手機硬件“五臟六腑”：深析

手機雖小，五臟俱全。讓吾等如解剖課般，逐一識此等精密組件。

SoC（系統級芯片）：手機之“大腦 + 心臟”

SoC 全稱 System on Chip，譯之即“將整個系統塞入一塊芯片”。其不唯 CPU（中央處理器），乃將 CPU、GPU（圖形處理器）、NPU（神經網絡處理器）、ISP（圖像信號處理器）、基帶（司通信）等全部集於一體。

此猶將一城之政府大樓、發電廠、通信中心、警察局皆建於一棟樓內，效率極高。

CPU：通用計算之核心

CPU 乃 SoC 中最核心之組件，司處理絕大多數計算任務。手機 CPU 採 ARM 架構，與計算機之 x86 架構異，ARM 架構以精簡指令集（RISC）爲特色，功耗更低，更適合移動設備。

CPU 大小核設計：今之手機 CPU 普遍採“大小核”架構（ARM 之 big.LITTLE 技術），將核心分爲不同性能等級：

- **超大核/大核**：性能最強，司重度任務。如遊戲加載、應用啓動、複雜計算。頻率最高（3GHz+），然功耗亦最大。
- **中核**：性能與功耗平衡，處理日常中等負載任務。刷微博、觀視頻、多任務切換主要賴之。
- **小核/能效核**：功耗最低，處理輕量級後臺任務。待機、聽音樂、接收消息等，幾乎不費電。

三等級之核心協同工作：系統調度器依任務負載自動分配——輕量任務交小核省電，重活交大核猛幹，幹畢復切回小核。此即何以手機既能流暢打遊戲，又能待機一整日。

各品牌架構特點：

- **高通驍龍 8 Elite：**採 Oryon 自研 CPU 架構（源自 Nuvia 收購），2 個超級核（4.32GHz）+ 6 個性能核（3.53GHz），皆基於臺積電第二代 3nm 製程。徹底別 ARM 公版架構，單核與多核性能皆實現跨越式提升。GPU 爲 Adreno 830，採切片架構，圖形性能提升 40%，Hexagon NPU 算力約 45 TOPS，AI 推理性能提升 45%。（注：三星 Galaxy 定製版頻率更高，超級核可達 4.47GHz。）
- **聯發科天璣 9400：**1 個超大核（Cortex-X925，3.62GHz）+ 3 個性能核（Cortex-X4，3.3GHz）+ 4 個能效核（Cortex-A720，2.4GHz），全大核架構延續，臺積電第二代 3nm 製程。GPU 爲 Immortalis-G925（12 核），峯值性能提升 41%，光線追蹤性能提升 40%。AI 處理器爲第八代 NPU 890，支持端側大模型運行、端側 LoRA 訓練與端側視頻生成。
- **蘋果 A18 Pro：**2 個性能核（約 4.04GHz）+ 4 個能效核（約 2.4GHz），臺積電第二代 3nm 製程。6 核 GPU 支持硬件級光線追蹤，16 核 NPU（約 35 TOPS）爲端側 AI 與 Apple Intelligence 提供強大算力。蘋果自研架構於單核性能與能效比上依然領先，統一內存架構爲 AI 推理提供高帶寬優勢。
- **華爲麒麟 9020：**搭載於 Mate 70 系列，採自研泰山 CPU 架構，NPU 與 ISP 能力持續增強，支持 5.5G 與衛星通信，代表國產芯片之最新突破。

GPU：圖形渲染引擎

GPU（Graphics Processing Unit）專司圖形與並行計算任務。其核心數量遠多於 CPU（動輒數百上千個核心），然每個核心之功能相對簡單，宜同時處理大量相似之計算。

圖形渲染流程：

1. **頂點處理：**計算 3D 模型中每個頂點之位置座標。
2. **光柵化：**將 3D 圖形轉爲 2D 屏幕上之像素點。
3. **像素着色：**爲每個像素計算顏色、紋理、光照效果。
4. **輸出合成：**將所有圖層合成最終畫面，送至屏幕顯示。

GPU 對日常使用之影響：不唯遊戲！界面滑動、動畫效果、視頻播放、照片濾鏡、AR 應用皆賴 GPU。GPU 性能弱之手機，滑動界面或掉幀，遊戲更直接卡頓。

各品牌 GPU：高通用自研 Adreno GPU；聯發科用 ARM Mali GPU；蘋果用自研 GPU，性能與能效皆行業標杆。

NPU：AI 加速引擎

NPU（Neural Processing Unit）乃專爲神經網絡運算設計之處理器。較 CPU 與 GPU，NPU 處理 AI 任務時效率更高、功耗更低。

神經網絡運算方式：NPU 之核心運算乃矩陣乘法與卷積運算——此正神經網絡中最基礎、最頻繁之計算。NPU 藉硬件級優化，可並行處理大量矩陣運算，速度較 CPU 快數十倍，功耗卻低得多。

AI 處理流程：

1. **數據輸入：**攝像頭圖像、麥克風音頻、傳感器數據等。
2. **預處理：**調數據格式與尺寸，適配神經網絡輸入要求。
3. **推理計算：**NPU 加載訓練好之模型，對輸入數據進行前向計算，輸出結果。
4. **後處理：**將 AI 輸出轉爲可用之結果，如人臉座標、文字內容、場景標籤。

實際應用場景：

- **計算攝影：**夜景合成、人像虛化、HDR 處理、AI 美顏。
- **語音助手：**語音識別、自然語言理解、語音合成。
- **每日爲上班族做早餐之烹飪博主：**AI 字幕、對話翻譯。
- **智能識別：**文字識別（OCR）、物體識別、場景識別。
- **端側大模型：**最新旗艦芯片已支持於手機端運行小型 AI 大模型（7B 參數級別），實現離線 AI 對話、文檔摘要、智能寫作等功能。驍龍 8 Elite 之 Hexagon NPU 算力約 45 TOPS，天璣 9400 之 NPU 890 同樣支持端側大模型（LLM 推理速度提升 80%），蘋果 A18 Pro 之 16 核 NPU 提供約 35 TOPS 算力。
- **AI 圖像處理：**AI 消除（一鍵去除照片中之路人）、AI 擴圖（擴展圖片邊界）、AI 風格轉換、文字生成圖片等。

ISP：影像信號處理器

ISP 處理攝像頭傳來之原始數據，將光線信號化爲好看之照片。

性能何以觀？觀型號！蘋果 A18 Pro、高通驍龍 8 Elite、聯發科天璣 9400、華爲麒麟 9020 皆旗艦級別。中端觀驍龍 8s Gen3/7+ Gen3、天璣 8400。勿唯觀核心數，架構與製程（納米數）更重要。3nm 製程乃今最先進者，能效比遠優於 5nm 與 4nm。

運存（RAM）：臨時工作臺

運存猶如汝之辦公桌，愈大，能同時攤開之文件愈多。12GB 乃今之主流，16GB 宜重度用戶，24GB 以上則爲遊戲玩家與極客之選。

運存類型亦重要：LPDDR5 較 LPDDR4X 更快更省電。蘋果雖運存不大，然系統優化佳，故看似“夠用”。

運存不夠將何如？App 將爲系統殺掉後臺，切換時需重新加載。此即何以有時切個微信歸來，遊戲已沒。

存儲（ROM/閃存）：長期倉庫

存儲猶如汝之書櫃，裝所有照片、視頻、App 之所。今主流乃 UFS 3.1 或 UFS 4.0，速度較老式之 eMMC 快數倍。

存儲速度何以重要？其決 App 打開速度、遊戲加載速度、照片保存速度。用久之後，慢速存儲將使手機愈來愈卡。

屏幕：汝之眼睛

屏幕乃汝與手機交互之主要窗口，好壞直接影響體驗。

- **材質：**OLED 色彩豔麗、對比度高、更省電（顯示黑色時像素不發光），然低亮度下或有頻閃傷眼；LCD 更護眼、成本低，然對比度與色彩不如 OLED。
- **分辨率：**1080P 乃主流，2K 更清晰然更耗電。蘋果之”視網膜屏”概念即：於一定距離下，人眼分辨不出像素點。
- **刷新率：**60Hz 乃標準，90Hz/120Hz 更流暢，滑動時絲般順滑。高刷新率將耗電，故多數手機支持自適應刷新率（LTPO），依內容動態調整。
- **觸控採樣率：**影響觸控響應速度，遊戲玩家須關注，愈高愈跟手。
- **調光方式：**DC 調光較 PWM 調光更護眼，低亮度下不易眼累。今多數手機支持高頻 PWM 調光，亦爲護眼。

屏幕 HDR 技術

HDR（High Dynamic Range）使屏幕同時展現更亮之亮部與更暗之暗部，畫面層次感大幅提升。

- **實現原理：**普通屏幕之亮度範圍有限（SDR），最亮與最暗間之差距不夠大。HDR 屏幕之峯值亮度更高（通常 1000 尼特以上），對比度更大，色域更廣，能顯示更多細節。
- **主流標準：**HDR10（最基礎，免費開源）、HDR10+（三星推動，動態元數據）、Dolby Vision（杜比視界，畫質最佳然需授權）。手機支持何標準取決於屏幕硬件與系統授權。
- **實際效果：**觀 HDR 視頻時，陽光更刺眼、陰影更深邃、色彩更豐富。前提乃視頻本身爲 HDR 製作，屏幕亦支持 HDR 顯示。

影像 HDR 技術

拍照時之 HDR 與屏幕 HDR 乃兩回事。影像 HDR 乃通過算法解“亮處過曝、暗處死黑”之問題。

- **算法原理：**手機於按下快門時，實拍多張不同曝光之照片（欠曝保留亮部細節、過曝保留暗部細節、正常曝光保留中間調），而後將之合爲一張高動態範圍之照片。

- **處理流程**：多幀捕獲對齊補償（防手抖）曝光融合/色調映射色彩校正輸出最終照片。
- **效果優化**：今手機已能做”實時 HDR”——取景器中所見畫面即 HDR 處理後之效果，所見即所得。蘋果之 Smart HDR 與安卓之自動 HDR 皆甚成熟。

電池與充電：能量之源

電池乃手機最“短命”之部件，用兩三年即明顯老化。今主流容量乃 5500-7500mAh。

- **充電功率**：安卓陣營已卷至 100W 甚至 200W，十餘分鐘充滿；蘋果尚於 20-30W 徘徊。快充傷電池乎？用原裝配件問題不大，手機有溫控保護。
- **無線充電**：方便然效率低、發熱大。宜辦公室隨手充，不宜急出門時用。
- **反向充電**：手機化爲充電寶，給耳機、手錶充電，應急用。

攝像頭：光與影之魔法

今手機攝像頭動輒三四個，各有分工：

- **主攝**：最常用者，傳感器最大，畫質最佳。
- **超廣角**：拍風景、建築，視野更寬。
- **長焦**：拍遠處，有光學變焦與數碼變焦之分。
- **微距**：拍極近距離之小物。

像素數非全部！傳感器尺寸（底大一級壓死人）、光圈大小（f/1.8 較 f/2.4 大）、防抖（OIS 光學防抖甚重要）皆影響畫質。計算攝影（AI 算法）今愈來愈重要，能將多張照片合爲一張佳照。

潛望式長焦鏡頭

傳統長焦鏡頭需較長之光路，將使手機變得甚厚。潛望式長焦通過一 90 度之棱鏡將光線折射，使光路於手機內部橫向展開，從而於保持手機輕薄之同時實現 5 倍甚至 10 倍光學變焦。

- **結構原理**：光線自鏡頭入棱鏡折射 90 度沿手機寬度方向穿過鏡片組到達傳感器。相當於將一豎着之望遠鏡橫放。
- **光學特性**：光學變焦倍數高（5x-10x），畫質遠超數碼變焦。然傳感器尺寸通常較主攝小，暗光表現一般。
- **成像優勢**：拍月亮、拍遠處建築、拍舞臺特寫，乃普通手機無法企及者。

OIS 光學防抖 vs EIS 電子防抖

防抖乃影響拍照與錄像畫質之關鍵技術，兩種方式原理全異：

- **OIS（光學防抖）**：通過物理移動鏡頭或傳感器以抵消手抖。鏡頭內部有微型陀螺儀檢測抖動方向與幅度，而後驅動馬達反向移動鏡片組。優點乃不損失畫質，缺點乃成本高、佔用空間。
- **EIS（電子防抖）**：通過軟件算法裁切與偏移畫面以抵消抖動。手機錄製稍大之畫面範圍，而後依陀螺儀數據裁切出穩定之區域。優點乃不需額外硬件，缺點乃將裁切畫面（視野變小），暗光下或模糊。
- **實際選擇**：拍照優先觀 OIS，錄像兩者結合效果最佳。今多數旗艦手機支持”Sensor-Shift”傳感器位移防抖，較傳統鏡頭防抖效果更佳。

光學變焦 vs 數碼變焦

- **光學變焦**：通過物理移動鏡片組改變焦距，畫質不損失。手機上之光學變焦通常唯有固定倍數（如 2x、3x、5x、10x），不能如相機般連續變焦。
- **數碼變焦**：放大畫面中心區域，本質乃裁切 + 插值放大。放大愈多，畫質愈差。100 倍數碼變焦看似甚酷，然實際畫質已甚糊。
- **混合變焦**：結合光學與數碼，於光學變焦倍數間用多幀合成算法彌補畫質損失。效果較純數碼佳，然仍不如光學。

音頻：揚聲器與獨顯芯片

單揚聲器 vs 雙揚聲器：

- **單揚聲器**：唯底部一個發聲單元，聲音偏向一側，無立體感。中低端手機常見。
- **雙揚聲器（立體聲）**：頂部與底部各一揚聲器，形成立體聲場。觀視頻、聽音樂時左右聲道分明，沉浸感強甚多。聽筒兼做頂部揚聲器乃常見方案。
- **音質差異**：雙揚聲器不唯” 聲音更大”，關鍵乃空間感與層次感。用單揚聲器觀大片，總覺似缺何物。

遊戲手機獨顯芯片：

：

- **插幀**：將 30fps 之遊戲畫面通過算法插值至 60fps 或 120fps，畫面更流暢。此過程於獨顯芯片上完成，不佔用 SoC 之 GPU 資源。
- **超分**：將低分辨率畫面通過 AI 算法提升至高分辨率，畫質更清晰。
- **降功耗**：因插幀與超分由獨顯芯片負責，SoC 之負載降低，發熱與功耗皆減，遊戲更穩定。

GPS 定位：單頻與雙頻

手機定位主要賴 GPS 等衛星導航系統。

- **單頻 GPS**：唯接收 L1 頻段信號，定位精度約 3-5 米。於開闊地帶夠用，然於高樓密集之城市峽谷中，信號反射將致定位偏移較大。
- **雙頻 GPS**：同時接收 L1 與 L5 兩個頻段信號。L5 頻段波長更長、抗干擾能力更強，雙頻聯合解算可消除大部分電離層誤差，定位精度可達 1-3 米。於城市中導航、打車定位時優勢明顯。
- **多星座支持**：今之手機不僅支持 GPS，還支持北斗（中國）、GLONASS（俄羅斯）、Galileo（歐洲）等多個衛星系統。同時連接之衛星愈多，定位愈快愈準。

傳感器：手機之小觸角

手機中藏十數個傳感器，默默工作：

- **加速度計 + 陀螺儀**：感知手機姿態，玩遊戲轉向、自動旋轉屏幕。
- **光線傳感器**：自動調屏幕亮度。
- **距離傳感器**：打電話時手機貼近耳朵自動熄屏。
- **指紋傳感器**：今主流乃屏下光學或超聲波指紋。
- **面部識別**：蘋果之 Face ID 乃 3D 結構光，安全級別高；多數安卓乃 2D 照片識別，安全性一般。
- **一、摺疊屏今已發展至何程度?**：近場通信，刷公交卡、門禁卡、銀行卡。
- **氣壓計**：測海拔高度，爬樓梯皆能檢測到。
- **霍爾傳感器**：檢測磁吸殼之開合。

二、操作系統：安卓 VS iOS VS 鴻蒙

硬件乃身體，操作系統乃靈魂。三大系統各有特色：

- **安卓**：猶如開源社區，手機廠商可深度定製，選擇多。汝可換主題、裝第三方應用商店、折騰各種玩法。然亦因開放，碎片化嚴重，不同品牌體驗差異大。
- **iOS**：猶如精心打理之花園，蘋果軟硬件一手抓，體驗統一流暢。應用質量高，隱私保護好，然自由度低，多數事須按蘋果之規矩來。
- **鴻蒙**：華爲之新物種，主打分佈式能力，可使不同設備無縫協同。後吾等將專講。

三者生態皆極其豐富，擇汝所好之生活方式即可。無絕對好壞，唯適不適合。

三、手機接口全解析

手機接口乃連接外部世界之橋樑，不同接口差異甚大：

Micro USB

- **物理特性**：扁平梯形接口，分正反面（插反了插不進）。已於手機上基本淘汰，然部分低端設備與老設備仍在使用。
- **傳輸限制**：唯支持 USB 2.0 標準，最高傳輸速率 480Mbps，充電功率通常不超過 10W。
- **缺點**：正反面不通用、插拔壽命短、傳輸速度慢、充電功率低。

Lightning（蘋果閃電接口，已淘汰）

- **物理特性**：蘋果自研之 8 針接口，正反皆可插。iPhone 14 及更早機型使用，現已全面淘汰。
- **傳輸限制**：Lightning 接口物理上唯支持 USB 2.0 標準，最高傳輸速率 480Mbps。即使使用 USB-C 轉 Lightning 線，傳輸速度亦受限於 Lightning 接口本身。
- **淘汰原因**：歐盟通過法規要求 2024 年起所有手機統一使用 USB-C 接口，蘋果自 iPhone 15 系列始全面改用 Type-C。Lightning 接口正式退出歷史舞臺。
- **注意**：蘋果 MFi 認證之數據線內含認證芯片，非認證線或無法正常充電或傳輸數據。若汝仍在使用舊款 iPhone，建議備好 Lightning 數據線。

USB Type-C

- **物理特性**：24 針腳之橢圓形接口，正反皆可插。已成爲手機與多數電子設備之標準接口。歐盟已立法要求 2024 年起所有手機、平板、耳機等便攜設備統一使用 USB-C 接口，全球其他地區亦在跟進。
- **傳輸協議**：Type-C 唯接口形態，實際傳輸能力取決於內部協議。支持 USB 2.0（480Mbps）、USB 3.0/3.1/3.2（5-20Gbps）甚至 USB4（40Gbps）。
- **重要區分**：並非所有 Type-C 接口皆支持 USB 3.0！甚多手機雖用 Type-C 接口，但內部唯連接矣 USB 2.0 之引腳，傳輸速度與老接口一樣慢。iPhone 16/17 標準版僅支持 USB 2.0 速度，Pro 版方支持 USB 3.0（需使用 USB-C to USB-C 線）。
- **充電能力**：支持 USB PD 等快充協議，最高可支持 240W 充電。
- **視頻輸出**：支持 DisplayPort Alt Mode，可通過轉接線輸出視頻至顯示器。

5mm 音頻接口

- **物理特性**：圓形 3 段或 4 段插孔，3 段傳輸左右聲道音頻，4 段多一麥克風通道。
- **最高速率**：模擬信號傳輸，即插即用，不需驅動或轉換。音質穩定，延遲爲零。
- **現狀**：大部分旗艦手機已取消 3.5mm 接口，需 Type-C 轉接線或使用藍牙耳機。部分中端機與遊戲手機仍保留。

四、手機產品體系與命名規則

面對各品牌眼花繚亂之產品線，搞清楚命名規則能助汝速定適合自己之機型。

主流品牌產品系列：

- **蘋果**：iPhone（標準版）/ iPhone Plus（大屏版）/ iPhone Pro（專業版）/ iPhone Pro Max（大屏專業版）/ iPhone SE（平價版）/ iPhone 16e（平價 AI 版，搭載 A18 芯片與蘋果自研 C1 基帶）。2026 年 iPhone 17 系列全面搭載 A19 Pro 芯片，AI 能力進一步提升。
- **華爲**：Mate 系列（商務旗艦）/ P/Pura 系列（影像旗艦）/ Mate X 系列（摺疊旗艦）/ nova 系列（年輕中端）/ 暢享系列（入門）
- **小米**：小米數字系列（旗艦）/ 小米 MIX（探索旗艦/摺疊）/ Redmi K 系列（性能旗艦）/ Redmi Note 系列（中端）/ Redmi 數字系列（入門）
- **OPPO**：Find X 系列（旗艦）/ Reno 系列（中端）/ A 系列（入門）/ 一加數字系列（性能旗艦）
- **vivo**：X 系列（影像旗艦）/ iQOO 數字系列（性能旗艦）/ S 系列（自拍中端）/ Y 系列（入門）
- **三星**：Galaxy S 系列（旗艦）/ Galaxy Z Fold/Flip（摺疊旗艦）/ Galaxy A 系列（中端）

常見後綴含義：

- **Pro**：Professional 之縮寫，專業版。通常於標準版基礎上升級攝像頭、屏幕、處理器等核心配置。
- **Max**：最大版。屏幕最大、電池最大，然配置不一定比 Pro 高。
- **Ultra**：極致版。通常代表該系列之最高配置，堆料最猛，價格亦最高。如三星 S24 Ultra、小米 14 Ultra。
- **SE**：Special Edition 特別版，實多為平價之版。配置縮水，價格遂降。如 iPhone SE。
- **Plus**：加大版。屏幕較標準版大，電池亦大，然配置基本一致。

- **Lite/青春版**：輕量版。配置精簡，價格更低。
- **Note**：原本指大屏手機，今各品牌用法不一。Redmi Note 乃中端系列。
- **Neo**：新銳版。通常指性能取向之中端機型。

五、基帶與天線：手機通信之幕後功臣

手機能打電話、上網，賴基帶芯片與天線此對搭檔。

基帶芯片：

基帶乃手機中負責無線通信之處理器，其將數字信號調製成無線電波發出，亦將接收到之無線電波解調成數字信號。基帶性能直接影響通話質量、上網速度與信號穩定性。高通乃最大之手機基帶供應商，聯發科、三星亦有自研基帶。蘋果自 iPhone 16e 始搭載自研之 C1 基帶芯片，標誌蘋果正式進入自研基帶時代，C1 基帶採 4nm 製程，功耗表現優異；iPhone 16/16 Plus/16 Pro/16 Pro Max 仍用高通基帶（驍龍 X75 5G），預計後續機型將逐步過渡至自研基帶。更早之 iPhone XS/11 系列使用 Intel 基帶時信號表現不佳，乃不少用戶之痛點。

天線設計：

手機天線藏於機身內部，通常圍繞邊框佈置。天線設計直接影響信號接收性能：

- **MIMO 技術**：多入多出，用多個天線同時收發數據，提升速度與穩定性。4×4 MIMO 較 2×2 MIMO 速度更快。
- **天線數量**：旗艦機通常有十數個天線（蜂窩、Wi-Fi、藍牙、NFC、GPS 各需獨立天線）。
- **金屬機身之影響**：金屬將屏蔽信號，故金屬機身手機需設計天線帶（邊框上之塑料條）。玻璃與塑料機身對信號影響較小。

網絡技術演進：

- **4G LTE**：理論峯值速率下行 1Gbps/上行 500Mbps（Cat.18），實際體驗通常 50-150Mbps。覆蓋最廣，夠用。
- **5G NR**：理論峯值速率下行 10Gbps+。Sub-6GHz 頻段覆蓋較好，mmWave 毫米波速度極快然覆蓋極小。
- **5G-A（5G Advanced）**：5G 之增強版，亦叫 5.5G。下行速率可達 10Gbps，上行 1Gbps，連接數密度與定位精度大幅提升。2024 年起中國移動、電信、聯通開始於重點城市部署 5G-A 網絡，華爲 Mate 70/80 系列、小米 15/16 系列等旗艦手機已支持 5G-A。5G-A 尚帶來矣" 通感一體" 能力，基站可同時通信與感知環境，爲低空經濟（無人機配送）與車聯網提供基礎設施。

網絡頻段分類：

手機支持之網絡頻段決其能在哪些國家、哪些運營商之網絡上使用：

- **FDD-LTE Band（B 系列）**：如 B1、B3、B5 等，4G 頻段。不同國家用不同頻段，中國主要用 B1/B3/B5/B8/B34/B38/B39/B40/B41。
- **TDD-LTE Band**：4G 之另一制式，中國大量使用 B38/B39/B40/B41。
- **5G NR Band（N 系列）**：如 n1、n3、n28、n41、n78、n79 等。n78（3.5GHz）乃中國 5G 主力頻段，n41（2.5GHz）移動使用較多。
- **購買建議**：若需出國使用，購買前確認手機是否支持目標國家之頻段。國行手機通常對中國頻段支持最全，然或缺少部分海外頻段。

六、汝之手機，到底在演幾何角色？

其早已不唯電話：

1. **通訊工具**：電話、短信、微信、郵件……
2. **智能相機**：拍照、錄像、掃描文檔……
3. **錢包**：支付、公交卡、銀行卡、會員卡……
4. **信息門戶**：新聞、天氣、股票、知識……
5. **遊戲機**：自消消樂至 3A 大作……
6. **移動辦公室**：文檔、表格、會議、協作……
7. **健康助手**：計步、心率、睡眠、運動……
8. **智能家居遙控器**：控制燈、空調、電視、掃地機器人……
9. **數字身份證**：電子身份證、電子駕照、健康碼……
10. **娛樂中心**：音樂、視頻、電子書、播客……

思之，一臺小小之設備，承載幾何生活！故，學善用之，即學更好地生活。

實操一番：了解硬件配置

實操一番：了解硬件配置

第一章：敬禮，吾之私用電腦

買手機時，汝常見“*12GB+8GB* 運存”此等宣傳——似憑空多了 *8GB* 運行內存。然其真能使手機變快乎？會否反傷手機？此節吾等即將“內存拓展”此技術徹底講清楚。

一、本質定義

安卓手機之“內存拓展”（又稱內存擴展、內存融合、RAM Plus 等）**並非真正物理增加運行內存芯片**，乃一項系統級軟件技術：通過將手機部分內部存儲空間（ROM/UFS 閃存）動態劃轉，模擬爲虛擬運行內存（RAM）使用。

，乃一項系統級軟件技術：通過將手機部分內部存儲空間（ROM/UFS 閃存）動態劃轉，模擬爲虛擬運行內存（RAM）使用。

二、技術原理

主流廠商之實現方式主要有二：

zRAM 壓縮

於 RAM 內部劃分一塊壓縮數據空間，將不活躍之應用代碼進行壓縮存儲，壓縮率約 2:1。數據仍在 RAM 中，速度快、不損耗閃存，然需 CPU 參與壓縮解壓。

swap 分區

直接於 UFS 閃存中劃分固定空間作爲虛擬內存，將後臺進程數據寫入存儲。依賴存儲帶寬，讀寫速度遠低於物理內存。

直接於 UFS 閃存中劃分固定空間作爲虛擬內存，將後臺進程數據寫入存儲。依賴存儲帶寬，讀寫速度遠低於物理內存。

三、速度差距乃關鍵瓶頸

此技術之根本侷限在於速度差異。今主流之 UFS 4.0 閃存順序讀速約 4200MB/s，僅爲 LPDDR5X 物理內存之 1/10-1/15。故於輕負荷之 zRAM 壓縮階段速度接近真實 RAM，然一旦進入高負載場景，速度將驟降至幾十分之一。

四、適用場景與收益邊界

內存拓展並非萬能，效果因機型與使用場景而異：

- **有用之情況**：6GB 物理內存 +UFS 3.1 以上、剩餘空間 ≥30GB 之機型，開啓 3GB 拓展後後臺保活數可多 3-4 個，重載率降低 68%，宜外賣小哥、網約車司機等重度多任務用戶。
- **日常場景**：對微信多開、網頁標籤保活、即時通訊消息響應等改善顯著，後臺應用冷啓動耗時明顯縮短。
- **有限場景**：對大型遊戲加載、4K 視頻剪輯等高帶寬場景提升有限。
- **有害之情況**：8GB 內存 +UFS 2.2 且剩餘空間不足之機型，開啓後遊戲幀率波動反擴大，刷短視頻多耗電，切換卡頓。

- **無效之情況**：16GB 以上大內存機型收益幾乎爲零，最多多保活 1 個 APP，卻將使 CPU 負載增加 12%、發熱上升 2-3℃。對 RAM 大於 12GB 之機型，建議直接關閉以避免無意義之壓縮計算。

五、各廠商命名與開啓方式

不同品牌叫法各異：小米稱“內存擴展”、華爲謂“內存融合”、三星稱“RAM Plus”、vivo 命名爲“Extended RAM”、一加稱作“Memory Extension”。此功能屬系統原生能力，與安卓底層版本無直接綁定關係。

一般開啓路徑爲：進入【設置】【更多設置】尋【內存】或【性能】子項開啓“內存拓展”並擇容量檔位（通常 2GB-7GB），重啓後生效。開啓後系統將顯示如“8GB+4GB”之內存組合。

注意之事

- 內存拓展將佔用部分機身存儲空間，請確保剩餘存儲不少於 20-30GB，否則或影響系統流暢度。
- 閃存作爲虛擬內存將有讀寫損耗，雖今 UFS 閃存壽命較長，然長期高頻 swap 仍或加速存儲老化。
- 閃存作爲虛擬內存將有讀寫損耗，雖今 UFS 閃存壽命較長，然長期高頻 swap 仍或加速存儲老化。

小結

小結

來動手：觀汝手機有無內存拓展

1. 啓【設置】，檢索” 內存擴展”、” 內存融合”、”RAM Plus” 或”Extended RAM”。
2. 尋後觀當前是否開啓，及拓展容量設置幾何。
3. 尋後觀當前是否開啓，及拓展容量設置幾何。
4. 嘗試關閉或調低拓展容量，觀察一週後臺保活與發熱變化，尋最適自己之設置。

第一章附：鴻蒙操作系統專題

鴻蒙（HarmonyOS）乃華爲自主研發之操作系統，自 2019 年發佈至今，已歷數個大版本。其不唯“又一安卓替代品”，乃有獨特設計理念之新物種。讓吾等一同深入識此正在崛起之操作系統。

一、鴻蒙爲何物？何以與衆不同？

鴻蒙之核心理念乃“萬物互聯”。異於傳統操作系統主要服務於單一設備，鴻蒙自設計之初即考慮了多設備協同。

分佈式架構乃鴻蒙之靈魂。簡言之，其可將多個設備虛擬成一個“超級終端”。汝之手機、平板、計算機、手錶、電視，不再是孤立之個體，乃一個整體之“器官”，可互相調用對方之能力。

譬如：汝用手機觀視頻，欲於大屏上觀，傳統方式乃投屏；而於鴻蒙上，汝可直接將視頻“流轉”至電視上續觀，手機化爲遙控器。此非簡單之投屏，乃應用本身於跨設備運行。

二、鴻蒙 4.3：基于 AOSP 之成熟版本

鴻蒙 4.3 乃當前華爲手機上廣泛部署之版本。須說明者，鴻蒙 4.3 兼容安卓開源項目（AOSP）之代碼框架，故其能完整兼容安卓應用——汝於華爲應用市場下載之大部分 App，本質上仍是安卓應用，通過鴻蒙之兼容層運行。然鴻蒙 4.3 並非簡單套殼安卓，其於 AOSP 基礎上加入了大量華爲自研之分佈式能力、方舟引擎等技術。

核心特性：

- **安卓應用兼容**：由於基於 AOSP 開發，鴻蒙 4.3 可正常運行幾乎所有安卓應用，用戶幾乎感受不到差異。
- **方舟引擎**：華爲自研之編譯器與運行時，使應用運行更流暢、更省電。較早期版本，流暢度提升明顯。
- **超級終端**：於控制中心，可直觀地見周圍之其他鴻蒙設備，拖拽即可連接。手機調用計算機之鍵盤鼠標、平板調用手機之攝像頭，皆甚方便。
- **服務卡片**：類似 iOS 之小組件，然更靈活。可上滑 App 圖標展開卡片，顯示關鍵信息，不用打開 App。
- **純淨之模**：唯允許從官方應用市場安裝應用，減惡意軟件風險。
- **隱私中心**：集中管理應用權限，觀應用行爲記錄。

用戶界面特點：

- **雜誌鎖屏**：鎖屏界面顯示精美圖片與文章，拂去即入桌面。
- **萬能卡片**：桌面圖標可展開成卡片，顯示更多信息。
- **超級中轉站**：一個臨時存儲區，可將文字、圖片、文件暫存，而後拖至任意 App 中用。跨應用分享變得無比絲滑。
- **智慧搜索**：全局檢索，可搜應用、文件、設置、網頁。

三、鴻蒙 NEXT（鴻蒙 6）：華爲自主操作系統之真正起點

鴻蒙 NEXT（HarmonyOS NEXT 5.0）與鴻蒙 4.3 有根本性之別——其採華爲自研之鴻蒙微內核，不再依賴 Linux 內核與 AOSP 代碼，亦**不再直接支持安卓應用**。此乃鴻蒙自“兼容安卓之過渡形態”正式走向“完全獨立之操作系統”之關鍵一步。

重大變革：

- **完全自主內核：**鴻蒙 NEXT 用華爲自研之內核，不再包含任何 AOSP 代碼。系統更精簡、更安全、更高效。
- **不再支持安卓應用：**此乃最大之變化。鴻蒙 NEXT 唯能運行原生鴻蒙應用，無法安裝與運行 APK 文件。升級前請確認汝常用之 App 已有鴻蒙原生版本。
- **原生應用生態：**華爲正全力推動開發者將應用遷移至鴻蒙原生平臺。目前主流 App 如微信、支付寶、淘寶等已推出鴻蒙原生版本，然部分小衆應用或尚未適配。
- **星盾安全架構：**全新之安全體系，自硬件至軟件之全鏈路保護。支持更細粒度之權限控制，譬如” 模糊定位”——應用唯知汝大概何處，不知精確位置。
- **AI 深度融合：**系統級 AI 能力，不唯某個 App 有 AI。譬如 AI 字幕可實時翻譯視頻，AI 摳圖可智能識別主體。
- **跨設備無縫流轉：**更進一步之分佈式能力。應用狀態可於設備間無縫切換，汝於手機上編輯之文檔，行至計算機前自動於計算機上打開。
- **元服務：**無需安裝即可用之輕量級服務。譬如掃二維碼直接打開某個功能，不用先下載 App。

升級鴻蒙 NEXT 前必讀

升級鴻蒙 NEXT 前必讀

操作邏輯變化：

- **手勢導航：**左右邊緣滑動返回，底部上滑回桌面，上滑停頓進多任務。
- **智慧多窗：**支持分屏、懸浮窗、平行視界等多種多任務形態。
- **暢連通話：**華爲之視頻通話服務，支持屏幕共享、美顏、背景虛化。

四、鴻蒙生態：設備與應用

鴻蒙不唯手機系統，乃一個覆蓋多種設備之生態系統。

支持之設備類型：

- **手機與平板**：核心設備，功能最完整。
- **智能手錶與手環**：獨立運行或與手機協同。
- **智慧屏（電視）**：家庭娛樂中心。
- **智能音箱**：語音交互入口。
- **汽車**：鴻蒙座艙，智能駕駛艙體驗。
- **智能之家**：冰箱、空調、洗衣機等。

應用生態現狀：

鴻蒙應用市場已有數百萬應用，主流 App 基本皆有。華爲亦於積極推動開發者開發原生鴻蒙應用。截至 2026 年中，微信、支付寶、淘寶、抖音、美團、京東等頭部應用均已推出鴻蒙原生版本，日常使用基本無障礙。部分小衆應用與行業應用或仍於適配中，升級前建議確認汝常用之 App 是否已支持。

五、鴻蒙實用技巧

- **超級終端**：自屏幕右上角下滑啓控制中心，點” 超級終端”，可見周圍之其他設備，拖拽連接。
- **超級中轉站**：長按文字/圖片/文件，擇” 複製到中轉站”，而後於其他 App 中長按粘貼，或拖拽出來。
- **服務卡片**：上滑 App 圖標展開卡片，長按卡片可調大小或添至桌面。
- **智慧多窗**：自屏幕左右邊緣向內滑動並停頓，啓側邊欄，點應用以懸浮窗啓。
- **隱私保護**：設置 > 隱私，可觀應用權限使用記錄，設模糊定位等。

注意之事

注意事項

今即試之：體驗鴻蒙特色功能

1. 開控制中心，尋” 超級終端”，觀四周有無他鴻蒙之設備。
2. 開控制中心，尋” 超級終端”，觀四周有無他鴻蒙之設備。
3. 上滑數個常用 App 圖標，觀其服務卡片所顯何內容。
4. 往設置 > 隱私，查閱近時何應用曾用敏感權限。

第二章：整理汝之數字之室——文件管理

手機用久則亂，善習始於文件之管理。猶汝之室須定時整理，汝之數字空間亦須打理。井然有序之手機，尋物既便，運行亦暢。

一、三層存儲之體：汝之數字倉

1. **機身存儲**：核心之區，速度迅捷，系統與 App 皆在於此。此乃汝手機至寶之空間，宜精打細算。
2. **SD 卡（若支持）**：擴展之衣櫃，速度稍緩，宜於存放電影、照片等大文件。注：今諸多旗艦機已不支持 SD 卡。
3. **雲存儲**：保險之櫃兼同步之網盤。防丟失，亦能於多設備間同步。iCloud、百度網盤、OneDrive 皆常用之選。

二、識“文件管理器”

安卓一般有 [文件] 或 [文件管理] App，iOS 乃自帶之 [文件] App。乃汝檢視手機諸文件之“資源管理器”也。

常見文件夾之義：

- **DCIM**：相機所攝之照片與視頻。
- **Pictures**：他圖，如截圖、所存之圖。
- **Download**：瀏覽器所下之文件。
- **Music**：音樂文件。
- **Movies**：視頻文件。
- **Documents**：文檔文件。
- **Android**：安卓系統之文件夾，內爲各 App 之數據，勿手動刪之。

三、文件系統之結構：深入知之

安卓本於 Linux，文件系統有層級之構。雖常人不必深究，然知其基本之義亦有所助。

存儲分區：

- **/system**：系統分區，存操作系統之文件。常人無權訪問，須 root 之權。
- **/data**：數據分區，存用戶數據與應用數據。此乃汝日常所用之空間。
- **/sdcard 或 /storage/emulated/0**：用戶可訪問之存儲空間，汝之照片、下載文件皆在於此。

App 數據之存儲：

- **內部存儲**：App 之私有空間，他 App 不能訪問。卸載 App 時則刪之。
- **外部存儲**：App 於用戶可訪問處所存之文件，如照片、下載之物。卸載 App 時未必刪之。

四、數據之備份與恢復：爲汝數據上保險

數據無價，備份乃必養之習。

備份之策：

1. **雲備份**：自動而便，然須網絡與雲空間。iCloud、華爲雲、小米雲等皆支持。
2. **本地備份**：以數據線連電腦，用官方之具備份。速快、空間大，然須手動操作。
3. **雙重備份**：雲備份加本地備份，至爲安全。

備份何物？

- **照片與視頻**：最佔空間，亦最珍貴。
- **通訊錄**：存於雲賬號中最爲便。
- **微信聊天記錄**：用微信自帶之備份功能。
- **重要文檔**：存于云盤或電腦上。

五、文件之安：護汝之隱私

- **敏感文件加密**：諸手機有” 安全文件夾” 或” 私密空間” 之功能，可將敏感文件納其中，須密碼或指紋方能訪問。

- **刪除非銷燬**：常之刪除僅標記空間可覆，數據猶在。欲徹底刪除，須用專門之安全擦除之具，或多次覆寫。

- **云端之安**：擇信譽佳之雲服務商，開雙重之認證，定期查登錄之設備。

六、空間不足乎？來一番大掃除

- **清理緩存**：App 之臨時記憶，清之則騰地。於設置 > 存儲中可清理。

- **尋大文件**：用文件管理器按大小排序，揪出佔空間之”元兇”。

- **刪殘留**：卸載 App 後，時有垃圾之文件殘留，宜手動清之。

- **清理相冊**：刪重複之照、模糊之照、不需之截圖。

- **清理微信**：微信乃空間之大戶，定期清理聊天記錄與緩存甚爲要。

七、常見文件格式全解析

手機中諸文件後綴名令人頭眩，明乎其義可助汝善管文件：

格式	類	類	特性
JPG/JPEG	圖	圖	相冊/任意圖覽器
相冊/任意圖覽器	圖	圖	相冊/任意圖覽器
相冊/任意圖覽器	音	音	有損壓縮，體小，兼容性極佳。最通用之音頻格式。
任意音樂播放器	音	音	無損壓縮，音質與 CD 同，體比 MP3 大三至五倍。發燒友之首擇。
音樂播放器/專業播放 App	音	音	無壓縮之原始音頻，音質至佳而體巨。專業錄音常用。
音樂播放器	音	音	AAC 之編碼，同碼率下音質勝於 MP3。蘋果生態常用。
Apple Music/音樂播放器	MKV	視頻	最通用之視頻封裝格式，兼容性極佳。手機錄視頻之默認格式。
相冊/任意視頻播放器	MKV	視頻	開源之封裝格式，支持多音軌、多字幕。電影資源常用。
VLC/nPlayer	PDF	文檔	Excel 電子表格。XLSX 乃壓縮格式。
WPS/Office	PPT/PPTX	演示	Excel 電子表格。XLSX 乃壓縮格式。
WPS/Office	XLS/XLSX	表格	Excel 電子表格。XLSX 乃壓縮格式。
WPS/Office	PDF	文檔	跨平臺之格式，排版固定不可編輯。合同、論文、電子書常用。
Adobe Reader/WPS	RAR	壓縮	最通用之壓縮格式，跨平臺兼容性佳。
系統自帶/解壓 App	RAR	壓縮	壓縮率高於 ZIP，然須專用軟件解壓。
解壓 App	ISO	鏡像	光盤鏡像之文件，完整複製光盤之內容。系統安裝包常用。
解壓 App/虛擬光驅	HTML	網頁	網頁之文件，用瀏覽器開之。所存之網頁常用此格式。

練一練：整理”下載”文件夾

練一練：整理“下載”文件夾

八、圖與文件系統之管理：使資產井然有序

上節吾等習於“識文件夾”，然識之猶未足也。本節所講者，**何以主動建一套可久行之管理規範**，使汝之文件隨時可尋、不失、不爲人窺。

文件之分類與命名之規

亂之源乃無規。建規則實爲易事，要在於**堅持**。

- **”時間 + 事件”命名之法**：文件夾之名帶時間戳與事件之述，便於日後回憶與檢索。如 `202606_` 旅行票據、`2025` 年終總結、`2024` 孩子生日。如此既見時間之線，亦可一目知其內容。
- **三級分類之體**：
  - **一級**：按文件之類（文檔 / 圖 / 視頻 / 音）
  - **二級**：按使用之場（工作 / 生活 / 學習）
  - **三級**：按具體之事件或項目（如”2026Q2_ 項目 A”）
- **月度清理之計劃**：每月固定一日（建議發薪之日或月初一日），對”下載”目錄集中整理：**有用者歸檔、無用者刪、珍貴者轉存**。一年十二次小整理，勝於積一年而大掃除。

本月任務清單

本月任務清單

1. 于文件管理器中建 `2026Q2_ 項目 A` 形之本月工作文件夾；
2. 形之本月工作文件夾；
3. 爲今年之家庭照建 `2025 家庭`、`2026 家庭`此形之年度文件夾。

冗餘文件處理之策

手機用久則“長胖”，多半乃冗餘文件之作祟。

- **七十二時之自動清理**：微信、QQ 等社交 App 所接收之臨時文件（表情包、聊天圖、小視頻），大多觀一次即無用。於 [微信] > [設置] > [通用] > [存儲空間] 中開”自動清理”，將過期緩存設爲七十二時。手機頓然清爽，且不丟重要之記錄。
- **存儲空間之定向清理**：安卓 [設置] > [存儲] > [清理加速]，iOS [設置] > [通用] > [iPhone 存儲空間]，系統將按”緩存 / 臨時文件 / 可卸載 App / 大文件”分類列出。重點清理：

(1) 緩存文件；(2) 已過期之安裝包（升級 App 後所留之舊版安裝包）；(3) 重複之文件（同名同尺寸之照）。

- **自動分類之規**：諸品牌相冊 App 支持智能分類：人物、風景、文檔、截圖。開啓後，新拍之合影自動歸入” 人物 > 張三”，掃描之合同自動歸入”PDF”。**配以每月之整理，可大降” 尋物不得” 之概率**。

圖之安全處理之規

圖乃手機中最“敏感”之數據類型——一張隨手之拍即能暴露汝之位置、設備、時間。吾等建議**建“雙重保護”之工作流**：

發送證件照/敏感截圖之標準流程

1. **可見信息打碼**：用系統自帶或第三方之具（如美圖秀秀、Snapseed）**塗抹身份證號之中間數位、車牌號、門牌、姓名、金額**等關鍵字段；
2. **元數據之剝離**：發送前於相冊中點” 分享” 或” 導出”，系統將問是否保留位置之信息，擇” **不保留**”。如此拍攝之設備型號、GPS 座標、拍攝之時間即不隨圖同發；
3. **臨時水印**（可選）：于敏感截圖上加” 僅供 XX 辦理使用” 之文字水印，避被濫用。

元數據（EXIF）清理之常用方法（基礎操作，深度防護見第六章“照片脫敏處理”）：

- **系統自帶**：iOS [設置] > [隱私與安全性] > [相機]，可閉” 精確位置”；分享時擇” 僅當前圖” 將**自動剝離大部 EXIF**。
- **安卓**：諸品牌相冊 App 于” 分享” 時將提示” 是否移除位置之信息”，勾選可也。
- **專業之具**：ExifTool（電腦端）、Metapho（iOS）、Photo Metadata Remover（安卓）可徹底清除所有元數據。

對於需**長期保存**之敏感圖（身份證、銀行卡、戶口本之掃描件、護照），建議存於**加密相冊**：

- iPhone：[照片] > 選中照片 > 隱藏（僅 Face ID 驗證後可覽）；
- iPhone：[照片] > 選中照片 > 隱藏（僅 Face ID 驗證後可覽）；
- 第三方加密 App：**360 安全相冊、騰訊相冊管家（隱私相冊）、KeepSafe** 等，單獨密碼保護，隱蔽性更佳。

第三章：使手機接“生命之線”——網絡

無網，智能手機則“智障”其半。網絡乃現代手機之生命之線，解之、用之、護之，乃每個手機用戶之必修課。

〇、聯網不迷——分清 Wi-Fi 與流量之“路”與“車”

衆人之手機屏幕上方同時亮着 Wi-Fi 與信號之圖標，然於二者之本區別卻一頭霧水。最經典之誤即：**連着 Wi-Fi 卻刷不出頁**——蓋因路由器斷網，而手機未自動切迴流量。欲徹底明此問題，須建一“路與車”之認知之模。

核心之喻：路與車

視互聯網如一城。**“路”**乃汝手機至互聯網之間之通道，**“車”**乃行於路上之數據。Wi-Fi 與流量乃兩條不同之路，然數據（車）之本身則一也。

本質之別：本地通道 vs 遠程高速

Wi-Fi 本質上乃一種**局域網之技**，猶汝家樓下無線鋪設之路，司將數據自路由器傳至手機。然若寬帶欠費或光纜斷，路雖猶在，外之“高速公路”卻已斷，自無處可往。而網絡流量（移動數據）則經運營商之 4G/5G 基站傳輸，相當於汝直開於運營商之高速專線，有信號即直連互聯網。

費用與場：固定月租 vs 按量計費

Wi-Fi 通常爲一次性之投（寬帶月費），但電費不斷，全家無限用，甚宜下電影、玩遊戲等大流量之場。而流量則按用量計費，超套餐或降速或額外扣費，更宜於地鐵、出差等移動之場處理輕量之事。

來一試：檢汝之流量套餐

開手機之 [設置] > [移動網絡] > [流量管理]，觀今月尚餘幾何流量、何 App 最耗流量。心中有數，出門方不慌。

安之慮：公共 Wi-Fi 乃隱之“釣魚場”

於安之上，移動數據通常勝於公共 Wi-Fi。移動數據由運營商加密，而公共 Wi-Fi 實易爲黑客搭爲“僞熱點”截獲未加密之流量。然此處須分清：於 HTTPS 全普及之今，攻擊者能截獲者乃**汝所訪之網站（域名）**，以及**未加密之 HTTP 內容**；而微信消息、銀行 App 之登錄密碼等經 HTTPS 加密之數據，攻擊者**不能直讀**——除非施更復雜之 SSL 剝離攻擊且用戶略之證書之警。然即便如此，於外行支付之事，直用手機流量仍爲最穩之擇。

安全提醒

於機場、商場、咖啡廳等公共之所，勿連名號可疑之免費 Wi-Fi（如“Free_WiFi_2024”）。縱連之，亦絕勿於其上登銀行 App 或輸密碼。

手機端之判：解狀態欄之“暗語”

何以知手機此刻用何聯網？以華爲手機爲例，狀態欄中運營商信號旁或 WLAN 信號旁現**雙向箭之圖標**，即表正用對應之方上網；若二者同時亮，則網絡加速生效，手機正同時用兩路。

iPhone 用戶則更直觀：Wi-Fi 圖標亮即在用 Wi-Fi，不亮時觀右上角之信號格即在用流量。部分安卓手機亦於狀態欄顯“5G”或“4G”之字，徑告汝今之移動網絡之類。

一、移動網絡（4G/5G）與流量

移動網絡乃汝隨時隨地聯網之基。自 2G 至 5G，速度增上千倍，然其理實同。

網絡之制：

- **4G LTE**：今覆蓋最廣，速度足用（理論上最高百 Mbps+），耗電相對低。
- **5G**：速快（理論上最高 10Gbps），然覆蓋猶在完善，耗電高。宜下大文件、觀高清視頻。
- **5G SA vs NSA**：SA 爲獨立組網，NSA 乃依託 4G 基站。SA 之體驗更佳，然須運營商與手機皆支持。

流量之管：

- **流量何物**：即手機上網所生之數據量。觀視頻、刷圖最費流量，文字與語音相對省。
- **省流之小貼**：於設置中開” 省流量模式”，Wi-Fi 下再更 App 或觀高清視頻。
- **信號格 $\neq$ 網速**：信號強僅表連接佳，網速猶視基站之忙閒、汝距基站之遠近。

重要之警：出國前請閉【數據漫遊】!

於國外用移動數據或生天價之賬單！出國前務於設置中閉數據漫遊，或先購當地流量卡/國際漫遊套餐。

二、Wi-Fi：連接、分享與安

Wi-Fi 乃家與辦公室聯網之主將，速快、不費流量。然 Wi-Fi 實亦不斷進化，不同之標差甚大。

連 Wi-Fi：

往 [設置] > [WLAN] 擇網絡輸密碼。今諸多手機支持掃碼連接，更爲便。

分享密碼：

安卓與 iOS 皆能使已連之手機速分享密碼於友。安卓生二維碼，iOS 可靠近自動分享。

Wi-Fi 之標演進：

Wi-Fi 之標由 IEEE 制定，每代皆顯著提升：

標準	協議	頻段	最高速率	特點
Wi-Fi 4	802.11n	2.4/5GHz	600Mbps	老舊之標，覆蓋尚可
Wi-Fi 5	802.11ac	5GHz	3.5Gbps	今最普及，速度足用
Wi-Fi 6	802.11ax	2.4/5GHz	9.6Gbps	多設備之能強，延遲低
Wi-Fi 6E	802.11ax	2.4/5/6GHz	9.6Gbps	增 6GHz 頻段，不擁堵
Wi-Fi 7	802.11be	2.4/5/6GHz	46Gbps	極速傳輸，多鏈路聚合

- **Wi-Fi 6 之關鍵技**：OFDMA（正交頻分多址）使多設備同傳，MU-MIMO（多用戶多入多出）提升多設備併發之效，TWT（目標喚時）使設備更省電。
- **Wi-Fi 6E**：於 Wi-Fi 6 之基增 6GHz 頻段，供更多信道，減干擾。須手機與路由器皆支持。
- **Wi-Fi 7**：引入 MLO（多鏈路操作），可同時使用多個頻段傳輸數據，大幅降低延遲。320MHz 超寬信道使速率飛躍。驍龍 8 Elite Gen5、天璣 9400 等旗艦芯片平臺均已支持 Wi-Fi 7，小米 17、華爲 Mate 80、三星 S26 等機型可使用 Wi-Fi 7 網絡（需配合 Wi-Fi 7 路由器）。
- **2.4GHz vs 5GHz**：2.4GHz 穿牆佳而速慢、干擾多；5GHz 速快而穿牆弱。Wi-Fi 6E/7 之 6GHz 頻段速最快，而穿牆最弱，宜同一室中用。

公共 Wi-Fi 之險：

公共 Wi-Fi 或爲黑客竊聽數據或設釣魚熱點。守則：

- 避登銀行賬戶、輸密碼。
- 最佳用 VPN 加密連接。
- 勿自動連陌生 Wi-Fi。
- 用 HTTPS 網站（地址欄有鎖之圖標）。

三、藍牙：短距無線通訊之萬能手

藍牙或爲汝手機上用頻最高之無線技之一——無線耳機、智能手錶、車載系統、鍵鼠，皆賴其連。

技之理：

藍牙工作於 2.4GHz ISM 頻段，用跳頻擴頻之技（FHSS）將數據散於七十九頻道上速切傳，有效避干擾。藍牙之通訊距分三級：Class 1 約百米、Class 2 約十米（手機常用）、Class 3 約一米。

版本之演進：

- **藍牙 4.0**：引入低功耗藍牙（BLE），使手環、心率帶此類小電池之設備亦能無線連，續航自天變月。
- **藍牙 4.2**：提升傳輸之速與隱私之護，支持 IPv6 直連。
- **藍牙 5.0**：傳輸之速倍增（2Mbps），傳輸之距四倍（理論三百米），廣播之能八倍。支持雙音頻（同時連兩副耳機）。
- **藍牙 5.1**：引入方向尋址之能（AoA/AoD），定位之精可達釐米級。
- **藍牙 5.2**：LE Audio 新標，支持 LC3 之編碼（音質更佳、功耗更低）、多流音頻、廣播音頻。
- **藍牙 5.3/5.4**：進一步優化功耗與連之穩，增強週期性廣播。

LE Audio 之普及況：2025-2026 年，LE Audio 已進入快速普及階段。主流品牌之新款真無線耳機（如華爲 FreeBuds Pro 5、小米 Buds 6 Pro、三星 Galaxy Buds3 Pro 等）均已支持 LE Audio 與 LC3 編碼，提供更低延遲與更好音質。購買新耳機時，建議優先選擇支持 LE Audio 之產品。

應用之場：

- **音頻之傳**：無線耳機、車載之響。藍牙 5.2 之 LE Audio 使音質與延遲皆有質之飛躍。
- **數據之傳**：近距傳文件、照。雖速不如 Wi-Fi Direct，然勝於簡。
- **設備之配**：智能手錶、手環、鍵鼠、遊戲手柄。
- **位置之服**：藍牙信標（Beacon）用於室內導航、物之追蹤（如蘋果 AirTag）。
- **智能之家**：藍牙 Mesh 網控燈泡、門鎖等。

四、紅外遙控：手機化萬能遙控

紅外遙控乃最古之無線控技之一，然於手機上猶爲實用。部分安卓手機（如小米、華爲部分機型）內置紅外發射器，可遙控電視、空調、機頂盒等家電。

工作之理：

紅外遙控經紅外發光二極管發特定頻（通常 38kHz）之紅外光脈衝信號，家電之紅外接收器解此脈衝而執對應之操。不同之品牌用不同之編碼之協議，常見者有 NEC、RC5、RC6、SONY 等。手機中之紅外 App 內置各品牌之編碼庫，擇品牌與型號即能配。

局之限：

- **方向之性**：紅外乃直線傳，須對設備發之，不能隔牆。
- **距之短**：有效之距一般五至八米。
- **蘋果不支**：iPhone 無紅外發射器，不能用此功。

五、NFC：碰一碰之法

NFC（Near Field Communication，近場通訊）乃一種極短距（十釐米以內）之高頻無線通訊之技，頻十三點五六 MHz，傳輸之速最高 424Kbps。

通訊之理與模：

NFC 本於 RFID 技而變，支持三作之模：

- **讀寫之模**：手機讀 NFC 標籤中之信，如讀 NFC 貼紙而觸動作。
- **點對點之模**：兩臺 NFC 設備互傳數據，如 Android Beam（已停用）。
- **卡模擬之模**：手機模擬一智能卡，此乃最常用之模——公交卡、門禁卡、銀行卡皆此模也。

安之機：

- **超短之距**：十釐米內之通訊距本身即物理之安，遠竊幾乎不能。
- **安之芯**：手機內置獨立之安之芯（SE）或用 HCE（主機卡模擬），支付之鑰存於安之區，即手機被 root 亦不能提。
- **指紋/面容之驗**：支付時鬚生物之識以認，即手機失亦不能盜刷。
- **令牌之化**：Apple Pay、華爲 Pay 等不傳真卡號，乃用一次之令牌，即被截亦不能複用。

典型之用：

- **公交卡/門禁卡**：將實體卡之信寫於手機，刷手機即行。
- **移動之付**：Apple Pay、華爲 Pay 等，將銀行卡綁於手機，碰一碰即付。
- **設備之配**：NFC 碰一碰速配藍牙耳機、音箱。
- **身份之驗**：NFC 身份證之讀、電子駕照之驗。

六、個人熱點：將手機變 Wi-Fi 路由器

已流量多時，可開熱點以予平板、電腦用。

- **開啓之法**：設置 > 個人熱點/網絡共享，開啓之。
- **注意之事**：甚耗電與流量，宜設一複雜之密碼，並限連設備之數。
- **USB 之共**：以數據線連電腦，較無線之點更穩、更快。

動手一試：設並管熱點

1. 開 [設置]，尋” 個人熱點” 或” 網絡共享”，開啓之，設好名與密碼。
2. 用另一設備（如電腦）搜而連此 Wi-Fi。
3. 於手機端觀之，能否理已連之設備，或踢去不識之設備。

第四章：安裝與管理 App——手機之“技能包”

App 使手機無所不能，然亦須管之。猶汝予新員工安排工位、分配權限也，裝 *App* 亦須一套規範之流程。

一、官方之店：至安之“應用超市”

- **安卓**：Google Play（國外）或各品牌自帶之店（如華爲應用市場、小米應用商店）。
- **iOS**：惟 App Store。
- **鴻蒙**：華爲應用市場。

核心之安之義：經審覈、更有所保、支付之安。行之指：**首選惟用官方之店下載！**

二、純淨/防護之模：開手機之“防毒牆”

主流之國產品牌（華爲、小米、OPPO、vivo、榮耀）皆內置了**純淨之模**或**防護之模**，此乃系統級別之安之開關。**強烈建議諸用戶皆開之**。

其究何物？

純淨之模（亦稱“防護之模”）之**核心之功乃阻第三方 App 之安**：當其開時，系統將**惟允自官方應用之店裝 App**，自瀏覽器、二維碼、聊天軟件等渠道所下之安裝包將直被攔，從源頭杜惡意之軟件。

適用之人

- **年老之用戶**：分不清”官方”與”假冒”之告，純淨之模乃至穩之護符。
- **對應用之權不敏之用戶**：不需裝小衆 App、測試版、外服遊戲。
- **欲升設備安之諸用戶**：縱爲發燒友，開之亦勝於不——真有臨需，再獨閉可也。

開啓之徑（以華爲/小米爲例）

- **華爲/鴻蒙**：[設置] > [系統和更新] >**[純淨之模]**，擇” 開”。
- **小米**：[設置] > [隱私保護] >**[保護隱私]** 或 [設置] > [應用設置] > [應用安裝]，閉” 安裝未知應用”。
- **OPPO/vivo**：[設置] > [安全] >**[安裝外部來源應用]**，逐個閉授權。

應用之店之安檢測之機

應用之店非“擺設”，其背有完之安之流水線：

- **病毒之掃**：每 App 上架前經數家殺毒引擎聯合掃，識已知木馬與病毒之特徵。
- **權限之核**：店將審 App 所請之權限是否” 合”——一電筒 App 要通訊錄之權？直駁或下架。
- **開發者之認**：上架須企業之資、個人身份證或信用卡之認。出問題能尋人，非” 黑 App”。
- **持續之監**：已上架之 App 亦被持續復掃，新發之惡意行將強下架。

三、第三方之來與“電子寄生蟲”：險之極高！

自網頁或不明之鏈下 App，或夾帶病毒、竊隱私、盜刷支付。此處吾等特提一新概：**“電子寄生蟲”**。

何爲“電子寄生蟲”?

指彼等**被植木馬之“破解版”軟件**——汝以爲“省了錢”，實則“中了招”。常見之形：

- ” 破解版”、” 免費版”、” 去告版” 付費軟件；
- ” 破解版”、” 免費版”、” 去告版” 付費軟件；
- 羣聊中” 小告” 所薦之所謂” 內部版”App。

此等程序表面能用，背或在**偷通訊錄、讀短信、監麥克風、後臺挖礦**。純淨之模乃防之**第一道防線**——直不使汝裝。

第二道防線之建議：

- **定期全盤病毒掃**：用手機自帶之安中心（如華爲手機管家、小米安中心）每月掃一。
- **慎予敏感之權**：通訊錄、短信、位置、相機、麥克風——此五” 高危之權” 必嚴審。
- **關注耗電與流量之異**：手機突燙、電速掉、流量半夜跑——八成後臺藏” 寄生蟲”。

若萬不得已須裝第三方 App：

1. 保來源絕對可信。
2. 保來源絕對可信。
3. 事後用安 App 掃。

終極之建議：於系統設置中，閉“安裝未知應用”之權。

四、裝、更、卸

- **裝**：店中點” 獲取” 或” 裝”，裝畢首開將問汝要權。
- **更：必更！**主爲修安之洞。建議開自動更。
- **卸**：長按圖標拖至卸區。欲清更徹，往 [設置] > [應用管理] 中尋而卸之。

五、權限之管：汝乃 App 之“老闆”

App 要權（如通訊錄、位置）有時合（地圖要位置），有時不合（電筒要通訊錄）。

- **權限之擇**：” 始終允”、” 僅用時允”、” 禁止”、” 每問”。
- **管之則**：
  1. 最小化：非必不與。
  2. 最小化：非必不與。
  3. 情境化：似打車軟件，惟用時予位置之權。

重點盯防之權

- 通訊錄/短信：或被用以發垃圾短信。
- 實時位置：或被追蹤行蹤。
- 實時位置：或被追蹤行蹤。
- 文件訪問：或讀汝隱私之文件。

六、軟件安裝包之構成要素

汝曾否思之，自應用之店所下之安裝包，內究裝何物？知安裝包之構成，可助汝更解 App 之作，亦能於遇問題時知何排查。

安卓 APK 文件之結構

APK（Android Package Kit）本乃一 ZIP 壓縮包，解壓可見以下核心之目錄與文件：

- **AndroidManifest.xml**：應用之" 身份證" 與" 說明書"。錄應用之包名、版本號、所需之權、支持之系統版、入口 Activity 等核心之信。系統裝應用時首讀此文件。
- **classes.dex**：應用之核心碼。Java/Kotlin 源代碼編譯後變 Dalvik 字節碼，存此文件中。一應用或有多 dex 文件（如 classes2.dex、classes3.dex）。
- **resources.arsc**：編譯後之資源索引表。錄所有資源之 ID 與對應之關係。
- **res/目錄**：資源之文件夾。含圖、佈局文件、字符串、顏色定義等。不同辨率之圖放不同 drawable 文件夾中（如 drawable-hdpi、drawable-xhdpi）。
- **assets/目錄**：原始資源之文件夾。存不編譯之資源，如網頁文件、字體、音頻等。App 運行時可經 AssetManager 讀之。
- **lib/目錄**：本地庫之文件夾。存編譯好之.so 文件（C/C++ 碼編譯之產），按 CPU 架構分類（如 arm64-v8a、armeabi-v7a）。遊戲與需高性能算之 App 通常有此目錄。
- **META-INF/目錄**：簽名信之文件夾。含 MANIFEST.MF、CERT.SF、CERT.RSA 三文件，用於驗 APK 之完與簽名。

iOS IPA 文件之結構

IPA（iOS App Store Package）同爲壓縮包，結構略異：

- **Payload/目錄**：核心內容之所在。內有.app 文件夾，含應用所有內容。
- **Info.plist**：應用之配置文件，似安卓之 AndroidManifest.xml。錄應用 ID、版本、權之需、支持之設備等。
- **可執行文件**：與.app 文件夾同名之主程序，乃編譯後之 Mach-O 格式二進制文件。
- **Frameworks/目錄**：應用所賴之動態庫與框架。
- **資源文件**：圖、音、視頻等資源，通常存於.app 文件夾內。
- **_CodeSignature/目錄**：碼之簽名信，用於驗應用之完與源。

簽名驗證之機

何故自官方之店所下之 App 更安？簽名驗證爲關鍵。

- **安卓之籤**：開發者以私鑰對 APK 籤，系統裝時以公鑰驗。若 APK 被篡（如植病毒），簽名即失效，系統拒裝。Google Play 猶須 App Signing，由 Google 保管簽名之鑰，進一步護應用之安。

- **iOS 之籤**：蘋果之審更嚴。每 IPA 必具蘋果所頒之證書籤，否則不能裝。此即 iOS 惟自 App Store 或企業證書裝應用之由。企業證書籤之應用有效期有限，且蘋果可隨吊銷。
- **籤之作**：
  1. 驗源：認應用確來自所聲之開發者。
  2. 驗完：保應用於傳中未被篡。
  3. 驗完：保應用於傳中未被篡。

安裝包之大與優化

汝或發見，應用之店所顯之 App 大與裝後佔之空間不一。蓋因：

- **下載之大**：壓縮後之 APK/IPA 大。
- **安裝之大**：解壓後之大，通常遠大於下載之大。
- **運行時之佔**：App 運行時猶生緩存、數據文件，佔更多之空間。

現代 App 之優化之方：

- **App Bundle（安卓）**：Google Play 之動態分發之技。用戶下載時惟得適己設備之資源（如惟下載對應屏辨率之圖），大減下載之大。
- **App Thinning（iOS）**：類之技，蘋果將據設備自動裁不需之資源。
- **按需之下**：遊戲等大型應用可先下核心包，後續再下資源包。

小知識：何有之 App 安裝包特大？

小知識：何有之 App 安裝包特大？

實操之環：審三 App 之權

實操之環：審三 App 之權

第五章：使手機更知汝意——基礎設置

花數分設，體驗大升。此等設置猶爲手機“調音”，使更合汝用之習。

一、聲與震

設一喜之鈴聲與通知聲。活用靜音之模（開會）、勿擾之模（寢時），世界頓清。

- **鈴聲**：擇一汝能聞、不尬者。
- **震之模**：靜音時賴震以醒。
- **勿擾之模**：可設例外，如允通訊錄來電。
- **音量之離**：媒體之量、鈴聲之量、鬧鐘之量可分調。

二、顯與亮

- **自動之亮**：使手機據環境之光自調，省心。
- **護眼/夜覽之模**：濾藍光，夜間觀手機舒。
- **深色之模**：省電（于 OLED 屏尤效），且夜間不刺眼。諸多 App 亦支持深色之模。
- **屏刷新之率**：部分安卓機可調，越高（如 120Hz）滑越絲滑。
- **自動鎖屏**：設十五秒或三十秒，安且省電。
- **字體之大**：覺字小？調大可也。

三、壁紙、主題與桌

換一佳之壁紙（靜態或動態），心情皆變。安卓品牌手機常有主題之店，能整套換圖標與風。

桌面小組件：天氣、日程、音樂之控置桌面，一步直達。iOS 與安卓皆支持，長按桌面空處加之。

四、效率之設：使手機爲汝之“外掛腦”

此部分乃“一設則難返”之功，每皆能使汝日常之效大升。

專注/免打擾之模：奪被通知所奪之注

- **核心之思**：非” 少觀手機”，乃” 當專注時專注，當聯時再聯”。
- **用之建議**：
  • 工作之模：惟允” 星標” 聯繫人與必要 App 過；
  • 睡眠之模：自動開，惟允鬧鐘；
  • 健身/閱/會議：據當前活動定製。
- **進階之玩**：用自動化加位置/時觸，使模自切。如：iOS 快捷指令可設” 至公司即自動入工作之模”。

通知之管：使紅點不“奪命連環 call”

- **閉營銷通知**：九成之推實汝不需。
- **按要之序**：將人際通訊之類（電、短信、即時通訊）置頂；新聞、購物、社交平臺之類可設” 靜默推”。
- **分組通知**：將同類通知折於一，避一解鎖被數十條淹沒。

快捷之手：使操快人一步

- **截屏**：電源加音量下（iOS 全面屏）/電源加音量下（安卓通用）。
- **截長圖/滾動截屏**：常截後點” 長截圖” 之擇。
- **一鍵掃碼**：控制中心/快捷設置中加” 掃一掃” 之圖標。
- **輕點背面（iPhone）**：於 [設置] > [輔助功能] > [觸控] > [輕點背面] 中綁” 截屏” 或” 鎖屏” 等動作。
- **語音之喚**：Siri/小愛同學/小藝/小布，喊一聲即能查天氣、設鬧鐘。

輸入法：最被低估之效神器

- **擇對輸入法**：
  • 搜狗/百度：詞庫大，流行語更速；
  • 訊飛：語音輸入識別之率至高，開會記筆記之神器；
  • Gboard（谷歌）：簡、跨平臺同步強、支持滑行輸入；
  • iOS 自帶：隱私至佳，與系統深集成。
- **必備之開關**：智能糾錯、聯想詞、雲詞庫同步、上滑輸入數字符號。

- **隱私之提**：慎用”整句聯想”或”個性化推”之功，或上輪輸入內容。

桌與多任：使工作流順暢

- **App 分屏/畫中畫**：觀視頻時回消息，上網時記筆記。
- **小窗/懸浮窗**：微信/抖音支持”小窗之模”，回覆畢即收。
- **常用 App 置頂**：Dock 欄（屏底）惟放最高頻之四，減查尋之時。

”上工第一日”檢之單

“上工第一日”檢之單
(1) 鎖屏密碼加生物識 (2) 通知分組加閉營銷推 (3) 字體之大/刷新之率/深色之模
(4) 默認輸入法/瀏覽器/郵件 (5) 緊聯人/醫療卡 (6) 備份設置

五、輔助功能與無障：科技向善之力

莫爲“輔助功能”之名所嚇，此等功能人人能用。其初爲殘障者設，然於常人亦有用——臨手傷抱娃、地鐵上耳機壞不聞、戴老花鏡觀小字……皆能救場。說到底，無障之設體現了**科技向善**之理念：使每個人皆能平等地享數字之生活。

視障用戶之“眼”——屏閱讀器全攻略

屏閱讀器乃視障用戶用手機之核心之具。其將屏上**所有可視內容朗而出**，用戶經手與語音以操手機。下分別介 iOS “旁白”與安卓 TalkBack 之用法。

iOS 旁白（VoiceOver）

開啓之徑：[設置] > [輔助功能] > [旁白]，開開關。亦可三擊側邊鍵（或主屏鍵）速開。
基本之手：

1. 單指滑：逐元素覽屏內容；
2. 單指滑：逐元素覽屏內容；
3. 雙指輕掃：上下/左右整頁滾；
4. 三指輕掃：切頁；
5. 三指輕掃：切頁；

個性化之設：[旁白] > [語音]，可調**朗之速**（默認約五成，可調至七至八成聽感更流）、**音調**、切不同語音包。iOS 自帶“增強語音”與“語音控制器”，前者能減呼吸聲與停頓，聽感更自然。

、切不同語音包。iOS 自帶” 增強語音” 與” 語音控制器”，前者能減呼吸聲與停頓，聽感更自然。

開啓之徑：[設置] > [輔助功能] > [TalkBack]，開開關。或長按音量上下鍵三秒速啓（部分品牌支持）。

基本之手：

1. 單指左右滑：切焦點之元素；
2. 單指左右滑：切焦點之元素；
3. 單指雙擊：激活元素；
4. 雙指上下/左右滑：滾；
5. 四指單擊：暫停/續朗。

個性化之設：[TalkBack] > [文字轉語音設置]，可調**語速**、**音調**、切**語音引擎**（建議裝 Google 文字轉語音引擎，朗更自然）。

使小字變大、使模糊變清——視覺之調

放大之手

- **iOS**：[設置] > [輔助功能] > [縮放]，開後**三指輕點屏**即放大。默認全屏縮，可擇窗口縮（局部放大窗）。**雙指開合可調倍（1.5x 至 15x）**，三指雙擊鎖當前縮之比。
- **安卓**：[設置] > [輔助功能] > [放大]，三指點擊放大，再三指點擊退；或直**三指點擊屏三下**啓臨放大（最多八倍）。

高對比之字/粗體文本

- **iOS**：[設置] > [輔助功能] > [顯示與文字大小]，開” 粗體文本”、” 增強對比度”、” 降低透明度” 等開關。” 智能反轉” 可於淺色底使文字保持黑，避白底白字。
- **安卓**：[設置] > [顯示] > [字體大小]，可調” 小、普通、大、超大” 四檔。華爲/小米額外供” 粗體” 與” 高對比文本” 之開關。

色彩濾鏡（色盲之友）

- **iOS**：[設置] > [輔助功能] > [顯示與文字大小] > [色彩濾鏡]，供**紅/綠/藍/黃四色盲之模**，及” 灰度” 與” 反轉顏色” 之擇。建議先做一次色覺之測（系統自帶）再擇對應濾鏡。
- **安卓**：[設置] > [輔助功能] > [顏色校正]，含**三色盲之模**加” 反轉顏色” 之開關。

AI 圖像之述：使 AI 爲汝之“眼”

當屏閱讀器遇圖時，傳統之做法乃跳或惟讀“圖”二字。然今 AI 可**以一語述圖之內容**，於識驗證碼、讀圖、識菜單甚有助。

諸系統 AI 圖像之述支持之況

- **iOS 15+**：於 [設置] > [輔助功能] > [旁白] > [語音描述] 中開” 圖像之述”。系統將調**本地 AI 之模**自動生圖之文述（須 iPhone 12 及以上）。
- **新版 TalkBack 應用**：Google TalkBack 于較新版中集**圖之問**之功，可問” 此圖中有何”，AI 實時答（不嚴賴安卓系統之版，更 TalkBack 應用即能體驗）。
- **第三方 App**：”Seeing AI”（iOS）、”Lookout”（安卓）、” 小藝幫幫”（華爲）皆爲優之 AI 視覺之助，可識物、讀文件、識人臉之表情。

使用之場

使用之場**此乃真之“科技使生活更平等”**。

聽障用戶之“耳”——實時字幕全攻略

實時字幕（Live Captions）乃**系統級之語音轉文字**之功，聽障用戶於噪之環、圖書館、外語之場皆特有用。

實時字幕之開與優化

iOS：[設置] > [輔助功能] > [實時字幕]，開開關。系統將檢設備所有音頻（通話、視頻、播客、FaceTime、語音備忘），自動顯字幕於屏上。
安卓 12+：按**音量鍵**點擊彈出框中之“實時字幕”圖標；或於 [設置] > [輔助功能] > [實時字幕] 中開。
準確率優化之巧：

1. 盡量于**靜之環**用，遠離空調、機之轟鳴；
2. 言者**咬字清、語速適**時識別之率至高；
3. 時識別之率至高；
4. 支持中文普話、粵語、英語等主流之種，**小語種識別之率較低**；
5. ；

通話之場雙道：語音轉字加字轉音

當用戶**既不聞亦不能言**時（如突失聲、喉術恢復期、重口喫），手機猶能當“傳話員”。

通話之實時轉譯（TTY / RTT）

iOS：[設置] > [輔助功能] > [RTT/TTY]，開後通話時屏上顯鍵盤，對方之音被實時打成字，**汝所打之字亦被念出**予對方聽。
安卓：[設置] > [輔助功能] > [實時轉錄] 加 [聲音通知]。“實時轉錄”將對方言實時顯爲字；“聲音通知”於不便語音時將汝之入轉爲語音放。
技術路線：

- **RTT（Real-Time Text）**：雙方皆須支持，字邊打邊顯，無延遲，主流運營商已支持；
- **TTY（電傳打字）**：須專用 TTY 設備，已漸爲 RTT 所代；
- 微信/QQ 之**語音轉字之功**亦爲替之方：長按語音消息”轉字”。

向所有人之“小而實用”之功

- **輕點背面**（iOS 14+）：[設置] > [輔助功能] > [觸控] > [輕點背面]，可綁**雙擊/三擊**觸截圖、鎖屏、控制中心等動作。雙手持快遞時特好用。
- **輔助之觸/懸浮球**：iOS [輔助功能] > [觸控] > [輔助之觸]，安卓 [輔助功能] > [懸浮導航]。虛按鈕替手或物理之鍵，鍵不靈或單手操時救場。
- **切換之控**：經外部開關、單次點擊、頭部動以控手機，**爲肢體障者設**。iOS [輔助功能] > [切換之控]，安卓 [輔助功能] > [開關之控]。
- **聲之識**（iOS）：[輔助功能] > [聲之識]，開後手機將”聞”門鈴、嬰哭、火警、警笛等聲，**識到則震加屏通知**，於聽障用戶與睡期間甚有值。
- **LED 閃之醒**：iOS [輔助功能] > [音頻/視覺] > [LED 閃以醒]，安卓 [設置] > [通知] > [閃光燈提醒]。來電與通知時閃光燈閃，**噪之環無聲亦能感**。
- **單聲道之音**：[設置] > [輔助功能] > [音頻] > [單聲道之音]，將左右聲道合。單耳聽力損者不失另一半聲之信。

蘋果之無障設之哲學

蘋果之無障設之哲學

花三十秒略體驗之

花三十秒略體驗之

1. 開**旁白/TalkBack**，試不賴眼用三指滑以覽微信；
2. 開**實時字幕**，對手機言以觀識別之準率；
3. ，對手機言以觀識別之準率；**三指點擊**放大一段字至二百%，體驗視障者之日用；
4. 放大一段字至二百%，體驗視障者之日用；

第六章：守護汝之數字生活——安全與隱私

手機之安 = 隱私之安加財產之安。於數據即資產之時，護手機即護汝之己。

一、鎖屏：第一道物理防線

鎖屏乃汝手機之第一道防線，亦乃最易被忽者。

安之級之序：

1. **密碼/PIN 碼**：安度至高，乃最終之保（忘指紋/臉時賴之）。
2. **指紋之識**：安與便之完之衡。
3. **人臉之識**：至便，然安之性賴於技（3D 結構光安，2D 照片易騙）。
4. **圖案之解**：便而易留痕被人猜。

設強鎖屏：

- 務必設一複雜之密碼（勿用生日、123456）。
- 開生物之識。
- 開生物之識。

二、“查找吾手機”：終極防盜之具

必開！其能：

- **定位**：手機失於何，於圖上尋之。
- **放聲**：使手機響鈴，宜家中尋時。
- **丟失之模**：遠鎖死手機並顯聯之號。

- **遠擦除**：最後一招，清所有數據（然擦後不能再定位）。

往 [設置]，蘋果尋“查找吾 iPhone”，安卓尋“查找吾設備”或品牌類之服，保其爲綠開之態。

三、系統之更：打補丁，堵洞

系統之更首旨在修安之洞，猶爲屋補牆洞。故，見系統更之提，盡即時升之。

系統之更首旨在修安之洞，猶爲屋補牆洞。故，見系統更之提，盡即時升之。

四、識與防電信詐

於 AI 技迅猛之今，傳統之“眼見爲實”之識之模已全失效。詐者正用深僞（Deepfake）等 AI 技，實現自“廣撒網”式詐至“精圍獵”式罪之全升。

AI 時代最常之數詐之套

(1) AI 換臉擬聲之詐

(1) AI 換臉擬聲之詐**突車禍、緊借、遭綁**等。此類之詐用親情、友情等情之紐，**迷之性極強**。

。

(2) 冒權威/客服之詐**造恐慌之心或權威之壓**，誘受害者轉或開屏共之功，從而竊賬戶之信。

(3) AI 投資理財之詐

(3) AI 投資理財之詐**終捲款而逃**。

破局之法——“照妖鏡”防護之思

AI 能克聲與面，然亦非無懈，有數破可利用：

- **動作驗證法**：行視頻通話時，主動要對方完預設之連貫動作之令，如” 用手於臉前速揮過”、” 側臉示耳之廓” 或” 連速眨五次”。今之 AI 換臉技於處複雜動態之遮、速動之景與光影之變時，**極易現畫之卡頓、面之扭或邊之穿模**等破。

- **暗號覈驗法**：建並用預設之私核問，如” 吾上次同聚時汝點何菜”、” 吾送汝之首物何” 等惟雙方知之細。若對方顯遲疑或答非所問，應**即止通話並啓安之預**。

- **二次覈實法**：嚴守公安部網安局所發之” **四不一核**” 之則——不輕信陌之信、不隨轉匯、不泄個人敏信、不傳可疑之內容。凡涉錢財往來，務必經**已知之原號回撥或當面認**。行轉之事時，建議擇” 二十四時到” 之模，爲潛詐之識與資之挽預足時窗。

傳統常之套（仍須警）：

- **中獎/刷單返利**：利誘，先予小甜再騙大錢。
- **網戀之詐**：殺豬盤，養情後騙錢。
- **冒熟人**：盜號後借（AI 時代更難辨）。

手機自帶之護：

- 騷擾電話之攔。
- 驗證碼防竊。
- 驗證碼防竊。

防騙之訣：

不輕信、不泄、不點、不轉、有疑問先核！遇“急事”先冷五分，幾所有詐於此時將露馬腳。

五、御“開盒”與社工攻：斬隱私泄之鏈

“開盒”（社會工程學攻）之核之理在**系統性集、析並拼個人數字之片，構建完之個人之像**。防之要在爲個人隱私劃明紅線，建多層次之護之體。

照之脫敏處

基礎 EXIF 清理方法見第二章“圖片與文件系統管理”。於社交媒體分享照片前，尚需進一步**徹刪照之元數據（EXIF 信）**，含攝之時、設備之型、GPS 位之座標等敏之數據。於照中所含之**門牌號、車牌之信、車票之條碼、身份證號**等關鍵之信嚴打碼或模糊處，避無中泄個人行蹤之軌。

- iOS：[設置] > [隱私與安全性] > [相機] 中可閉” 精確位置”；分享時擇” 僅當前照” 可剝元數據。
- 安卓：部分相冊 App（如 Google 相冊）分享時將提示是否保留位置之信。
- 通用之具：ExifTool、Metapho 等可查/編 EXIF。

應用權限之收

基礎權限管理方法詳見第四章“權限管理”。於“防開盒”場景下，尚需進一步定期審查並關閉電商類、工具類 App 之**麥克風、通訊錄、位之信**等非必之權，防 App 於後臺“偷聞”日之對或非法集個人關係之網；對久置而猶佔高權之應用宜即卸，從源頭減隱私泄之險點。

- 見” 通訊錄之權” 請時，先問之：此 App 果必知吾所有聯人乎？
- 三月未用之 App，果卸。權限乃越用越緊，非越用越松。

智能設備安之設防

智能攝頭、智能門鎖、智能音箱等 IoT 設備於初用時必**即改默認登之密**，設含大小寫字母、數字與特符之高強獨立之密；不常用之遠訪功宜常閉，定期檢設備固件之更並即補安之洞，**防黑客侵併窺家隱私之空**。

真案之警

真案之警**不改默認密 ＝ 將家大門敞**。

六、隱私之護：汝之數據乃汝之財

手機中之照、聊天記、位之軌，本質上皆汝之“數字之資”。一泄，輕則告擾，重則人肉、詐甚至敲詐。

App 權限之“最小化之則”：

- **位置**：打車/地圖用” 始終允”；他 App 用” 僅用時允”。
- **通訊錄**：除非必（如微信加好友）方予，否則一律禁。
- **相冊/全部照**：iOS 14+ 起可授” 選定之照”，安卓 13+ 同支持精授。
- **麥克風/攝頭**：平閉，需時再開。疑被偷錄時，滑出控制中心觀有無” 小綠點/小橙點”。
- **後臺之跟**：iOS 14.5+ 起強要 App 用 IDFA 前必問；安卓可閉” 廣告 ID”。

隱私之管實戰：

- **隱私之報（iOS）**：[設置] > [隱私與安全性] > [App 隱私之報]，觀何 App 於偷訪汝之位、相機、麥克風。
- **權限用之記（安卓）**：[設置] > [隱私] > [權限用之記]，觀誰於何時調了敏之權。
- **限廣告之跟**：於系統設置中閉” 廣告 ID” 或” 限廣告之跟”，能減” 被精廣告擾”。

七、失/被盜後之“急救”單

手機失勿慌，依序處之：

1. **即凍 SIM 卡**：聯運營商掛失，避被用以收驗證碼。
2. **遠鎖**：經雲服（iCloud/品牌雲）開” 丟失之模”。
3. **改要之密**：登過手機之所有要之賬號（支付、銀行、郵箱）即改密。
4. **報案**：往派出所報案，錄 IMEI 號。手機尋回時亦便於證身份。

5. **告親友：**防騙以汝名借錢。

6. **（最後）遠擦：**確認尋無望時再操，擦後定位之功失效。

八、SIM 卡與 eSIM 之安

- **設 SIM 卡 PIN：**開後，每重啓手機或換卡皆須輸 PIN 碼，防”拔卡換手機”以繞鎖屏。
- **eSIM 之優：**無實體卡可被拔，安之性更高；同時支持多號切，旅時甚爲便。
- **二次換卡之險：**警”運營商二次換卡”之騙——騙者冒用汝之身補辦 SIM 卡，從而接管汝之手機號。設”補卡須機主本人持身份證至營業廳辦”乃最效之防。

切勿爲之事

- 用生日/123456/qwerty 等弱密；
- 將驗證碼予任何人（縱爲”客服”）；
- 將驗證碼予任何人（縱爲”客服”）；
- 於公共 Wi-Fi 下登網銀或支付；
- 予陌人開”屏共”或”遠控”。

試啓並擬”遠鎖”

1. 保汝”查找吾手機”之功已開。
2. 保汝”查找吾手機”之功已開。
3. 尋”查找設備”之功，試對汝己之手機發一”放聲”或擬”鎖”之令，熟全操之流。

第七章：前沿功能全解析

科技日新月異，手機之功亦不斷進化。衛星通、無網通、車禍檢、星閃之技……此等聞之甚幻之功，實已入吾等之生。今吾等同探此等前沿之技。

一、衛星通：無信亦能求救

衛星通乃近年手機之域一大突。當汝於荒郊野嶺、深山老林，手機全無信時，衛星通或乃救命之草。

工作之理：

手機直與軌道上之通信衛星建連，發求救之信。今主支持發短信與位之信，不能打電話。

支持之機型：

- **蘋果**：iPhone 14 系列及以後，支持通過衛星發送緊急 SOS。iPhone 17 系列進一步支持衛星發送 iMessage 與短信，且擴展矣更多地區之覆蓋。
- **華爲**：Mate 50 系始支持北斗衛星消息（惟文），Mate 60 Pro 系進一步支持天通衛星通話（可打電話），Mate 70 系衛星通之能進一步強。
- **小米/OPPO/vivo**：2024 年起旗艦機型續支持北斗衛星消息之功。
- **發展之勢**：衛星通正自高端旗艦專之功向下普及，未來中端機型亦有望支持。中國北斗衛星系統於國產手機上之支持至廣。

使用之場：

- 戶外探險、登山遇險。
- 戶外探險、登山遇險。
- 海上遇險。

使用之法：

1. 保於開之野，能見天。
2. 保於開之野，能見天。
3. 依屏之指，對衛星之方。
4. 發預設之求救之信與位。

小知

小知

二、無網通：近距通訊之新方

“無網通”指於無移動網與 Wi-Fi 之況下，兩臺手機之間直通訊之技。

技之理：

用藍牙或專用之無線電之頻，於短距內（通常數百米至數公里）建點對點之連。

應用之場：

- **戶外之動**：徒步、露營時隊友間之通。
- **災之援**：地震、洪水後通中時。
- **大型之動**：演唱會、體育賽事人多信差時。

支持之台：

- **蘋果**：iOS 17 之”NameDrop” 與” 離線找友” 之功。
- **華爲**：暢連 App 支持無網通。
- **小米**：小米妙享中心之部分功。

三、車禍之檢：要時救命

車禍之檢乃用手機之感器檢重車禍，並自動撥緊電之功。

工作之理：

手機經加速度計、陀螺儀、氣壓計、麥克風等多感器，合判是否生重撞。檢到車禍後，手機將：

1. 彈倒計時之界，問汝是否無恙。
2. 彈倒計時之界，問汝是否無恙。
3. 向緊聯人發汝之位。

支持之機型：

- **蘋果**：iPhone 14 系及以後。
- **安卓**：部分高端機型續支持。

注意之事：

- 功須開方能用。
- 功須開方能用。
- 或誤觸（如過山車、劇動），即取可也。

四、星閃之技：下代之無線連

星閃（SparkLink）乃華爲牽頭研之新代無線通訊之技，目標乃於某場替藍牙與 Wi-Fi。其於關鍵之標對標甚超現之方：空口之速可達九百 Mbps（遠超藍牙之五十 Mbps），延遲低至二十 μ s（藍牙約十五 ms），單網支持四千九十六設備同連（藍牙通常八以內）。

技之特：

- **超低之延**：端至端之延惟二十微秒（零點零二 ms），較藍牙低兩量級，遊戲與樂之同步幾乎無可感。
- **高之速**：單鏈峯值之速九百 Mbps，乃藍牙音頻之數十倍，可傳無損音頻甚高清視頻。
- **低之耗**：待機之耗與藍牙當，而傳之效更高，單位數據之能耗更低。
- **廣之覆加高之並**：連之距較藍牙遠五成以上，單網支持上千設備同在。

應用之場：

- **無線之機**：更低之延，玩遊戲音畫同步。
- **智能之表**：更穩之連。
- **智能之家**：更多設備同連。
- **車載之聯**：更流之手機-車機之連。

支持之設備：

今主要爲華爲生態之設備，如華爲 Mate 60 系、部分華爲耳機與表。未來將有更多之品牌支持。

技之望

技之望

往略檢之：汝手機是否支持此等功

1. 往設置中搜” 衛星”，觀有無衛星通相關之擇。
2. 搜” 車禍檢” 或” 緊急 SOS”，檢是否開。
3. 若汝手機乃華爲，觀是否支持星閃設備之連。
4. 解此等功之用法，要時或能派上用場。

二、軟件篇（第 8-14 章）

第八章：通信：短信、電話與社交

通信者，手機最原始、最核心之使命也。今已自簡單之通話與短信，演為含即時通訊、社交網絡與電子郵件於一體之複雜生態系統。

一、電話 App 之進階功能

今之手機電話應用，久已逾簡單撥號盤，集成錄音、騷擾攔截、智能識別諸強大工具。

通話錄音（留意法律合規）：

- 其用：記錄重要通話內容，以備工作紀要、採訪、客戶承諾或爭議留存。
- 法律提示：在中國境內，通話錄音之得為證據，前提乃取得對方同意，或用於維護自身合法權益。
- 單方錄音與雙方同意：就法律層面言，「未經對方同意」所得錄音非當然無效。最高人民法院相關司法解釋：錄音取得方式若未嚴重侵害他人合法權益、未違法律禁止性規定、未悖公序良俗，仍可為證據呈堂。然部分省份法院於民事審判中，對「偷偷錄音」採信較嚴；刑事案件則通常要求更嚴之合法性審查。
- 各地之異：北京、上海、廣東等地法院於單方錄音為民事證據較寬容；而涉商業祕密、個人隱私之錄音，無論何省皆須特別謹慎，恐因侵害隱私權而被排除。
- 操作之法：安卓通常於通話界面有明確「錄音」按鈕；iOS 系統不提供原生通話錄音功能，須藉第三方 App，或用帶錄音功能之藍牙耳機。

錄音使用紅線

- 不得以錄音敲詐、傳播或公然侮辱他人。
- 涉第三方隱私（如偶錄他人對話）之內容，勿得隨意擴散。
- 重要場景宜於通話開始時口頭告知「本次通話將被錄音」，既合規亦更具證據效力。

騷擾電話治理深度內容：

- 原理：電話 App 經網絡查詢龐大號碼庫，來電時顯示「騷擾電話」「詐騙電話」或公司名稱。

- 開啟之法：安卓入電話 App ＞設定＞騷擾攔截；iOS 入設定＞電話＞來電阻止與身分識別。各品牌詳細開關路徑與推薦開啟項，見第十三章「實用系統工具」中品牌對照表。

- 主流標記平臺：國內號碼標記主要由三家數據服務商提供——騰訊手機管家、360 手機衛士、電話邦（Tapir）。運營商（移動 / 聯通 / 電信）之「機主姓名」顯示、「騷擾攔截」功能亦常調用此等數據。手機廠商（華為、小米、OPPO、vivo）之標記庫多與上述服務商合作。

- 被誤標記奈何：若爾正常工作號碼被誤標為「外賣」「快遞」「騷擾電話」，可循以下渠道申訴：電話邦：訪號碼申訴平臺（haoma.tk.cn）提交身份證明。

- 360：經 360 手機衛士 App 內「號碼申訴」功能提交。

- 騰訊：經「騰訊手機管家」小程序「號碼申訴」頁面提交。

- 三大運營商：中國移動用戶可發短信「KTFSR」至 10086 開通防騷擾功能；電信用戶關注「天翼防騷擾」公眾號；聯通用戶於「聯通手機管家」中設置。

防騷擾進階：開啟運營商「高頻呼叫攔截」（如移動之高頻呼叫限制），可自動攔截 AI 語音外呼。

手機自帶攔截常見能力：來電識別 / 智能攔截：據號碼庫自動標記並攔截騷擾、詐騙、廣告推銷。

黑名單 / 白名單：手動拉黑已知騷擾號；以通訊錄、快遞外賣號加入白名單，免誤殺。

短信關鍵詞過濾：含「中獎」「貸款」「點擊鏈接」等關鍵詞之短信，自動歸入攔截箱。

iOS 靜音未知來電：不在通訊錄之來電，僅靜音入語音信箱，不打斷爾；宜騷擾電話多之用戶，然須留意漏接陌生重要電話。

勿擾模式聯動：夜間或開會時僅許通訊錄來電，配合定時開關，睡眠質量立時改善。

實用軟件如何配合：先用系統自帶攔截，再疊加國家反詐中心與運營商防騷擾；清理類可擇清濁等輕量工具，然同類安全 / 清理軟件裝一套即可，詳見第十三章。

VoLTE / VoNR 為何物、如何開啟：

- VoLTE（Voice over LTE）：基於 4G 網絡之語音通話技術，通話時不復回落 2G / 3G，能邊打電話邊上網，音質較傳統通話更清晰，接通更快。

- VoNR（Voice over NR）：基於 5G 之語音通話，乃 VoLTE 下一代之演進。延遲更低、音質更佳，且能支持高清視頻通話。須 5G 手機 +5G 套餐 + 當地 5G 覆蓋三者具備方能使用。

- 開啟之法：iOS 於「設定＞蜂窩網絡＞蜂窩數據選項＞語音與數據」中開啟「VoLTE」；安卓於「設定＞移動網絡＞移動數據＞高級」中尋「VoLTE 高清通話」開關。運營商通常默認開啟，若通話時網絡斷開、無法上網，多半 VoLTE 未啟。

附識

若爾手機通話時上網驟緩或斷網，多半 VoLTE / VoNR 未開啟，回落至 2G / 3G 網絡。開啟後即可「邊打電話邊刷視頻」。

二、短信與 iMessage / RCS

傳統短信並未被淘汰，接收驗證碼、銀行通知與官方信息，仍賴之為可靠渠道。

- iMessage：蘋果設備間經互聯網所發之消息，顯示為藍色氣泡。支持高質量圖片視頻、已讀回執等，全然免費且端到端加密。
- RCS：全球運營商與谷歌所推動之下一代通信協議。功能類 iMessage 或微信，然基於電話號碼。
- iMessage 與 RCS 之互操作性：自 iOS 18 起，蘋果始支持 RCS 標準，是故 iPhone 與安卓手機之間可經 RCS 發高清圖片、已讀回執與打字狀態提示，告別「綠色氣泡」體驗之斷層。然前提乃雙方運營商均開通 RCS，且手機系統版本足新。

RCS 於國內部署之情狀：

- 中國移動：部署 RCS 較早，部分省份已支持「5G 消息」業務，可發富媒體消息、互動卡片。須手機終端支持且於營業廳開通。
- 中國電信：以「電信 5G 消息」提供服務，部分套餐默認支持，老用戶須發指定短信指令開通。
- 中國聯通：於主要城市部署 5G 消息，然終端覆蓋參差，許多安卓機型仍須手動於短信 App 中開啟「增強信息」或「5G 消息」開關。
- 實際體驗：國內 RCS 推廣之速緩於所期，蓋受微信生態強勢、終端適配不統一諸因所影響。日常通信仍以微信為主，RCS 多任「官方推送」之角色（如銀行通知、政務消息）。

驗證碼安全紅線

- 勿轉發驗證碼：凡令爾「讀出驗證碼」或「轉發短信」者皆可能是詐騙之徒，雖自稱「客服」「快遞員」「公檢法」者亦然。驗證碼者，爾之錢袋也。
- 防釣魚短信：警惕「ETC 過期」「醫保卡凍結」「快遞丟失理賠」等短信中之鏈接，正規機構必不經短信鏈接索要銀行卡號、密碼。
- 號碼識別：銀行官方短信通常來自 95 開頭短號或 1069+ 企業簽名，凡從普通 11 位手機號發來之「銀行通知」殆皆詐騙。
- 雙因子備份：綁定重要帳號之手機號勿隨意更換；若必須換號，務先解綁所有關鍵帳號，不然新機主可能經短信驗證碼重置爾之密碼。

三、微信 / QQ：國民級社交應用之管理

微信與 QQ 已逾通信工具，成集社交、支付、獲取信息於一體之「超級 App」。

聊天記錄備份與遷移：

1. 於電腦登錄 PC 版或 Mac 版微信。
2. 點擊左下角菜單＞［遷移與備份］＞［備份與恢復］。
3. 擇「備份聊天記錄至電腦」，於手機擇須備份之聊天。
4. 遷移至新手機：於原手機微信［我］＞［設定］＞［通用］＞［聊天記錄遷移與備份］＞［遷移］＞［遷移至手機 / 平板］，掃碼即成。

存儲空間清理深度之法：

- 常規入口：微信［我］＞［設定］＞［通用］＞［存儲空間］，可清理緩存，並按時間、檔案類型精細清理聊天記錄。
- 聊天內清理：入某聊天＞右上角「…」＞［聊天記錄］＞［管理存儲空間］，可單獨清理某聊天中之圖片、視頻、檔案，保留文字。
- 原圖 / 原視頻清理：群聊中接收之高清原圖視頻最佔空間，可於［管理存儲空間］中勾選「原圖」批量刪除。
- 關閉自動下載：［設定］＞［通用］＞［照片、視頻、檔案和通話］，關閉「自動下載」，免 WiFi 下默默緩存大量群檔案。
- 小程序緩存：於［發現］＞［小程序］長按不常用之小程序刪除，可釋其本地緩存。
- 聊天記錄壓縮備份：電腦版微信備份時勾選「壓縮」，可將數 GB 聊天記錄壓縮至原體積之 30%-50%。

羣管理技巧：

- 群公告：群主 / 管理員於群聊右上角「…」＞［群公告］發布，全員強提醒，宜重要通知。新成員入群時自動彈出。
- 群待辦：群主可將置頂消息設為「群待辦」，所有成員頭上現紅色提示，點擊完成後自動消失。宜活動報名、任務收集。
- 群接龍：於輸入框輸入「# 接龍」可發起接龍，宜點餐、統計報名。接龍結果自動彙總至群聊頂部，免刷屏。
- 群直播：群聊右上角「+」＞［群直播］，可發起僅本群可觀之直播，宜企業內訓、家族聚會、產品內測。

- 折疊該群聊：於群列左滑群名＞［折疊］，可收納不重要之群於「折疊的群聊」，僅新 @ 爾時提醒。
- 僅群主 / 管理員可 @ 所有人：群成員 500 人以上時，普通成員無法 @ 所有人，免被打擾。

朋友圈權限精細化管理：

- 分組可見性：發布時點擊「誰可以看」，擇「部分可見」或「不給誰看」，基於預先設置之標籤分組（如「同事」「家人」「客戶」）精準推送。
- 允許朋友查看範圍：［我］＞［設定］＞［朋友權限］＞［朋友圈］＞［允許朋友查看朋友圈的範圍］，可擇「最近三天 / 一個月 / 半年 / 全部」，建議設為「半年」以平衡隱私與社交。
- 屏蔽某條朋友圈：長按朋友圈某條動態＞［不看其朋友圈］或［不再關注］，對方不會知曉。
- 朋友圈廣告管理：［我］＞［設定］＞［關於微信］＞［個人信息與權限］＞［個性化廣告管理］，可關閉基於行為之個性化推薦。
- 陌生人十條：［朋友權限］＞［朋友圈］＞［允許陌生人查看十條朋友圈］，關閉可免陌生人翻看爾之過往動態。

四、視頻通話對比

視頻通話工具各有所重，因場景擇合宜之器，溝通更順暢。

- 微信視頻：國民級覆蓋，老少鹹宜。宜親友日常聯絡、快速溝通。至多支持 20 人視頻（須小程序「群組視頻」），弊在網絡差時畫質下降明顯。
- FaceTime：蘋果生態專屬，畫質優、端到端加密。自 iOS 15 起支持以「鏈接」邀安卓 / Web 用戶加入，然主持人必為蘋果設備。宜家庭跨設備溝通。
- Zoom：國際會議首選，支持至多 100 人免費 40 分鐘會議。屏幕共享、虛擬背景、分組討論室等功能完善，宜跨國 / 遠程協作。
- 騰訊會議：國內辦公首選，與微信深度整合，掃碼即入。支持 300 人會議（付費版），雲端錄製、實時字幕、會議紀要功能強大，宜企業內訓與大型會議。
- 使用場景建議：1 對 1 日常聯絡擇微信；蘋果用戶之間擇 FaceTime；正式商務會議擇 Zoom 或騰訊會議（國內擇騰訊，跨境擇 Zoom）。

網絡帶寬要求與流量消耗參考：

- 微信視頻通話：約需 500Kbps-1Mbps 帶寬，標準畫質每分鐘約耗 3-5MB 流量，高清約 8-12MB。

- FaceTime：標清約 1Mbps，720p 約 2Mbps，1080p 須 4Mbps 以上。每分鐘流量約 5-15MB。
- Zoom / 騰訊會議：720p 約 1.5-3Mbps，1080p 須 3-5Mbps。每小時流量約 300-800MB，多人會議主持人流量更大。
- 建議：長時視頻通話盡量用 WiFi；移動網絡下可主動降低畫質（於 App 設定中關閉「高清視頻」）以省流量。

視頻通話畫質優化之技：

- 光線：背對窗戶、面朝光源（自然光最佳），免逆光致人臉變黑。
- 角度：手機攝像頭略高於眼睛，免雙下巴；橫屏通話時雙手持握更穩。
- 音頻：用耳機可免回聲與嘯叫；安靜環境勿開「揚聲器」模式。
- 網絡：優先 WiFi；4G / 5G 視頻時盡量靠近窗戶以免信號衰減；多人會議可關閉其他視頻畫面，僅留發言人。
- 虛化背景：騰訊會議、Zoom 均支持 AI 虛化，可遮蔽雜亂背景。宜穿純色衣物，免與背景色相近而致「綠幕穿幫」。

小貼士

網絡卡頓時，先關視頻保留語音，較徑直掛斷更禮貌。重要會議前可用「騰訊會議」自帶「網絡檢測」功能先測帶寬。

五、電子郵件：于手機配置與使用

- 添加郵箱帳戶：iOS 於設定＞郵件＞帳戶＞添加帳戶；安卓通常有「電子郵件」App。
- 使用之技：為不同郵箱帳戶設不同通知方式；善用滑動手勢速歸檔、刪除或標記郵件。

IMAP vs POP3 vs Exchange 協議之別：

- POP3：自服務器下載郵件後默認刪除原件，僅本地留存。宜只用一台設備、須離線閱讀之場景。弊在換設備後不見歷史郵件，已不推薦。
- IMAP：郵件恆存於服務器，本地僅作緩存。多設備（手機 / 電腦 / 平板）同步狀態一致——一端已讀，全部已讀。乃當前主流推薦之協議。
- Exchange（含 ActiveSync）：微軟協議，除郵件外兼同步日曆、聯繫人、任務。企業郵箱首選。Outlook、網易企業郵箱、騰訊企業郵箱均支持。
- 擇用建議：個人郵箱用 IMAP；公司郵箱先 Exchange，次 IMAP；盡量勿用 POP3，除非有特殊歸檔之需。

企業郵箱配置之法：

1. iOS：［設定］＞［郵件］＞［帳戶］＞［添加帳戶］＞擇「Microsoft Exchange」或「其他」。
2. 安卓：於自帶「電子郵件」App 或 Outlook App 擇「添加帳戶」，輸入完整郵箱地址與密碼。
3. 服務器填寫（以 Exchange 為例）：服務器地址通常為「mail. 公司域名.com」或「outlook.office365.com」，域名填公司簡稱。
4. 若提示「無法驗證服務器」，可手動輸入：入站服務器（IMAP）端口 993，SSL 加密；出站服務器（SMTP）端口 465 或 587，TLS 加密。
5. 登錄後建議於［帳戶設定］開啟「推送」（Push）以實時接收郵件，並關閉「郵件預覽」以護商業機密。

小示

企業郵箱通常須管理員開通「客戶端授權密碼」或「IMAP / SMTP 授權碼」，不能徑用網頁登錄密碼。登錄失敗時，先至網頁版郵箱設置中生成授權碼，再填入手機。

實操：備微信聊天記錄至電腦一過

1. 於電腦啟動備份：開電腦版微信＞左下角菜單＞［遷移與備份］＞［備份與恢復］。
2. 擇備份至電腦：點擊「備份聊天記錄至電腦」。
3. 於手機擇備份內容，候備份完成。

第九章：移動辦公與學習

吾昔亦以爲手機辦公不過“湊合”而已，直至某次出差忘攜電腦，強以手機成標書之改與合同之籤。實言之，手機辦公之瓶頸往往不在手機本身，而在君不諳其能之邊界也。

一、文檔之處理：於掌中編輯與創作

於手機屏上編輯文檔，與電腦上之感確有不同。最大之患乃屏小、指粗、排版易偏。然有數術可使體驗大善：

- **WPS Office**：本土化最善，盡兼容微軟之格式（docx/xlsx/pptx），內置 PDF 工具甚豐。吾尤愛其” 語音備註” 之能——閱合同時不便打字，徑對手機言數語，即自動轉爲文字批註附於文中。又 WPS 內置海量免費模板，制簡歷、彙報 PPT 時可省甚多之時。
- **Microsoft 365**：若君之公司用正版 Office，此乃最佳之選。與桌面版格式兼容幾近完美，無 WPS 開復雜排版微移之患。深融 OneDrive，君於電腦上改至一半之文檔，手機上啓即最新之版。
- **石墨文檔/騰訊文檔**：此二者乃” 在線協同” 之代表。宜多人同編一文檔，每人所改之處實時而顯。吾嘗以石墨文檔與友共制旅行攻略，二人同填信息於表格，效率遠勝一人獨爲。

實用之工作流：吾今之習乃——要緊之件存 OneDrive，日常編輯用 WPS，團隊協作開石墨文檔。文件既上雲端，再無“忘攜 U 盤”之尷尬矣。

二、筆記之用：自隨手記至知識體系

記筆記之事，衆人初即欲構繁複之系統，然不三日即棄。吾之議乃先輕後重——先養“記”之習，再慮“管理”之難。

- **隨手記——系統自帶即足**：iOS 備忘錄啓之極速，且支持掃描文檔、手寫、錄音。安卓各品牌之便籤亦不遜，鎖屏即可速啓。此等工具宜記臨時之想、購物之單、會議之要。其要在啓之速，思及即可記，無需待 App 加載。
- **稍後讀——收集信息**：微信之” 浮窗” 實乃隱藏神器。見佳文未及細閱，點右上角” 浮窗”，即縮爲小圓點貼於屏邊，隨時可點開續讀。較轉發至” 文件傳輸助手” 便利多矣。又，Cubox 或 Pocket 此等稍後讀之器，可跨平臺收集君於各處所見之佳文。

- **構體系——Notion 與飛書：**當君覺零散之筆記難覓時，即可上 Notion。此物甚靈——君可建” 讀書筆記數據庫”，自動以書名、作者、評分組織之；亦可搭” 項目看板”，使工作任務之進可視化。飛書之” 多維表格” 亦類此，且國內訪問更速。吾以 Notion 理所有寫作素材、學習筆記與項目進度，已用三年餘矣。

一實用之筆記策：吾己之習乃“三件套”——臨時之想用系統便籤（秒開），待讀之文用微信浮窗（順手），需長期積累之識放 Notion（體系化）。勿初即欲將諸物塞入一器，如此反易棄也。

三、掃描與 OCR：以紙質之世搬入手機

手機掃描之事，善用之誠可省卻諸多奔走打印店之勞。吾前次辦簽證需一堆材料之電子版，皆以手機成之。

- **iPhone 之用戶：**iOS 備忘錄自帶之掃描已甚善。開備忘錄點相機圖標選” 掃描文稿”，對準文件即自動識邊框而矯透視。拍畢可調色——選” 黑白” 之式出效最似掃描儀，” 灰度” 宜有圖之資料。
- **安卓之用戶：**多品牌之相機自帶” 文檔掃描” 之式。若不得，往應用商店下載 Microsoft Lens（免費，且與 OneDrive 相通）。
- **OCR 文字識別：**拍畢之掃描件，若需將其上之文字提取以編輯，即需 OCR 矣。微信自帶之” 提取文字” 最簡——長按圖片即可用，識別準確率尚可。若大量處理，可用白描 App 或全能掃描王。於中文識別，百度之 OCR 引擎效較佳；WPS 之圖片轉文字亦甚準。
- **掃描小術：**置於光線充足之處拍之，最好有臺燈自側面打光，免於文檔上留君之手影。若文件折角，先以手壓平再拍。多頁文件可連掃後合爲一 PDF，無需一頁頁獨存。

四、網盤何以擇：無最好者，惟最適合君者

擇網盤之事吾思之甚久，終覺實則視君之“主力生態”爲何。

- **蘋果全家桶 iCloud：**同步至使君不覺其存。君於手機上拍之照，數分鐘後電腦上即自有。且蘋果之隱私保護最嚴。然其弊亦顯：跨平臺分享予安卓或 Windows 用戶較繁，免費僅 5GB，擴容需費。
- **Windows 主力 OneDrive：**若君用 Windows 電腦 +Office 365，OneDrive 乃天作之合。Word 中存文檔，自動即存於雲端。1TB 空間配 Office 訂閱甚劃算。手機端 App 之自動備份照片之能亦佳。
- **國內分享百度網盤：**實言之，百度網盤之下載限速久爲人詬，然其分享生態誠無人能敵。中文互聯網上大量學習資料、影視資源皆以百度網盤分享。若僅偶下載他人分享之件，不充會員亦可湊合。若君爲重度資源收集者，會員該開即開，所省之時值錢多矣。

- **阿里雲盤/誇克網盤**：此二年乃後起之秀。阿里雲盤上傳下載不限速（目前如是），誇克網盤與誇克瀏覽器深綁，搜資料時即可直存至網盤，甚便。

吾之組合方案：iCloud 理個人照片與日常文件（隱私爲先），百度網盤存資源與受分享（生態爲先），OneDrive 理工作文檔（Office 集成爲先）。三網盤各司其職，互不相擾。

五、一真實之移動辦公場景

述吾己之真實經歷。嘗有一次出差至外地，客戶臨時欲改合同條款。吾之工作流如是：

1. 於酒店收客戶微信發來之修改意見（文字消息 + 一張手寫批註之照）。
2. 啓 OneDrive 尋合同之原件，以 WPS 啓而編輯，依客戶微信之消息逐條改條款。
3. 改畢以 WPS 之”輸出爲 PDF”導出，發至客戶微信羣確認。
4. 客戶提第二輪之意見，吾以 WPS 之畫筆之器徑於 PDF 上圈修改之處，配語音批註以明改動之思。
5. 確認無誤後，以 WPS 之”電子簽名”在線籤之，發予對方亦籤。
6. 終以籤畢之合同 PDF 存檔於百度網盤與 iCloud 各一，並於 Notion 之工作日誌記一筆。

全過程自始至終約四十分鐘，全程未開電腦。實言之，三年前吾斷不信手機能爲此等事，然今誠可。要在預諳此等工具，勿待需時方手忙腳亂下載註冊。

試爲之：搭建君之移動辦公工作流

1. 擇定君之主力文檔 App（WPS 或 Microsoft 365），登錄並關聯雲盤。
2. 開備忘錄或便籤，以掃描之能拍一份文檔，導出爲 PDF 發予己。
3. 擇一網盤 App，設爲手機照片自動備份（擇”僅 Wi-Fi 下備份”以省流量）。
4. 於 Notion 或飛書新建一頁，試搭一簡之”工作日誌”模板。

第十章：移動支付與生活之務

實言之，吾父母那一輩於“手機付錢”之事，初乃抗拒者。“錢在手機中，爲人所竊奈何?”此慮甚常，然實情乃：君褲兜中所揣之現金，遺失之概率遠高於手機支付被盜刷之概率。手機支付非以錢存入手機，乃一鑰匙耳，一開君銀行賬戶之鑰匙也。

一、究竟何以綁定銀行卡?

無論君用支付寶抑或微信，首次綁卡皆須行此數步：

1. 開 App，覓” 我的” 或” 錢包”。點” 添加銀行卡”。
2. 開 App，覓” 我的” 或” 錢包”。點” 添加銀行卡”。
3. 輸卡號。今多 App 皆支持直拍照識別卡號，無需一一數字敲之。
4. 銀行將發一驗證碼短信至君預留之手機號。填入之，設一 6 位數字之支付密碼。

有一小竅門：若慮安全之患，可往銀行 App 開一 II 類或 III 類電子賬戶，專以綁定手機支付。此卡中惟放平日零花之錢，縱有失，損亦有限。

二、付款碼乃“誰見誰收錢”

衆人多惑付款碼與收款碼之別，此別甚要。

- **付款碼**：君示予收銀員掃之彼條形碼 + 二維碼。此碼相當於君之” 電子錢包”，誰掃之誰即能自君賬上劃錢。故**切勿截圖發予任何人**。此碼每分鐘自動刷新一次，正爲防盜用。
- **收款碼**：他人掃此碼以轉賬予君。此碼可截圖存之，可發予他人，安全。

識此別，可避諸多詐騙之阱。凡有人令君“將付款碼截圖發來”者，百分之百乃騙子也。

三、此等安全設置，費五分鐘設好可省甚多之心

- **指紋/面容支付**：於支付寶與微信之支付設置中，開生物支付。如此每次付款須按指紋或掃臉，縱手機遺失，拾得者亦無法費君之錢。

- **安全鎖**：支付寶與微信皆有”安全鎖”之能。開之後，入 App 之”錢包”頁需額外驗身一次。路徑：支付寶我的設置賬號與安全安全中心安全鎖。
- **免密支付之查**：衆人簽約各種免密支付，己亦忘之。往支付寶與微信之”支付設置”中，覓”免密支付/自動扣款”，視簽約何服務，將不需者關之。
- **登錄設備之理**：定期視何設備曾登君之賬號。若見不識之設備，立踢之。路徑：支付寶設置賬號與安全登錄設備管理。

四、公交地鐵：手機即交通卡

今出門乘公交地鐵，已鮮有人用實體交通卡矣。手機刷閘機，有二路可行：

- **手機錢包中之虛擬交通卡**：於蘋果錢包、華爲錢包、小米錢包中直開一虛擬交通卡。此方案之利乃：刷時無需解鎖手機，直靠之即可，響應最速，且無網亦能刷。部分城市之虛擬交通卡尚有與實體卡同之換乘優惠。
- **支付寶/微信之乘車碼**：於 App 中搜”乘車碼”，開君所在城市之公交或地鐵乘車碼。此方案之利乃：無需充值，直自餘額或綁定之銀行卡扣。然需聯網，且刷前須開 App 覓碼。

吾個人更薦用手機錢包之虛擬交通卡。雖首次須充十塊二十塊，然用之誠便甚多，尤趕地鐵之時，無需停下解鎖手機覓 App。

五、生活繳費、外賣、打車

- **水電氣繳費**：於支付寶或微信之”生活繳費”中，將君家之水、電、燃氣戶號綁好。後每月收賬單，直點入即可交。戶號一般於君家之紙質賬單可得，或打客服電話問之。
- **外賣比價**：同一店於美團與餓了麼之價與優惠常異。下單前兩邊皆視之，有時可省數塊。又，美團有”拼好飯”之能，與友同點，價可便宜不少。
- **打車安全**：於滴滴或高德打車中，記設好”緊急聯繫人”與”行程分享”。打車之時，App 自動將君之行程分享予所設之聯繫人。又，深夜打車宜擇”出租車”而非”快車”，出租車有行業監管，安全係數更高。

六、以手機理醫保、社保、政務

此數年之變甚大，昔須奔大廳排隊之諸多事，今手機上數分鐘即畢。

- **醫保電子憑證**：於支付寶或微信中搜”醫保電子憑證”，激活之後，往醫院看病、藥店買藥直刷手機即可，無需帶實體醫保卡。首次需人臉識別驗證，後每次用時將生一動態碼。
- **社保查詢**：下載”掌上 12333”App 或以支付寶中之”市民中心”，可查社保繳納記錄、養老金賬戶餘額、失業金申領等。

- **個稅：**每年三月至六月之個稅彙算清繳，於” 個人所得稅”App 上即可畢。記視有無可申報之專項附加扣除（子女教育、房貸利息、贍養老人等），此等皆可助君少交稅或退稅。

記此數條，防騙保平安

- 公檢法、銀行、客服，不會令君轉賬至” 安全賬戶”。
- 凡令君將付款碼截圖發去者，皆騙子也。
- 凡令君將付款碼截圖發去者，皆騙子也。
- 網購退款將原路返，無需君提供銀行卡號。

今即爲之：爲君之支付 App 作一次安全體檢

1. 開支付寶與微信，確認已啓指紋/面容支付。
2. 覓” 安全鎖” 之能並開之。
3. 覓” 安全鎖” 之能並開之。
4. 視” 登錄設備” 之列，移除不識之設備。
5. 視” 登錄設備” 之列，移除不識之設備。

第十一章：攝影與圖像管理

實言之，衆人以手機拍照，所出之效遠不如手機本身之潛力。非君技不行，乃諸多小設置與小習君未察也。此章不述虛者，惟言君可立用之物。

一、君是否亦犯此等錯？

先言最常見之數患，君視中了幾條：

- **鏡頭乃髒**：此誠最普遍之患。掏手機之時指紋按於鏡頭上，拍出霧濛濛，還以爲手機壞矣。養習，拍照前以衣角拭鏡頭。
- **從不調曝光**：默認之式下，稍點屏乃對焦，然若點畢上下滑，可調亮度。衆人所拍之照非暗即亮，即不知此操也。
- **閃光燈永開**：手機 LED 閃光燈所拍之效誠醜，光又硬又冷，把人拍如鬼。除非真伸手不見五指，否則關之。暗光之下，以夜景模式遠勝閃光燈。
- **放大賴兩指拉伸**：於屏幕上雙指縮放乃數碼變焦，本質就是裁切畫面而後放大，畫質將明顯下降。若欲拍遠處之，直接用取景框上之變焦按鈕（0.5x/1x/2x/3x）。其中 0.5x（超廣角）與 1x（主攝）乃原生光學鏡頭，不損失畫質；2x/3x 則多爲" 光學品質變焦"——通過高像素傳感器裁切實現，畫質接近光學變焦但並非獨立鏡頭。超過 3x 之變焦通常就是混合變焦或數碼變焦矣，畫質將有明顯下降。

二、數立升出片率之小術

此等無需何理論知識，用一次即可見效：

- **三分法構圖 + 網格線**：於相機設置中開" 網格線"（亦稱參考線）。拍照時將君欲突出之物置四線之交點上，畫面立" 對" 甚多。此乃攝影中最簡亦最效之一規，無甚玄學，即合人眼之視覺習慣。
- **覓" 引導線"**：拍風景或街景之時，留意畫面中有無自然之線——馬路、欄杆、牆壁、一行樹——使此等線自畫面邊緣延至主體。此術善用，路人觀君所拍將以爲君學過攝影。
- **勿老立而拍**：衆人拍照永爲己眼之高，故所出之視角千篇一律。試蹲下自低角度拍之，或覓高處俯拍，同地所出之感全異。拍小孩與寵物尤需蹲下，平視方能拍出自然之感。

- **光線乃第一要素**：午時大太陽下拍人像，面上之影又硬又醜。最佳之光乃清晨與傍晚之”黃金時段”，陽光斜、色偏暖，拍何皆佳。陰天之光雖不出彩，然勝在均勻柔和，拍人像反較大太陽下易出片。
- **夜景模式使用要領**：開夜景模式之後，手機自動連拍多張而後合成。此時最要者乃——手絕不可抖。能靠牆即靠牆，能放桌上即放桌上，實在不可即屏息。夜景模式需君穩住二至四秒，抖則畫面糊矣。

三、人像模式與專業模式

- **人像模式**：能拍出背景模糊、主體清晰之效（淺景深）。手機因鏡頭物理尺寸小，真實虛化之能有限，故賴 AI 算法以成——簡言之即 AI 將君之人摳出，繼而模糊背景。用時保持一至二米之距效最佳，離太遠或太近算法皆易出錯。又，拍人像之時，注意背景中勿有太複雜之線，否則 AI 扣圖將於髮際邊緣”翻車”。
- **專業模式**：若君欲全控畫面之效，可試專業模式。數個關鍵參數當知：ISO 越低畫質越淨，然畫面將暗；快門越慢進光越多，然手抖將糊；白平衡管色之冷暖，調不佳照將偏藍或偏黃。常人無需常年用專業模式，然解此數參數至少可使君知相機在爲何。

四、後期修圖：勿過度，恰到好處即足

衆人對修圖有誤解，以爲修即不真。實則相機所拍之照本與肉眼所見不同，適當調整乃爲還君當時所見之畫面，非造假也。

- **Snapseed**：谷歌出之免費修圖工具，功能專業然無廣告。吾日常修圖之流乃：先以”調整圖片”拉亮度與對比度以”突出細節”銳化之以”白平衡”糾偏色終”裁”成所欲之構圖。Snapseed 之”局部”甚佳，可僅調畫面中某一區域，無需全圖同動。
- **醒圖/美圖秀秀**：此二者宜修人像。然吾以爲有一則：美顏濾鏡拉至三成內即佳，逾此數即假，膚如塑料。自然較完美更好看。
- **Lightroom 手機版**：若君對色有追求，此乃最專業者。有曲線調整、HSL 分色調節此等高級之能。然於多數人而言稍殺雞用牛刀。
- **修圖工作流之議**：吾己之習乃先拍好原片，歸家後於電腦屏或 iPad 上慢慢修，手機屏太小，色亦不準。若必於手機上修，記將屏亮度調至中偏上，勿於省電之式下修圖。

五、照片太多奈何？整理之策

手機相冊中堆數千張照片，欲覓一張特定者翻半日——此體驗衆人皆有。數實用之整理法：

- **善用搜索**：今 iPhone 與大部分安卓手機之相冊皆支持智能搜索。君可搜”貓””海邊””身份證””花”此等關鍵詞，系統將以 AI 識別畫面內容助君覓出。又可搜日期與地點，如”去年五月””北京”。此能知者不多，然誠甚佳。

- **定期清理垃圾**：截圖、重複之照、拍糊之廢片，此等乃佔空間之元兇。iOS 相冊有”重複項目”自動合併之能；安卓可往文件管理中搜”screenshots”文件夾，批量刪不需之截圖。吾每月花五分鐘清理，即可騰出數 G。

- **建相簿，勿全賴默認排序**：系統默認按時排序無錯，然於”孩子之成長照”“旅行照片”“重要證件”此等跨時段之內容，君當手建相簿歸類。蘋果之”共享相簿”尚可與家人同往其中加照，一人建，全家皆可觀。

- **備份乃底線**：照片乃最不可替之數據——文件失可重建，聊天記錄無亦即無，然孩子之照、老人之視頻失即真無矣。必開雲備份。iCloud 照片最省心，安卓這邊各品牌皆有己之雲服務，或以百度網盤/阿里雲盤自動備份。原則乃：至少有一份雲備份，勿將雞蛋置於同一籃中。

六、截圖與錄屏

- **截圖**：同按電源鍵 + 音量加鍵，乃所有手機通用之法。多安卓手機尚支持三指下滑截圖、指關節雙擊截圖。長截圖（滾動截圖）於安卓上甚成熟——截畢一張後點”滾動”按鈕，手機自動往下翻頁並拼爲一張長圖，宜截聊天記錄與網頁。

- **錄屏**：自控制中心添”屏幕錄製”，稍點始錄，再點止。錄前長按錄製按鈕可擇是否錄麥克風之聲——君欲作教程解說即開麥，僅錄操作畫面即關之。錄畢之視頻將自動存至相冊。

練之：以手機成一次完整之拍照 + 修圖 + 整理

1. 開相機設置中之”網格線”，覓一有自然光之處拍三張照片（注意拭鏡頭、調曝光）。
2. 擇最佳之一張，以 Snapseed 作基礎調整（亮度、對比度、裁剪）。
3. 爲此照建一專屬相簿，名爲”吾之作品”。
4. 爲此照建一專屬相簿，名爲”吾之作品”。

第十一章附：專業影像技術之識

欲真解手機攝影與視頻之奧，君需解若干專業之影像技術之識。此等識不僅能助君更善用手機相機，尚能使君於擇設備時更具判斷力。

一、分辨率、幀率與碼率：視頻質量之三要素

分辨率（Resolution）：

分辨率定畫面之清晰度，以像素數表之。

- **720P (HD)**：1280×720 像素，入門級清晰度。

- **1080P (Full HD)**：1920×1080 像素，今最主流之分辨率。
- **2K (QHD)**：2560×1440 像素，較 1080P 更清晰。
- **4K (UHD)**：3840×2160 像素，超高清，細節豐富。
- **8K**：7680×4320 像素，未來之標準，今設備支持有限。

分辨率越高，畫面越清晰，然文件亦越大，對存儲與處理之能求越高。

幀率（Frame Rate）：

幀率定畫面之流暢度，以每秒幀數（fps）表之。

- **24fps**：電影標準幀率，有獨特之”電影感”。
- **30fps**：電視與網絡視頻常用，流暢自然。
- **60fps**：高幀率，宜運動場景與遊戲錄製，甚流暢。
- **120fps/240fps**：超高幀率，可用於慢動作視頻。

幀率越高，畫面越流暢，慢動作之效越佳，然文件亦越大。

碼率（Bitrate）：

碼率定視頻之數據密度，以每秒數據量（Mbps）表之。

- 碼率越高，畫質越佳，細節保留越多，然文件亦越大。
- 碼率過低將致畫面現馬賽克、色塊等壓縮之痕。
- 碼率過低將致畫面現馬賽克、色塊等壓縮之痕。

二、視頻編碼格式：H.264、H.265 與 ProRes

H.264 (AVC)：

- **最高速率**：最廣用之編碼格式，兼容性極佳。
- **優點**：幾所有設備皆能播，壓縮效率適中。
- **缺點**：較新格式，壓縮效率較低，文件較大。
- **適用場景**：網絡視頻分享、普通用戶日常拍攝。

H.265 (HEVC)：

- **最高速率**：H.264 之升級版，壓縮效率提升約五成。
- **優點**：相同畫質下文件更小，支持更高分辨率與幀率。
- **缺點**：編碼與解碼需更強之處理器，部分老設備不兼容。
- **適用場景**：4K 視頻拍攝、存儲空間有限之場景。

Apple ProRes：

- **最高速率**：蘋果開發之專業視頻編碼，幾無損壓縮。
- **優點**：畫質極高，後期處理餘地大，色彩準確。
- **缺點**：文件極大，需大量存儲空間，處理需專業設備。
- **適用場景**：專業視頻製作、電影級拍攝。

ProRes RAW：

- **最高速率**：合 ProRes 與 RAW 格式之長。
- **RAW 爲何**：未經處理之原始傳感器數據，保留最大信息量。
- **優點**：後期調整餘地極大，可改曝光、白平衡等參數而不損畫質。
- **缺點**：文件極其巨大，需專業後期軟件處理。
- **適用場景**：高端專業製作。iPhone 15 Pro 系列及後支攝 Apple ProRes 視頻（最高 4K 60fps），iPhone 17 Pro 進一步支 ProRes RAW 徑機內錄製。

三、HDR：高動態範圍技術

HDR（High Dynamic Range）技術可同時保留畫面中最亮與最暗部分之細節。

- **HDR10**：最基礎之 HDR 標準，兼容性好。
- **HDR10+**：三星推動之標準，支持動態元數據。
- **Dolby Vision**：杜比開發之頂級 HDR 標準，畫質最佳，然需授權。
- **HLG**：廣播級 HDR 標準，兼容普通顯示器。

拍 HDR 視頻時，需支持 HDR 之顯示器方能見完整之效。

四、色彩空間與色深

色彩空間：

- **sRGB**：標準色彩空間，兼容性最佳。
- **Display P3**：蘋果推廣之廣色域標準，色彩更豐。
- **Rec.2020**：HDR 視頻之標準色彩空間，色域最廣。

色深：

- **8-bit**：1670 萬色，普通視頻標準。
- **10-bit**：10.7 億色，專業視頻標準，色彩過渡更平滑。
- **12-bit**：680 億色，高端專業標準。

實用之議

- 日常拍攝：1080P/4K + 30fps + H.265 編碼，平衡畫質與文件大小。
- 運動拍攝：1080P/4K + 60fps，保流暢度。
- 運動拍攝：1080P/4K + 60fps，保流暢度。
- 存儲之議：高碼率視頻甚佔空間，議定期備份至電腦或雲盤。

探之：君之手機視頻設置

1. 開相機 App，切至視頻模式。
2. 開相機 App，切至視頻模式。
3. 視分辨率與幀率之項，試不同之組合。
4. 拍數段不同設置之視頻，比畫質與文件大小。

五、RAW 格式與 HEIF/JPEG 對較

何乃 RAW 格式：

RAW 格式乃相機傳感器記錄之“原始數據”——未經任何壓縮與處理之純原始信息。它遂像生鮮食材，須後期“烹飪”才能成品。手機蓋因存儲與算力有限，默認拍出來之皆乃已“烹飪好”之 JPEG 或 HEIF，RAW 須專門開啓。

RAW vs HEIF vs JPEG 對較：

- **JPEG**：最老之格式，兼容性最好（任何設備皆能打開）。8-bit 色深，有損壓縮，文件最小（一張照片 2-5MB）。後期空間有限，強行提亮暗部容易出現色塊。

- **HEIF/HEIC**：蘋果主推之新格式，同樣畫質下文件只有 JPEG 之一半（一張照片 1-3MB）。支持 10-bit 色深與透明通道。缺點乃部分老設備與 Windows 須安裝插件才能打開。

- **RAW**：未經壓縮之原始數據，文件最大（一張照片 20-100MB）。色深通常 12-16bit，後期空間最大。缺點乃須用專業軟件（Lightroom、Snapseed 等）才能打開與處理，無法直接分享。

何故專業攝影師要拍 RAW：

- **白平衡可後期無損調**：JPEG 拍錯矣白平衡（比如白熾燈下偏黃），後期糾正將損失畫質；RAW 可重新設置色溫，完全無損。

- **曝光容差大**：JPEG 欠曝 2 檔之暗部基本乃死黑一片，提亮後全乃噪點；RAW 能自欠曝 3-4 檔之暗部" 救" 出可用細節。過曝同理。

- **暗部與高光細節更豐富**：RAW 之動態範圍遠大於 JPEG，亮部然則曝、暗部不死黑之餘地大得多。

- **銳化、降噪、鏡頭矯正皆在後期可控**：相機之機內處理乃" 一錘子買賣"，RAW 可反覆調整，每次皆自原始數據出發。

手機何以開啓 RAW：

- **iPhone ProRAW**：僅 iPhone 12 Pro 及以上 Pro 機型支持。打開" 設置相機格式" 中開啓"Apple ProRAW"。而後於相機界面頂部將出現一個"RAW" 按鈕，點亮即可。文件約 25MB/張，建議有需求時再開。

- **安卓 Motion Raw / Ultra RAW**：各品牌叫法不同。小米/OPPO/vivo 旗艦機型於相機" 專業模式" 或" 更多" 裏能找到 RAW 開關；華爲於" 專業模式" 中開啓 RAW；三星 Galaxy S 系列於" 專業" 模式裏有 RAW 選項。文件大小通常於 20-50MB。

- **通用宜**：日常隨手拍沒必要開 RAW，文件太大佔空間；只有於重要場景（風光、人像、須後期）時方開啓。

六、算攝影理

現代手機攝影已離不開“計算攝影”——亦就是靠算法而非獨乃鏡頭與傳感器來生成照片。理解這幾個核心原理，能使汝更明白手機於按下快門之瞬間作了何。

多幀合成：

手機按下一次快門，實則拍矣多張照片。算法以多幀對齊、加權融合，得到一張高質量成片。

- **夜景模式**：1-3 秒內連拍多張欠曝照片，對齊後疊加——噪點互相抵消、亮度累加。結果就是暗光也能拍出乾淨之畫面。
- **HDR（高動態範圍）**：分別拍攝欠曝、正常、過曝三張（或多張），合成一張亮部與暗部皆有細節之照片。逆光場景尤其有效。
- **Deep Fusion（蘋果）/Photonic Engine（光子引擎）**：針對中低光場景，逐像素分析多幀並融合，重點提升紋理與細節，毛衣、毛髮這類細節將明顯更清晰。

語義分割：

AI 不只是以整張圖一刀切處理，而乃先“觀懂”畫面裏有何——天空、人臉、草地、建築、衣服——而後對每個區域分別應用不同之處理策略。

- **天空**：單獨增強藍色、壓低亮度，避過曝。
- **人臉**：識別膚色區域，單獨提亮、平滑膚色，不動五官與頭髮細節。
- **草地/樹葉**：增強綠色飽和度，升細節銳化。
- **背景虛化**：人像模式下，AI 識別主體輪廓，背景模糊、主體清晰。

深度學習之應用：

- **AI 場景識別**：相機自動識別” 美食”” 寵物”” 夜景””PDF” 等場景，並對應優化參數。比如美食模式將增強暖色調和飽和度，使食物看起來更誘人。
- **AI 人像虛化**：手機鏡頭物理上無法作至單反那種淺景深，靠深度學習模型預測主體邊緣，再對背景作模糊。最新算法已能正確處理頭髮絲、玻璃邊緣此細節。
- **AI 消/AI 擴圖**：選中畫面中之路人或雜物，AI 基於上下文” 補全” 爲遮擋之部分。這乃生成式 AI 於影像領域之應用。

算攝影 vs 光學攝影：

- **計算攝影之優勢**：小體積也能得到好畫質，暗光、逆光等極端場景也能出片，普通用戶不須懂攝影知識遂能拍出” 好觀” 之照片。
- **計算攝影之劣勢**：畫面有時” 過度處理”——銳化過度、色彩失真、天空顏色不自然；多幀合成於主體移動時將留下重影；細節乃” 算出來之” 而非真實記錄之。
- **光學攝影之優勢**：大底傳感器 + 大光圈鏡頭能真實捕捉光線，畫質自然，後期空間大；未能蓋因算法” 腦補” 產生僞影。
- **趨勢**：現代旗艦手機乃” 光學 + 計算” 結合——用大底傳感器獲取真實信息，用算法優化呈現。兩者並不對立。

七、鏡頭類型與焦段

現代旗艦手機通常配三至四顆鏡頭，每顆對應不同焦段。理解焦段，能使汝於不同場景下選對鏡頭。

常見鏡頭與等效焦距：

- **超廣角（13-24mm 等效）**：視野最廣，適合風光、建築、室內空間。能拍至眼前所見之大場面，但邊緣將有畸變（直線變彎），拍人像時人臉將爲拉伸。適合" 大場景"，不適合" 特寫"。
- **主攝（24-28mm 等效）**：日常最常用之鏡頭，畫質最好、光圈最大。兼顧視野與畫質，拍人拍景皆能用。大多數手機之"1x" 就是主攝。
- **長焦（70-200mm 等效）**：用以拍遠景、人像特寫。長焦能壓縮空間感（遠處之山與近處之人看起來更近），拍人像能獲得更自然之五官比例與背景虛化。
- **潛望式長焦（200mm+ 等效）**：通過反射式光路設計，於手機厚度內實現更長焦距。能於甚遠距離拍清楚細節，適合演唱將、體育賽事、野生動物。旗艦機型方有。

傳感器尺寸與像素大小之權衡：

傳感器乃“底”，決定能接收幾何光線。行業裏曰“底大一級壓死人”——同樣像素數下，傳感器越大，單個像素亦越大，進光量越多，畫質越好。

- **大底傳感器**：進光量大，夜景畫質好，動態範圍寬。缺點乃手機要作厚（鏡頭模組凸起），成本高。
- **高像素小底**：白天光線充足時，高像素能拍出更清晰之照片，方便後期裁切。但暗光下每個像素進光不足，噪點多。
- **主流旗艦擇**：主攝多爲 1/1.3 英寸至 1 英寸（部分機型已用上 1 英寸大底），像素數 5000 萬左右——這乃畫質與體積之平衡點。

像素融合技術（Pixel Binning）：

爲了兼顧白天高像素與夜晚大像素，手機廠商用“像素融合”——以多個小像素合併成一個大像素輸出。

- **四合一像素（Quad Bayer）**：5000 萬像素傳感器，輸出 1250 萬像素照片。每個輸出像素由 4 個相鄰像素合成，進光量升 4 倍，畫質更乾淨。
- **九合一像素（Nonacell）**：用以 1 億像素以上之傳感器，輸出約 1100-1200 萬像素。同樣原理，進光量更大。
- **實際效果**：默認拍照模式下，汝拿至之就是融合後之低像素照片——畫質好、文件小。須極致細節時（比如拍文檔、遠景），可手動切至" 高像素模式" 輸出全分辨率。

八、特殊攝模式

除矣日常拍照，手機裏還有幾個特殊模式，用對矣能拍至平時拍不出之畫面。

全景模式：

- 拍攝時按提示箭頭緩慢平穩地橫向移動手機，勿快——快矣將出現拼接錯位。
- 保持手機垂直方向穩定，唯橫向平移。若上下晃動，地平線將變波浪。
- 有移動之人或車時慎用——拼接時將出現" 半個身體" 或" 鬼影"。
- 全景模式亦可豎着拍——豎拍橫拼接，適合拍高聳之建築或瀑布。

慢動作與延時攝影：

- **慢動作**：用 120fps 或 240fps 高幀率攝，回放時變成慢鏡頭。適拍水滴、運動瞬間、寵物跳躍。光要充，慢動作模式下進光時短，暗光畫質差。
- **延時攝影**：間隔拍攝多張照片合成快進視頻。適合拍車流、雲彩移動、日落、花朵開放。拍攝時長幾分鐘至幾小時不等，務必用三腳架固定手機，手持必糊。
- **小術**：延時攝影拍雲彩時，把手機架好，拍 15-30 分鐘，終得到 10 秒左右之延時視頻，效果震撼。

星空/天文模式：

- 須遠離城市光污染，找完全沒有路燈干擾之郊外或山區。城市裏拍不出銀河。
- 須用三腳架，手持必糊。手機要穩穩架住幾秒至幾十秒。
- 天氣要晴朗無雲，月亮勿太大（滿月將蓋住星光）。農曆月初或月末之暗夜最適合。
- iPhone 之" 夜間模式" 將自動識別星空並延長曝光時間（最長 30 秒）；安卓旗艦之" 星空模式" 或" 天文模式" 效果類似。要耐心等待，按下快門後勿動手機。

微距模式：

- 用以拍攝花朵、昆蟲、珠寶、文字特寫等小物體。能貼近至 1-2 釐米距離對焦。
- 部分手機有專門之微距鏡頭（多爲 200 萬像素或 500 萬像素之小底鏡頭），畫質一般；旗艦機用超廣角鏡頭兼顧微距，畫質更好。
- 貼近拍攝時鏡頭離主體甚近，將遮擋光線，必要時用臺燈或手電補光。
- 對焦距離極近，景深甚淺，主體前後移動幾毫米遂或脫焦。拍攝時手要穩，最好使主體保持不動。

靜態攝影實用宜

- 日常攝：HEIF/JPEG 默認即可，文小、畫質足用。
- 重要場景（風光、人像、須後期）：開啓 RAW，保留最大後期空間。
- 擇對鏡頭：風光用超廣角，人像用長焦，日常用主攝。
- 夜景與星空：務必固定手機，開夜景模式或天文模式。
- 大底手機（如 1 英寸傳感器機型）畫質上限更高，但操作習慣仍乃關鍵。

動手試試：摸清汝之相機能力

1. 查看汝之手機是否支持 RAW 拍攝（設置相機格式），試着開啓並拍攝一張，對比同一場景之 JPEG 與 RAW 文件大小。
2. 切換至超廣角、主攝、長焦三個焦段，分別拍同一處遠景，體會視角差異與畫質差異。
3. 找一個有夜景模式或星空模式之場景，固定手機後拍攝一張夜景照片。
4. 嘗試用全景模式橫拍一張大場景，留意持手機平穩移。

第十二章：影音娛樂與閱讀

手機今基本上已代 *MP3*、電視遙控器、*Kindle* 與掌上遊戲機。然實言之，衆人僅用此等 *App* 最基本之能，尚有諸多佳處未察。此章即論何以使手機娛樂更順、更適。

一、聽音樂：擇何 App，何以聽方適

音樂 App 今功能皆相仿，然各有側重，擇對可省不少心：

- **網易雲音樂**：其強項乃歌單社區與評論區。君欲聽" 宜雨天聽之歌" 或" 學倦放鬆用之純音樂"，搜之即有他人整理好之歌單。評論區有時比歌更精彩。弊乃部分歌曲版權不全，有些歌聽着聽着即灰矣。
- **QQ 音樂**：版權最全，周傑倫、五月天此等熱門歌手基本上惟 QQ 音樂有。音質選項亦豐，會員可開無損音質。若君對" 欲聽之歌皆能搜到" 此事甚在意，QQ 音樂乃最穩之選。
- **Apple Music**：蘋果用戶之首選。界面淨、無廣告、音質佳，且支持無損與杜比全景聲。然其推薦算法不如國內 App 諳中文用戶之好，歌單文化亦不如網易雲發達。宜喜簡潔體驗、不太在乎社交功能之人。
- **Spotify**：全球用戶最多，推薦算法公認最強。" 每週發現" 與" 日推" 準確率高得嚇人，用越久越諳君。弊乃免費版廣告多，且某些中文歌版權覆蓋不全。需特殊網絡方可用。

數聽歌小術：若君用藍牙耳機，往音樂 App 設置中視有無“音效調節”或“均衡器”，稍調低音與高音，效提升明顯。又，流媒體聽歌甚費流量，記於設置中將“僅 WiFi 下播放無損”開之，或預下載好喜之歌單離線聽。

二、播客（Podcast）：通勤路上之“耳朵伴侶”

播客此物，未聽者覺與己無關，一旦始聽即回不去。本質即“電臺節目之 Netflix 版”——有人聊科技，有人講歷史，有人讀故事，君欲聽何類皆有。

- **小宇宙 App**：國內做得最佳之播客 App，界面清爽，無雜亂之能。其" 每日推薦" 與" 編輯精選" 助君發現新節目，評論區亦活躍。吾個人聽最多者乃《忽左忽右》（文化歷史類）與《不合時宜》（社會議題類），每期如旁聽數聰明人聊天。

- **蘋果播客**：iPhone 自帶，無需額外下載 App。與 CarPlay 之集成最佳，開車之時甚便。
- **收聽之術**：大部分播客 App 皆支持 1.2 倍速或 1.5 倍速播放，習之後可省不少時。吾一般 1.2 倍聽中文，1.5 倍聽英文。又必開” 自動下載新節目” 與” 播畢自動刪除”，如此君之手機不會堆滿聽過之音頻文件。

三、看視頻：自播放器至流媒體之正解

衆人不知，手機上可播之格式遠比流媒體 App 所供者多。一佳之本地播放器可解諸多問題。

- **VLC 播放器**：開源免費，幾何格式皆能播——MKV、AVI、FLAC、甚至 ISO 光盤鏡像。若君下載一視頻於電腦上能播然手機上播不了，八成乃格式之患，裝一 VLC 即解。其尚支持外掛字幕，將.srt 字幕文件與視頻置同一文件夾，VLC 將自動加載。
- **nPlayer**：若君欲更精緻之體驗，此付費 App 值得買。界面美觀，手勢控制甚順——左邊上下滑調亮度，右邊上下滑調音量，左右滑快進快退。支持 DTS 與杜比音效，看大片之時臨場感強多矣。
- **Bilibili（B 站）**：已不只是二次元網站矣，今上之學習資源多到離譜。考研、編程、攝影、烹飪，何教程皆有。彈幕文化乃 B 站之魂，看視頻之時開着彈幕，感如有一羣人陪君同觀。然看學習視頻之時議關彈幕，不然易分心。
- **投屏至電視**：手機屏再大亦不如電視看着適。同一 WiFi 下，開視頻 App 右上角之”TV” 投屏按鈕，即可將畫面投至電視上。大部分智能電視與投影儀皆支持，看劇看電影體驗佳多矣。iPhone 用戶尚可 AirPlay 直鏡像整個屏幕。

四、電子書閱讀：手機上亦能好好看書

有人言手機上讀書傷眼、易分心，此等誠有理。然手機閱讀之優勢亦顯——隨時隨地，碎片之時利用最大化。吾於地鐵上、排隊之時、睡前，以手機讀了不少書。

- **微信讀書**：目前國內體驗最佳之閱讀 App。書庫甚全，出版書與網文皆有。其以” 閱讀時長換書幣” 之式，看得多即可免費換書，對學生黨甚善。其” 劃線” 之能可觀他人在何段作了標記，有時見他人標註之佳句，誠有共鳴。然” 閱讀排行榜” 此物有點卷，易變爲” 爲刷時而讀書”，議關之。
- **Kindle App**：若君有 Kindle 設備，手機端 Kindle App 可同步閱讀進度。Kindle 之正版電子書排版質量比盜版 PDF 佳甚多，然 Kindle 中國區書城已關，需切至海外區。
- **Apple Books（圖書）**：iPhone 自帶，宜讀 EPUB 與 PDF。界面極簡，翻頁動畫甚適。若君自網上下載電子書文件，直” 分享至圖書” 即可導入。
- **閱讀設置之議**：無論用何 App，必以背景設爲暖黃色調（護眼模式），字體調大一點，行間距拉開。夜讀開” 深色之模”——黑底白字，眼適甚多。切勿白底黑字小字體於暗光下讀太久，眼疲乃積累者也。

五、手機遊戲：玩得爽，亦不傷手機

吾對手機遊戲之態乃：玩可，然勿令其反控君。合理用遊戲工具，玩得更適，亦不誤正事。

- **遊戲模式/遊戲空間**：今主流手機皆有此能。開之後手機自動清後臺、屏通知、防誤觸——君正打團戰，忽一電話彈出，此等事遊戲模式可助君擋之。部分手機尚支持性能模式/E-sports 模式，強處理器跑滿，然代價乃發熱與耗電皆更速。
- **畫質與幀率何以調**：勿無腦拉滿。先保幀率（流暢度），再追畫質。60 幀比 30 幀適甚多，然 120 幀對手機壓力甚大，中端機無需強求。若君玩乃競技類遊戲（比如王者、喫雞），幀率優先級遠高於畫質——流暢之操作手感比好看之草皮重要百倍。
- **遊戲時手機發燙奈何**：此乃正常，然長期高溫將損電池壽命。數降溫之法：摘手機殼散熱；將屏亮度降至七成左右（亮度乃高發熱之源）；勿一邊充電一邊打遊戲（發熱疊加，電池死得速）。若手機燙至手握不住之程度，速退出令其歇一會兒。
- **防沉迷小工具**：iOS 之"屏幕用時"與安卓之"數字健康"皆可爲遊戲 App 設每日時限。吾爲己設乃工作日最多一時，週末二時。至時 App 圖標將變灰，點不進。此限真有用，非裝樣子。
- **雲遊戲**：若君手機性能不夠然又欲玩大作，可試雲遊戲平臺（比如騰訊 START 雲遊戲、網易雲遊戲）。遊戲於遠程服務器上跑，畫面以視頻流之式傳至君之手機上。對網絡求高然對君手機性能幾乎無求。弊乃延遲感不可避免，宜玩策略類、RPG 類，不宜 FPS 射擊類。

動手爲之：打造君之手機娛樂系統

1. 於音樂 App 中下載君最喜之 3 張專輯或歌單，確保離線可聽。
2. 往小宇宙或蘋果播客訂閱 3 個感興趣之頻道，下載一期試聽。
3. 裝 VLC 播放器，試以之播一段電腦拷來之視頻文件。
4. 裝 VLC 播放器，試以之播一段電腦拷來之視頻文件。
5. 若君玩遊戲，開遊戲模式視有何設置可優化（免打擾、幀率優先等）。

第十三章：工具類 App 推薦

此章精選效率、健康、出行類工具，並補充實用系統與跨設備之能：清濁等清理與防騷擾、手機自帶攔截開關、天氣與地震預警；向日葵 / *ToDesk* 遠控電腦、*Trae* 等 *AI* 辦公、人工桌面、*LocalSend* 等局域網快傳、跨設備剪貼板、*Aegis* / *Microsoft Authenticator* 動態口令、滴答清單日程與紙質掃描。

一、效率工具：管理汝之時與注意力

效率工具乃數字生活之「骨架」，助爾將零散之想法、任務、資料整理得井井有條。下按場景分類推薦。

待辦清單（任務管理）

- 滴答清單（TickTick）　所宜之人：須將工作、生活、學習任務與日程統一記錄者；此節重點推薦之一。其要：跨手機 / 電腦 / 平板 / Web 同步；清單 + 日曆 + 番茄鐘三合一；自然語言輸入「明天下午 3 點開會」「每週一交週報」自動識別時間與重複規則；支持子任務、標籤、優先級、附件與協作清單。宜「日程安排 + 任務打卡 + 每日回顧」一體化管理。
- 滴答清單如何用得更順：收集箱：先悉數投入「今天 / 收集箱」，晚間再歸類至工作、家庭、學習清單。
- 日曆視圖：會議、截止日拖至日曆觀其衝突，較徒視待辦列表更直觀。
- 重複任務：服藥、繳費、健身設為規則任務，毋每次手填。
- 番茄鐘：於任務上徑啟專注，統計本週「果成幾何」。
- 與手機功能聯動：語音助手言「提醒我…」可入系統提醒；重要截止日兼啟系統日曆以為雙保險。

Todoist　所宜之人：求極簡、跨平臺之用戶。其要：界面克制清爽，項目層級清晰（任務子任務子子任務），karma 積分激勵習慣養成。

Microsoft To Do　所宜之人：辦公族、Outlook 用戶。其要：完全免費、無廣告，「我的一天」功能每日清空、減輕心理之負，與微軟生態深度集成。

系統便箋 / 提醒事項　所宜之人：不欲裝第三方 App 之輕度用戶。其要：原生體驗、零延遲，Siri / 小愛語音徑直添加，與系統 widget 聯動。

番茄鐘（專注力訓練）

- Forest　所宜之人：管不住手、易刷手機者。其要：專注時種樹，提前離 App 則樹枯，以「虛擬生命」建立正向反饋，可與好友合種。
- 潮汐　所宜之人：須白噪音輔專注 / 入睡者。其要：自然音效（雨聲、海浪、森林）+ 番茄鐘 + 冥想三合一，界面極簡療癒。
- 原則：經典 25 分鐘專注 +5 分鐘休息，可因自身狀態調為 50/10 或 90/20。

日曆（時間可視化）

- 系統日曆　所宜之人：所有用戶。其要：原生推送最穩、耗電極低，可與 Siri / 小愛語音添加。
- Google Calendar　所宜之人：多設備用戶、須與他人共享日程者。其要：「目標」功能自動為爾安排健身 / 學習時間，與 Gmail 智能識別郵件中之事件。

筆記類（知識沉澱）

- Notion　所宜之人：須統一管理筆記、任務、數據庫、wiki 者；學生、自由職業者、團隊協作。其要：全能知識庫，Block 模塊化編輯，可搭建個人主頁、閱讀清單、項目看板，模板生態豐富。
- Obsidian　所宜之人：深度知識管理者、研究者、寫作者。其要：本地 Markdown 存儲，雙向鏈接 + 關係圖譜，使爾筆記成「第二大腦」，數據完全歸爾所有。
- flomo　所宜之人：欲記錄碎片靈感而不欲寫長文者。其要：卡片式筆記，如發朋友圈然記錄想法，支持 # 標籤分類，按時間線回溯。
- 蘋果備忘錄　所宜之人：iPhone / Mac 用戶、輕度記錄之需。其要：啟動快、支持掃描文檔 / 手繪 / 清單 / iCloud 同步，免費且與系統深度集成。
- 印象筆記　所宜之人：須大量網頁剪藏、資料歸檔之資深用戶。其要：網頁剪藏插件強大，OCR 識別圖片文字，老牌穩定，然高級版須付費。

PDF 工具

- PDF Expert　所宜之人：iPad / iPhone 用戶、須批註合同之商務人士、學生。其要：iOS 上公認最強 PDF 編輯器，標註流暢、支持簽名、合併拆分、填寫表單。
- WPS　所宜之人：須同時處理 Word / Excel / PPT / PDF 之辦公族。其要：一站式辦公套件，國內網盤 / 微信文件兼容好，免費版功能已足日常。

- Adobe Acrobat　所宜之人：於 PDF 格式規範要求高者（法律、設計、出版）。其要：官方標準、轉換 PDF/A 格式無誤，跨設備同步與電子簽名功能專業。

掃描類（紙質文件數字化）

紙質合同、發票、作業、白板筆記、身份證複印件……手機攝像頭已能替代大部分掃描儀。核心流程：拍清楚自動裁邊與矯正導出 PDF / 圖片 OCR 隨選。

- 掃描全能王（CamScanner）　所宜之人：屢須掃描合同、票據、證件之商務人士與學生。其要：邊緣自動識別、透視矯正、OCR 文字識別，可生成多頁 PDF 並雲同步。
- 微軟 Lens　所宜之人：學生、須將白板 / 書頁掃成 Word 者。其要：免費無廣告，可一鍵導出至 OneNote / Word，識別文字準確率高。
- 蘋果備忘錄掃描　所宜之人：iPhone 用戶、輕度掃描之需。其要：無需裝 App，備忘錄內點相機圖標即可掃描，自動去陰影、簽名標註，免費好用。
- 白描 / 掃描全能王網頁版 / 品牌相機「文檔掃描」：華為、小米、OPPO、vivo 系統相機大多內置「文檔 / 掃描」模式；安卓亦可用 Google 相機或系統掃一掃。宜應急拍一頁合同。
- 拍照小技：光線均勻、手機與紙面平行、對準四角；多頁文件按序拍畢再合併 PDF；敏感證件留意水印與保存位置，用畢可自「最近刪除」徹底清理。

二、健康運動

健康類 App 乃手機最被低估之「私人助理」——能持續追蹤爾之睡眠、心率、運動量，較偶往醫院一次更能反映長期狀態。

運動健身

- Keep　所宜之人：居家訓練者、零基礎新手、欲減脂塑形者。其要：海量免費課程，自 HIIT 至瑜伽全覆蓋，AI 自動生成訓練計劃，社區氛圍濃厚。
- Nike Training Club（NTC）　所宜之人：求專業教練指導、望系統訓練者。其要：Nike 官方出品，課程全部免費，動作演示由專業教練錄製，按肌群 / 器械分類清晰。
- 蘋果健身（Fitness）　所宜之人：Apple Watch 用戶、欲以閉環激勵自己動起來者。其要：每日三活動圓環（移動 / 鍛煉 / 站立），目標可視化，與手表心率數據無縫打通。

睡眠追蹤

- AutoSleep　所宜之人：Apple Watch 用戶、欲知每晚深睡 / 淺睡比例者。其要：戴表入睡自動追蹤，無需手動啟停，予睡眠質量評分與就寢建議。

- Sleep Cycle　所宜之人：不戴表睡覺、欲以手機監測者。其要：以麥克風分析翻身 / 呼嚕聲，於淺睡階段智能喚醒，使爾起牀不苦。

- 小米運動健康　所宜之人：小米手環 / 手表用戶。其要：免費且與小米手環深度聯動，心率 / 血氧 / 睡眠一體追蹤，性價比高。

冥想放鬆

- Calm　所宜之人：壓力大之上班族、入睡困難者。其要：「每日平靜」冥想引導 + 睡眠故事（名人朗讀），界面唯美，英文內容豐富。

- 潮汐　所宜之人：須白噪音專注 / 入睡之中文用戶。其要：國產精品，自然音效 + 冥想課程 + 番茄鐘，中文引導更親切。

- 小睡眠　所宜之人：須助眠音效、記錄夢話 / 呼嚕聲者。其要：海量白噪音可疊加組合（雨聲 + 風扇 + 篝火），睡前定時關閉，夢話錄音有趣又實用。

女性健康

- Flo　所宜之人：須記錄經期、備孕或關注女性健康之用戶。其要：AI 預測下次經期 / 排卵日準確度高，社區可匿名提問，界面設計精緻。

- 美柚　所宜之人：國內女性用戶、望看中文社區與科普者。其要：本地化內容豐富，按經期 / 備孕 / 孕期 / 育兒分階段提供對應知識。

用藥提醒

- 用藥助手　所宜之人：須長期服藥之慢性病患者、須照顧老人用藥之子女。其要：掃碼識別藥品、自動建立用藥清單，按時彈窗提醒，記錄服藥情況可分享予家屬。

數據中樞

- 蘋果健康 / Google Fit　所宜之人：所有智能手機用戶。其要：匯聚來自手機、手環及第三方健康 App 之所有數據（步數、心率、睡眠、經期等），乃統一之「健康檔案中心」。

三、旅行出行

出門在外，數款趁手 App 能使爾少踩無數之坑。下按出行全流程推薦。

航班高鐵

- 航旅縱橫　所宜之人：屢乘飛機之商務人士。其要：官方民航數據，航班動態 / 登機口變更 / 延誤預警最權威，可查機型與座位圖。

- 12306　所宜之人：所有國內火車出行者。其要：官方唯一可信購票渠道，支持候補購票（有人退票自動搶），學生票 / 團體票辦理方便。

- 飛常準　所宜之人：於航班準點率敏感、須接送機者。其要：航班延誤預測準確，可分享行程予家人實時查看爾之位置與到達時間。

地圖導航

- 高德地圖　所宜之人：國內城市用戶、自駕 / 叫車族。其要：實時路況準確、避堵能力強，叫車 / 合乘一站搞定，公共交通換乘方案豐富。

- 百度地圖　所宜之人：於地點信息、室內導航有需之用戶。其要：全景地圖 / AR 實景導航，景區 / 商場室內地圖細緻，語音包豐富。

- Google Maps　所宜之人：出國旅行者。其要：全球覆蓋最全、本地評價真實，街景與離線地圖乃異國他鄉之救星。

翻譯

- 谷歌翻譯　所宜之人：出國旅行者、跨語言學習者。其要：「對話」實時雙向翻譯、「相機」即時取詞翻譯乃旅行神器，支持離線語言包下載。

匯率換算

- 極簡匯率　所宜之人：出國旅行、海淘、跨境購物者。其要：界面極致清爽，支持 160+ 貨幣，匯率實時更新，可同時對比多種貨幣，長按數字速換算。

天氣

- 系統天氣 / 中國天氣網　所宜之人：所有人，查預報與官方氣象預警之首選。其要：廠商天氣 App 與「中國天氣」等官方渠道對接氣象臺預警，暴雨、颱風、高溫等預警以橫幅 / 通知推送，權威性高。

- 彩雲天氣　所宜之人：須精準至分鐘級降雨預報者。其要：「未來兩小時是否下雨」準確度業內領先，降雨帶動態地圖直觀可見，雨前主動推送提醒。

- Carrot Weather　所宜之人：求趣味與豐富 widget 之天氣發燒友。其要：毒舌 AI 語音播報天氣、超多自定義 widget 桌面組件、雷達圖專業，付費版功能更全。

- 預警要點：天氣類 App 須允許通知方收得到預警；預警分藍、黃、橙、紅四級，紅色示災害性天氣風險最高，宜按提示調整出行。更多開啟方式見本章第七節「天氣預警 / 自然災害預警 / 地震預警」。

旅行攻略

- 馬蜂窩　所宜之人：好作詳細攻略、自駕遊 / 深度遊者。其要：用戶原創遊記內容豐富，目的地玩法 / 路線 / 避坑指南詳盡，可下載離線攻略。

- 小紅書　所宜之人：欲看真實體驗、尋小眾打卡點 / 美食之年輕旅行者。其要：圖文短視頻形式直觀，「XX 城市 3 天攻略」等內容更新快、時效性強，搜索體驗好。

四、自動化工具啟蒙

自動化乃「以一次配置換長期省事」之槓桿，能將爾每日重複數十次之操作壓縮為零。此節僅作概念啟蒙，具體玩法與詳細教程請見第二十三章：編程思維與自動化啟蒙。

- iOS「快捷指令」：蘋果官方自動化工具，無需編程，可一鍵完成「到家開 WiFi+ 亮度調低 + 播報天氣」等組合動作。

- 安卓自動化：系統級場景聯動（如小米「自動任務」、華為「情景智能」）上手最簡；Tasker 功能強大然學習曲線陡峭，宜極客。

- 核心思路：先識別「每天都要重複做之操作」，再思「能否以一個觸發條件 + 多個動作代之」。詳細示例與進階玩法見第 23 章。

五、瀏覽器與搜索引擎：爾之信息入口

瀏覽器乃爾通往互聯網之窗，搜索引擎乃爾於信息海中導航之羅盤。善用之，能使爾少走許多彎路。

瀏覽器工作之原理：

爾於瀏覽器中輸入網址後，瀏覽器則：(1) 向 DNS 服務器查詢域名對應之 IP 地址；(2) 向該 IP 地址之服務器發送 HTTP / HTTPS 請求；(3) 服務器返回網頁代碼（HTML、CSS、JavaScript）；(4) 瀏覽器解析代碼並渲染為爾所見之頁面。

主流瀏覽器推薦：

- 微軟 Edge：基於 Chromium 內核，速度快、兼容好。內置 Copilot AI 助手，可總結網頁、翻譯、對話。支持跨設備同步，個人強烈推薦作為主力瀏覽器。

- Safari：蘋果設備默認瀏覽器，與系統深度集成，省電高效。隱私保護好，然擴展插件較少。

- Chrome：全球使用率最高，擴展豐富。然佔記憶體大，手機端相對耗電。

搜索引擎之擇：

- Bing：微軟旗下搜索引擎，與 Edge 深度集成。AI 摘要功能可徑直給出答案而非僅鏈接，搜索效率甚高。圖片搜索體驗亦佳。
- Yandex：俄羅斯最大之搜索引擎，圖片搜索與反向圖片搜索能力甚強。找相似圖片、識別物體時甚為好用。
- 百度：國內信息搜索之主力，然廣告較多，須一定辨別之能。
- Google：全球最精準之搜索引擎，然國內須特殊網絡方能使用。

高效信息檢索指南：

- 善用 AI 搜索：Bing 之 AI 摘要、Edge 之 Copilot 可徑直回答問題，助爾從海量結果中提煉關鍵信息，較逐條點擊鏈接高效多矣。
- 識別虛假信息與廣告：搜索結果中標「廣告」或「贊助」之條目乃付費推廣，非自然搜索結果。留意甄別信息來源之權威性，優先擇官方網站、知名媒體之內容。
- 正確下載應用：於搜索引擎中輸入「應用名 + 官網」，如「微信官網」，優先點擊帶官方域名標識之結果。勿從第三方下載站安裝軟件，彼常捆綁流氓軟件。
- 官網鑑別之技：下載 Windows 11 應訪 microsoft.com，下載 Chrome 應訪 google.com。視域名與品牌一致與否，乃最基本之鑑別法。

網站域名知識：

域名者，網站之「門牌號」也，識之能助爾判斷網站性質與來源：

- 常見後綴之義：.com —商業機構，最常見
- .cn —中國國家域名
- .org —非營利組織
- .net —網絡服務機構
- .edu —教育機構
- .gov —政府機構
- .xyz/.tech/.info —新興通用域名，門檻低，須慎

國家 / 地區域名：.jp —日本

.kr —韓國

.uk —英國

.de —德國

.tw —臺灣

.hk —香港

安全提示：.gov 與.edu 後綴之網站通常更可信，以申請門檻高也。.xyz、.top 等廉價域名易為釣魚網站所利用，訪時須多留心眼。

廣告攔截：

- AdGuard　所宜之人：為各類彈窗、貼片廣告、應用內廣告所擾之所有用戶。其要：系統級廣告攔截（不限於瀏覽器），可過濾 App 內橫幅廣告與開屏追蹤，省流量、加速頁面加載，付費版支持全設備。

隱私瀏覽器：

- Brave　所宜之人：重隱私、不欲為廣告追蹤之日常瀏覽者。其要：基於 Chromium 內核，開箱即用屏蔽廣告與追蹤器，自帶 Tor 隱身模式，速度較 Chrome 快、佔記憶體更低。

- DuckDuckGo　所宜之人：望「搜一個詞不被全網追着推廣告」之用戶。其要：自帶同名搜索引擎（不記錄搜索歷史），一鍵清除所有瀏覽痕跡（「燃燒按鈕」），跨平臺體驗統一。

瀏覽器對比一覽：

瀏覽器	其要	適用之人
微軟 Edge	Chromium 內核 + 內置 Copilot AI，可總結網頁 / 翻譯 / 對話	求效率之辦公族、AI 深度用戶（推薦主力）
Safari	與蘋果生態深度集成，省電高效，隱私保護強	iPhone / Mac 用戶，重續航與系統聯動
Chrome	全球使用率最高，擴展插件生態最豐富	須大量擴展插件之進階用戶、跨設備同步黨
Brave	開箱即用屏蔽廣告與追蹤器，速度更快	重隱私、不裝插件亦欲屏蔽廣告之用戶
DuckDuckGo	同名搜索引擎不記錄歷史，一鍵清除痕跡	不欲被個性化追蹤之隱私敏感用戶

試創建一個「睡前放鬆」之自動化流程

1. 創建觸發條件：特定時間（如 22:30）。
2. 添加執行動作：調低音量＞開啟勿擾模式＞播放放鬆歌單＞調低亮度。
3. 保存並測試。

六、實用系統工具：清理、防騷擾與安全防護

效率與健康類 App 之外，實影響日常體驗者往往是「少被打擾、存儲夠用、騙子進不來」此三事。此節先盤點常用實用軟件，再單獨展開手機自帶之騷擾攔截功能——多人購機多年，卻從未開啟此等開關。

系統清理與存儲優化

- 清濁　所宜之人：安卓用戶，欲輕量清理緩存、殘留檔案、空目錄，又不欲被「手機管家」全家桶打擾者。其要：體積小、無廣告之擾，支持深度掃描應用殘留、大檔案定位、存儲佔用分析；較商業「一鍵加速」更克制，宜動手能力強之用戶定期維護。留意：清理前先確認路徑，微信、相簿、文檔目錄勿盲目全選；系統級緩存盡量先用品牌自帶存儲清理，第三方工具僅作補充。

- 系統自帶存儲清理　所宜之人：所有人，尤長輩與怕折騰之用戶。其要：安卓［設定］>［存儲］>［清理加速］；iOS［設定］>［通用］>［iPhone 存儲空間］。系統更諳自家 App 緩存結構，誤刪風險最低，宜為第一選擇。

- 微信 / QQ 內置存儲管理　所宜之人：聊天大戶。其要：微信［我］>［設定］>［通用］>［存儲空間］，可按聊天、檔案類型精細清理；較外部「清微信」類工具更安全。

- 原則：能用系統 / App 自帶者，即不用第三方；能按路徑 / 類型勾選者，即勿「一鍵全清」。清理之目標在釋放空間，非將重要數據一併掃除。

防騷擾與安全類實用軟件

- 國家反詐中心　所宜之人：所有人，尤長輩、財務相關崗位、屢網購 / 接陌生電話者。其要：官方出品，可開啟來電預警、短信風險提示、可疑網址檢測；與公安反詐數據庫聯動，乃「三道防線」中最有官方背書之一環。

- 品牌安全中心 / 手機管家　所宜之人：對應品牌用戶。其要：華為手機管家、小米安全中心、OPPO 手機管家、vivo 手機管家等，集成騷擾攔截、病毒查殺、權限管理、支付保護，通常免費且與系統底層協同更好，不建議再裝第二套商業安全軟件致權限相軋。

- 運營商防騷擾服務　所宜之人：被高頻外呼、AI 語音電話轟炸之用戶。其要：移動發送 KTFSR 至 10086 開通；電信關注「天翼防騷擾」；聯通於「聯通手機管家」中設置。攔截發生於運營商側，手機不響鈴亦能擋住一批騷擾號。

- AdGuard　所宜之人：為各類彈窗、貼片廣告、應用內廣告所擾之所有用戶。其要：系統級廣告攔截（不限於瀏覽器），可過濾 App 內橫幅廣告與開屏追蹤，省流量、加速頁面加載，付費版支持全設備。

- 慎裝「清理加速 / 騷擾攔截」全家桶：部分第三方管家常駐後臺、反覆請求通訊錄與短信權限，攔截能力未必強於系統，反增耗電與隱私之險。先開系統自帶功能，再慮官方反詐 App。

實用軟件選型原則

- 系統優先：攔截、清理、權限管理，先開手機自帶開關。
- 官方補充：國家反詐中心、運營商防騷擾屬官方通道，可疊加。
- 第三方克制：同類功能裝一個即足，免權限相軋與後臺爭資源。
- 少即是多：工具裝得愈多，通知與權限之面愈廣，為「工具本身」所擾之概率愈高。

手機自帶騷擾攔截功能：各品牌怎麼開

下按品牌列出入口。名稱或隨系統版本微異，可於設置中徑搜「騷擾」「攔截」「未知來電」。與第八章「騷擾電話治理」互為補充：此處重開關位置，彼處重號碼標記與誤標申訴。

平臺	入口路徑	推薦開啟項
iPhone / iOS	［設定］>［電話］/［信息］	靜音未知來電；已阻止之聯絡人；信息中「過濾未知發件人」
小米 / Redmi	電話 App >設定>騷擾攔截；或安全中心	智能攔截、陌生號響一聲掛斷、短信關鍵詞過濾
華為 / 榮耀	電話>騷擾攔截；手機管家>騷擾攔截	來電識別、黑名單、短信智能攔截
OPPO / realme / 一加	電話 App >騷擾攔截；或本機>騷擾攔截	來電攔截、詐騙電話識別、短信攔截規則
vivo / iQOO	電話>騷擾電話攔截；i 管家	騷擾攔截、詐騙電話預警、信息過濾
三星	電話>設定>阻止號碼 / 來電顯示與垃圾電話	阻止未知號碼、垃圾電話識別（地區支持度不同）

三層防護疊加建議

日常推薦「系統攔截 + 運營商防騷擾 + 國家反詐中心」三層疊加：系統管標記與黑名單，運營商於網側擋高頻外呼，反詐中心補官方預警。三層不必皆開至最嚴，免將快遞、外賣、醫生電話亦誤殺——可對通訊錄與最近聯絡人放行。

動手練習：今日就把騷擾攔截開啟

1. 開電話 App 設定，尋得「騷擾攔截 / 靜音未知來電」，開啟智能攔截。
2. 為爸媽手機亦檢查一遍：同路徑 + 開通運營商防騷擾（移動可發 KTFSR 至 10086）。
3. 應用商店安裝「國家反詐中心」，註冊並開啟來電預警。
4. 試撥一個已被標記之測試號（或看攔截記錄），確認功能生效。

七、天氣預警、自然災害預警與地震預警

騷擾攔截所擋者乃「電話詐騙」，預警功能所擋者乃「天災與極端天氣」。多人只裝天氣 App 看溫度，卻未開通知、未開地震預警——暴雨紅色預警、颱風、地震來臨前數秒至數十秒，手機本可為爾爭得寶貴之避險時間。

天氣預警：暴雨、颱風、高溫怎麼收

- 官方渠道優先：中國氣象局 / 中央氣象臺 / 各省氣象局發布之預警最權威。可關注「中國天氣」微信公眾號、微博，或下載「中國天氣」等官方類 App。
- 系統天氣 App：華為天氣、小米天氣、OPPO 天氣、vivo 天氣、蘋果「天氣」等多會推送當地氣象臺預警；入天氣 App 設定，確認「氣象預警 / 災害性天氣」通知已開啟。
- 彩雲天氣等分鐘級降雨：宜通勤、接送子女前判斷「這半小時要不要帶傘」；預警能力仍以官方氣象臺為準。
- 預警顏色之義（通用）：藍色：注意防範，一般照常出行、關注更新。
- 黃色：加強防範，減少不必要之戶外活動。
- 橙色：嚴重，盡量留於室內，遠離危險區域。
- 紅色：特別嚴重，聽從停課、停工、避險指引，勿強闖積水、山洪、大風路段。

常見預警類型：暴雨、暴雪、颱風、高溫、寒潮、大風、雷電、冰雹、道路結冰、大霧、乾旱、地質災害風險。

自然災害預警：非止「下雨」

- 發布主體：氣象類由氣象臺負責；地質災害、山洪等常由自然資源、水利、應急管理部門聯合發布；地方政府亦經應急廣播、短信、12379 等渠道發布。
- 如何多渠道接收：手機系統通知 + 天氣 App 預警。
- 微信關注當地「應急管理局」「氣象局」公眾號。
- 12379 國家預警信息發布相關渠道（短信 / 服務，視地區開通情況）。
- 電視、應急廣播、村 / 社區大喇叭——山區、汛期勿只盯手機。

戶外與出行提醒：颱風天勿貼近廣告牌、勿行樹下；暴雨勿入地下車庫、隧道、下穿橋；山區旅遊遇「地質災害氣象風險」預警應立即離開陡坡、溝谷。

與急救手冊聯動：災後失聯、手機沒電、須定位求救時，見第三十二章地震 / 災害場景，以及第七章衛星通信與無網通。

地震預警：與「地震預報」非一回事

地震預警（EEW）者，於地震波（尤破壞力更強之橫波）到達目標區之前，利用電磁波快於地震波、以及臺站更快捕捉縱波之原理，向可能受影響區域提前數秒至數十秒推送警報。其不能預測地震是否發生，然能搶出「就近避險、切斷火源、衝出電梯」之窗口。距震中愈近，預警時間愈短，甚至可能來不及。

- 官方與行業方案：中國大陸常見接入方包括中國地震預警網相關能力，以及成都高新減災研究所等機構推動之預警服務；部分廠商手機會預裝或可聯網接收預警。
- 第三方 App：應用商店搜「地震預警」等官方 / 機構相關應用（安裝前看清開發者，避免山寨「假預警」App 要爾開一堆無關權限）。
- 收到警報後如何做：室內就近躲避（桌下、承重牆角、堅固傢具旁），護住頭頸；遠離窗戶、外牆、高大傢具；勿盲目奔樓梯 / 跳樓；在電梯者即刻於最近樓層離開；室外到開闊地，遠離電線、廣告牌、危房。震後檢查燃氣與電路，留意餘震。
- 勿擾勿蔽：地震預警應允許「突破靜音 / 勿擾」響鈴；睡前置手機於伸手可及處，鈴聲音量勿調至 0。

手機自帶地震預警：各品牌怎麼開

下列入口隨系統版本或微異，可於設置中徑搜「地震」「預警」「安全預警」。iOS 系統本身對國內地震預警支持有限，通常須安裝可靠之第三方預警 App，並保持定位與通知權限。

平臺	常見入口	說明
小米 / Redmi	手機管家 / 安全中心 > 地震預警；或設置搜「地震」	部分版本系統內置預警服務，須聯網並開啟定位
華為 / 榮耀	設定＞安全＞應急預警通知 / 地震預警	名稱或為「應急預警通知」，內含地震等
OPPO / realme / 一加	設定＞安全（SOS 緊急聯絡）＞地震預警	部分機型視地區服務覆蓋而定
vivo / iQOO	天氣 App 或設定＞安全 / 更多＞地震預警	以實際系統版本菜單為準
iPhone / iOS	系統設置中通常無完整內置開關；安裝可靠地震預警 App 並允許通知	App 須後臺運行與定位權限；系統勿擾勿屏蔽其警報
三星等	設定＞安全與緊急情況 / 無線緊急警報	「緊急警報」覆蓋範圍因地區與運營商而異

關於地震預警之常見誤區

- 預警 ≠ 預報：其不能告爾「明天會不會地震」，只在地震已發、波尚未到達時報警。
- 非處處皆能提前多秒：震中附近可能僅數秒甚至來不及，預警主要惠及周邊區域。

- 關通知即同未裝：再好之預警，靜音、殺後臺、不給定位，皆可能收不到。
- 山寨 App：警惕要通訊錄、短信權限之「地震預報」軟件，優先系統內置或正規機構方案。

預警能力如何搭配

日常建議：「系統天氣預警通知 + 官方氣象 / 應急渠道 + 地震預警（系統或正規 App）」。與防騷擾同，不必裝一堆同類軟件；關鍵在通知權限開啟、定位允許、勿擾能突破。為長輩手機做完騷擾攔截後，順手將天氣預警與地震預警亦檢查一遍。

動手練習：10 分鐘檢查預警設置

1. 開系統天氣 App，入設定，開啟「氣象預警 / 災害性天氣」通知。
2. 於系統設置搜「地震」或「預警」，尋得並開啟地震預警 / 應急預警（安卓）；iPhone 則安裝正規地震預警 App 並允許通知。
3. 關注當地氣象局或應急局公眾號 / 服務號，允許其推送。
4. 確認預警聲音不會被勿擾完全靜音；為爸媽手機同樣檢查一遍。

八、跨設備協同：遠控電腦、快傳與帳號安全

手機用得順，多半因其與電腦、平板「結隊」。此節補充數類高頻實用工具：手機遠控電腦、AI 辦公與遠程協作、動態壁紙、局域網快傳、跨設備剪貼板、動態口令 2FA，以及紙質掃描與日程任務之銜接。更系統之多設備聯動見第十七章。

手機遠控電腦：向日葵 / ToDesk

- 向日葵遠程控制　所宜之人：須於外出時操作家中 / 公司 Windows 或 Mac 之用戶；亦常用於助長輩遠程修手機（見第二十七章）。其要：國產生態成熟，手機 App 控制電腦桌面、傳檔案、開關機（部分須硬件）；免費版一般足個人輕度使用。
- ToDesk　所宜之人：求連接穩定、低延遲遠程辦公 / 運維之用戶。其要：設備碼 + 密碼連接，手機控電腦、電腦控手機（視版本）、檔案傳輸與多顯示器切換較順手，國內節點體驗通常較好。
- 基本用法：電腦端安裝客戶端並登錄 / 保持在線手機端裝 App 以設備識別碼 + 連接密碼（或同一帳號）建立會話手機觸控板 / 手勢操作桌面。
- 安全紅線（務必記住）：陌生人自稱「客服 / 公檢法 / 運營商」令爾裝向日葵、ToDesk 並遠程爾之手機或電腦，一概拒之——此典型詐騙手法也（詳見第二十七章）。
- 自用之時：開啟二次驗證、修改默認密碼、不用時退出帳號或關閉無人值守；公司電腦遵守單位安全規定。
- 公共場合遠程辦公，留意周圍是否有人偷窺屏幕與口令。

AI 辦公軟件：Trae / WorkBuddy / Dumate

- Trae　所宜之人：編程與文檔協作用戶，中文環境友好。其要：字節跳動推出之 AI IDE / 編程助手，SOLO 模式可一句話生成項目；日常寫代碼、改文檔效率高。若電腦端運行，可於移動端配合遠控或官方配套能力查看 / 跟進任務（以產品最新形態為準）。詳見第四十一章 AI 編程工具全景。

- WorkBuddy　所宜之人：望 AI 助整理會議紀要、郵件、匯報、辦公文檔之職場用戶。其要：面向辦公場景之 AI 助手，宜「起草—潤色—總結」此類高頻文案；部分工作流可於手機端發起，重活付電腦端完成。

- Dumate　所宜之人：須多人協作、文檔與知識管理一體化之團隊 / 個人。其要：AI 協作向辦公工具，重資料匯聚與協同編輯；手機端可查看與輕量編輯，複雜排版建議於電腦大屏處理。

- 「手機端遠程控制電腦」何解：AI 辦公能力多運行於電腦 / 網頁端；於手機經向日葵 / ToDesk 連回電腦，或用產品自身之手機 App / 網頁，則通勤時可「遙控」電腦上之 AI 任務。原則：重計算、長文檔付電腦；看進度、改數句付手機。

- 隱私提醒：上傳合同、身份證、客戶數據前看清產品隱私條款與企業合規要求；公司資料優先用單位指定工具。

人工桌面：米哈遊動態壁紙（手機 +PC）

- 人工桌面　所宜之人：喜米哈遊角色、欲免費動態壁紙之用戶。其要：米哈遊推出之免費動態壁紙應用，支持手機端與 PC 端；角色 Live2D / 動態演出可作桌面，部分機型 / 系統可設為鎖屏或桌面壁紙。

- 省電與性能：動態壁紙較靜態圖更耗電、佔 GPU；低電量或舊機型可只在充電時使用，或降低幀率（若 App 提供選項）。

- 安裝來源：優先官網 / 正規應用商店，避免第三方「破解版」捆綁木馬；PC 版留意安裝路徑與開機自啟是否需要。

局域網文件快傳：LocalSend / 飛鴿傳書 / 隔空投送

- LocalSend　所宜之人：安卓 / Windows / iOS / Mac / Linux 混用，欲跨平臺、不經公有雲快傳者。其要：開源、局域網直連，同一 Wi-Fi 下互傳照片視頻文檔，無需數據線、不經廠商服務器，速度取決於路由器。

- 飛鴿傳書　所宜之人：辦公室 Windows 電腦之間、舊局域網辦公場景。其要：老牌局域網傳輸 / 聊天工具，宜內網傳大檔；留意從正規渠道下載，企業內網遵守 IT 策略。

- 隔空投送（AirDrop）　所宜之人：iPhone、iPad、Mac 用戶。其要：蘋果生態一鍵分享，藍牙發現 +Wi-Fi 直連傳輸，免流量；可設「僅聯絡人」以降陌生人騷擾。

- 其他補充：小米互傳、華為分享、vivo 互傳等品牌方案於同品牌設備間甚快；微信「文件傳輸助手」方便然受限於文件大小與登錄設備，大文件更推薦 LocalSend ／隔空投送。

- 使用提示：雙方連同一 Wi-Fi；公共 Wi-Fi 傳敏感文件有風險，優先私人熱點；傳輸完成後於接收方確認文件是否完整。

跨設備剪貼板同步：KDE Connect ／手機電腦互傳

- 蘋果「通用剪貼板」　所宜之人：iPhone+Mac 用戶。其要：登錄同一 Apple ID、開接力後，手機複製 Mac 徑直粘貼（反之亦然），文字與部分文件皆可。

- KDE Connect　所宜之人：安卓 +Windows ／ Linux ／ Mac，欲開源剪貼板同步、通知鏡像、遠程輸入之用戶。其要：局域網配對後同步剪貼板、推送手機通知至電腦、以手機為幻燈片翻頁器 / 觸控板；數據不經第三方帳號。

- 品牌「手機電腦互傳」：華為多屏協同 / 超級終端、小米妙享、vivo 互傳、OPPO 互連、榮耀互聯等，常自帶跨屏剪貼板與文件拖曳，同品牌體驗最佳。

- Windows+ 安卓官方通道：「手機連接 / Link to Windows」（Your Phone 演進）可於電腦看手機通知、照片，並於支持之場景複製粘貼；三星等機型集成更深。

- 安全提醒：剪貼板可能殘留密碼、驗證碼；共用電腦上慎用全局同步，用畢於密碼框避免二次粘貼洩露；部分輸入法亦讀剪貼板，留意權限。

動態口令 2FA：Aegis ／ Microsoft Authenticator

- 何以用認證器而非短信：短信驗證碼可能遭遇 SIM 補卡劫持；TOTP 動態口令於本地按時間算法生成，無需依賴手機號信號，更抗釣魚與攔截（仍須防木馬與截屏）。

- Aegis Authenticator　所宜之人：安卓用戶，好開源、本地可控者。其要：開源免費，支持加密備份與導入導出，界面乾淨，可以指紋 / 人臉解鎖令牌列表。

- Microsoft Authenticator　所宜之人：用微軟帳號、Office 365 ／企業帳號之用戶；跨 iOS ／安卓。其要：推送批准登錄、密碼監控與帳號備份（視產品版本），與 Windows ／企業身份體系結合緊密。

- Google Authenticator ／ 2FAS 等：生態內亦常見；關鍵在有加密備份，換機不失令牌。

- 換機必為：舊機遷移或用服務中「備用碼 / 恢復密鑰」；備用碼打印於紙存兩處，勿只存同一部手機。

- 與密碼管理器：部分密碼庫亦可存 TOTP，方便然風險更集中；高價值帳號建議「密碼管理器 + 獨立認證器」分工。詳見第十四章。

推薦組合：一日之中怎麼串起來

實用工作流之例

- 晨：滴答清單看今日日程天氣 / 預警掃一眼電腦開 Trae / 文檔，手機置一旁。
- 晝：紙質材料用掃描類 App 轉 PDF，經 LocalSend / 隔空投送發至電腦；長文於電腦用 AI 辦公工具起草。
- 外出：用 ToDesk / 向日葵連回電腦繼續任務；重要帳號登錄走 Aegis / Microsoft Authenticator 動態口令。
- 夜：滴答清單勾選完成項；須時人工桌面換壁紙放鬆一下。

遠控與帳號安全口訣

- 他人令爾裝遠控軟件掛斷電話，自尋官方客服。
- 驗證碼、動態口令勿告任何人，包括「快遞」「客服」。
- 公司電腦遠控與安裝 AI 軟件前，先問 IT 能不能用。
- 公共網絡傳文件、遠程登錄時格外警惕。

動手練習：把「跨設備」鏈路跑通一次

1. 手機與電腦連同一 Wi-Fi，用 LocalSend（或隔空投送）互傳一張照片。
2. 於滴答清單建一條「今晚 8 點：整理掃描件」任務，確認手機與電腦都能看到。
3. 為郵箱或微信開啟 2FA（若支持），用 Aegis 或 Microsoft Authenticator 綁定，並保存備用碼。
4. 於自己電腦安裝向日葵或 ToDesk，用自己手機成功連一次，再退出或加固密碼，熟悉流程但不留安全後門。

第十四章：密碼管理器專題

於數字之世，密碼乃君家門之鑰匙。然鑰匙太多、太繁，奈何？密碼管理器乃君之“萬能鑰匙管家”。

一、何以君需密碼管理器？

今人平均有幾十甚至上百個網絡賬號。若每賬號皆用同密碼，一旦一網站被黑，所有賬號皆危。若每賬號皆用不同之繁密碼，根本記不住。

密碼管理器解此困：君只需記一主密碼，其助君記所有其他密碼。

二、密碼管理器之工作原理

1. **加密存儲**：所有密碼皆經高強度加密存儲，惟君能解密。

2. **主密碼**：君設一主密碼，以解鎖密碼庫。主密碼本身不存於服務器上。

3. **自動填充**：於網站或 App 登錄界面，密碼管理器自動填充賬號密碼。

4. **密碼生成**：可生隨機、繁復之密碼，提安全性。

三、加密理簡介

瞭解密碼管理器之加密原理，能幫汝判斷一個服務商是否值得信任。下面用通俗之語言解釋幾個核心概念。

AES-256 加密

AES（高級加密標準）乃美國國家標準與技術研究院（NIST）認可之對稱加密算法，爲銀行、軍方、政府廣泛使用。“256”指密鑰長度爲 256 位——理論上須超過宇宙年齡之時間才能暴力破解。主流密碼管理器（1Password、Bitwarden、KeePass 等）皆用 AES-256 加密汝之密碼庫。

零知架構（Zero-Knowledge Architecture）

“零知識”非說服務商何皆不存，而乃曰**加密和解密皆於汝之設備本地完成，服務器唯保存已加密之密文**。具體程：

1. 汝於客戶端輸入主密碼。
2. 客戶端用主密碼派生出加密密鑰（這一步於汝手機/電腦上完成）。
3. 只有加密後之密文方上傳至服務器。
4. 下次登錄時，服務器以密文下發，客戶端再次用主密碼解密。

鍵點：**主密碼本身自不離開汝之設備**，服務器上亦沒有。故縱服務商爲黑，黑客拿至之亦只是一堆無法解密之密文——除非彼等能猜出汝之主密碼。

何故”零知識”意味着服務商亦觀不至汝之密碼？

蓋因解密密鑰由汝之主密碼派生，而主密碼唯存在於汝腦中、唯於汝本地設備上使用。服務商之服務器上既沒有主密碼，亦沒有解密密鑰，只有密文。這就是何故 Bitwarden、1Password 等正規服務商皆明確聲明“我們亦無法看到汝之密碼”——非彼等不欲觀，乃技術上作不至。

密鑰派生函數：PBKDF2 與 Argon2

主密碼不可直接用作加密密鑰——它須經過“密鑰派生函數”處理。此函數之作用乃**故意使計算變慢**，使黑客暴力破解之成本高至不可行。

- **PBKDF2**：老牌標準，爲 iCloud 鑰匙串、Bitwarden、LastPass 等廣泛使用。通過反覆迭代（通常 10 萬次以上）哈希來拖慢破解。
- **Argon2**：2015 年密碼哈希競賽冠軍，比 PBKDF2 更現代。除矣”算力消耗”尚引入矣”內存消耗”——使 GPU/ASIC 等專用硬件亦難以加速破解。Bitwarden、KeePassXC 等已支持。

迭代次數越高越安全，但解鎖速度亦將變慢。正規服務商都將公開其使用之派生算法與迭代次數，便於安全社區審計。

案例之鑑：LastPass 2022 年數據之洩

2022 年 8 月，老牌密碼管理器 LastPass 公告其開發環境爲入侵；同年 11 月進一步披露，黑客拿走矣用戶之密碼庫備份（加密後之密文）。事件之關鍵教訓：

- **零知識架構守住矣底線**：由於密碼庫本身乃加密之，黑客拿至密文但解不開——前提乃用戶之主密碼足夠強。

- **弱主密用戶最危**：黑客可離線暴力破解弱主密碼，因此使用弱主密碼或重複主密碼之用戶面臨實質風險。
- **元數據亦泄露**：雖密碼內容加密，但站點 URL、備註等元數據或未加密，將暴露用戶使用矣何服務。

自 LastPass 事件學至之

第一，主密碼須長且獨特（建議 16 位以上，包含大小寫、數字、符號）；第二，優先選擇支持 Argon2id、迭代次數高之服務商；第三，對高價值賬號（銀行、郵箱）務必開啓獨立之二次驗證，勿唯依賴密碼管理器。

四、主流密管器薦

系統自帶：

- **iCloud 鑰匙串**：蘋果生態用戶首選。與 Safari、App 無縫集成，免費，安全。弊乃跨平臺使用不便。
- **Google 密碼管理器**：安卓用戶方便。與 Chrome、安卓系統集成，免費。
- **華爲密碼保險箱**：鴻蒙系統內置，宜華爲用戶。

第三方專業工具：

- **Bitwarden**：開源免費，功能強大，支持所有平臺。強烈薦予注重隱私之用戶。
- **1Password**：界面美觀，功能豐富，宜家庭與團隊。付費。
- **LastPass**：老牌密碼管家，免費之版其能有限。
- **KeePass**：完全本地存儲，開源免費。宜極客用戶。

五、密強度真實評估

密碼管理器普遍將顯示“密碼強度”指示條（紅/黃/綠），但此評估往往只觀長度與字符種類，並不全面。真正判斷密碼是否安全，須考慮“熵”與“是否已泄露”。

熵（Entropy）：密碼強度之科學度量

“熵”衡量密碼之不確定性，單位乃比特。熵越高，暴力破解所需嘗試次數越多。

- **”123456”**：熵極低，瞬間破解。
- **”P@ssw0rd”**：看似複雜，但出今凡字典攻擊列表裏，熵依然低。

- **”correct horse battery staple”**：四個隨機單詞，熵約 44 較特，破解難度遠超”P@ssw0rd”。
- **16 位隨機字符（含大小寫 + 數 + 符號）**：熵約 100 較特，目前算力無法暴力破解。

結論：**“長度 + 隨機性”較“複雜符號”更要**。這亦乃密碼管理器生成之隨機長密碼真正安全之原因。

HaveIBeenPwned：查汝之密碼是否已泄露

安全研究員 Troy Hunt 維護之 haveibeenpwned.com 收錄矣歷次大型數據泄露中流出之賬號與密碼。汝可：

1. 輸入郵箱，查汝之賬號是否於某次泄露中爲曝光。
2. 用 Pwned Passwords 功能查詢某個密碼是否於已知泄露庫中（密碼管理器與瀏覽器皆已內置此查詢）。

若一個密碼出現在泄露庫中，**無論它看起來多複雜皆須立即更換**——蓋因字典攻擊將優先嚐試此已知密碼。

k-Anonymity 查詢理

擔心於 HaveIBeenPwned 輸入密碼將再次泄露？不必擔心。現代查詢採用“k-匿名”協議：瀏覽器唯以密碼哈希之前 5 位發給服務器，服務器返回所有以這 5 位開頭之哈希列表，本地再比對。亦就是曰，服務器永遠觀不至汝之完整密碼哈希。1Password、Bitwarden 之“已泄露密碼檢測”皆乃這樣工作之。

密碼管理器內置之監控功能

- **1Password Watchtower**：實時監控弱密碼、重複密碼、已泄露密碼、過期密碼、不使用 2FA 之網站，並以紅/橙/黃分級展示。
- **Bitwarden Vault Health**：免費版即可觀弱密、重複密；付費版可檢測已泄露密（基於 HaveIBeenPwned）。
- **iCloud 鑰匙串” 安全宜”**：iOS 設置密碼安全建議，將提示汝何密碼出現在已知泄露中、何重複使用。
- **Google 密管器” 密檢工具”**：Chrome 內置，可一鍵檢已泄露、弱、重複密。

現在遂查：汝之密碼是否安全

1. 打開 haveibeenpwned.com，輸入汝之常用郵箱，觀看出現在何泄露事件中。
2. iPhone 用戶：設密安全宜，逐條處紅色警。
3. Chrome 用戶：設密檢密，依示改。

4. 1Password/Bitwarden 用戶：打開 Watchtower/Health Reports，優先處理” 已泄露” 與” 重複” 項。

六、Passkey：無密登錄之未來

2024-2026 年，密碼技術方經歷一場革命——**Passkey（通行密鑰）**有望徹底淘汰傳統密碼。Apple、Google、Microsoft 三大平臺聯合推動，FIDO 聯盟與萬維網聯盟（W3C）背書，這乃數十年來登錄方式最大之一次變革。

何乃 Passkey？

Passkey 基於 FIDO2 / WebAuthn 標準，用**設備上之密鑰對**替代密碼。原理可簡化爲：

- 汝之手機生成一對密鑰：**公鑰**（註冊賜網站）+ **私鑰**（永遠存在汝設備之安全芯片中）。
- 登錄時，網站發一個” 挑戰”，汝之手機用私鑰簽名，再由網站用公鑰驗證。
- 簽名前須汝通過生物識別（Face ID / 指紋）或設備 PIN 確認身份。
- 整個程中，**沒有任何密碼於網絡中傳輸**。

Passkey vs 傳統密

維度	傳統密	Passkey
抗釣魚	差，釣魚網站可竊取密	天然抗釣魚，私鑰唯對真實域名簽名
服器泄露響	密碼哈希或爲破解	服務器唯存公鑰，泄露無影響
便性	需記憶/輸入/管	Face ID/指紋一秒登錄
重放攻擊	或爲中間人截獲重放	每次簽名一性，無法重放

各平臺支情況

- **Apple iCloud 鑰匙串 Passkey**：iOS 16+ / macOS Ventura+ 原生支持，Passkey 可於所有登錄同一 Apple ID 之設備間同步，新設備登錄網站時無需重新創建。
- **Google 密管器 Passkey**：安卓 + Chrome 原生支，Passkey 存 Google 賬戶中，跨安卓設同步；2023 年起 Google 個人賬號已支用 Passkey 登錄。
- **Microsoft**：Windows 11 / Edge 已支持 Passkey；微軟賬號、Outlook、Office 等服務陸續開放 Passkey 登錄，並可於 Microsoft Authenticator 中管理。
- **1Password / Bitwarden**：已支持 Passkey 之創建、存儲、跨平臺同步，使汝不受限於單一生態。

跨設備同步乃關鍵突破

早期 Passkey 之最大痛點乃“綁定單臺設備”——換手機遂要重新創建。2023 年起 Apple、Google、Microsoft 實現矣 Passkey 之雲端同步：iCloud 鑰匙串同步跨 Apple 設備，Google 密碼管理器同步跨安卓設備。1Password 等第三方則打通矣跨生態同步。這意味着 Passkey 已具備替代密碼之實用性。

何以於手機上創建與使用 Passkey

iOS（iPhone）徑：

1. 於支持 Passkey 之網站（如 Google、微軟、亞馬遜、eBay、PayPal）賬號安全頁選擇” 添加通行密鑰 / Add passkey”。
2. 系統彈出 Face ID / Touch ID 驗證窗口，通過後 Passkey 自動生成並保存至 iCloud 鑰匙串。
3. 下次登錄該網站時，輸入用戶名後擇” 用 iPhone 登錄”，Face ID 一刷即可。
4. 管入：設密找至對應網站，可見 Passkey 條目。

安卓徑：

1. 於支持 Passkey 之網站賬號安全頁選擇” 添加通行密鑰”。
2. Google 密碼管理器彈窗選擇汝之 Google 賬號屏幕鎖驗證（指紋/圖案/PIN）。
3. Passkey 自動保存至 Google 密碼管理器，跨同賬號安卓設備同步。
4. 下次登錄時，擇” 用 Passkey”，屏幕鎖一解即可。
5. 管理入口：安卓設置密碼、密鑰與自動填充密碼管理器 google.com。

Passkey 跨設登錄

若汝於 Windows 電腦上登錄一個只有 iPhone 上有 Passkey 之網站，可掃碼授權：

1. 網站登錄頁擇” 用 Passkey”。
2. 擇” 用其他設”。
3. 電腦顯二維碼，用 iPhone 掃碼。
4. iPhone 彈出 Face ID 驗證，確認後網站於電腦上完成登錄。

這一過程通過藍牙 + 互聯網雙重加密通信，Passkey 私鑰未能離開汝之 iPhone。

現在遂試：體驗無密碼登錄

1. 登錄 Google 賬號安全通行密鑰添通行密鑰，體驗 Face ID/指紋登錄。
2. 登錄微軟賬號安全高級安全項添通行密鑰。
3. 於淘寶、京東、eBay、PayPal 等已支持 Passkey 之服務上嚐鮮。
4. 保留傳統密碼作爲後備，逐步遷移高價值賬號至 Passkey。

過渡期宜

Passkey 並未悉取代密——許多網站仍處於“Passkey + 密”並存階段。宜：高價值賬號優先開啓 Passkey，但**勿立即刪除傳統密碼**，留作備用；同時繼續使用密碼管理器，蓋因非所有網站皆支持 Passkey。以 Passkey 觀作“密碼管理器之強力補充”而非“立刻替換”。

七、硬件安全密鑰

若汝對安全之要求高於普通用戶——比如持有大量加密貨幣、乃高管/記者/活動人士、或保護企業核心賬號——硬件安全密鑰乃比短信驗證碼更可靠之選擇。

何乃硬件安全密鑰？

硬件密鑰乃一個物理設備（形似 U 盤），內部安全芯片存儲汝之私鑰，須物理接觸或 NFC 才能完成驗證。代表產品：

- **YubiKey**（Yubico 公司）：最主流，支 FIDO2/WebAuthn、U2F、OTP、PIV、OpenPGP，USB-C / USB-A / NFC 等多種接口。5 系列 50-70 美元。
- **Google Titan**：Google 自家出品，USB-C + NFC / 藍牙，約 30-40 美元。
- **OnlyKey**：開源替代品，支多種協議。
- **國產替代**：如飛天誠信 ePasskey，價更親民。

何故硬件密鑰比短信驗證碼安全？

- **抗釣魚**：與 Passkey 一樣，硬件密鑰唯對真實域名響應，釣魚網站無法騙取驗證碼。
- **抗 SIM 劫持**：短信驗證碼可爲 SIM-swap 攻擊截獲，硬件密鑰無需手機號。
- **抗中間人**：FIDO2 協議每次簽名皆包含挑戰值，無法重放。
- **物理隔離**：私鑰永遠於硬件芯片內，惡意軟件縱攻陷汝之電腦亦讀不至。

適場景

- 加密貨幣交易所賬號、財包復。

- 企業 SSO、遠程辦公 VPN、SSH 登錄服器。
- 郵箱、GitHub、社交媒體等” 主賬號”——一旦爲攻破可重置所有其他賬號。
- 記者、律師、高管等高敏感人羣。

何以配置硬件密鑰作爲二次驗證

以 Google 賬號 + YubiKey 爲例：

1. 購買 YubiKey（建議一次買兩以，一以日常用、一以備用，存放於不同地點）。
2. 登錄 myaccount.google.com 安全兩步驗。
3. 選擇” 添加安全密鑰” 以 YubiKey 插入 USB-C 口或貼近手機 NFC 感應區。
4. 依示觸摸 YubiKey 金屬觸點畢註冊。
5. 建議注冊第二以備用密鑰。
6. 於” 兩步驗證” 設置裏，以硬件密鑰設爲首選驗證方式，短信驗證碼作爲備用。

主流支持之網站：Google、GitHub、Microsoft、Apple ID、Facebook、Twitter、Dropbox、GitLab、AWS、Coinbase、Binance 等。

雙密鑰策

硬件密鑰最大之風險乃“丟失”。建議：(1) 同時註冊兩以不同之密鑰；(2) 一以隨身攜帶（鑰匙扣），一以存放於家中保險櫃或銀行保管箱；(3) 部分網站允許以“備用恢復碼”打印出來離線保存。這樣縱主密鑰丟失，亦未能爲永久鎖於賬號外。

購機渠要慎

務必自官方店鋪或授權代理購買，勿買二手或來源不明之密鑰——理論上預置過之硬件或已爲植入後門。YubiKey 之固件於出廠後無法更新，但 Yubico 將持續推送安全公告，關注其官網即可。

八、家/團隊密共享

家庭場景下，Netflix、家庭 Wi-Fi、路由器後臺、智能門鎖密碼須全家共用；團隊場景下，服務器、數據庫、社交媒體賬號須多人協作。**用微信發密碼、寫於共享文檔裏皆乃高危作法**——一旦聊天記錄或文檔權限泄露，密碼遂泄露矣。

1Password 之“共享保險庫”（Vault）

1Password 乃家庭與團隊共享之標杆產品。它的核心機制乃**多保險庫**：

- **個人保險庫**：只有汝能觀，如銀行、郵箱等私人賬號。

- **共享保險庫**：家人或團隊成員皆能觀，如 Netflix、家庭 Wi-Fi、共享信用卡。
- **權限分級**：團隊版可設定某些成員”唯查看”或”唯查看但不編輯”。

家人或團隊成員無需共享主密碼，各自用自己之賬號登錄，但可看到爲授權之共享保險庫內容。密碼修改後所有成員實時同步。

Bitwarden 之“組織”功能

Bitwarden 之免費版支持個人密碼共享（通過“發送 Send”功能限時分享單條密碼）；付費版可創建“組織”，以若干條目共享賜家庭成員或團隊同事，權限可細分。Bitwarden 尚提供自託管選項，適合對數據主權要求高之企業。

家庭場景：共享 Netflix、Wi-Fi 等密碼之安全方式

- **Netflix / 視頻網站**：用 1Password 家庭版之”共享保險庫”存登錄密碼，家庭成員各自登錄即可，無需於羣裏發。
- **家 Wi-Fi 密**：iOS 設備間可通過 AirDrop 直接共享 Wi-Fi 密碼賜來訪之客人，無需念出來；安卓 12+ 也有”附近分享 Wi-Fi”功能。
- **智能門鎖、智能家電**：儘量用廠商 App 之”家庭成員”功能爲每人單獨創建賬號，而非共享一個管理員賬號。
- **家用路由器後台**：管理員密碼存至 1Password 共享保險庫，訪客 Wi-Fi 單獨開一個 SSID。

1Password 家庭版之性價比

1Password Families 約 4.99 美元/月，支持最多 5 位家庭成員，每人皆有獨立之私人保險庫 + 4 個共享保險庫。算下來人均不至 1 美元/月，乃目前家庭共享體驗最好之方案。Bitwarden 之家庭版 10 美元/年支持 6 人，性價比更高，但界面體驗略遜。

團隊場景：服器/數據庫密共享

- **1Password Teams / Business**：可按部門、項目組創建多保險庫，細粒度權限控制，凡訪問行爲可審計。
- **Bitwarden Teams / Enterprise**：開源策，可自託，企業版支 SSO 集成、目錄同步。
- **職責分離則**：管員密不應祇由一人掌握——設 2-3 名”應急聯繫人”，保鍵人員離職/休假時業務不中斷。
- **離職程**：員工離職時立即撤銷其密碼管理器訪問權限，必要時旋轉所有他接觸過之高敏感密碼。

緊急訪問（Emergency Access）：萬一汝出事

想象一個場景：汝獨自管理家庭所有重要賬號，但意外住院或不幸離世，家人無法登錄銀行、郵箱、保險賬號。**“緊急訪問”能**正是爲此設計——授權一位信任之家人/朋友，於汝無法響應時獲取汝之密碼庫。

Bitwarden 緊急訪問設：

1. 登錄 Bitwarden Web Vault 設緊急訪問。
2. 添加信任聯繫人之郵箱，邀請對方接受。
3. 設定” 雲端雙因素認證”（建議 7-14 天）：自對方發起請求至汝確認之時間窗口。
4. 若汝於等待期內不響應（拒絕或不操作），到期後對方自動獲得訪問權。
5. 汝隨時可手動拒絕請求或撤銷緊急訪問權限。

1Password 緊急訪問：

1. 1Password 不直接叫” 緊急訪問”，但通過” 家庭/團隊組織” 功能可實現——以家人加入家庭版後，將部分條目放入” 共享保險庫”。
2. 對個人版用戶，建議以主密碼告知一位高度信任之家人，或寫入密封信封存放於保險櫃。
3. 1Password 尚提供” 恢復碼”，開啓雙重認證時務必打印保存。

現在遂設：爲家人留一以” 備用鑰匙”

1. 選定一位信任之家人作爲” 緊急聯繫人”。
2. 于密碼管理器中設置緊急訪問（Bitwarden）或加入家庭共享保險庫（1Password）。
3. 以主密碼 + 恢復碼寫於紙上，封進信封，存放於家中保險櫃或告知可信家人。
4. 每半年回顧一，確認緊急聯繫人是否仍然適用。

九、密管器用最佳實踐

- **設強主密碼：**主密碼乃最後防線，務必設一長且繁之密碼，並牢記。
- **啓雙重認證：**爲密碼管理器賬號啓雙重認證，增一層保護。
- **定期查弱密碼：**大多數密碼管理器有” 安全審計” 之能，助君覓出弱密碼與重複密碼。
- **勿忘主密碼：**主密碼無法找回，忘即意味着失所有密碼。議寫於紙上，置於安全之處。
- **緊急訪問：**設緊急聯繫人，萬一君無法訪問，信任之人可取君之密碼。

重要提醒

重要提醒

今即試之：始用密碼管理器

1. 擇一密碼管理器（薦 Bitwarden 或系統自帶）。
2. 注冊賬號，設一強主密碼。
3. 注冊賬號，設一強主密碼。
4. 導入現有密碼（若有）或始添新密碼。
5. 查安全審計報告，覓需改之密碼。

三、進階篇（第 15-20 章）

第十五章：電池之理與續航之優化

電池乃君手機之“能量心臟”。此章帶君解電池之小脾氣，學科學之充電與續航之術，徹底告別電量焦慮。

一、電池乃何物？循環與健康度

手機電池（鋰離子或鋰聚合物）如能量海綿：充電即吸水（存電），放電即擠水（用電）。有數特點值得記之：

- **隨用隨充**：與舊電池不同，其無記憶效應，無需非用完再充。
- **怕熱**：高溫乃電池頭號殺手。
- **會老化**：用久，最大容量即降，此乃正常之象。

核心指標：循環次數與健康度

- 一次循環非充一次電。比如今日用 60% 充滿，明日用 40% 充滿，總耗 100% 電量方算一次循環。
- 電池健康度即當前最大容量佔全新時之百分比。低於 80% 時，君將明顯覺續航變短。
- **何以看？**iPhone：設置 > 電池 > 電池健康與充電；安卓：一般往設置中搜電池健康。

二、充電玄學？快充、無線充與黃金之則

- **快充傷電池？**用原裝或認證之快充配件，傷甚微。手機與充電頭有溫控，過熱將自動降功率。
- **無線充電**：便然效率較低，發熱更明顯。宜整夜充電。
- **黃金養生之則**：盡量使電量保持于 20%-80% 之間，勿老令其餓（0%）或撐（100%）。

iPhone”80% 充電限制” 功能——以黃金法則變成系統設置

iPhone 15/16 系列（iOS 17 以上）已以“20%-80% 法則”作成矣系統功能。開啓後，手機充至 80% 將自動停止充電，從而大幅延緩電池老化。設置路徑：

設 > 電池 > 電池健康與充電 > 充電優 > 80% 限
三個選項之含義：(1) **不限**——充至 100%；(2) **優電池充電**——學習汝之作息，唯於凌晨終一段延遲到 100%；(3)**80% 限**——直接停於 80%，最護電池。長期於家/辦公室有充電條件之人，強烈建議選 80% 限制；屢出差、須滿電出門之人，可選優化電池充電。

GaN 充電器——出差黨之好搭檔

汝或注意至現在之快充頭越作越小，背後功臣乃 **GaN（氮化鎵）**這種新型半導體材。相較傳統硅基充電器，GaN 充電器有幾個明顯優勢：

- **體積小**：同等功率下，GaN 充電器可作至傳統硅基之 1/2 甚至 1/3 大小，65W 功率之 GaN 頭比蘋果老之 5W 頭尚小。
- **效高、發熱低**：氮化鎵之開關損耗遠低於硅，能量轉換效率可達 95% 以上，充電時不容易燙手。
- **適出差**：一個 65W 或 100W 之 GaN 多口充電器，能同時賜手機、平板、筆記本充電，比帶三塊”磚頭”輕便得多。
- **選購宜**：認準支持 PD 協議之 GaN 充電器（65W 起步可充筆電），知名品牌如倍思、安克、努比亞、小米等均可。注意覈對數據線是否匹配（高功率需用 E-Marker 芯片之 Type-C 線）。

三、快充協議全解：何以君之充電器充不快？

君或遇此情：明明手機支持快充，換充電器卻變得甚慢。此乃因快充非簡單“功率大即行”，乃需充電頭、數據線、手機三方“說同一種語言”——此即快充協議。

快充之基本原理

充電功率 = 電壓 × 電流（$P = U \times I$）。欲提功率，或提電壓，或提電流，或二者皆提。不同協議擇不同之技術路線。

快充之工作流程：

1. **握手階段**：手機與充電頭通過數據線通信，交換各自支持之協議與功率信息。
2. **協商階段**：雙方確定一共同支持之最高功率檔位。
3. **充電階段**：充電頭按協商好之電壓電流輸出，手機端進行電源管理。
4. **調整階段**：隨電量增加，手機將逐步降充電功率，護電池。

主流快充協議詳解

USB PD（Power Delivery）——通用協議之王

- **最高速率**：USB-IF 組織制定之官方標準，覆蓋範圍最廣。PD 3.0 支持最高 100W（20V/5A），PD 3.1 擴展至 240W（48V/5A）。
- **電壓檔位**：5V、9V、15V、20V，PD 3.1 尚支持 28V、36V、48V。
- **技術路線**：iPhone 全系、谷歌 Pixel、部分安卓旗艦、筆記本電腦、Switch 遊戲機皆支持 PD 協議。
- **優勢**：通用性強，一充電頭可予多種設備充電；支持 PPS（可編程電源），可精細調節電壓電流。
- **劣勢**：需 Type-C 接口與專用芯片，成本較高。

高通 QC（Quick Charge）——安卓陣營之元老

- **QC 3.0**：支持 3.6V-20V 以 0.2V 爲步進之可調電壓，最高 18W。
- **QC 4.0/4+**：兼容 USB PD 協議，支持最高 27W，加入雙路充電技術。
- **最廣泛**：支持最高 100W+，0-50% 僅需 5 分鐘，大幅改善發熱。
- **技術路線**：用高通驍龍處理器之手機大多支持，然各品牌通常會優先用己之私有協議。

OPPO VOOC/一加 WARP/真我 Dart——低壓大電流之代表

- **最高功率**：低壓大電流（5V/4A 起步），發熱更小，效率更高。
- **VOOC 1.0**：20W（5V/4A），2014 年發佈，開創快充時代。
- **SuperVOOC**：50W-65W，採用雙電芯串聯設計。
- **SuperVOOC 2.0**：65W，30 分鐘充滿 4000mAh 電池。
- **150W/240W 超級閃充**：目前最快之量產快充技術。
- **最高速率**：需專用充電頭與數據線（有額外觸點），不兼容其他協議。

華爲 SuperCharge——高壓快充之演進

- **FCP（Fast Charge Protocol）**：9V/2A，18W，兼容 QC 2.0。
- **SCP（Super Charge Protocol）**：支持更高功率，自 22.5W 至 66W 再至 100W。
- **技術特點**：採用電荷泵技術，將高壓轉爲低壓大電流輸入電池，效率高達 98%。
- **技術路線**：華爲/榮耀手機專用，部分充電器支持 PD 協議。

小米澎湃快充——自研協議之崛起

- **澎湃 P1 芯片**：小米自研之快充管理芯片，支持 120W 有線快充。

- **澎湃 G1 芯片**：電池管理芯片，與 P1 配合實現更高效之快充。
- **技術特點**：採用雙電芯串聯 + 電荷泵方案，支持多種協議兼容。

華爲/榮耀——保守然穩定

- **標準**：僅支持 USB PD 協議。
- **功率**：iPhone 8 及以後支持快充，最高約 27W（iPhone 15 Pro Max 可達 27W+）。
- **最高速率**：功率相對保守，然充電曲線優化好，發熱控制出色。
- **配件要求**：需 USB-C to Lightning 線（或 USB-C to USB-C 線）與 PD 充電頭。

快充協議對比表

標準	協議	最高功率	技術路線
USB PD 3.1	VOOC/Dart	240W	高壓/多檔位
最廣泛	QC 5.0	100W+	高壓可調
高通機型	VOOC/Dart	240W	低壓大電流
OPPO 系專用	SuperCharge	100W	電荷泵
華爲/榮耀	蘋果快充	~27W	PD 協議

無線充電協議

無線充電亦有己之協議標準：

- **Qi 標準**：WPC（無線充電聯盟）制定，最通用之標準。基礎 Qi 支持 5W-15W。
- **Qi2 標準**：2023 年發佈，引入磁吸定位（類似蘋果 MagSafe），支持 15W+。
- **私有協議**：各品牌有己之高速無線充電，如小米 50W 無線充、華爲 50W 無線充等，需專用充電器。

快充選購指南

- **視協議匹配**：購充電器時，確認其支持君手機之快充協議。第三方充電器通常會標註支持之協議。
- **視數據線**：大功率快充需專用數據線。普通數據線或只支持 60W 以下。
- **視認證**：蘋果用戶議擇 MFi 認證之數據線；安卓用戶擇品牌原裝或知名第三方。
- **視發熱**：快充時手機發熱乃正常，然若燙手即不正常矣，議暫停充電或換充電器。

快充安全提示

- 避邊充邊玩大型遊戲，將加劇發熱。
- 避邊充邊玩大型遊戲，將加劇發熱。
- 充電時取掉厚重之手機殼，助散熱。
- 充電時取掉厚重之手機殼，助散熱。

四、電池新術：硅碳負極電池

若汝留意 2024-2026 年之旗艦手機發布將，“硅碳負極”此詞幾成矣高端電池之代名詞。這乃鋰電池負極材料之一次重要升級，原理實則不復雜：

- **傳統石墨負極**：手機電池始終用之乃石墨作負極，技術成熟、穩定，但能量密度已接近理論上限（約 372mAh/g）。
- **摻入硅材**：硅之理論比容量乃石墨之 10 倍左右（約 4200mAh/g）。以少量硅摻入石墨裏作成”硅碳複合負極”，遂能顯著提升整塊電池之能量密度。
- **能量密度升**：同等體積下，硅碳負極電池之容量可提升約 **15%-20%**。這正是 2024-2026 年旗艦機能於 7-9mm 機身裏塞進 5500-6000mAh 大電池之關鍵原因。

何機型用上矣？

- **榮耀**：Magic 系列率先商用硅碳負極，Magic6 Pro、Magic7 系列、Magic V3 折屏均用。
- **小米**：小米 14 Ultra、小米 15 系列搭載新一代硅碳負極電池。
- **vivo**：X100 Ultra、X200 Pro 系列用高密度硅碳負極策。
- **華爲/OPPO 等**：2025-2026 年旗艦普遍與進，硅碳負極方自”黑科技”變成高端機之標配。

對用戶之實際影響：硅碳負極帶來之最大好處乃“同體積電池容量更大”——手機可作得更薄，同時續航不縮水；或機身不變厚，但電池自 4500mAh 漲至 5500mAh。換句話說，汝不用爲續航付“厚重”之代價矣。須注意之乃，硅碳負極電池於低溫和滿充管理上仍需配合快充芯片作優化，這亦乃廠商建議長期保持 80% 充電之原因之一。

五、這方乃真正之省電術

- **屏幕（耗電之王）**：調低亮度、縮短自動鎖屏之時。OLED 屏手機開深色模式可省不少電。
- **信號差**：於電梯、地下室，手機將拼命搜信號，耗電極快。
- **5G**：若信號不佳，手切回 4G，續航將佳多矣。

- **定位服務**：非導航類 App，定位權限設爲” 僅使用時允許”。
- **限後臺活動**：iPhone 於設置 > 通用 > 後臺 App 刷新；安卓於應用電池設置中。

查之：爲君之電池作個體檢

1. 視健康度：覓並記下君之電池最大容量。
2. 視健康度：覓並記下君之電池最大容量。
3. 定對策：限後臺耗電異常之 App。

第十六章：賬號體系與雲端之務

吾有一友，去年手機被竊。其第一時間所思非“手機值幾何”，乃“吾內之照片還能覓回否”。甚憾，其未開雲備份。手機乃身外之物，然賬號內之數據，乃真失即再無回者。此章即論何以給君之數字生活上一真正之保險。

一、君之手機賬號，究竟管幾何物？

衆人覺賬號乃用以下載 App 者。實則，一手機賬號背後所牽之物，遠較君所想爲多：

- **通訊錄**：君存了幾年甚至十幾年之電話號碼。
- **照片與視頻**：自孩子出生至畢業典禮，自第一次約會至結婚旅行。
- **備忘錄**：君隨手記之靈感、日記、賬號密碼（對，多人將密碼記於備忘錄中）。
- **App 購買記錄**：花了數百塊買之付費 App 與內購。
- **查找手機**：手機失之後，此乃唯一能助君定位、鎖機、遠程抹數據之工具。

若君用蘋果手機，以上所有物皆綁於君之 Apple ID 上。若君用安卓，即綁於君之華爲/小米/OPPO/vivo 賬號上，尚有君之谷歌賬號。此等賬號即君數字生活之“戶口本”，失戶口本，諸多物即說不清矣。

二、三必爲之事，今即爲之

1. **啓雙重認證**：無論君用蘋果抑或安卓，第一事即往賬號設置中將” 雙重認證” 或” 兩步驗證” 開之。此物如爲君之門加一反鎖——光有密碼不夠，尚需君之手機收一驗證碼。縱有人於網上猜到君之密碼，無君之手機，彼亦進不去。
2. **綁一可靠之恢復方式**：雙重認證甚安全，然亦有一險——君已之手機失奈何？故系統將提示君設” 恢復密鑰” 或” 備用郵箱/手機號”。此乃君於緊急之時取回賬號之唯一途徑，最好記於紙上，勿存手機中。
3. **確認” 查找手機” 已啓**：此能於手機失之後能助君定位、鎖定、播放聲音、甚至遠程擦除所有數據。乃最後一道防線。多人買手機之時跳過此步，後欲用時覺未開，即晚矣。

三、iCloud：蘋果用戶之“空氣”

用蘋果手機之人，遲早須對一靈魂之問：iCloud 空間滿奈何？

用蘋果手機之人，遲早須對一靈魂之問：iCloud 空間滿奈何？

- **清理**：往設置君之名 iCloud 管理儲存空間，視誰佔了君之空間。一般而言，照片與備份乃大頭。可先刪不需之舊備份。
- **升級**：50GB 每月 6 塊錢，200GB 每月 21 塊錢。個人覺 50GB 於多數人足矣，主要用以備份與存照片之縮略圖。
- **曲線救國**：若君不欲花錢，可將照片定期導出至電腦上，而後關 iCloud 照片同步。然如此，君即失照片自動備份之便。

一重要之提醒：iCloud 照片圖庫非“網盤”——乃“同步”。君於手機上刪一張照片，iCloud 上亦將刪。若君所欲乃“備份至雲端而後清空手機”，則 iCloud 照片非君所欲，君需乃 iCloud 雲備份，或第三方之網盤。

iCloud+：付費用戶才能享受之隱私三件套

只要升級到任意付費 iCloud 檔位（50GB/200GB/2TB 等），遂自動成爲 **iCloud+** 用戶，解鎖三個隱私護能：

- **Private Relay（私密中繼）**：蘋果版之”VPN”。Safari 瀏覽之流量經蘋果服務器中轉後再發往網站，沒有人（包括蘋果）能同時看到汝乃誰、汝訪問矣何。適合於公共 Wi-Fi 下保護隱私。設置路徑：設置 > 汝之名字 > iCloud > 私密中繼。
- **Hide My Email（隱郵件）**：於網站/App 註冊時生成一次性隨機郵箱地址，郵件將自動轉發至汝之真實郵箱。不欲再收這家網站之郵件，直接關掉此隨機地址遂行，真實郵箱不暴露。設置路徑：設置 > 汝之名字 > iCloud > 隱藏郵件。
- **自定義電子郵件域名**：可用汝自己之域名（比如 @ 汝之姓.com）收發 iCloud 郵件，適合個人品牌或家庭郵箱管理。

提示：Private Relay 於某些國家/地區不可用，國內用戶或須其他方案補充。

iCloud 高級數據保護（ADP）——真正之端至端加密

2022 年底蘋果推出矣 **iCloud 高級數據護（Advanced Data Protection，ADP）**，以端至端加密（E2EE）自原來之健康、密碼等少數數據，擴展至了**絕大部 iCloud 數據**——照片、備忘錄、iCloud 雲備份、設信息等共 23 類數據。

- **何乃 E2EE**：加密密鑰唯存在汝之設備上，連蘋果服務器皆沒有。就算蘋果爲黑客攻破，或執法部門要求調取，蘋果亦拿不出能解密之明文數據。
- **開啓之徑**：設置 > 汝之名字 > iCloud > 高級數據保護，依示設復聯繫人/復密鑰（**必作**，丟矣密鑰汝之數據亦救不回來）。

- **注意之事**：(1) 開啓後務必妥善保管恢復密鑰，否則賬號丟失時無法找回數據；(2) 共享相簿、iCloud 共享圖庫等少數功能不參與 E2EE；(3) 中國大陸區 Apple ID 暫不支持 ADP，須切換至其他地區賬號。

一句話建議：若汝之賬號裏有重要隱私數據，且賬號地區支持，強烈建議開啓 ADP——這乃普通人能獲得之最強雲端數據保護。

四、同步與備份，勿混之

此二詞聽來相仿，然則果全異：

- **同步**乃雙向者：君於手機上刪之，雲端亦刪之；君於雲端刪之，手機上亦隨之刪。iCloud 照片、通訊錄、備忘錄此等皆乃同步。利乃多設備之間保持一致，弊乃" 刪即真無矣"。
- **備份**乃單向者：手機將當前狀態拍一" 快照" 存至雲端。後君於手機上刪物，雲端備份中之那份尚在。iCloud 雲備份、電腦本地備份，皆乃備份。利乃數據更安全，弊乃佔空間，且不能實時更新。

五、安卓用戶之賬號矩陣

安卓這邊之情較蘋果複雜些許，因君之數據或散於數個賬號中：

- **品牌賬號**（華爲/小米/OPPO/vivo）：管君之雲空間、查找手機、應用商店、錢包。每品牌皆有己之雲服務，通常乃基礎免費 + 擴容收費之式。
- **谷歌賬號**：若君用 Gmail、Google 相冊、Google 通訊錄，此等物皆在谷歌賬號中。谷歌免費予 15GB，較蘋果大方，然亦與 Gmail、Drive 共用。
- **微信/QQ**：其乃獨立於手機系統者。君之聊天記錄與微信藏，既不在品牌賬號中亦不在谷歌賬號中，只存於微信己之服務器上。

故安卓用戶作備份之時，須分別查：品牌雲備份、谷歌備份、微信聊天記錄備份，三者缺一不可。

六、家庭共享：令家人亦受益

蘋果之“家人共享”可使君與家人共享 App 購買、音樂、電影、iCloud 空間，尚能共享位置。最多可加 6 人。設之之法：設置君之名家人共享。華爲與小米亦有類似之能，於賬號設置中能覓“家共享”或“家庭雲”。

一甚實用之場景：爲爸媽之手機上設好家人共享，如此君即可隨時見其位置（前提乃其同意），萬一老人走失或手機聯繫不上，能有一定位。

賬號被盜之後果遠較君所想爲重

賬號被盜之後果遠較君所想爲重

實操：爲君之賬號作一次全面體檢

1. 開手機設置，入賬號頁，確認雙重認證已啓。
2. 開手機設置，入賬號頁，確認雙重認證已啓。
3. 入 iCloud 或雲空間管理，視何數據在同步，關不需者。
4. 確認” 查找吾之手機” 已啓。
5. 確認” 查找吾之手機” 已啓。

第十七章：手機與他設備之聯動

君手中之手機，實則乃一超級遙控器兼數據中心。其不光能打電話，尚能將電腦、電視、手錶、音箱、汽車串於一起，使此等設備不再是孤島。此章即論何以令其“組隊打怪”。

一、與電腦協同：手機與電腦不再是二世界

實言之，吾昔最煩者乃手機拍一照片，欲放至電腦上修之，尚須覓數據線、等傳輸。後覺，實則根本無需數據線，佳之聯動比數據線快十倍。

- **蘋果生態（AirDrop + 通用剪貼板）**：若君用 iPhone 配 Mac，隔空投送乃君第一當學之能。於手機上點分享隔空投送選君之 Mac，照片瞬間即過，不用流量不用 Wi-Fi，用乃藍牙與 Wi-Fi 直連。尚有一易忽之能曰” 通用剪貼板”——手機上覆制一段文字，於 Mac 上直貼即出，簡直如魔法一般。

- **安卓 +Windows（連接至 Windows）**：微軟有一叫” 連接至 Windows” 之 App，三星手機內置，其他安卓手機往應用商店下一即行。配對之後君可於電腦上直視手機通知、回短信、翻照片，甚至接打電話。吾已用二年，開會之時不用一直低頭視手機，電腦屏上彈一通知即足。

- **品牌多屏協同**：華爲之” 多屏協同” 與小米之” 小米妙享” 更狠——直將手機屏投至電腦上，君可鼠標拖拽文件自手機至電腦，亦能於電腦上刷抖音、回微信。跨設備複製粘貼、文件拖拽皆行，體驗甚絲滑。

二、投屏至電視：全家同觀

過年歸家之時，吾堂妹欲於電視上放其手機內之畢業旅行視頻。折騰半日，終覺實則甚簡：

- **iPhone 電視**：自右上角下拉控制中心，點彼兩方框疊一起之圖標（屏幕鏡像），覓君之電視名，一點即連上。前提乃電視與手機連同一 Wi-Fi，且電視支持 AirPlay（大部分智能電視皆支持）。

- **安卓電視**：各家叫法不同，華爲叫” 無線投屏”，小米叫” 投射”，OPPO/vivo 叫” 多屏互動”。一般於快捷開關面板中能覓，覓不到即往設置中搜” 投屏”。

- **視頻 App 直投**：若君僅欲於電視上看愛奇藝、B 站、YouTube，實則更簡——於播放界面覓一矩形帶 Wi-Fi 符號之小圖標，點之，選電視，視頻即過矣。此時手機可幹別的，視頻於電視上續放，互不相擾。
- **投屏不成功奈何**：九成之情乃未連同一 Wi-Fi。尚有一成乃電視之投屏能未開，往電視設置中覓" 多屏互動" 或"AirPlay" 開關。

三、與智能手錶/手環：健康管家長於手腕上

吾爲吾母買一手環，其初言“吾又不運動買此何爲”。然用二週，其覺己每日只睡五時多，始主動調作息矣。此即手錶/手環最實用之價值——將君不見之健康數據變爲可見之數字。

- **通知過濾器**：手機於包中或口袋中震個不停之時，抬腕視一眼——乃老闆之微信抑或拼多多之促銷，一秒判斷，不值得掏手機之消息即令其去。
- **健康數據同步**：手錶 24 時監測心率、血氧、睡眠、步數，此等數據將自動彙總至手機上之健康 App 中。久而久之君能見己之趨勢——此月是否走得太少，此周睡眠質量有否變佳。
- **交通卡/門禁卡同步**：於手機上綁好之公交卡與門禁卡，大部分品牌將自動同步至手錶上。乘地鐵之時無需掏手機，手腕一靠閘機即過，真有一種" 未來已來" 之感。
- **找手機神器**：手錶上一般皆有" 找手機" 之能，點之手機即響。吾已用此能之頻率遠超預期——手機隨手一放即忘在何處矣。

四、與智能家居：手機乃家中之“總開關”

智能家居聽來甚玄，然說穿即三事：以手機遠程控制、設自動化之則、語音操控。

述吾已之數個實用之法，君即知此物多實用：

- **空調**：夏歸家路上提前以手機將空調開之，進門即涼。花一百多買一空調伴侶插座即可實現，無需換空調。
- **燈**：睡前躺牀上，懶得起關燈，手機上點之或喊一聲" 嘿 Siri/小愛同學，關燈"。
- **掃地機器人**：出門前手機上點之" 開始清掃"，歸家地即淨矣。尚能設每日定時掃。
- **攝像頭**：家有老人小孩或寵物者，手機上隨時能視一眼家中的情況。

擇何生態？一言以蔽之：

- 用 iPhone 優先擇支持 Apple HomeKit 之設備（貴一點然隱私保護好）。
- 設備多求性價比小米米家生態最全，燈、插座、掃地機何皆有。
- 用華爲手機優先鴻蒙智聯，與華爲設備體驗最佳。

五、藍牙耳機與音箱：無線之自由

自有線耳機換爲藍牙耳機，於吾而言乃手機體驗升級中最明顯之一。再不會自包中掏出一團亂麻，亦無需憂線掛於門把手上矣。

- **配對甚簡**：耳機首次開機一般自動入配對之式（閃燈），手機開藍牙即能搜到，點之即連上。後每次取出自動連，何都不用做。
- **同品牌有加成**：AirPods 配 iPhone 將自動彈窗顯電量，尚能於多個蘋果設備之間無縫切。華爲 FreeBuds 配華爲手機、小米耳機配小米手機亦皆有類似之彈窗與優化。
- **藍牙音箱**：家庭聚會、戶外露營之時，一藍牙音箱可使氣氛翻倍。連上之後，手機即君之 DJ 臺。
- **注意延遲**：打遊戲之時若用藍牙耳機，或有輕微延遲（槍響半秒方聞槍聲）。若對延遲敏感，或用有線耳機，或買支持” 低延遲模式” 之藍牙耳機。

UWB 超寬帶技術——比藍牙更精準之” 空間感”

除矣藍牙，新款手機裏尚藏着一顆 UWB（Ultra-Wideband，超寬帶）芯片。它的能力乃**精確測距與空間定位**——能算出另一個設備離汝多遠、於哪個方向，精度可達釐米級。藍牙只能告訴汝“於附近”，UWB 能告訴汝“於汝左前方 1.2 米”。常見應用：

- **AirTag/尋精確**：iPhone 查找 AirTag 時顯示”7 米遠、向左走” 之箭頭導航，靠之就是 UWB。普通藍牙只能告訴” 於不於範圍內”。
- **數車鑰匙**：搭載 UWB 之 iPhone 與安卓旗艦，可當車鑰匙走近車輛自動解鎖、走遠自動鎖車，還能區分駕駛員與乘客位置，防中繼破解。
- **室內導航**：商場、機場裏經由 UWB 信標現” 過道級” 導航，較 GPS 準得多。
- **支持之機型**：iPhone 11 及以後機型均帶 UWB 芯片（U1/U2）；安卓陣營小米、三星、vivo、OPPO 部分旗艦亦已搭載。

LE Audio 與 Auracast——藍牙音頻之新一代標準

用矣二十年之經典藍牙音頻（A2DP）方爲 **LE Audio**（低功耗音頻）取代，這乃藍牙 5.2 以後之新一代音頻標準：

- **更省電、音質益善**：採用全新之 LC3 編碼，比老標準之 SBC 音質更好、功耗更低，耳機續航明顯延長。
- **雙路音頻**：左右耳塞獨傳，單耳模式切不掉線。
- **Auracast 廣播音頻**：LE Audio 之招牌功能。一個手機/電視可同時向**無數副**耳機廣播音頻——機場候機廳、博物館、健身房、教室裏皆能用一個發射源，多人用自己之耳機聽同一畫面。
- **現狀**：2024-2026 年新上市之旗艦耳機與手機普遍支持 LE Audio，老設備需硬件升級

無法軟件解鎖。買新耳機時可留意是否支持 LE Audio/Auracast 標識。

六、與車載系統：開車時之安全搭檔

開車視手機乃違法者，亦要命者。然有時確需導航、接電話、放音樂。此時 CarPlay（蘋果）與 Android Auto（安卓）即派上用場矣。

- **何以連**：若君之車支持（近幾年大部分新車皆支持），以數據線將手機插至車上之 USB 口，車載屏將自動切爲 CarPlay 或 Android Auto 界面。部分新車支持無線連接，更省事。
- **能何爲**：導航（比車載導航好用多矣）、打電話（方向盤接聽）、聽歌（手機內之音樂 App 直投至車上）、語音發微信（Siri 或小愛幫忙讀消息與回覆）。
- **老車亦能用**：若君之車較老不支持，花數百塊買一外掛之 CarPlay 屏，粘於儀表臺上，體驗相仿。

七、聯動中之安全與隱私

設備聯越多，隱私泄露之險面即越大。數點需注意：

- 公共場合投屏之前，先視手機通知欄有無私密消息，勿令全家見不該見之內容。
- 公共場合投屏之前，先視手機通知欄有無私密消息，勿令全家見不該見之內容。
- 將設備送予他人或賣掉之前，先解綁——往手機設置中覓彼設備，選” 移除” 或” 取消配對”。

試爲之：將手機屏投至電視上，並設一智能場景

1. 確保手機與電視連同一 Wi-Fi。
2. 確保手機與電視連同一 Wi-Fi。
3. 於電視上覓” 屏幕鏡像” 或”AirPlay” 之能並開之。
4. 選一張君最喜之照投至電視上，令家人觀之。
5. 選一張君最喜之照投至電視上，令家人觀之。

案例 1：電池續航優實戰

- **場景**：用戶小米 14 續航差，半天遂沒電。
- **步驟**：(1) 設電池觀耗電排行 (2) 識別異常後臺 App (3) 閉 5G 切 4G (4) 限後臺 (5) 開啓省電模式 (6) 充電時摘殼散熱。
- **預期果**：續航延 30-50%，電池健康度數據可讀。

第十八章：數據備份與遷移

吾一鄰人，手機掉入水池中撈出已開不了機矣。其最心疼者非手機本身，乃其中三年未備份過之照片——孩子自出生至會走路之全部記錄。聞其言畢吾心甚感怏怏，因此事完全可避免。此章即認認真真爲數據上一道保險。

一、先搞清：君手機內何物失即無矣

不同數據之“生存狀態”不同，君須分清：

- **存手機本地、失即真無者：**照片視頻（未開雲同步之話）、備忘錄、短信、通話記錄、下載之文件。
- **存 App 服務器上、換手機重新登即有者：**微信好友列表、淘寶訂單、銀行 App 內之交易記錄。此等物存於對應公司之服務器上，非存於君手機內。
- **處於灰色地帶、需尤操作者：**微信聊天記錄（雖微信服務器存了，然君已無法自服務器下載）、手機銀行綁定之設備憑證、兩步驗證 App 內之密鑰（Google Authenticator 那種）。

故君當重點護者，乃第一類與第三類。

二、整機備份：一次搞定所有物

整機備份如爲手機拍一完整之“快照”——當前系統設置、桌面佈局、App 數據、照片、通訊錄，全部打包。

方案一：電腦本地備份（最可靠，薦有電腦之人）

- **iPhone：**將手機以數據線連至電腦上，開 iTunes（Windows 舊版）或 Finder（Mac），於設備頁中選” 立即備份”。注意須勾” 給備份加密”，否則密碼與健康數據不會被備份入。
- **安卓：**各品牌有己之電腦端軟件，比如華爲手機助手、小米助手、三星 Smart Switch。連上電腦按提示操作即行。
- 電腦備份之利乃不用花錢買雲空間，且恢復之速比雲端恢復快甚多。弊乃需君記定期去插線備份——大部分人做不到此。

方案二：雲備份（最省心，薦所有人）

- **iPhone**：設置君之名 iCloud iCloud 雲備份開之。手機插着電源、連 Wi-Fi、鎖屏之時將自動備份。免費之 5GB 大概率不夠，議花 6 塊錢升級至 50GB。
- **安卓**：各家品牌雲皆有備份之能。華爲手機往設置搜" 雲備份"，小米往" 小米雲服務"，OPPO/vivo 亦類此。又尚有一谷歌自帶之備份（設置系統備份），此亦開之。
- 雲備份最大之利乃" 不用君操心"，自動搞定。最大之弊乃恢復之時須下載甚久，且免費空間一般不夠用。

三、分類備份之策：要緊者單獨另存一份

整機備份乃兜底者，然有些物太要緊，值得君單獨另存一份：

照片與視頻（最珍貴之數字資產）

- **雙保險方案**：開 iCloud 照片/Google 相冊自動同步，同時每隔數月手將照片複製一份至電腦或移動硬盤上。如此雲端一份、本地一份，互爲備份。
- **不欲花錢之方案**：Google 相冊免費予 15GB，可設爲" 壓縮畫質" 以省空間。或每隔一月以數據線將照片全部拷至電腦上。

通訊錄（易忽然極要緊）

- iPhone 通訊錄默認與 iCloud 同步，只要開同步即無需額外備份。然若君欲更安全一點，可往 iCloud.com 導出 vCard 文件存電腦上。
- 安卓通訊錄通常與谷歌或品牌云同步。往通訊錄 App 之設置中導出.vcf 文件，即一份完整之備份。

微信聊天記錄（單獨處理，重中之重）

此必單獨拿出言，因整機備份內微信聊天記錄甚多時候不完整，且換機工具根本搬不動其。

- **法一：遷移至新手機**。微信我設置通用聊天記錄遷移與備份遷移遷移至手機/平板。新舊手機連同一 Wi-Fi，掃碼即能傳。注意：遷移完成之前，舊手機勿退微信。
- **法二：備份至電腦**。電腦上裝微信 PC 版，左下角三條槓遷移與備份備份與恢復備份聊天記錄至電腦。後換手機了可再恢復至新手機上。
- **法三：備份至云端**。微信亦支持備份至 iCloud 或手機雲空間，然注意：此備份只能於君當前手機上恢復，不能跨設備還原。換手機之話，尚須用前二法。

四、換機指南：自舊手機至新手機

換新手機乃高興事，然若數據未搬過去即變頭疼事矣。三種情況分別對待：

同品牌升級（最簡）

- **iPhone 新 iPhone**：開機激活時將舊手機置旁，將自動彈” 快速開始”，以舊手機掃新手機上之圖案即行。所有數據、設置、App 佈局將原封不動搬過去，包括 Wi-Fi 密碼皆將自動帶去。大概需二三十分鐘。
- **安卓同品牌**：用品牌自帶之換機工具。華爲用” 手機克隆”，小米用” 小米換機”，OPPO 用” 手機搬家”，vivo 用” 互傳”。步驟皆相仿：新舊手機裝同一 App、掃碼連接、勾要搬之物、開傳。

跨品牌換機（安卓換安卓，不同品牌）

用第三方換機工具或用舊品牌自帶之換機 App（大部分品牌之換機 App 亦支持換至其他安卓手機）。能搬走者有：通訊錄、照片、短信、通話記錄、部分 App。搬不走者：App 內之登錄狀態（君須重新登）、微信聊天記錄（用微信己之遷移之能）。

跨平臺換機（安卓換 iPhone，或反過來）

- **安卓 iPhone**：蘋果有一” 轉移到 iOS”App，安卓手機上下載後，於新 iPhone 激活時擇” 自安卓設備轉移數據”。能搬走通訊錄、照片、短信等基礎數據，然 App 數據搬不了。微信聊天記錄尚須用微信己之遷移之能。
- **iPhone 安卓**：各安卓品牌有對應之遷移工具，然實言之，跨平臺遷移體驗不如同平臺佳。最要緊之事先單獨做之：通訊錄導出、照片備份、微信聊天記錄遷移。餘者慢慢來。

五、備份之黃金之則：三二一原則

此乃數據安全領域之一經典原則，簡而易記：

- **3**：要緊數據至少留三份（一份原件，兩份備份）。
- **2**：備份用二種不同之存儲介質（比如雲端一份，移動硬盤一份）。
- **1**：至少有一份備份置於不同之物理位置（防火災、盜竊等同時毀所有數據）。

於常人而言，做到“雲端一份、電腦一份”即已算甚佳矣。無需搞太繁，關鍵乃君真去做了。

微信聊天記錄尤注意

微信聊天記錄尤注意

練之：作一次完整之備份演練

1. 查 iCloud 雲備份或品牌雲備份有否開啓，確認上一次備份之時。
2. 查 iCloud 雲備份或品牌雲備份有否開啓，確認上一次備份之時。
3. 手備份照片：以數據線連電腦，將 DCIM 與 Pictures 文件夾複製至電腦。
4. 備份微信聊天記錄：開微信我設置通用聊天記錄遷移與備份備份至電腦。
5. 將導出之通訊錄 vcf 與照片文件夾複製一份至移動硬盤或 U 盤，成”三二一”內之”一份離線於別處”。

第十九章：故障排查與自救

手機如人，偶爾亦會“生病”。卡頓、閃退、黑屏、充不進電……大部分毛病實則未至那般可怕，君完全可於送修之前已先試之。此章乃君之“手機急診手冊”——先以低成本之法試，不行再覓專業醫生。

一、手機變卡、發熱——最常被問之問題

實言之，手機用一年左右始變卡，太正常矣。原因未必乃手機壞矣，更多時候乃“積灰”——垃圾文件、後臺進程、存儲空間慢慢被填滿。

先作診斷，勿急修：

1. **查存儲空間**：往設置存儲視剩餘空間。若不到 10% 矣，系統將變得甚卡，因其掙扎着騰地方。吾之經驗乃：保持至少 15%-20% 之剩餘空間，手機方能較順暢。

2. **查耗電排行**：往電池設置中視過去 24 時之耗電排行。若某個 App 佔了大頭然君根本未甚用，即其在後臺偷跑。直卸載或限後臺活動。

3. **聯想近來做了何**：乃升級系統後變卡？（大版本更新後前幾日變卡正常，系統於後臺重建索引）。抑或裝了某個新 App 之後變卡？（卸載試視）。

4. **視使用環境**：手機於太陽底下暴曬、套着厚厚之殼邊充邊玩遊戲，發燙乃必然者。高溫尚將致 CPU 自動降頻，君即覺手機突然變卡矣。涼下來再試。

治療之法（自簡至繁）：重啓手機（解六成之問題）清存儲空間（刪無用之 App 與視頻）某個 App 搞鬼即清其緩存或卸載重裝最後一招：備份數據後恢復出廠設置。

二、應用閃退、無響應——App 不聽話矣

某個 App 點開即閃退、或用着用着突然卡死不動矣。處理流程如醫生問診，一步步來：

1. **先關重開**：自後臺劃掉 App，再重新開。大多數臨時 bug 此下即好矣。

2. **更新 App**：往應用商店視有無更新。有時乃因 App 版本太老，與系統不兼容。

3. **查權限**：有些 App 需某個權限方能正常工作，然君未給。比如相機 App 未給相機權限開了即閃退。往設置中覓彼 App，將該給之權限皆給了。

4. **清緩存**：往設置應用管理覓彼 App 存儲清除緩存。注意乃" 清除緩存" 非" 清除數據"！清除數據將把君 App 內之登錄狀態與設置全刪了，相當於恢復出廠給 App 之狀態。

5. **卸載重裝**：最後一招。卸載之前先確認此 App 之數據會否丟——微信之話先備份聊天記錄，遊戲之話先確認賬號有否綁手機號。

三、充不進電或充電慢——恐非電池之鍋

充不進電乃尤易令人焦慮之問題，然甚多時候原因出於配件上，非手機壞矣。

- **先排查充電口**：以牙籤或不導電之小工具，輕輕將充電口內之灰塵與棉絮掏之。此吾己遇過——口袋裏掉了一根線頭剛好堵於充電口內，插進去以爲接觸不良，掏出即好矣。

- **換根線與充電頭試**：線之損壞率比充電頭高甚多，尤其經常捲來捲去之線。換一條確定沒問題之線與充電頭交叉測之，很快能定乃哪個配件壞矣。

- **無線充電之問題**：若君用無線充電，略查手機殼是否太厚，或殼後面有無金屬片（磁吸支架之類）。又，將手機準確地放至充電板之正中——歪了即或充不上。

- **充電慢**：確認充電頭功率夠不夠（iPhone 欲快充至少需 20W 之 PD 充電頭），線支不支持快充（非所有線皆能過 3A 以上電流），手機是否太燙（太燙將自動降功率護電池）。

四、網絡連不上、Wi-Fi 老是斷

於外面用流量好好的，歸家連不上 Wi-Fi；或 Wi-Fi 連着然就是沒網。按此序排查：

1. **開關飛行模式**：下拉控制中心，開飛行模式等 5 秒再關之。此等於令手機重新搜一次網，甚有效。

2. **Wi-Fi" 忘記" 再重連**：往 Wi-Fi 設置中，點彼網絡名旁之感嘆號（安卓乃齒輪），選" 略之此網絡" 或" 忘記"，而後重新輸密碼連之。

3. **重啓路由器**：多人家裏之路由器數月未關過，時久將不太穩定。拔掉電源等 30 秒再插回去，大部分 Wi-Fi 問題即解矣。

4. **查是否手機之問題**：別的設備連同一 Wi-Fi 正常否？若都正常即君之手機不行，則手機或需重啓或重置網絡設置（設置通用傳輸或還原還原網絡設置，不會刪君之數據，然將把所有 Wi-Fi 密碼皆清矣）。

五、屏幕問題：觸控不靈、閃屏、留印子

- **觸控不靈敏**：先撕掉鋼化膜視是否膜之鍋——便宜鋼化膜用久確實將影響觸控。屏上有水或手上有油亦將致觸控漂移。皆非之話，略重啓再試。

- **屏上有印子/殘影（燒屏）**：長時間顯同一畫面（比如一直亮屏刷抖音），將於 OLED 屏上留" 殘影"。此非屏壞矣，只暫時的。關屏休息半個時一般能恢復。

- **屏閃爍**：若是低亮度下閃爍，恐乃 PWM 調光之問題，多見於 OLED 屏。往設置中覓"DC 調光" 或" 防頻閃" 之項，開後將佳多。非所有手機皆有此能。

- **屏有一塊區域全黑/亮**：此較嚴重，大概率乃屏物理損壞，需換屏。非官方維修換之屏，色彩與亮度通常不如原裝，亦影響防水。

六、無法開機、卡於 Logo 界面——先勿慌

此情最易令人心跳加速，然冷靜，先試此數個：

- **充電半時再說**：甚多時候只是電徹底耗盡矣。插上原裝充電器，等至少 30 分鐘，再嘗試開機。有些手機於電耗盡後需充至一定電量方會有反應。

- **強制重啓**：iPhone 全面屏機型按一次音量加、一次音量減、長按側邊電源鍵不放，直至屏現蘋果 Logo（或要按住 10-15 秒）。安卓通用之法：長按電源鍵 10 秒以上，直至震動或亮屏。

- **卡於啓動界面反覆重啓**：此情叫" 白蘋果"（iPhone）或" 卡 Logo"（安卓）。若強制重啓不行，或需入恢復模式以電腦修復。iPhone 連電腦開 iTunes/Finder，入恢復模式後選" 更"（非" 恢復"），將重裝系統然保留數據。安卓進 Fastboot 或 Recovery 模式，各品牌組合鍵不同，議搜君手機型號 +" 進恢復模式"。

- **以上皆不行**：即只能求助專業維修矣。數據若備份過，即不用太憂。

七、聲音問題：喇叭不響、通話聲音小

- **喇叭無聲**：先確定非靜音模式（iPhone 側邊之靜音撥片）、勿擾模式、或音量被按至最低矣。聽來甚傻，然此情佔了四成。而後查藍牙是否連着某個耳機或音箱。

- **通話對方聽不到君言**：查話筒（麥克風）孔有無堵住——手機底部之小孔甚易被灰塵堵住。以軟毛刷輕輕清理。勿以針或牙籤硬捅，將捅壞防塵網。

- **聲沙啞/破音**：若是喇叭進水所致——勿以吹風機熱風吹！以紙巾吸乾表面水分，而後放着自然晾乾。網上言用聲波排水 App 有一定之效（播放低頻聲把水振出），可試之。

八、覓官方維修，避彼等坑

若以上之法皆試過猶不行，即須覓專業維修矣。然維修市場水甚深，述吾己踩過之坑與經驗：

- **覓真官方，勿覓”看起來像官方”者：**往品牌官網（Apple.com.cn、Huawei.com.cn 等）覓”授權服務商”列表。街邊掛個蘋果 Logo 之非授權店，用之或非原裝配件，修完君之防水即無矣。
- **送修前必做之事：**備份所有數據（勿指望修之時數據不會丟），退所有要緊賬號（Apple ID、微信、支付寶），關”查找吾之手機”。有些維修點將要求君提供鎖屏密碼，此事有隱私之險——君最好己先退賬號再送過去。
- **換屏之坑：**非官方屏分”原壓屏”（原裝玻璃換過）、”後壓屏”（非原裝玻璃）、”組裝屏”（全非原裝）。價差好幾倍，然顯示效與耐久度亦差甚多。若手機尚在保修期，老老實實走官方。
- **換電池之坑：**非原裝電池或虛標容量，且用半年即衰減得比原裝電池二年之損還大。蘋果換電池官方價比非官方貴不了太多，議走官方。
- **數據恢復之坑：**手機進水/摔壞後，往維修若對方言”數據恢復要加錢”，小心——有些不良商家將故意誇大損壞程度以多收錢。正規之數據恢復乃先報價再操作，且恢復不成功不收費。

萬能第一招：重啓手機

萬能第一招：重啓手機

今即記之：此等救命之操作當爛熟於心

1. **強制重啓：**全面屏 iPhone 按音量加音量減長按電源鍵至現 Logo。安卓長按電源鍵 10-15 秒直至震動。
2. **清理充電口：**覓一小工具（牙籤或手機取卡針），輕輕清理充電口之灰塵。
3. **Wi-Fi 重連：**往 Wi-Fi 設置中”忘記”君當前之 Wi-Fi，重新輸密碼連之。
4. **開/關飛行模式：**下拉控制中心，開飛行模式，心中數 5 秒，關之，視網絡有否恢復。
5. ：下拉控制中心，開飛行模式，心中數 5 秒，關之，視網絡有否恢復。

第二十章：開發者模式與高級權限

此章乃爲彼等欲深探手機“隱藏功能”之用戶所備。開發者模式、Root 權限、BL 鎖……此等聽來甚極客之概念，實則離普通用戶並不遠。

一、開發者模式：解鎖隱藏設置

開發者模式乃手機系統內置之一隱藏菜單，其中有諸多普通用戶用不到然開發者需之能。

何以啓之：

- **安卓**：入設置 > 關於手機 > 連點” 版本號”7 次，直至提示” 君已處於開發者模式”。
- **鴻蒙**：設置 > 關於手機 > 連點”HarmonyOS 版本”7 次。
- **iOS**：普通用戶無法啓完整之開發者模式，需付費之 Apple Developer 賬號。

常用能一覽：

- **USB 調試**：連接電腦進行高級操作（如刷機、數據恢復、ADB 命令）之必開開關，詳見本章”ADB 實用命令” 小節。
- **顯示觸摸操作**：屏幕上顯示觸摸點（一個小圓點與隨手指），適合錄製操作教程或演示。

性與流暢度相關：

- **動畫速度調（窗口動畫縮放 / 過渡動畫縮放 / 動畫程序時長縮放）**：這三個選項控制打開 App、切換頁面時之動畫時長。默認乃 1x，調至 0.5x 將使手機” 看起來” 更快（實際上程序運行速度沒變，只是動畫變快矣），調至關閉則直接無動畫跳轉。多數極客將設爲 0.5x，既保留過渡感又顯快；完全關閉將使操作顯得生硬，且或掩蓋程序真實之卡頓，反而使汝誤以爲” 甚流暢”。
- **強制 GPU 渲染（Force GPU Rendering）**：強制所有 2D 繪圖通過 GPU 而非 CPU 處理。於老舊機型或某些 App 優化不佳時能提升滾動流暢度，但亦將增加 GPU 負載與耗電。現代手機默認已大量使用 GPU 渲染，此開關對中高端機型影響有限，更多乃歷史遺留選項，普通用戶保持默認即可。
- **顯 GPU 呈現模式分析（Profile GPU Rendering）**：於屏幕上以柱狀圖實時顯示每幀之渲染耗時，綠色橫線代表 16.6ms（60fps 之幀時間閾值）。柱子超過綠線遂說明掉幀矣。這乃開發者排查 App 卡頓之利器，普通用戶亦可用它來對比哪個 App 優化得差——比如某個 App 滾動時柱子屢飆至紅線以上，那基本就是它的鍋。

後臺與續航相關：

- **後臺進程限制**：可設置” 標準限制”” 最多 1 個進程”” 最多 2 個進程” 等。表面上觀限制後臺進程能省電，但實際上 Android 自帶之後臺管理已作得不錯，強行限制反而將導致 App 頻繁重新加載（冷啓動反而更耗電、體驗亦變差）。此選項更適合調試用，不建議普通用戶長期開啓，保持” 標準限制” 即可。
- **不留活動**：每離開一個界面遂立即銷燬它。極不推薦，將導致 App 頻繁白屏、重新加載，僅用以測試 App 之恢復邏輯。

注意之事

開發者模式裏之設置或影響系統穩定，不理解之選項勿亂改。開啓 USB 調試後，連接電腦時當小心，勿連不信任之電腦。

二、OEM 解鎖：開發者項裏之“刷機開關”

於開發者選項裏有一個不太起眼但甚重要之開關——“OEM 解鎖”（部分品牌叫“OEM 解鎖允許”或“設備解鎖”）。此開關本身未能解鎖 BL 鎖，但它乃一個“前置許可”：只有打開它，廠商方允許汝通過 fastboot 命令或品牌工具去解鎖 BL。

- **開關之作用**：相當於向廠商聲明”吾同意放棄保修、自負風險，請允許吾解鎖 BL”。若此開關乃關閉之，任何解鎖 BL 之命令都將爲直接拒絕。
- **與 BL 鎖之關係**：OEM 解鎖乃”申請資格”，BL 解鎖方乃”實際操作”。前者於手機設置裏點一下遂行，後者須去廠商官網申請解鎖碼、用電腦跑 fastboot 命令（如 `fastboot flashing unlock`），二者缺一不可。
- **恢復出廠設置將重置嗎**：將。”OEM 解鎖”開關於恢復出廠後將回至關閉狀態，但 BL 鎖一旦解鎖就是物理級別之狀態，未能蓋因恢復出廠而重新上鎖——須手動執行 `fastboot flashing lock` 方能覆上鎖。
- **何時候宜打開**：除非汝確定要刷機或 Root，否則保持關閉。它與”USB 調試”乃兩個獨立開關，刷機時通常須同時打開。

小知識：何故廠商要設置此開關

早期安卓沒有此開關，惡意軟件只要拿至權限遂能偷偷刷入系統。Google 後來要求所有安卓廠商於開發者選項中加入“OEM 解鎖”開關，作爲一道“用戶知情同意”的確認。這亦就是何故即便汝解矣 BL 鎖，若 OEM 解鎖開關爲關閉，重新刷機仍然將失敗。

三、BL 鎖（Bootloader 鎖）：刷機之首道門

Bootloader 乃手機啓動時加載操作系統之程序。BL 鎖即廠商爲安全而加之鎖，防用戶刷入非官方系統。

BL 鎖之作用：

- **安全保護**：防惡意軟件刷入系統。
- **防盜**：即使手機被刷機，無原賬號密碼亦無法用。
- **保修限制**：解鎖 BL 通常意味着失官方保修。

各品牌 BL 鎖之策：

- **小米**：可解鎖，需綁賬號等待期（7-30 日）。
- **華爲**：2018 年後止提供解鎖碼，基本無法官方解鎖。
- **一加**：可解鎖，相對寬鬆。
- **OPPO/vivo**：部分機型可解鎖，需申請。
- **三星**：可解鎖，然將觸發 Knox 熔斷，失 Samsung Pay 等功能。
- **蘋果**：iOS 設備無傳統意義上之 BL 鎖概念，然有類似之安全機制。

四、ADB 實用命令：通過電腦控制手機

ADB 全稱 Android Debug Bridge（安卓調試橋），乃 Google 官方所出之命令行工具，使電腦經 USB 或 Wi-Fi 與安卓手機相通。其本身不需 Root，但使手機啟“USB 調試”即可用。可視之為“以電腦馭手機之遙控”——刷機、裝 App、傳文件、查日誌，皆賴之。

何以通過 USB 連接：

1. 手機端：入開發者項開”USB 調試”。
2. 電腦端：自 Google 官方下載 Platform Tools（含 `adb` 和 `fastboot` 兩個可行文），解壓至任意目錄。
3. 用數據線連接手機與電腦，手機上將彈窗” 是否允許 USB 調試”，勾選” 始終允許” 並確定。
4. 于電腦命令行（Windows 之 PowerShell 或 cmd）里執行 `adb devices`，若輸出類似 `List of devices attached /xxxxxxxx device`，遂說明連上矣。

驅動與授權彈窗勿略之

Windows 用戶若 `adb devices` 顯示“unauthorized”或根本不列出設備，多半乃驅動沒裝好或忘矣於手機上點“允許”。三星/小米等品牌可裝各自之“USB 驅動”或“手機助手”來補齊驅動；Mac 與 Linux 一般免驅。若手機彈窗爲汝手滑點掉矣，拔插一下數據線遂將重新彈出來。

常用 ADB 命令：

- `adb install app.apk`：以電腦上之 APK 裝到手機（`-r` 表覆蓋裝並留數據；`-s` 裝至 SD 卡）。
- `adb uninstall com.xxx.xxx`：按包名卸 App。
- `adb push` 電腦路徑 `/sdcard/`目標路徑：以電腦文件推送到手機。

- `adb pull /sdcard/文件電腦路徑`：把手機文件拉至電腦（比發給自己微信再下載快得多）。
- `adb shell`：進入手機之 Linux Shell，可直接執行 `ls`、`pm`、`am` 等命令。
- `adb reboot bootloader`：重啓手機進入 Fastboot 模式（刷機之入口）。
- `adb reboot recovery`：重啓入 Recovery 模式。
- `adb shell pm list packages`：列出手機上所有已安裝應用之包名（找包名必用）。加 `-s` 只觀系統應用，加 `-3` 只觀第三方應用。
- `adb shell pm uninstall --user 0 包名`：爲當前用戶卸預裝 App，免 Root，詳見下方實戰。

免 Root 卸預裝 App 實戰：

這乃 ADB 最實用之功能之一：用 `pm uninstall --user 0` 命令可爲“當前用戶”卸載預裝 App，而不須 Root，亦不影響系統分區。爲卸載之 App 於系統裏“消失”，但實際還於系統分區裏，恢復出廠設置後將回來。

1. 先用 `adb shell pm list packages -s` 列出所有系統應用，找到汝欲卸載之包名。比如小米瀏覽器乃 `com.android.browser`，華爲應用市場乃 `com.vmall.client`。
2. 不確定包名對應哪個 App？可於電腦上用 `adb shell pm list packages | findstr` 關鍵字搜索，或裝一個第三方 App ”Package Viewer” 於手機上直接查看每個 App 之包名。
3. 行卸：`adb shell pm uninstall --user 0 包名`，較如 `adb shell pm uninstall --user 0 com.android.browser`。返 `Success` 遂成功矣。
4. 欲恢復何以辦？兩種方法：(1) 恢復出廠設置（最徹底但麻煩）；(2) 重新安裝：`adb shell cmd package install-existing 包名`。

千萬勿亂刪系統核心包

“pm uninstall –user 0”雖是免 Root 之安全方式，但亂刪關鍵系統應用（如 `com.android.systemui` 態欄、`com.android.phone` 電話、`com.android.settings` 設置）將導致系統崩潰、黑屏、無法開機等嚴重問題。刪之前先 Google 一下“此包名能不可刪”，寧可少刪勿錯刪。建議一次唯刪一兩個，確認沒問題再繼續。

小知識：–user 0 乃何意思

安卓 4.2 之後支持多用戶，每個用戶有獨立之 ID（機主乃 0，附加用戶自 10 開始）。`--user 0` 表示“唯對當前機主卸載”，系統分區裏之 APK 文件依然存在，只是不賜機主顯示矣。這就是何故它不須 Root——沒動系統文件，只是改矣“用戶配置”。這亦乃它與 Root 後卸載最大之區別：Root 卸載乃真正自系統裏刪掉，所有用戶皆不再有；ADB 卸載只是“對當前用戶隱藏”。

五、Root 權限：安卓之“超級管員”

Root 權限相當於 Linux 中之超級用戶（對/目錄擁有完全控制權，可讀寫任意文件），乃安卓系統之最高權限。它與 Windows 之 Administrator 有本質區別：Windows 出於安全考慮對管理員作了諸多限制（真正之最高權限實則乃 TrustedInstaller），即便是 Administrator 亦無法像 Linux root 那樣直接執行 `sudo rm -rf /*`。獲得 Root 權限後，汝可修改系統文件、卸載預裝應用、使用須 Root 之高級 App。

Root 之利：

- 卸無法卸之預裝應用。
- 用高級功能 App（如鈦備份、綠色守護）。
- 改系統界面與功能。
- 裝第三方 Recovery 與自定義 ROM。

Root 之險：

- **失去保修：**大多數廠商不保修 Root 過之手機。
- **安全風險：**惡意 App 或獲 Root 權限，造成嚴重後果。
- **系統不穩：**誤操作或導致系統崩潰。
- **銀行 App 不能用：**甚多銀行與支付 App 檢測到 Root 將拒運行。
- **OTA 更新失敗：**或無法正常收官方系統更新。

嚴重警告

Root 操作有一定風險，或令手機變磚（不能開機）。若汝乃新手，建議先充分了解相關知識，並確保已有完整備份。操作前當細讀教程，確認汝之機型與系統版本。

六、SafetyNet 與 Play Integrity 認證：何故 Root 後銀行 App 不可用

甚多 Root 用戶皆遇至過此樣之崩潰瞬間：Root 之後興致勃勃地裝矣一堆模塊，結果打開銀行 App——“您之設備不安全，無法使用”，Google Pay 亦用不了，連 Netflix 皆拒絕高清播放。此背後乃 Google 之兩套設備完整性認證機制：舊之 SafetyNet 與新之 Play Integrity API。

何乃 SafetyNet / Play Integrity：

- **SafetyNet Attestation（舊版，2017 年起）：**Google 早期推出之設備完整性檢查 API，應用通過它向 Google 服務器查詢” 此臺設備為否通過 CTS 兼容性測試、為否處於未修改之官方狀態”。返回結果分爲 Basic Attestation（基礎校驗）與 Strong Attestation（強校驗，依賴硬件密鑰）。Google 已宣佈 SafetyNet 將於 2024 年後逐步停用。

- **Play Integrity API（新版，2022 年起）**：Google 推出之替代方案，比 SafetyNet 更嚴格亦更全面。除矣設備完整性，尚包括” 應用完整性”（App 為否爲篡改）與” 賬號/授權完整性”（用戶賬號為否爲 Google 信任）。新 App 應遷移至 Play Integrity。

- **誰來用**：銀行、金融、支付類 App（如 Google Pay、各銀行之網銀、支付寶/微信支付之部分場景）、流媒體（Netflix 高清版權保護）、部分遊戲反作弊（如騰訊/網易之部分遊戲）、企業辦公 App（MDM 設備合規性檢查）。

Play Integrity 之三級認證結果：

- **MEETS_DEVICE_INTEGRITY（設全性經由）**：設備運行正版 Android 系統，Bootloader 未解鎖，無 Root，CTS 兼容性測試通過。可正常使用銀行、支付類 App。

- **MEETS_BASIC_INTEGRITY（基全性經由）**：設備基本可信，但或存在輕微異常。某些 App 將降級使用（如 Netflix 仍可播放但限制清晰度），某些嚴格之銀行 App 將直接拒絕。

- **MEETS_STRONG_INTEGRITY（強全性經由）**：最高級別，要求硬件密鑰背書（如 Google Pixel 系列之 Titan M 芯片），大多數第三方品牌機型縱未 Root 亦達不至此一級。

- **無凡標籤（NO_INTEGRITY）**：設備已 Root、解鎖 BL、刷矣第三方 ROM 或爲識別爲模擬器，絕大多數支付/銀行 App 將直接拒絕運行。

何故 Root 後此 App 將失效：

應用通過 Play Integrity API 向 Google 服務器發起認證請求，Google 將檢查設備之多項指標——Bootloader 鎖定狀態、系統分區為否爲修改、為否存在 su 二進制文件、SELinux 為否處於寬鬆模式等。任何一項異常皆將導致返回之認證級別降低或爲空，銀行 App 一觀“設備不安全”，自然遂拒絕服務矣。

今隱策：Magisk Hide / Shamiko / Zygisk：

爲了於享受 Root 便利之同時還能用銀行 App，社區開發矣一系列“隱藏 Root 狀態”之方案。核心思路皆乃：使檢測程序看到之非一個爲修改過之系統，而乃一個“看起來正常”之設備。

- **Magisk**：當前最主流之 Systemless Root 方案——不修改系統分區，而乃通過掛載之方式實現 Root。此樣設計之好處乃 OTA 更新失敗之幾率小、卸載比較乾淨，乃後續所有隱藏方案之基礎。但使用 Magisk 仍然須解鎖 BL 鎖。

- **Magisk Hide（已停更）**：Magisk 早期內置之隱藏模塊，能對指定 App 隱藏 Root。但 2021 年起已爲 Magisk 官方移除，原因乃 Google 持續升級檢測手段，雙方陷入” 貓鼠遊戲”。

- **Zygisk**：Magisk 24.0+ 引入之新機制，類似 Riru，通過注入 Zygote 進程實現模塊加載，比直接修改系統更隱蔽。開啓 Zygisk 乃使用現代隱藏方案之前提。
- **Shamiko**：當前最主流之隱藏模塊，配合 Zygisk 工作，可爲指定 App（"反向白名單"模式）隱藏 Root、隱藏 Magisk 自身、僞造 Bootloader 鎖定狀態。Shamiko 通過 LSPosed 倉庫或 GitHub 分發。
- **Play Integrity Fix（osm0sis）**：專門針對 Google Play Integrity 檢測之 Magisk 模塊，將修改設備指紋（fingerprint）使 Google 認爲汝之設備乃某個未 Root 之官方機型。

貓鼠遊戲永遠未能結束

Google 每隔一段時間遂將更新檢測規則，App 廠商亦將與進升級 Play Integrity 校驗。今天還能用之隱藏方案，明天或遂爲識別。若汝完全依賴銀行/支付 App（比如日常轉賬、掃碼付款），最穩妥之選擇仍然乃：勿 Root、勿解 BL 鎖。隱藏方案更適合"主力機不 Root + 備用機玩 Root"之玩家配置。

七、賜普通用戶之宜

- **開發者模式**：可開啓，體驗一下動畫加速、GPU 呈現分析等功能，但勿亂改其他不理解之選項。
- **OEM 解鎖與 BL 鎖**：若汝不打算刷機或 Root，兩個開關皆保持默認（關閉）狀態最安全。
- **ADB**：免 Root 之強大工具，普通人也能用——比如用 `pm uninstall --user 0` 卸載預裝 App。但刪之前一定要查清楚包名能不可刪。
- **Root**：普通用戶不建議 Root，風險大於收益。若確實須某些功能，可先了解有無不須 Root 之替代方案（比如用 ADB 卸載預裝、用 Shizuku 替代部分 Root 功能）。

動手爲之：啓開發者模式並調動畫速度

1. 入設置 > 關於手機。
2. 連點"版本號"7 次，直至提示已啓開發者模式。
3. 返設置，覓"開發者選項"（通常於設置底部或"系統"中）。
4. 覓得"窗口動畫縮放"、"過渡動畫縮放"、"動畫程序時長縮放"，皆設爲 0.5x 或關閉。
5. 略感手機為否變快矣。

第二十章附：Termux——Android 設備上之便攜式 Linux 終端環境

若君覺開發者模式尚不夠"硬核"，欲於手機上體驗完整之 Linux 命令行，則 Termux 正爲君而備。其無需 root 權限，即可使一臺尋常安卓手機化爲便攜之 Linux 工作站，乃進階用戶探索移動端開發與運維之利器。

一、何謂 Termux

Termux 乃一款專爲 Android 設備設計之開源終端模擬器與 Linux 環境應用程序，無需 root 權限即可於手機上提供完整之 Linux shell 體驗。作爲一輕量級應用（安裝包僅 30MB 左右），Termux 經 APT 包管理系統支持豐富之軟件生態，允許用戶於移動設備上運行 Python、Node.js、Git、SSH 等

Termux 之核心優勢在於輕量化與高度可擴展性，適於開發、學習及系統管理等多種場景。其將標準之軟件包與精確之終端仿真結合於一開源解決方案中。

二、核心功能與特性

完整之 Linux 環境

Termux **僅內置 Bash** 此一種 Shell。若欲使用 Zsh 或 fish 等其他 Shell，須通過包管理器手動安裝，例如：`pkg install zsh` 或 `pkg install fish`。同時 Termux 支持 Vim、Nano、Emacs 等編輯器，提供接近原生之命令行體驗。用戶可通過“pkg install”安裝 2000+ 軟件包，包括 GCC、OpenSSH、SQLMap 等開發與安全工具。

強大之包管理

使用 APT 包管理器（類似 Debian 與 Ubuntu），可安裝數千種 Linux 軟件包，涵蓋開發工具、網絡工具等。支持 Python、Ruby、Node.js、Go、Rust、Swift 等多種編程語言環境。

安全與網絡功能

- **SSH 遠程連接：**支持 SSH 客戶端與服務器功能，可遠程連接其他設備或使手機成爲 SSH 主機。
- **加密工具：**提供 OpenSSL 等加密工具，支持數據加密與安全通信。
- **服務器搭建：**可配置 Nginx 或 Apache 搭建靜態網頁服務器。

系統集成與擴展

- 訪問手機存儲權限，實現本地文件管理。
- 通過 Termux:API 插件調用傳感器、攝像頭等 Android 硬件功能。
- 支持藍牙鍵盤與外接顯示器連接，具完整之鼠標支持。

三、插件生態系統

Termux 之核心應用程序可通過幾個可選之插件應用進行擴展，提供額外之功能與集成：

插件名稱	功能描述
Termux:API	向命令行程序公開 Android API 功能
Termux: Boot	允許於設備啓動時運行腳本
Termux: Float	提供浮動終端窗口
Termux: Styling	爲終端啓用顏色方案與字體自定義
Termux: Tasker	將 Termux 腳本與 Tasker 自動化應用集成
Termux: Widget	允許自主屏幕小部件運行腳本

重要提示

Termux 及其插件必須自同一來源安裝，因其共享相同之 sharedUserId。混合來自不同來源（如 F-Droid 與 GitHub）之安裝將導致安裝錯誤。

四、安裝與配置指南

安裝步驟

1. **下載應用：**從官方渠道（F-Droid 或 GitHub）下載安裝 Termux，無需 root 權限。
2. **初始化系統：**首次啓動後等待”Installing bootstrap packages” 完成。
3. **獲取存儲權限：**執行 `termux-setup-storage` 獲取手機文件系統權限。
4. **更新軟件源：**執行 `pkg update && pkg upgrade` 更新軟件包列表。

基礎優化

- **換鏡像源：**使用 `termux-change-repo` 命令通過圖形界面更換鏡像源（推薦清華大學鏡像源）。
- **快捷鍵設置：**音量減鍵模擬 Ctrl，音量加鍵模擬 Esc，提升輸入效率。
- **界面美化：**安裝 Zsh + Oh My Zsh，配置主題與插件。

五、典型應用場景

移動開發測試

於手機上搭建完整之開發環境，編寫並運行 Python 腳本，使用 GCC 編譯 C 程序，通過 Git 管理代碼倉庫，結合 SSH 與遠程服務器同步。

服務器運維

通過 SSH 遠程管理服務器，執行系統維護、日誌分析等任務，隨時隨地處理服務器問題。

學習 Linux 操作

提供真實之 Linux 環境，乃學習命令行與系統管理之理想平臺，可練習 Shell 腳本編寫、Linux 文件權限管理（如 `chmod 755` 設 rwxr-xr-x 權限位、`chown user: group` 改文歸屬、`ls -l` 查看 owner/group/other 三組九位權限）等基礎命令。注意此乃 Unix/Linux 經典之“三組九位”權限模型（owner / group / other × 讀 / 寫 / 執行），與 Windows NTFS 之 ACL 訪問控制列表乃完全不同之兩套體系。

網絡安全測試

安裝 Nmap、SQLMap、Hydra、Metasploit 等安全工具，進行網絡掃描與滲透測試。

六、版本與兼容性

Termux 之最新穩定版本乃 v0.118.2，適用於 Android ≥ 7 之設備。需注意者，Termux 於 Android 12 及更高版本上或存不穩定性，建議留意官方更新日誌以獲取最新兼容性信息。

總結

Termux 將強健之 Linux 終端體驗攜至 Android 設備之上，使手機化爲便攜之開發與修習利器。無論開發者、系統管理員抑或技術愛好者，皆可借 Termux 於移動設備上盡享完整之命令行體驗，以成“袖中 Linux”之理念。

試爲之：於手機上運行首條 Linux 命令

1. 自 F-Droid 下載並安裝 Termux，初啓以待其初始化完畢。
2. 執行 `pkg update && pkg upgrade` 以更新軟件包。
3. 安裝 Python：`pkg install python`。
4. 輸入 `python3 -c "print('Hello from Termux!')"`，以領略於手機上運行代碼之樂。

，不隨關閉窗口而消失。

第二十一章：移動網絡與信息安全深探

吾輩日以手機上網、支付、閒談，甚覺便利。然君曾思之乎？君之聊天記錄、支付密碼、照片……此等信息於網上“裸奔”之時多矣？實言之，衆人未嘗思及此問題，然一旦生變——譬如賬號被盜、銀行卡被刷——屆時追悔已晚。本章當詳論，何以使君之數字生活不至如此“透明”。

一、VPN：網絡中之“隱形斗篷”

VPN，全稱虛擬專用網絡（Virtual Private Network）。君可想象其爲於君家與所欲往之網站間，築一條專屬且加密之地下隧道。君於隧道中往來，外人不辨君手中何物、欲往何處。

- **尋常上網**：君之手機公共網絡目標網站。中間所經之路由器、交換機，任一環節皆或有人”偷窺” 君所發之數據。尤以咖啡廳、機場等公共 Wi-Fi 爲甚，言之難聽，直如” 裸奔”。
- **使用 VPN**：君之手機 VPN 加密隧道 VPN 服務器目標網站。數據經加密封裝，” 偷窺者” 所見唯亂碼而已。且目標網站所見者乃 VPN 服務器之 IP 地址，非君真實之 IP。

然 VPN 究能何爲？簡言之：其一，**護君於公共 Wi-Fi 上之隱私**；其二，**繞過地域限制**（譬如觀看僅於特定國家上線之內容）；其三，**防君之網絡運營商（ISP）記錄君之瀏覽習慣**。然吾須提醒一句：**切莫用免費 VPN**！免費之背後往往意味着君之數據即其“商品”。吾有一友嘗用某免費 VPN，結果手機莫名始彈廣告，後察之，乃該 VPN 將其瀏覽數據打包售予廣告商也。故擇 VPN，或擇付費之大品牌（如 NordVPN、ExpressVPN），或乾脆不用。

再者，VPN 非萬能。其唯能護“傳輸中”之數據，若目標網站本身不安全、或君之手機已中惡意軟件，VPN 亦無能爲力。

二、公共 Wi-Fi 之“甜蜜陷阱”——比君所想更險

君或以爲，不過連一免費 Wi-Fi 耳，能有多大風險？吾告君一事：安全研究者曾做實驗，於機場設一名爲“Starbucks_Free”之 Wi-Fi 熱點，未及一時辰已有上百人連接。實則此熱點乃研究者自設，凡經此熱點之**未加密流量**——譬如君所訪之 HTTP 網頁內容、未啓用 TLS 之 App 通信——彼等確能截獲而觀之。

然須注意一關鍵前提：今君日常所用之微信、網銀、支付寶，絕大多數已走 HTTPS 加密。換言之，攻擊者縱截獲數據包，所見亦唯亂碼，**無法直接讀出君之聊天內容或支付密碼**。然彼

等仍可知君訪問了哪些網站（域名），若某些老舊 App 或網頁尚用 HTTP 明文傳輸，則內容將盡露。此等攻擊名曰“中間人攻擊”（Man-in-the-Middle），其理乃攻擊者於君與真正 Wi-Fi 路由器之間“插足”。更可畏者，多人之手機設爲“自動連接已知 Wi-Fi”，攻擊者但需將己之熱點名設爲君曾連過之某 Wi-Fi（譬如“ChinaNet”或“CMCC”），君之手機即自動連上。

何以防之？數條實用之策：

- **認準 HTTPS**：地址欄開頭爲 https://，所多之”s” 即 Secure 之意。訪問任何需輸密碼之網站時，若見 http://（無 s），立止勿輸！今主流瀏覽器皆於非 HTTPS 網站前顯” 不安全” 之警，見此標記當警惕。
- **關閉” 自動連接”**：無論 iPhone 抑或安卓，皆於 Wi-Fi 設置中關閉” 自動加入” 或” 自動連接”。寧每次手動點之，勿使手機自斷。
- **以手機流量行敏感操作**：轉賬、登錄網銀、輸入身份證號……此等操作老老實實用己之 4G/5G 流量，勿省那點流量錢。
- **忘記不常用之 Wi-Fi**：出差歸來、旅遊歸來，將之前臨時連過之酒店 Wi-Fi、機場 Wi-Fi 皆” 忘記” 之。如此下次手機不會自動連至同名假熱點。

三、HTTPS 何以護君？——SSL/TLS 握手

方纔提及 HTTPS，其究竟何以運作？每次君訪問 HTTPS 網站，瀏覽器與服務器間將行一次“SSL/TLS 握手”，其全過程大約如此：

1. 君之網絡運營商
2. 服務器回復：” 善，吾自君之清單中擇此加密方案，此乃吾之數字證書，內含吾之公鑰，由某某 CA 機構簽名認證。”
3. 瀏覽器驗證證書之真僞：為否由可信之 CA 簽發？為否過期？域名為否相符？
4. 驗證既過，雙方以公鑰加密協商出一” 會話密鑰”，此後之通信皆以此密鑰加密。

此猶如君往銀行辦事：先遞上身份證（證書），櫃員覈實君之身份，而後爾等約定一暗號，此後之對話唯爾等能解。縱有人於旁偷聽，亦唯聞加密後之亂碼。

然須注意：HTTPS 唯保“傳輸過程”加密，不保目標網站爲善類。釣魚網站亦可用 HTTPS——彼小鎖圖標唯示君所連乃“某個網站”，不代表其即君所欲往者。故仍須自核網址為否無誤。

四、DNS 安全：君之“網絡電話本”安全乎？

DNS（域名系統）猶如互聯網之電話本：君輸入“baidu.com”，DNS 爲君查得其對應之 IP 地址（譬如 110.242.68.66），而後瀏覽器方能連之。問題於於，傳統 DNS 查詢乃**明文**、**未加密**者！為謂君之 ISP、Wi-Fi 提供者、或任何中間節點，皆能見君訪問過哪些網站。

更可惡者乃“DNS 劫持”：攻擊者於 DNS 環節做手腳，將“baidu.com”指向一假釣魚網站。君明明輸入正確網址，卻被引至一模一樣然實則虛假之登錄頁面，一輪賬號密碼即遭殃。

幸今已有加密 DNS 之策：

- **DNS over HTTPS（DoH）**：將 DNS 查詢藏於 HTTPS 流量中，混於正常網頁瀏覽流量之內，根本無從分辨何爲 DNS 查詢。
- **DNS over TLS（DoT）**：用 TLS 加密協議專門保護 DNS 查詢，端口獨立，網管能知君用了 DoT 然觀不到內容。

吾私薦用 Cloudflare 之 1.1.1.1 或 Google 之 8.8.8.8 作 DNS 服務器，二者皆支持加密 DNS。於安卓之“私人 DNS”設置中徑填“one.one.one.one”或“dns.google”即可，甚簡。iPhone 稍繁，需裝一描述文件或經 App 設置，然亦值得爲之。

五、“無痕瀏覽”真能隱身乎？

先言結論：**無痕模式乃君家之窗簾，非隱身斗篷。**

其所作：不留痕跡於本地——關閉無痕窗口後，瀏覽器不存君之瀏覽歷史、Cookies、表單填寫記錄、搜索記錄。借他人電腦查一物、或買禮物不欲爲家人自瀏覽記錄所覺，用無痕模式爲宜。

其不能作：

- 君所訪之網站**照舊能見君來過**，君之 IP 地址、設備信息、瀏覽行爲，網站端一清二楚。
- 君之網絡運營商**照舊能見君訪問了哪些網站**，唯不見具體內容（若用了 HTTPS）。
- 公司/學校之網絡管理員**一樣能監控**君之上網行爲。
- 君所下載之文件、所添之書籤將**保留下來**，不會隨着關閉窗口而消失。

故當記住：無痕模式乃“本地擦除器”，非“網絡隱身衣”。若君真需隱藏上網痕跡，須用 VPN+Tor 瀏覽器之組合拳，然於日常使用而言，知此足矣。

六、瀏覽器隱私：勿令瀏覽器賣君

君或未察，瀏覽器本身即一巨大之信息收集器。Cookies、瀏覽歷史、緩存、自動填充、位置權限……每一樣皆或泄露君之隱私。數條建議：

- **定期清理瀏覽數據**：每隔一段時日，往設置中將 Cookies、緩存、歷史記錄清之。非爲防”被追蹤”——彼靠清 Cookies 已不太管用——乃爲防手機愈來愈卡。瀏覽器緩存可佔數 G 之空間。
- **勿信”請勿追蹤”之幻想**：瀏覽器中之”發送’請勿追蹤’請求”選項，本質乃”請”網站勿追蹤君。然網站聽否全然另一事，多數網站徑直略之。勿指望之。

- **善用隱私保護瀏覽器**：Firefox Focus、DuckDuckGo 瀏覽器、Brave 等瀏覽器默認攔截追蹤器與廣告，比 Chrome 與 Safari 之默認設置乾淨多矣。
- **搜索引擎亦要緊**：以 DuckDuckGo 代百度/Google 搜索，其不錄君之搜索歷史，不建用戶畫像。實話言，搜索結果確不如 Google 精準，然日常查資料足矣。
- **檢查 App 權限**：往手機設置中觀君之瀏覽器被授予何權限。其真需訪問君之位置、麥克風、通訊錄乎？大概率不需。

七、實用安全清單：自今始護己

言之多矣，終予君一簡明之安全清單，費半小時了之，君之手機安全性將有質之飛躍：

1. **關閉 Wi-Fi 自動連接**：iPhone 於設置-無線局域網中，安卓於 Wi-Fi 設置中。
2. **換一加密 DNS**：安卓徑設私人 DNS 爲"dns.google"，iPhone 以 App 或描述文件設 1.1.1.1 之 DoH。
3. **開啓雙因素認證（2FA）**：微信、支付寶、Apple ID、Google 賬號，能開 2FA 者皆開。短信驗證碼已為最低級別之 2FA，能用 TOTP（如 Google Authenticator、Microsoft Authenticator）更佳。
4. **查一遍 App 權限**：將每個 App 之權限過一遍，尤以位置、通訊錄、短信、麥克風、相機此五項。君將發現有些手電筒 App 竟要君之通訊錄——速卸之。
5. **定期清理瀏覽器數據**：設一日曆提醒，每月一次，清緩存、Cookies。
6. **勿用公共充電樁之數據線**：USB 口既能充電亦能傳數據，惡意充電樁（名曰"Juice Jacking"）可趁君充電時竊數據。出門自帶己之充電頭與線。

即刻爲之：爲君之手機做一次"安全體檢"

1. **DNS 升級**：安卓：設置 > 連接與共享 > 私人 DNS > 填入"one.one.one.one"或"dns.google"。iPhone：以 Safari 搜"1.1.1.1 DNS profile" 安裝描述文件，或下載 Cloudflare 之 1.1.1.1 App。
2. **Wi-Fi 審計**：打開 Wi-Fi 列表，將不識、不常用、名怪之熱點皆" 忘記" 之。
3. **權限大掃除**：入設置 > 隱私/權限管理，逐個 App 審查權限。標記出權限要求不合理（譬如計算器要通訊錄）之 App，考慮卸載或用替代品。
4. **檢查賬號安全**：打開 Apple ID 或 Google 賬號之安全頁面，觀有無不明設備登錄，確認 2FA 已開啓，檢查恢復聯繫人為否可信。

第二十二章：AI 如何於手機中工作

歡迎至手機最奇之“魔術現場”，主角乃：人工智能。君每日所用之“一拍即得”美照、張口即來之語音助手、越用越懂君之輸入法，背後皆有此隱形大腦於焉。

一、AI 相機：君之“後期暗房”已入手機

今之 AI 計算攝影，更似將一專業修圖師 + 攝影師團隊徑塞入快門鍵中。於君按下快門前，AI 已始工作：掃描場景，識出“此乃藍天”、“此乃人臉”、“此乃夜景”。

二、語音助手：君之“聲音祕書”何以懂君？

1. **聽清（語音識別）**：將聲波轉爲文字。
2. **聽懂（自然語言理解）**：理解意圖，提取關鍵實體。
3. **執行與回答**：調用對應 App 或服務，再以語音合成” 說” 與君聽。

三、端側 AI vs. 雲端 AI

端側 AI 將計算留於手機中，速快、隱私高、可離線。雲端 AI 需聯網，然能處理更繁雜之任務。

四、AI 大模型與手機：君之隨身智能助手

2025-2026 年，AI 大模型（如 DeepSeek R1、ChatGPT-5、文心一言 5.0、通義千問 3 等）之爆發深刻改變矣手機之使用方式。手機不再只為執行指令之工具，而乃能理解、推理與創作之智能夥伴。端側 AI 芯片（NPU 算力突破 80 TOPS）使大模型推理首次於手機本地跑通，無需聯網也能用 AI。

何爲 AI 大模型？

AI 大模型（Large Language Model，LLM）乃經海量文本數據訓練而出之 AI 系統，能理解並生成自然語言。君可想象其爲一讀過互聯網上幾乎所有文字之“超級學霸”，能答問、撰

文、翻譯、編程、析數據。

手機上之 AI 大模型應用：

- **對話助手**：DeepSeek、ChatGPT、豆包、Kimi 等 App，可隨時提問、翻譯、撰文案、總結文章。比傳統搜索引擎更智——其不只予君鏈接，乃徑予君答案。
- **寫作與創作**：令 AI 助君撰郵件、總結會議紀要、潤色文章、生成創意文案。
- **圖像生成與編輯**：AI 一鍵消除照片中之路人、擴展圖片邊界、風格轉換、甚至依文字描述生成圖片。
- **實時翻譯**：通話實時翻譯、面對面對話翻譯、拍照翻譯，出國旅行不再有語言之礙。
- **學習輔導**：AI 可釋概念、出題練習、批改作業，乃隨身家教。

端側大模型：手機自運 AI

最新之旗艦手機已能於本地運行小型 AI 大模型（通常 7B 參數左右），所致之變乃：

- **不需網絡**：斷網亦能用 AI 功能，飛機上、地鐵裏皆可。
- **隱私更安全**：君之對話與數據不會上傳雲端，全於手機本地處理。
- **響應更速**：無網絡延遲，AI 之答幾乎即時。
- **當前限制**：端側模型之能尚不及雲端大模型，繁雜任務仍需聯網。然隨芯片 AI 算力之升，端側模型將愈強。

系統級 AI 集成：

- **蘋果 Apple Intelligence**：iOS 18 引入之 AI 系統，集成寫作工具、圖像魔法棒、Siri 增強、通知摘要等功能。端側處理優先，繁雜任務方上雲端。
- **華爲盤古大模型**：集成於鴻蒙系統，小藝助手得大模型之能，支持文檔摘要、智慧識屏、AI 修圖等。
- **小米澎湃 AI**：系統級 AI 之能，支持 AI 搜圖、AI 摘要、AI 實時字幕、端側大模型對話等。
- **三星 Galaxy AI**：通話實時翻譯、筆記 AI 摘要、圖片生成編輯、搜索圈選等功能。

五、數個典型應用場景與實操

爲令君更直觀感受手機 AI 之“超能”，下文以五個具體場景，示 AI 何以於日常生活中“出手相救”：

場景一：實時翻譯——出國旅行之隨身翻譯官

實際表現：於巴黎點餐不諳法語？iPhone 之“翻譯”App 支持面對面雙人對話模式，雙方各觀己方語言之字幕，甚至可同聲傳譯。安卓陣營之三星 Galaxy AI 通話實時翻譯能令君於電話中徑說中文，對方所聞乃英文，反之亦然。

背後原理：語音識別（ASR）機器翻譯（NMT）語音合成（TTS）三步流水線，端到端延遲已可控於 1 秒內。

場景二：AI 消除——拯救被路人“毀掉”之遊客照

實際表現：於景點拍一絕美自拍，然背景中擠滿他客？用“魔法消除”圈選路人，手機將自動識別並“塗去”路人，再智能填充背景，效果常令人驚豔。

背後原理：圖像分割（Inpainting）模型先識別並刪除選中區域，而後依周圍像素與語義理解“腦補”出合理之填充內容。此背後之 AI 模型訓練數據量達數百萬張圖片。

場景三：AI 修圖——一鍵成“攝影大師”

實際表現：拍了一張陰天灰濛濛之風景照？AI“魔法天空”可一鍵將天空換爲晴空、晚霞甚至星空。AI 擴圖可令豎構圖變爲橫構圖，手機“腦補”出畫面外之場景。

背後原理：基於生成式 AI（Generative AI）之擴散模型（Diffusion Model），理解圖像之語義內容後生成新像素。此乃當前 AI 圖像處理之“最前沿”。

場景四：文檔處理——辦公族之效率倍增器

實際表現：會議畢，將會議錄音予 AI，其能自動整理出結構化之會議紀要、提取待辦事項、生成摘要。一份 10 頁之英文 PDF 報告，AI 可於 10 秒內爲君提煉出核心結論。

背後原理：長上下文大模型（Long Context LLM）可“讀完”數萬字之文檔，行多輪推理與信息抽取。Apple Intelligence、華爲小藝、WPS AI、騰訊元寶等皆具此能。

場景五：AI 搜圖與識物——君之“視覺搜索引擎”

實際表現：於街上見一種不識之花，相機一開，AI 立告君“此乃木槿花”。相冊中翻尋去年於海邊所拍之照片，搜索“大海”，AI 自動將所有含海之照片尋出，無需手動翻尋。

背後原理：計算機視覺（CV）模型對圖片內容行向量化（Embedding），再與文字查詢行相似度匹配。此等“多模態搜索”乃手機相冊管理之一大飛躍。

六、提示詞優化：令 AI 聽君指揮之“溝通公式”

AI 答之質，大抵繫於君何以“問”。同一之問，問法異，則 AI 所答天壤之別。下文吾輩介紹一套**最實用、最易上手之“角色-背景-任務-限制”溝通思路**，並配以大量實例。

何故需框架？

多人用 AI 如用百度——敲數關鍵詞入之。然 AI 非搜索引擎，其乃一“閱讀理解後回答”之語言模型。**予其更多上下文、說清君欲何、限其勿跑偏**，其即能予君精準之結果。框架乃將君腦中“好答案長何樣”譯爲 AI 能解之指令。

角色（Role）：告 AI 其爲何人

予 AI 一具體之“身份”，其即調用對應領域之專業知識與表達方式。**角色設定愈具體，輸出愈專業**。

實用示例：

- ”君乃一位**有 10 年經驗之小米手機維修工程師**。”（技術問題）
- ”君乃一位**擅長將繁雜概念講簡之中學物理老師**。”（科普問題）
- ”君乃一位**審美好、閱書多、自有風格之圖書編輯**。”（寫作問題）
- ”君乃一位**每日爲上班族做早餐之烹飪博主**。”（生活問題）
- ”君乃一位**耐心、溫和、永不哂新手之中醫**。”（健康問題）

何以有效？角色設定了 AI 之**知識範圍、表達風格**與**專業深度**。無角色之 AI 乃一“全才然啥皆不精”之通才；有角色，其能“入”彼身份，如那位專業人士般思考。

背景（Background）：告 AI 君之情況

將“君爲何人、君之現狀、君之約束條件”先說清楚，AI 方能予**真適合君**之答案，而非泛泛而談。

實用示例：

- ”吾之手機乃**小米 14，購半年，近玩遊戲時發燙嚴重**。”（設備情況）
- ”吾乃**一 60 歲之退休婦人，目力不佳，不善用智能手機**。”（用戶畫像）
- ”吾正**備 2026 年 6 月之英語六級考試，基礎一般**。”（學習背景）
- ”吾開了一家**三線城市之小咖啡店，月流水 3 萬，僱兩名員工**。”（商業背景）
- ”吾**甫誕嬰在家坐月子，睡眠嚴重不足**。”（生活階段）

何以有效？背景信息令 AI 知**“爲誰服務”**與**“於何條件下”**。同一題，予學生與予家長講法不同；予城人與予鄉人建議亦異。背景缺，則 AI 只能予“最大公約數”之答案，常不痛不癢。

任務（Task）：明告 AI 做何事

此步當以**動詞開頭**（寫、列、對比、翻譯、解釋、總結……），將任務描述得**具體、可執行**。含糊之任務唯換來含糊之答案。

實用示例：

- ”請**列出 5 個吾應立即關閉之權限**，並說明緣由。”（動作明確）
- ”請**將下段英文郵件譯爲中文**，保持商務口吻。”（任務具體）
- ”請**對比此四款 3000 元價位之手機**，予出一張優缺點對比表。”（格式明確）
- ”請**以小學生能懂之言解釋‘何爲 5G’**。”（風格明確）
- ”請**爲吾之早餐店寫三條短視頻文案**，分別針對上班族、情侶、帶孩子之家長。”（細分明確）

何以有效？任務部分乃 AI 行動之“指令中樞”。**動詞愈清晰、要求愈具體，AI 愈知“交付何物”**。模糊之“稍弄”、“搞個”、“看看”乃 AI 大忌——其將揣測君意，結果常跑偏。

限制（Constraints）：告 AI 勿做何事

“限制”聽之似“約束”，實乃**令 AI 收斂至最適合君之輸出**之利器。常見維度有：字數、格式、語氣、避免項、必含項。

實用示例：

- ”**不超過 200 字，分點列出**。”（長度 + 格式）
- ”**語氣當親切自然，如與友聊天**，勿用‘先、次之、再次’此等官話。”（語氣）
- ”**勿用專業術語**，遇專業名詞當順便釋之。”（用詞限制）
- ”**每段不超過 3 行**，方便手機閱讀。”（排版）
- ”**必含：使用場景、優缺點、與吾預算之匹配度**。”（必含項）
- ”**勿予具體投資建議**，只釋原理。”（邊界）

何以有效？無限制，AI 易“自由發揮”——或太學術、或太長、或跑題。明確之限制 = 明確之“交付標準”，雙方對齊預期，結果方可控。

提示詞質量評估：好提示詞之 5 個特徵

寫好提示詞非一次性之事，**寫完先自查此五條**，再發與 AI：

1. **具體而非抽象**：非” 寫篇文章”，乃” 寫一篇 800 字、3 個小節、面向新手媽媽之手機選購指南”。

2. **含背景而非孤立**：告 AI” 吾爲何人、何故問此”，其方能予” 爲君定製” 之答案。

3. **有明確輸出格式**：爲表格、列表、段落、抑或 JSON？先說之。

4. **限制到位**：字數、風格、避項，三選一即有明顯提升。

5. **可被 AI 驗證**：問畢，令 AI 複述一遍君之要求；說不清楚即未說清楚。

常見問題與解法

症狀	解法
回答太泛、不實用	加背景（君爲何人/何場景）
回答太學術/太官方	加語氣限制 + 拒” 官話” 清單
回答跑題/答非所問	以動詞重述任務，並要求” 只答此”
回答太長看不完	硬性限字數（200 字內）
AI 一本正經胡說	加” 若不確定請標‘吾不確定’” + 重要事實自查
第一版不滿意	勿重寫，於原提示詞後追加” 再…一些”

三個完整場景案例

場景一：生活助手（早餐搭配建議）

提示詞：

君乃一位擅長營養搭配之早餐博主（**角色**）。吾乃一位上班族媽媽，每日晨只有 15 分鐘做早餐，家有 6 歲之孩子與 60 歲之婆婆（**背景**）。請**列出 7 天不重樣之早餐搭配**（**任務**）。每份早餐要求：(1) 準備時間不超過 15 分鐘 (2) 營養均衡（蛋白 + 碳水 + 蔬果）(3) 老少皆宜 (4) 食材於普通超市能買到（**限制**）。

何以有效：角色定專業度，背景予三類人共性，任務明交付物，4 條限制保可執行。

場景二：學習輔助（英語六級備考）

提示詞：

君乃一位**經驗豐富之英語六級老師**（角色）。吾乃一普通本科生，**四級 480 分，六級目標 500，尚餘 3 月**（背景）。請**爲吾制定一份可執行之 12 周備考計劃**（任務）。要求：(1) 每週明聽說讀寫之時間分配 (2) 予出每月之重點突破方向 (3) 推薦 3 本合適之複習資料（中文說明緣由）(4) 表格形式輸出（限制）。

何以有效：角色定專業背景，背景予出分數與時間錨點，任務明“分階段計劃”，限制要求“表格化”與“可執行”。

場景三：工作效率（會議紀要整理）

提示詞：
君乃一位**資深行政助理**（角色）。吾剛開完一個 1 小時之部門會議，**討論 Q2 營銷推廣方案**（背景）。**請將以下會議記錄整理成結構化紀要**（任務）。要求：(1) 分“決議事項 / 待辦事項 / 風險與異議”三部分 (2) 待辦事項標出負責人與截止時間（若未提及則寫“待確認”）(3) 總字數控於 600 字內 (4) 以 markdown 格式輸出（限制）。
何以有效：角色定行政視角，背景予會議主題，任務明結構化輸出，4 條限制令結果徑可複用。

進階玩法：令 AI 爲君之“私人助理”

試此等進階用法，以體驗 AI 之真正實力：

- **多輪對話優化：**對 AI 之回答不滿意時，可曰” 此太專業矣，請以小學生能懂之言再釋一次”。
- **鏈式思考（CoT）：**令 AI” 一步步思考”，回答質量通常更高：” 請一步步析此數學題”。
- **多 AI 協作：**以 A 寫初稿，B 潤色，C 翻譯——組合使用效果更佳。
- **系統提示詞：**於 AI 設置中預設君之身份、偏好、忌諱等，令回答更合君之需。

七、AI 之侷限與挑戰：保持清醒之判斷力

AI 雖強，然絕非“無所不能”。清醒識其侷限，方能更好用之：

- **” 幻覺” 問題：**AI 或一本正經地胡言（專業術語曰 Hallucination）。其將編造不存在之歷史事件、錯誤之引言、虛假之統計數據。重要信息必須人工覈實。
- **知識截止：**多數 AI 之訓練數據有截止日期（如 2024 年初），其不知之後發生之事。
- **推理能力有限：**繁雜之多步推理、數學證明、邏輯陷阱，AI 仍易出錯。
- **隱私風險：**於雲端 AI 服務中，君之對話或被用於模型訓練。敏感信息當優先用端側 AI。
- **偏見與價值觀：**AI 或帶訓練數據中之偏見，需保持獨立思考。

AI 使用小貼士

- AI 生成之內容或不準（” 幻覺” 問題），重要信息務必覈實。
- 勿向 AI 輸入個人隱私信息（身份證號、密碼、銀行信息等）。
- 善用提示詞（Prompt）：愈具體、愈清晰之描述，AI 之回答質量愈高。
- AI 乃工具非權威，保持獨立思考與判斷力。
- 遇關鍵決策（醫療、法律、財務），諮詢專業人士而非依賴 AI。

來挑戰：三道 AI 實戰題

1. **挑戰一：複雜指令：**對語音助手曰：” 明晨 7 點提醒吾帶傘，並將今日午後 3 點之日曆改至 4 點。” 觀其能否一次完成多步操作。
2. **挑戰二：AI 修圖：**拍一張有路人之遊客照，用” 魔法消除” 或”AI 橡皮擦” 功能，將路人清除乾淨。觀 AI 對背景之” 腦補” 效果。
3. **挑戰三：提示詞優化：**先以簡單提示詞” 寫一段旅遊攻略” 問 AI，再以 CRISPE 框架寫一詳細提示詞。較兩次回答之質量差異。

案例 2：換機數據遷移全程

- **場景：**iPhone 13 小米 15 跨品牌換機。
- **步驟：**(1) 舊機 iCloud/微信自帶備份 (2) 退 Apple ID + 閉” 尋” (3) 新機用” 小米換機”App 掃碼 (4) 擇遷移項 (5) 微信單獨用自帶” 聊錄遷移” (6) 驗鍵數據（通訊錄/相冊/微信）(7) 舊機加密擦。
- **預期果：**數據零丟失，舊機可安全轉賣。

第二十三章：編程之思與自動化啓蒙

前兩章，吾輩爲手機築起了安全之“邊界牆”，亦見識了牆內日益聰明之“*AI* 大腦”。今，吾輩解鎖手機之終極潛能之一：令其如程序員般“思考”，並自動完成君設定之連續任務。君或曰：“吾不會寫代碼，此與吾何干？”放心，本章所講根本不需君寫一行代碼——現代手機已把“編程”之門檻降至拖拖拽拽即可搞定之程度

一、自動化到底是個啥？

以大白話言之，自動化即：**令手機於某條件滿足時，自動爲君做一連串事**。君不需每次手動操作，亦不必記住該做何——手機會自斷時機，自執行。

舉例言之：每日晨君醒來，要手動關掉勿擾模式、打開微信觀消息、打開天氣 App 觀今日穿何、打開網易雲放首歌。若將此等步驟打包成一個自動化流程，設置成“每日晨 7 點自動執行”，則君醒後只需拿起手機，一切皆已備好。想想即爽？

自動化本質上乃“若……則……”之邏輯鏈，編程中之 if-else。學明此思維方式，縱君一生不寫代碼，處理問題之條理性亦將強甚多。核心要素：

1. **觸發條件**：何時始？可爲”每日晨 7 點”、”當吾連至家中 Wi-Fi”、”當吾至某地”、”當手機電量低於 20%”。

2. **執行動作**：具體做何？可爲發一條微信、打開某 App、調音量、換壁紙、播音樂、發通知……凡君能於手機上手動做者，自動化皆能做。

3. **中間邏輯**（可選然甚強大）：若情況 A 則做 X，否則做 Y。譬如”若為工作日則播新聞播客，週末則播音樂”，此曰條件分支。

二、iOS“快捷指令”——蘋果用戶之自動化神器

蘋果之“快捷指令”（Shortcuts）App 乃自帶之應用，不需下載。其核心概念乃將各種操作如搭積木般拼起。先言數個關鍵概念：

- **”快捷指令”**：一個完整之自動化流程，可手動點擊運行，亦可經 Siri 語音喚醒。譬如君曰”嘿 Siri，回家模式”，其即自動執行君設定好之一連串操作。

- **”自動化”**：由觸發條件自動運行之指令，不需君手動觸發。譬如”當手機連至車載藍牙時，自動打開地圖 App”。自動化於”自動化”標籤頁下設之，與”快捷指令”分開。

- **”動作”**：最小之執行單元。打開 App 乃一動作，發送消息乃一動作，獲取天氣乃一動作，調整亮度乃一動作。君可將成百上千個動作串於一處。

予君觀數個實實在在好用之例，君隨做一遍即明：

實例一：一鍵“早安流程”

每日晨點之（或對 Siri 曰“早上好”），手機將自動：獲取今日天氣朗讀天氣與氣溫顯示今日之日曆安排打開君常用之音樂 App 始播放。操作步驟：新建快捷指令添加動作“獲取當前天氣”添加“朗讀文本”添加“獲取日曆事件”添加“顯示提醒”添加“打開 App”選網易雲音樂。即此簡單，全程拖拽。

實例二：自動“省電模式”

於“自動化”標籤頁，新建自動化選“電池電量”設“低於 30%”添加動作“設定低電量模式爲開啓”、“降低屏幕亮度至 30%”、“關閉藍牙”。從此手機電量低於 30% 即自動入省電狀態，不需君操心。

實例三：掃碼自動記賬

每次君以微信/支付寶掃碼支付後，手機將讀取支付通知，自動將金額與商家記至一備忘錄或 Numbers 表格中。此需用快捷指令之“自動化”中之“收到短信/通知”觸發條件（iOS 17+ 支持讀取錢包交易），配“追加至 Numbers”動作。對於欲記賬又懶於手動記者，直如救星。

尚有一多人不知之巧用：君可將快捷指令添至主屏幕，變爲一 App 圖標。譬如做一“打車回家”之快捷指令，圖標放桌面上，點之即徑開滴滴並將目的地設爲家，比君自開 App 慢慢輸入快多矣。

三、安卓自動化——Tasker 和它之朋友們

安卓此邊，情況稍繁，然天花板亦更高。先言簡單者：

入門級：自動點擊器

多數安卓手機自帶“自動點擊器”或可於應用商店下載。其理甚簡：君手動操作一遍，其將君所點之座標與時間間隔錄下，而後可高速回放。譬如某 App 每日要領簽到獎勵，君錄一遍點擊流程，此後每日自動跑一遍即罷。缺點乃：其只認座標，若界面佈局變了（譬如彈了個廣告），其即點歪矣。

進階級：Tasker——安卓自動化之王

Tasker 乃安卓平臺上公認最強之自動化工具，常年霸榜。實話言其學習曲線稍陡，界面不算好看，然一旦上手，君將覺手機忽變得無所不能。Tasker 之核心亦“觸發任務”之式：

- **配置文件（Profile）**：iOS 之” 到達/離開位置” 觸發用者爲地理圍欄技術，準確度取決於 GPS 信號與系統優化。有時君已到家十分鐘方觸發。建議用 Wi-Fi 連接或基站觸發代位置觸發，更可靠亦更省電。
- **任務（Task）**：定義” 觸發後做何”。一個任務可含多個動作，按序執行。動作類型包括：調整系統設置（音量、亮度、Wi-Fi、藍牙、飛行模式）、發送短信、HTTP 請求、執行 Shell 命令、控制媒體播放、彈出對話框等等。

予君數個 Tasker 之經典實用場景：

- **到家自動開 Wi-Fi**：配置觸發條件爲” 連接至特定基站”（因君於家附近會連至固定之基站）任務” 開啓 Wi-Fi”。不用 GPS 亦不耗電，手機一進小區範圍即自動開 Wi-Fi。
- **會議模式**：觸發條件爲” 日曆事件標題含’ 會議’” 任務” 設置靜音模式、關閉媒體音量、自動回覆微信（需插件配合）”。再也不用憂開會時手機忽響。
- **深夜護眼**：觸發條件爲” 時間 22:00-06:00” 任務” 開啓護眼模式、降低亮度至 20%、將屏幕設爲黑白（灰度模式）”。黑白屏幕對刷短視頻之慾望有奇效——畫面一灰，君即不想刷矣。
- **耳機插入自動播客**：觸發條件爲” 有線耳機插入/藍牙耳機連接” 任務” 打開小宇宙 App 並始播放最新一集”。戴上耳機即能聽，不用解鎖手機覓 App。

若君覺 Tasker 上手太難，可試其簡化版兄弟：**MacroDroid**。界面友好許多，模板豐富，適不願折騰之用戶。另外今多數國產安卓系統（小米之 MIUI、華爲之鴻蒙、OPPO 之 ColorOS）皆自帶“場景”或“智能助手”功能，雖不及 Tasker 強大，然日常自動化完全夠用。

四、自動化思維之日常應用

學自動化最大之收穫，實則非省那點時間，乃**改君思考問題之方式**。一旦君習慣“若發生 X，即自動做 Y”之思維模式，君將覺生活中一大堆重複勞動皆可被省去。譬如：

- 每次拍完照片，自動備份至網盤，然後將本地照片壓縮成小尺寸以省空間。
- 每日晨自動自日曆讀取今日之安排，生成一段文字經微信發與家人。
- 收到快遞簽收短信後，自動於備忘錄中添一條待辦” 取快遞”。
- 手機電量充滿時自動發語音提醒拔掉充電器（避免過充傷電池）。
- 每日下班時間至，若尚於公司附近（經基站定位），自動提醒” 該下班矣”。

此等聽來甚高級，然實則君只要掌握“觸發 + 動作”此二基本概念，配合快捷指令或 Tasker，皆能做出。關鍵是**君要先意識到“此事可以自動化”**——多人非不會，乃根本未往那方面想。

五、踩坑指南：自動化之常見問題

搞自動化亦非一帆風順，有幾個坑吾先爲君踩過矣：

- **位置觸發或不靈**：iOS 之”到達/離開位置”觸發用之為地理圍欄技術，準確度取決於 GPS 信號與系統優化。有時君已到家十分鐘方觸發。建議用 Wi-Fi 連接或基站觸發代位置觸發，更可靠亦更省電。
- **快捷指令之”自動化”需授權**：iOS 裏，”自動化”標籤下之觸發多數需君手動確認方能運行（iOS 之安全機制）。然 Wi-Fi 連接、藍牙連接、鬧鐘停止此數個觸發爲可真正全自動運行者，不需確認。
- **Tasker 需開啓無障礙權限**：多數高級功能（譬如模擬點擊）需無障礙服務權限，此將令手機於設置中顯一常駐通知。此乃安卓之安全設計，無法去之。
- **勿將自動化搞得太繁**：剛開始做自動化，從”2-3 個動作”之小流程始。勿一上來即搞一個 20 步之超級流程——調試起來將瘋。

小貼士：自動化入門之正確姿勢

先費 5 分鐘思之：君每日於手機上重複最多之 3 件事爲何？譬如每日晨之例行操作、每次連藍牙耳機後之習慣、每次出門前之設置……自此等高頻小事入手，各做一簡單之自動化。哪怕每個自動化只省君 5 秒鐘，日積月累亦可觀。且做出後之滿足感，真會上癮。

練一練：三個入門自動化挑戰

選一個最適合君者完成，做完君即入門矣：

1. **挑戰一：早安自動化**：iOS：於快捷指令 App 中新建快捷指令添加”獲取當前天氣”“朗讀文本”“打開 App 選網易雲”。完成後君可對 Siri 曰”早上好”以觸發。安卓：於 Tasker 中新建任務添加”獲取天氣”“朗讀”“打開 App”設觸發時間爲每日 7:00。
2. **挑戰二：低電量自動省電**挑戰二：低電量自動省電
3. **挑戰三：連接車載藍牙自動導航**：iOS：自動化新建”藍牙”選擇君之車載藍牙添加”打開 App 選高德地圖”+”媒體音量調至 80%”。安卓：TaskerProfile 狀態藍牙已連接選擇車載藍牙任務”打開高德”+”媒體音量調高”。

案例 3：自動化”早安流程”

- **場景**：每日 7:00 自動播報天氣 + 關閉靜音 + 打開新聞。
- **步驟**：iOS: 快捷指令自動化時間觸發添加” 獲取天氣”+” 設定音量”+” 打開 App”;Android: MacroDroid 觸發器 (時間 7:00) 動作 (媒體音量/打開 URL/TTS 朗讀天氣)。
- **預期果**：免安裝即可實現，降低上手門檻。

第二十三章附：GitHub 入門指南——從代碼託管到協作平臺

上章吾輩講了“編程之思”，而真要將編程之思落地、與他人共做項目，即繞不開一名——*GitHub*。無論君為計算機系學生要參賽、欲存己之代碼作品，抑或單純欲觀全球開發者折騰何物，*GitHub* 皆君必須認識之平臺。此節吾輩自最基礎之概念講起，助君建一清晰之認知。

一、GitHub 究竟爲何物？

簡單言之，GitHub 乃一基於 Git 之**代碼託管與協作平臺**。其於 2008 年上線，2018 年爲微軟所收購，至今仍為全球最大之開源代碼託管平臺，託管着數億個倉庫，覆蓋幾乎所有主流編程語言。

若以一類比理解之：**Git 乃君電腦上之日記本，GitHub 則一公開之博客平臺**——君可發佈己之作品，關注他人之項目，參與討論與協作。其不僅為“程序員之後臺倉庫”，更似全球開發者共建之一座“技術城”——項目於此誕生、迭代、被討論、被複用。

於普通人而言，GitHub 亦非遙不可及。多數非程序員會用其託管筆記、電子書、個人博客，甚至用以備份配置文件。只要君有一絲“欲將己之事物整理好並分享出去”之需，GitHub 皆值得君瞭解。

二、Git 與 GitHub 之別（關鍵概念！）

多人初觸時常混此兩詞，此處爲君梳理清楚：

對比項	Git	GitHub
本質	版本控制工具（軟件）	代碼託管平臺（網站）
運行位置	本地電腦上	雲端（瀏覽器訪問）
核心職責	記錄文件之每次變更（何以改、誰改、何時改）	將 Git 倉庫放至雲端，方便存儲、展示、協作
為否需網絡	不需，離線可用	需聯網訪問
歸屬	開源軟件，由 Linus Torvalds（Linux 之父）創建	微軟旗下產品
同類競品	SVN、Mercurial	GitLab、Gitee（碼雲）、Bitbucket

簡言之：**Git 管“何以記錄變更”，GitHub 管“變更放於何處、何以協作”**。Git 乃工具，於本地運行；GitHub 乃平臺，將 Git 倉庫放至雲端，方便協作與展示。君可只用 Git 不用 GitHub（譬如自己寫代碼做版本管理），然用 GitHub 必用 Git（因 GitHub 即基於 Git）。

三、GitHub 之核心概念與功能

基礎概念

動手之前，先識數個高頻詞彙，否則觀 GitHub 之界面將一頭霧水：

- **倉庫（Repository，簡稱 Repo）**：一個項目對應一個倉庫，內中存放此項目之所有代碼文件、歷史版本與文檔。可理解爲一” 項目文件夾”。
- **提交（Commit）**：對代碼之一次修改記錄。每次 Commit 皆附說明信息，記” 此次改了何、何故改”。此乃 Git/GitHub 能” 時光倒流” 之基礎。
- **分支（Branch）**：自主線上分出之獨立開發線。譬如君欲加一新功能，可開一分支單獨開發，開發畢再合併回主線，不影響他人。
- **克隆（Clone）**：將雲端倉庫完整複製一份至本地電腦，此後即可於本地修改代碼。
- **推送（Push）/拉取（Pull）**：Push 乃將本地代碼上傳至 GitHub，Pull 乃自 GitHub 下載最新代碼至本地。此乃團隊協作時同步代碼之兩個核心動作。
- **合併請求（Pull Request，簡稱 PR）**：當君於一分支上改完代碼，欲合併至主分支時，發起一 PR，令團隊成員審查代碼後再合併。此乃 GitHub 協作之核心機制。
- **議題（Issue）**：用以記錄 bug、功能需求、討論話題。每個 Issue 如一討論帖，可指派負責人、打標籤。
- **Fork**：將他人之倉庫複製一份至己之賬號下，君可於己之副本上隨意修改，不影響原倉庫。改好後可予原作者發 PR，請求合併君之改進。

核心功能

GitHub 所供不止代碼存儲，尚包括：

- **版本控制**：記錄每次代碼更改之歷史，改壞代碼隨時回至上一正常版本。
- **團隊協作**：支持多用戶對同一項目之並行開發，多人同時開發不會覆蓋、不會衝突。
- **問題跟蹤**：用 Issue 記錄 bug 與功能請求，方便團隊跟進。
- **項目管理**：集成了待辦事項、項目看板（Projects）等管理工具，相當於內置一輕量級 Trello。
- **自動化**：經 **GitHub Actions** 實現代碼提交後自動編譯、測試、打包、部署，乃現代"CI/CD（持續集成/持續部署）" 之核心工具。
- **文檔管理**：README 介紹項目，Wiki 存放詳細架構與開發規範，Pages 甚至可徑託管靜態網站。
- **代碼託管與展示**：每個倉庫皆有漂亮之網頁展示，支持語法高亮，乃君技術作品之" 門面"。
- **開源社區**：君可 Star（藏）喜歡之項目、Watch（關注）項目動態、Follow（關注）開發者，參與全球開源協作。

四、註冊與上手：五分鐘創建君之第一個倉庫

註冊賬號

1. 訪問 github.com，點擊"Sign up"。
2. 填寫郵箱、設置密碼與用戶名（用戶名將出現於君之個人主頁地址中，建議用英文，譬如 `github.com/yourname`）。
3. 完成郵箱驗證與人機驗證。
4. 選免費方案（Free）即可，對個人用戶完全夠用。

創建第一個倉庫

1. 登錄後點擊右上角"+" 號，選"New repository"。
2. 填寫倉庫名（Repository name），譬如 `my-first-project`。
3. 寫一段簡短描述（Description），令他人知此倉庫爲何用。
4. 選公開（Public）或私有（Private）。公開倉庫所有人可見，私有唯己與協作者可見。免費賬號亦支持無限私有倉庫。
5. 勾選"Add a README file"——README 乃項目之" 說明書"，每個倉庫皆建議有之。
6. 點擊"Create repository"，君之第一個 GitHub 倉庫即誕生矣！

小知識：README 爲何物

README.md 乃一 Markdown 格式之文件，乃他人打開君倉庫時第一眼所見之內容。一個好之 README 通常含：項目簡介、安裝方法、使用示例、許可證、聯繫方式。寫好 README 乃程序員之基本素養，亦項目能否爲他人所用之第一道門檻。

五、基本工作流：自本地至雲端

理解 GitHub 之工作流，核心乃記此循環：**本地修改暫存提交推送雲端更新**。下以大白話釋每一步：

1. **修改文件**：於本地電腦上以編輯器寫代碼或改文檔。
2. **git add**：將修改之文件加入”暫存區”，相當於告 Git”此等改動吾準備記錄下來”。
3. **git commit**：將暫存區之改動正式記成一”提交”，附上一句說明，譬如”修復登錄 bug”。此步唯於本地完成。
4. **git push**：將本地之提交上傳至 GitHub 雲端，如此他人即能見之。
5. **git pull**：於始工作前，先自雲端拉取最新代碼，確保本地與團隊其他人保持同步。

於新手而言，記此口訣：**“先 pull 後改，改完 add，commit 帶說明，最後 push 上去”**。養此習慣，團隊協作即不會亂。

六、團隊協作流程：Pull Request 何以運作

GitHub 最強之處於其協作機制。一個典型之團隊協作流程乃：

1. **開分支**：自主分支（通常叫 main 或 master）開一新分支，譬如 `feature/login`，於此分支上開發新功能。
2. **開發並提交**：於新分支上寫代碼，多次 commit 記錄進度。
3. **推送至 GitHub**：將分支 push 至雲端倉庫。
4. **發起 Pull Request**：於 GitHub 網頁上發起 PR，請求將君之分支合併至主分支。寫清楚此 PR 做了何改動、何故改。
5. **代碼審查（Code Review）**：團隊成員查看君之代碼，提出意見或建議修改。君可根據反饋繼續 commit，PR 將自動更新。
6. **合併**：審查通過後，點擊”Merge”將代碼合併至主分支。君之貢獻即正式進入項目矣！

此套流程保證了**任何代碼進入主分支前皆經審查**，大大降低出錯概率。開源項目亦用此套機制接受全世界陌生人之貢獻——君完全可於某知名開源項目上提交 PR，被採納後君即此項目之貢獻者矣。

七、GitHub 之典型應用場景

學生參賽與課程項目

計算機系學生參加 ACM、黑客松、畢業設計，幾乎皆離不開 GitHub。其既能作團隊協作平臺，又能作作品展示窗口——比賽評委可徑觀君之倉庫，瞭解代碼質量與協作過程。多數公司招聘時亦會要求附上 GitHub 主頁，作爲技術能力之證明。

個人知識庫與筆記託管

非程序員亦能用 GitHub。多人用其託管學習筆記、技術博客（配 GitHub Pages 可免費建站）、電子書。Markdown 格式寫作 + Git 版本管理 + 雲端備份，三合一。

參與開源項目

GitHub 乃全球開源運動之核心陣地。君可 Fork 喜歡之項目，修改後發 PR 回饋社區；亦可單純 Star 藏好項目，跟蹤更新。參與開源乃提升技術、積累聲譽之絕佳途徑。

自動化部署與 CI/CD

借 GitHub Actions，可實現“代碼一 push，自動測試 + 自動部署至服務器”之全自動流程。多數個人博客、文檔站點皆如此搭建，寫完文章 push 之，網站即自動更新矣。

代碼備份與多設備同步

將代碼倉庫當作雲盤用，換電腦、多設備辦公時，git pull 之即能拿到最新代碼，再不用 U 盤拷來拷去。

八、予新手之數點建議

- **提交說明要寫清楚**：勿寫”update” 此等廢話說明。好之 commit message 寫” 修復用戶登錄後跳轉錯誤頁面之問題”，半年後回觀亦知當時幹了啥。
- **勤提交，小步快跑**：每完成一個小功能即 commit 一次，而非寫了一整日方提交一次。如此出問題更易定位。
- **用分支開發，勿徑改主分支**：哪怕為個人項目，養成開分支之習慣，未來團隊協作時即不會手忙腳亂。
- **善用.gitignore**：此文件用以告 Git 哪些文件勿納入版本管理（譬如密碼配置、編譯產物、依賴文件夾），避免將垃圾文件傳上去。
- **多觀優秀開源項目**：逛逛 GitHub Trending，觀高星項目之代碼結構、README 寫法、協作流程，乃提升最快之免費方式。

- **申請 GitHub 學生包**：學生認證後可免費獲得大量付費工具與服務之額度（Copilot、域名、雲服務等），價值上千美元。

安全提醒

- 千萬勿將密碼、API 密鑰、數據庫連接串等敏感信息提交至 GitHub，尤以公開倉庫爲甚——一旦泄露，將爲爬蟲秒抓，後果嚴重。
- 不小心提交了敏感信息，光刪除文件不夠（歷史記錄裏尚有），需用 `git filter-branch` 或專業工具清理歷史，並立即更換泄露之密鑰。
- 私有倉庫亦非絕對安全，敏感信息最好用環境變量管理，不進代碼倉庫。

國內訪問 GitHub 之注意事項

GitHub 於國內訪問有時不穩，或出現打不開、克隆慢之情況。可嘗試：(1) 用國內鏡像站（如 GitClone、ghproxy）加速克隆；(2) 配置 Hosts 或用代理；(3) 若主要面向國內協作，亦可考慮國產替代品 **Gitee（碼雲）**，功能類似且訪問速快。多數團隊會同時於 GitHub 與 Gitee 上託管，兼顧國際化與國內速度。

進階玩法：以 GitHub Pages 搭免費個人博客

GitHub Pages 允許君免費託管一靜態網站，配 Jekyll、Hexo、Hugo 等靜態站點生成器，可輕鬆搭建個人博客。寫畢 Markdown 文章 push 至倉庫，博客自動更新，且自帶 HTTPS 與自定義域名。無數技術博主之博客皆如此搭成，零成本、無廣告、版本可控。

九、總結：何故每人皆該瞭解 GitHub

GitHub 早已不只是“程序員之工具”，其更像是一**數字時代之工作方式**數字時代之工作方式

對計算機系之學生而言，GitHub 更是必備技能：它是君之作品集、協作平臺、學習資源庫，亦乃未來求職時最有說服力之“簡歷”。早點註冊賬號，將君之課程作業、小項目傳上去，慢慢積累，數年後君將擁有一份令面試官眼前一亮之技術履歷。

試爲之：完成君之 GitHub 初體驗

1. 註冊一 GitHub 賬號，完善個人資料（頭像、簡介、感興趣之技術領域）。
2. 創建一名爲 `hello-github` 之公開倉庫，勾選”Add a README file”。
3. 進入倉庫，點擊 README.md 之編輯按鈕（鉛筆圖標），用 Markdown 寫一段自我介紹，提交更改。
4. 於 GitHub 搜索欄搜一君感興趣之關鍵詞（譬如 `python-tutorial`），Star 兩個君喜歡之項目。
5. （進階）於電腦上安裝 Git，將倉庫 Clone 至本地，修改 README 後 push 回雲端，完成一次完整之” 本地雲端” 工作流。

第二十四章：Root 權限與 BL 鎖深析

此章乃對第二十章內容之深入擴展，適彼等欲真正理解手機底層機制、甚至嘗試刷機之用戶。請注意，此等操作有一定風險，請謹慎行之。

一、深入理解 Bootloader

Bootloader 乃手機啓動時首個運行之程序，其任務乃初始化硬件、加載操作系統內核。君可將 Bootloader 視爲手機之“守門人”。

BL 鎖之工作原理：

- **驗證簽名**：BL 鎖將驗證啓動鏡像之數字簽名，唯廠商簽名之系統方能啓動。
- **防止降級**：防用戶刷入舊版本系統以利用已知漏洞。
- **保護數據**：即使刷入新系統，無原賬號密碼亦無法訪問用戶數據。

硬件級存儲與驗證

BL 鎖之鎖定狀態（Locked/Unlocked）及廠商之公鑰，通常存儲於受保護之硬件區域（如高通之 devinfo 分區、eFuse 或 RPMB 分區），防止爲軟件篡改。每次啓動時，Bootloader 將從此安全區域讀取狀態，並據此決定為否執行嚴格之簽名驗證。

數字簽名驗證

廠商於發佈系統更新或鏡像時，將用自己之私鑰對其進行簽名。設備啓動時，Bootloader 將使用內置之公鑰對系統鏡像（如 boot.img、recovery.img）進行驗證。它之工作方式乃：

1. 對系統鏡像復算哈希值。
2. 用公鑰解密鏡像自帶之簽名，得到一個哈希值。
3. 比較此兩個哈希值，若一致則通過驗證。此確保矣系統鏡像未爲第三方篡改。

安全啓動鏈

BL 鎖乃整個安全啓動鏈之第一環。它驗證通過後，方將加載並執行下一階段之引導程序（如內核），從而層層確保系統啓動環境之可信。

鎖定 vs 解鎖：BL 鎖之雙面影響

爲了更直觀地理解 BL 鎖之利弊，我等用一個對比表來展示它於 5 個核心維度上之差異：

核心作用	鎖定狀態（默認）	解鎖後狀態
系統安全	防止刷入未簽名之非官方系統，保護系統完整性與底層安全 高安全：僅加載官方簽名鏡像，有效抵禦惡意固件篡改	安全降低：允許刷入第三方 ROM，系統完整性校驗機制失效，惡意軟件若獲取 Root 權限可深度篡改系統
系統穩定性	確保手機運行經過廠商測試與優化之官方系統，保障穩定流暢 穩定：使用官方固件，功能完整，OTA 升級可靠	風險增加：第三方 ROM 或存在兼容性問題，或導致系統崩潰、變磚、硬件功能異常（如攝像頭、快充），OTA 更新或失敗
保修權益	廠商通過鎖定 BL 提供穩定之官方系統，以提供保修服務 保修有效：享受官方免費保修服務（非人爲損壞）	保修失效：絕大多數廠商將解鎖 BL 視爲主動放棄保修之行爲，縱重新上鎖，亦或爲檢測出歷史解鎖記錄而拒保
數據安全	解鎖過程通常強制清除數據，防止舊數據於新系統環境下爲惡意利用 數據安全：日常使用中，BL 鎖限制了惡意程序對系統分區之修改能力	數據風險：解鎖時將強制清除所有用戶數據；若解鎖後未正確加密，數據存儲或不安全；丟失後更易爲刷機提取數據
功能限制	確保依賴安全環境之功能（如支付、銀行 App）正常運行 功能完整：所有官方功能均可正常使用	功能受限：部分安全敏感應用（如 Google Pay、銀行 App）或因檢測至 BL 解鎖狀態而拒絕運行；DRM 服務或降級，影響高清視頻播放

二、解鎖 BL 鎖之詳細流程（以小米爲例）

1. **申請解鎖權限：**於小米解鎖工具網站申請解鎖。
2. **綁定賬號：**於手機設置中綁定小米賬號。
3. **等待期：**依賬號等級，須等待 7-30 日。

4. **解鎖操作**：用小米解鎖工具連接電腦解鎖。

三、Root 之多種方案

- **Magisk**：現代 Root 方案，Systemless 設計，可隱藏 Root 狀態。
- **Kernelsu**：基於內核之 Root 方案，更底層，隱藏性更好。
- **SuperSU**：老牌 Root 方案，目前已停止更新。

四、刷機入門

刷機乃指予手機安裝新之操作系統。常見之刷機類型：

- **刷官方 ROM**：安裝官方系統，用於救磚或降級。
- **刷第三方 Recovery**：如 TWRP，用於備份恢復、刷入第三方 ROM。
- **刷第三方 ROM**：如 LineageOS、Pixel Experience 等，體驗不同之系統。

五、三星 Knox：BL 解鎖之不可逆代價

與其他安卓廠商不同，三星手機之 BL 解鎖機制有一個獨特之“熔斷”設計：**對於三星手機來說，解鎖將觸發 Knox 熔斷，永久失去部分安全功能（如 Samsung Pay）；將清除所有受保護數據；影響保修**。具體來說：

- **熔斷器乃一次性之**：手機內部有一顆 eFuse 熔絲，一旦檢測至 BL 爲解鎖遂將永久燒斷，無法恢復。
- **受影響之功能**：Samsung Pay（公交卡/銀行卡）、Knox 安全文件夾、企業 MDM 管控、部分健康保險類 App 皆將失效。
- **強制清空數據**：熔斷觸發後手機會強制恢復出廠設置，所有本地數據爲擦除（雲端數據不受影響）。
- **保修影響**：官方明確表示 Knox 熔斷後不再享受保修服務。

三思而後行

三星之 Knox 熔斷乃**不可逆**之，一旦觸發遂回不去原廠狀態。若汝只為欲用一些原本須 Root 之功能，建議先查查有無不須 Root 之替代方案。

重要警告

刷機有風險，操作需謹慎。或導致：

- 手機變磚（無法開機）

- 失去保修
- 數據丟失
- 支付功能異常

刷機前務必做好完整備份，細讀教程，確認機型與版本匹配。

推薦資源

- **XDA Developers**：全球最大之安卓開發者社區，有豐富之刷機教程與 ROM 資源。
- **酷安**：國內知名之數碼社區，有多中文教程與討論。
- **各機型論壇**：如小米社區、一加社區等，有官方與民間教程。

六、常見問題解答（Q& A）

Q1：解鎖 BL 鎖後還能恢復嗎？

A：可重新上鎖。通常須刷入官方完整之線刷包，並使用特定之刷機腳本（如小米之 `flash_all_lock.bat`）即可重新上鎖。但注意，重新上鎖後，因解鎖而觸發之一些安全熔斷（如三星之 Knox）或無法逆轉。

Q2：解鎖後何以 Root？

A：解鎖 BL 鎖只為第一步。Root 通常須：

1. 刷入自定義 Recovery（如 TWRP）。
2. 經由 Recovery 刷入 Magisk 等 Root 管理器來修補 boot 鏡像，遂得 Root 權限。

第二十五章：二手手機處理與數字遺產

嘿，諸位，經前二十餘章之“修煉”，信君已自一“數字小白”成長爲一名相當從容之手機使用者矣。然科技生活總有輪迴——新歡終會變舊愛。當君準備令老夥計“退役”時，君知何以體面、安全地告別乎？

一、舊手機之“安樂死”：安全出售或轉贈指南

核心原則：數據清理非“刪除”，而乃“覆蓋”。許多人以爲以照片拖進回收站、清空相冊遂萬事大吉，結果手機轉手後，新主人用數據恢復軟件一掃，老照片、聊天記錄、身份證照片全回來矣——此種“社死現場”每年皆於上演。

1. **自我”大掃除”**：完成終一次完整備份，退出所有賬號（iCloud、華爲/小米/三星賬號、Google 賬號），關閉”查尋吾之手機”功能——否則下一位使用者將爲激活鎖攔於門外。

2. **執行終極”格式化”**：現代手機之”恢復出廠設置”通常包含加密擦除——它銷燬之乃加密密鑰本身，密鑰一沒，整盤數據瞬間變爲亂碼。

3. **最後之”障眼法”（僅老設備需要）**：現代手機（iOS 8+、Android 6+）已默認全盤加密（FBE），恢復出廠設置將銷燬加密密鑰，數據理論上不可恢復，**無需此步驟**。僅對極老舊之非加密設備（2015 年前），方須於恢復出廠設置後用垃圾文件填滿存儲，再執行一次恢復出廠設置。

各品牌數據擦除路徑速查表：儘管原理相通，但每個品牌以“恢復出廠設置”藏之位置略有不同，關鍵差異如下。

品牌	擦除入口	關鍵差異與留意事項
華爲	設置 > 系統與更新 > 重置 > 恢復出廠設置	需先退出華爲賬號並關閉”查尋吾之手機”，否則將觸發激活鎖；純淨模式下或須密碼驗證。
小米 / Redmi	設置 > 吾之設備 > 恢復出廠設置	底部有”格式化虛擬 SD 卡”與”清除全部內容與設置”選項，二者皆要勾選；退出小米賬號前先關閉”查尋設備”。
OPPO / 一加	設置 > 其他設置 > 還原手機 > 徹底清除全部數據	ColorOS 提供”徹底清除”模式，將多次覆蓋存儲；需輸入鎖屏密碼確認。
vivo	設置 > 系統管理 > 備份與重置 > 清除所有數據	勾選”格式化手機存儲”方將清理 SD 卡區；需退出 vivo 賬號，關閉”查尋手機”。
三星	設置 > 通用管理 > 重置 > 恢復出廠設置	需先移除 Samsung 賬號並關閉”查尋吾之手機”；部分機型有”自動覆蓋”選項，建議開啓。
iPhone	設置 > 通用 > 傳輸或還原 iPhone > 抹掉所有內容與設置	須先退出 Apple ID，否則下一位用戶爲 iCloud 激活鎖攔住；iOS 15+ 支持”退出時保留 iCloud 數據”以備後查。

最容易被忽略之三件事

- **未退出賬號**：手機賣賜下家後爲遠程鎖定，自己亦下不來，終只能持身份證去官方售後解鎖。
- **SD 卡未取出**：恢復出廠設置未能格式化外置 SD 卡里之照片，記得手動取出或單獨格式化。
- **U 盤/OTG 外設未斷開**：擦除過程中若連着外設，或誤清外設數據，亦或導致擦除中斷。

二、數字遺產：君之社交靈魂何以安放？

手機裏之數據，早遂非“虛擬”之矣。它承載着汝之資金、社交關係、創作成果，乃至汝之“數字身份”。**一個人去世後，此資產該何以處理？**此不再乃一個抽象問題，而乃越來越多家庭真實面臨之難題——親屬不知道密碼、平臺不使繼承、加密貨幣永遠丟失。

法律依據：個人信息保護法第 49 條

2021 年施行之《中華人民共和國個人信息保護法》第 49 條明確規定：**自然人死亡之，其近親屬爲了自身之合法、正當利益，可對死者相關個人信息行使本章規定之查閱、複製、更正、刪除等權利；死者生前另有安排之除外。**
此意味着：

- **近親屬有法律依據**向平臺申請處理死者之賬號與個人信息。
- ”合法、正當利益”通常包括辦理遺產繼承、關閉賬號、刪除敏感隱私等。
- 死者生前若有遺囑或協議另行安排（如指定遺產聯繫人），自其約定。
- 平臺不可以”用戶隱私”爲由一概拒絕配合，但具體流程各家不同。

微信與支付寶：賬號不可繼承，但資金可以

此兩個中國人最常用之 App，規則各不相同：

- **微信賬號**：依據《騰訊微信軟件許可及服務協議》，賬號所有權歸騰訊，用戶僅享有使用權，**賬號本身不能繼承、不能轉讓**。但賬號內之**零錢、零錢通資金**屬於個人財產，近親屬可憑死亡證明、親屬關係證明、繼承公證書向財付通申請提現或轉移。
- **支付寶**：賬號同樣不可直接繼承，但支付寶餘額、餘額寶、理財通等**資金屬於可繼承財產**。繼承人可經由支付寶”客服熱線”或線下提交死亡證明、繼承公證書辦理；螞蟻財富類理財產品需走基金公司流程。
- **聊天記錄**：法律上屬個人隱私，平臺未能主動提供予家屬。如涉及遺產線索，可通過訴訟或公證申請調取。

切勿爲之事

- **勿以死者賬號密碼直接交予家人”遂用”**——此違反平臺協議，資金爲凍結甚至觸發風控時舉證更難。
- **勿嘗試用死者手機收驗證碼”提現”**——或爲認定爲盜用，反而觸犯法律。
- **勿拖延**：部分賬號長期未登錄將爲自動註銷，資金尋回將更麻煩。

Apple“遺產聯繫人”：預先指定，事半功倍

iOS 15.2 起蘋果推出“遺產聯繫人”功能，允許用戶提前指定信任之聯繫人，去世後該聯繫人可憑**死亡證明 + 遺產聯繫人訪問密鑰**申請訪問死者之 iCloud 數據（照片、備忘錄、通訊錄、備忘錄等）。操作步驟：

1. 於 iPhone 上進入 [設置] > [汝之姓名] > [登錄與安全] > [遺產聯繫人]。
2. 點擊”添加遺產聯繫人”，使用面容 ID 確認身份。

3. 選擇聯繫人（自通訊錄挑選），系統將生成一份**訪問密鑰**。
4. 將訪問密鑰通過短信、郵件或打印紙質件交給對方，並提醒對方妥善保存。
5. 可添加多個聯繫人作爲備份，避免單點失效。

爲指定人無需 Apple 設備，憑密鑰與死亡證明即可於蘋果官網申請訪問。整個流程通常於 7 天內完成。

Google“不活躍賬戶管理器”：自動處理身後事

Google 提供“不活躍賬戶管理器（Inactive Account Manager）”，能於用戶長期未登錄時自動執行預設操作——此乃目前最完善之數字遺產機制之一。

1. 訪問 **myaccount.google.com/inactive**，登錄 Google 賬號。
2. 設置超時期限：3 / 6 / 12 / 18 個月未登錄後觸發。
3. 添加受信任聯繫人（最多 10 人），並設置每位可訪問何數據（Gmail、相冊、雲端硬盤等）。
4. 可選：設置自動回覆郵件，告知來信者汝已無法處理。
5. 最後選擇：到期後是**分享數據給聯繫人**，抑或**直接刪除賬戶**。
6. 設置手機號作爲兜底驗證，避免誤觸發。

國內社交賬號：紀念化與註銷流程

國內主流社交平臺對逝者賬號之處理分兩種思路：**“紀念化”（保留賬號使親友緬懷）**和**“註銷”（徹底刪除）**。家屬可憑死亡證明、親屬關係證明、申請人身份證向平臺客服申請。

- **微博**：支持賬號” 紀念化”，由親屬提供死亡證明申請，賬號將爲鎖定無法登錄但內容保留，主頁將顯示” 該賬號已設置爲紀念賬號”。亦可申請註銷，需提供繼承公證。
- **抖音**：通過” 吾 - 設置 - 反饋與幫助 - 賬號與安全 - 賬號註銷” 或人工客服，提交死亡證明、親屬關係證明、申請人身份證，審覈通過後可註銷或轉爲紀念狀態。
- **B 站**：客服渠道提交逝者證明及親屬證明，可申請賬號註銷或保留爲紀念頁；UP 主之歷史稿件處理需結合版權聲明。
- **小紅書**：經由官方客服申訴，提交死亡證明、親屬關係證明，可申請註銷賬號並刪除內容。

關鍵提醒

- 各平臺規則將更新，辦理前先查閱官方客服最新指引。
- 所有證明文件建議先掃描備份再提交，原件妥善保管。
- 跨平臺一次性辦理時，可統一申請一份**繼承公證書**，省去反覆跑手續。

數字資產之繼承難題

除矣社交賬號，越來越多之人持有**真正有價值之數字資產**，其繼承問題更加複雜：

- **加密貨幣（比特幣、以太坊等）**：基於" 私鑰即所有權" 原則，**無私鑰遂等於資產消失**。繼承人無法通過法律文件向鏈上" 申訴"。建議：將助記詞/私鑰分片存放於多個物理安全位置（如銀行保險箱 + 家裏保險櫃），並於遺囑中註明訪問方式。
- **交易所賬戶**：與銀行賬戶類似，可憑繼承公證書向交易所（如幣安、OKX）申請處置，但跨境交易所流程漫長。
- **域名**：屬於可繼承財產，繼承人於域名註冊商（如阿里雲、GoDaddy）提交繼承公證後可辦理過戶。
- **知識產權（原創文章、視頻、專利、軟件著作權）**：著作權中之財產權利可繼承（人身權不繼承），收益權可由繼承人享有至作者死後 50 年。
- **雲存儲與網盤**：內容數據歸屬個人，可繼承；但部分平臺協議規定賬號本身不可轉讓，需先下載再註銷。

加密貨幣“消失”之真實代價

據 Chainalysis 報告，全球已有**超過 300 萬枚比特幣**因私鑰丟失而永遠無法尋回，相當於現有總量之六分之一。若汝持有加密資產，請務必使至少一位汝絕對信任之人知道**恢復路徑**——否則此財富將永遠沉睡於區塊鏈上。

一份“數字遺囑”模板

傳統之財產遺囑管不了數字資產。建議汝每年更新一份“數字遺囑”附件，與正式遺囑一併存放。以下模板供參考：

《個人數字資產處置附件》（模板）

第一部分：賬號清單（僅錄賬號名與屬平臺，**不寫密碼**）

- 社交類：微信（賬號 xxx）、微博、抖音、B 站、小紅書等
- 支付類：支付寶、微信支付、各銀行卡 App
- 雲端類：iCloud、華爲雲、小米雲、Google 賬號、微軟賬號
- 郵箱類：QQ 郵箱、網易郵箱、Gmail 等
- 其他：網盤、訂閱服務、遊戲賬號等

第二部分：密碼託管方案

- 主密碼存放方式：使用密碼管理器（如 Bitwarden、1Password），主密碼寫於紙質件中，封存於家中保險櫃。

- 恢復密鑰/助記詞：分片存放，至少兩處物理位置（如銀行保管箱 + 家中保險櫃）。
- 授權查詢人：姓名、身份證號、與本人關係、聯繫方式。

第三部分：處置意願

- 各類賬號希望” 紀念化” 抑” 註銷”。
- iCloud / Google 數據是否指定” 遺產聯繫人” 或” 不活躍賬戶管理器”。
- 加密資產由誰繼承，私鑰獲取路徑說明。
- 版權類作品收益由誰享有。

第四部分：聯繫人與平臺

- 信任執行人姓名、聯繫方式、備用聯繫人。
- 常駐平臺客服入口與所需證明材料清單。
- 本附件更新日期：_____ 年 ___ 月 ___ 日，下次複核日期：_____ 年 ___ 月 ___ 日。

關於” 密碼” 之法律提示

將密碼直接寫進遺囑存在泄露風險，且遺囑公示後所有繼承人皆能看到。**更穩妥之作法乃：遺囑裏唯寫 “賬號清單 + 處置意願 + 密碼託管位置”，真正之密碼通過密碼管理器主密碼單獨傳賜信任執行人**。此樣既符合法律要求，亦降低矣密碼泄露風險。

三、環保之最後一程

一部手機裏含金、銀、銅、稀土等數十種金屬，隨意丟棄既污染土壤，亦浪費資源。正確之回收渠道有三類，各有側重：

渠道	回收價	數據安全	適用場景
官方以舊換新（華爲/蘋果/小米等）	中等，常疊加補貼券	最高，官方有規範流程	方欲換同品牌新機時最劃算
三方回收平臺（愛回收、轉轉、回收寶等）	略高，可比價	較高，有數據清除報告	欲直接變現、不綁定品牌
公益捐贈（如鄉村學校、愛心機構）	無經濟回報	需自行徹底擦除	設備仍可用、欲作善事
發揮餘熱（自留改造）	省下新設備錢	原機主本人續用，無風險	設備性能尚可、家有改造需求

- **官方回收/以舊換新**：最推薦之方式，流程規範，數據安全有保障。蘋果、華爲、小米等品牌官網皆有” 以舊換新” 入口，評估後可抵扣新機款。

- **正規回收平臺**：選擇信譽良好之大型回收平臺，下單後順豐上門取件，檢測後打款。務必選擇**提供”數據清除報告”之**平臺，並留憑證。
- **公益捐**：可聯繫當地鄉村學校、社區公益組織，舊手機可改裝成教學平板或老人機使用。**捐贈前務必執行完整之擦除流程，並卸除所有 SIM/SD 卡**。
- **發揮餘熱**：改成車載導航儀、智能家居中控、監控攝像頭、寶寶監視器、電子相框。一臺舊手機可再戰好幾年。

電子垃圾之真相

一部手機之電路板裏含有**金、銀、鈀、銅**等貴金屬，每噸廢舊電路板含金量乃金礦石之數十倍。但若隨意丟棄，其中之鉛、汞、鎘將滲入土壤與地下水，污染持續數十年。**正確回收，既保護環境，也能使稀缺金屬循環再生**。

實操：爲君之舊手機行一場”標準葬禮”

1. 終一次完整備份：照片上傳雲端、重要文件複製至電腦、聊天記錄導出存檔。
2. 退出所有賬號：iCloud / 華爲 / 小米 / Google 賬號，關閉”查尋吾之手機”。
3. 取出 SIM 卡與 SD 卡，斷開所有 OTG 外設。
4. 執行加密恢復出廠設置，必要時手動勾選”格式化存儲”。
5.（僅老設備）用垃圾文件填滿存儲，再恢復出廠設置一次。
6. 擇它之下一站：官方以舊換新、正規回收平臺、公益捐贈，或轉贈家人。
7. 同步更新汝之”數字遺產清單”，以已淘汰之設備自清單中劃掉。

本月任務清單

1. 爲汝目前之主力手機指定 Apple”遺產聯繫人”或設置 Google”不活躍賬戶管理器”。
2. 用本節模板起草一份《個人數字資產處置附件》，與家人溝通存放位置。
3. 檢查家中是否有 2 臺以上閒置舊手機，預約一次官方或正規平臺回收。

第二十六章：當下手機技術前沿

聊完了舊手機之“身後事”，使我等以目光拉回當下。*2024-2026* 年乃手機技術之一個集中爆發期：*AI* 自雲端塞進矣口袋，衛星通信自旗艦下放至了中端機，摺疊屏自“嚐鮮”變成矣“常用”，電池技術亦迎來矣多年未見之材料級突破。此一章，我等遂來盤點此方發生之技術前沿，幫汝判斷“而今該不該換機”以及“換何機”。

一、AI 手機：乃營銷噱頭，抑真變革？

2024 年起，幾所有安卓廠商皆於旗艦機上打出“AI 手機”之旗號，蘋果亦於 iOS 18 中上線矣 Apple Intelligence。但到底何樣之手機方算“AI 手機”？業內大致形成矣三條判斷標準：

- **端側大模型**：手機本地能跑參數量於 7B 左右之語言模型（如 Qwen、Llama、DeepSeek 之輕量版本），不依賴雲端遂能完成文本摘要、翻譯、文案生成等任務。判斷方法：飛行模式下若還能用 AI 功能，則說明乃端側模型。
- **NPU 算力（TOPS）**：NPU（神經網絡處理單元）之算力以 TOPS（每秒萬億次操作）爲單位。2024 年旗艦芯片（如驍龍 8 Gen3、天璣 9300）之 NPU 算力於 40-70 TOPS 區間，2025-2026 年之新一代芯片普遍突破 100 TOPS，此乃端側跑大模型之硬件門檻。
- **AI Agent 場景**：非獨將” 問答”，還能跨 App 執行任務鏈——例如” 幫吾以昨天開將之照片整理成一個相冊並分享賜張三”，系統能自動調用相冊、文件、微信等多個 App 完成。此乃 AI 手機區別於” 帶 AI 助手之手機” 之核心標誌。

二、各廠商 AI 戰略對較：誰於認真作，誰於貼牌？

各家之 AI 路線差異明顯，選購時可重點關注：

- **蘋果 Apple Intelligence（iOS 18+）**：強調整合系統級應用——郵件摘要、通知優先級、寫作潤色、相冊 AI 修圖。端側模型爲主，複雜任務上雲至 Private Cloud Compute。優勢乃隱私保護與系統流暢度，劣勢乃中文場景上線較慢，部分功能 2025 年方逐步支持。
- **谷歌 Gemini Nano**：Pixel 8/9 系列內置，支持” 畫圈搜索”、Magic Compose、錄音摘要等。Gemini Nano 乃真正意義上之端側小模型，開源生態友好，乃安卓 AI 手機之參考實現。

- **實用示例：**：接入鴻蒙小藝，支端側文案生、AI 修圖、AI 消（去路人）、通話摘要。鴻蒙 NEXT 5.0 後進一步系統級整合，AI 能力與系統深融合。
- **小米澎湃大模型（MiLM）**：HyperOS 內置，支 AI 寫真、AI 摘要、AI 畫師、超分辨率。小米走” 端雲協同” 路線，端側奔輕量模型，重任務上雲。
- **OPPO（安第斯大模型）與 vivo（藍心大模型）**：ColorOS 15 與 OriginOS 5 深度接入，主打 AI 消除、AI 文檔摘要、AI 識圖問答、自然語言搜照片（” 尋去年於海邊拍之照片”）。

選購宜：若汝屢須 AI 處理工作文檔、整理信息，優先選端側算力強、AI 功能系統級整合之機型；若只為偶爾用 AI 修圖，主流中端機已夠用。

三、衛星通信：自旗艦“噱頭”至中端標配

衛星通信乃此兩年最實於之技術下沉之一。2022 年華爲 Mate 60 與 iPhone 14 首發時尚只為旗艦專屬，2025 年已下放至 2000-3000 元檔。

- **北斗短報文下放中端機**：北斗衛星之短報文功能（可發送 1000 漢字以內之短信 + 位置）已下放至 Redmi、realme、iQOO 等中端機型，無需月租，免費使用。適合戶外徒步、自駕遊等場景。
- **天通衛星通話**：華爲 Mate 60/70 系列、榮耀 Magic6 系列等支持天通一號衛星通話功能，可撥打/接聽衛星電話、發送衛星消息。須開通運營商衛星套餐（約每月 10-200 元），適合屢進無人區之用戶。
- **iPhone 衛星消息（SOS）**：iPhone 14 及後機型支衛星 SOS，可向緊急服構發文字消息並共享處。免費供 2 年服，部地區續費可用。
- **實用之議**：城市用戶基本用不上，但戶外愛好者、出差至偏遠地區之商務人士強烈建議選帶衛星功能之機型——關鍵時刻能救命。

四、折屏生態成熟度：能當主力機矣嗎？

摺疊屏已自“科技嚐鮮”走向“主力機可用”。判斷摺疊屏為否成熟，觀三點：硬件耐用性、App 適配度、交互生態。

- **硬件耐用性**：鉸鏈壽命普遍達 50 萬次以上（每天開合 100 次可用 13 年），水滴鉸鏈減少屏幕摺痕，IPX8 級防水已成標配。內屏 UTG 玻璃 + 保護膜方案使屏幕不再” 嬌貴”。
- **App 適配**：主流 App（微信、淘寶、抖音、B 站、Office、WPS）已針對大屏作適配，但仍有約 30% 之長尾 App 未適配，顯示爲拉伸或黑邊。建議購機前先試用汝常用之幾個 App。
- **懸停態交互**：摺疊至 90°-120° 時，部分 App 支持” 上半屏顯示內容、下半屏操作” 之懸停模式，適合視頻通話、自拍、觀片。相機懸停自拍乃摺疊屏獨有之體驗。

- **分屏與多任務**：大屏下可同時開兩個 App 分屏，甚至三個 App 小窗並存。HyperOS、ColorOS、OriginOS 之分屏邏輯已比較成熟，但 iOS（iPadOS 風格）之分屏於 iPhone 上仍未開放。
- **價格**：橫折（書本式）自萬元級下探至 5000-7000 元檔，豎折（翻蓋式）已下探至 4000-5000 元檔。2025 年部分廠商推出 3000 元檔豎折，摺疊屏真正進入大衆市場。

選購宜：若汝乃重度閱讀、文檔處理、視頻用戶，橫折屏值得考慮；若汝追求便攜與社交屬性，豎折屏更適合。建議至少去線下店以玩 30 分鐘再決定。

五、6G 與 Wi-Fi 7：下一代連之速度與邊界

雖 6G 尚沒真正落地，但相關技術已於路上矣：

- **6G 進展**：3GPP 計劃 2027 年完成 6G 標準制定，2030 年左右商用。6G 之理論峯值速率可達 1 Tbps（5G 之 50 倍），延遲低至 0.1ms，支持”通感一體化”（通信 + 感知），手機甚至能當低精度雷達用。目前各廠商還於原型驗證階段，離消費級產品尚遠。
- **Wi-Fi 7（802.11be）**：2024 年起成爲旗艦機標配，理論峯值速率達 46 Gbps（Wi-Fi 6 之 4 倍），支持 320MHz 頻寬、4K-QAM、MLO（多鏈路同傳）。實測於 Wi-Fi 7 路由器下，旗艦機傳輸速率可達 2-5 Gbps，比 Wi-Fi 6E 快 2-3 倍。若汝換矣 Wi-Fi 7 手機，記得配一臺 Wi-Fi 7 路由器才能發揮全部性能。
- **藍牙 5.4 與 LE Audio**：2024-2026 年新機普遍支持藍牙 5.4 及 LE Audio（低功耗音頻），可同時連兩副耳機聽同一首歌，Auracast 廣播音頻使多人於公共場合同步聽同一音源。

六、硅碳負極電池：續航終迎來材級突破

手機續航多年來爲詬病“一天一充”，2024 年起硅碳負極電池技術之普及使情況有矣實質改善。

- **技術理**：傳統石墨負極之理論容量約 372 mAh/g，硅之理論容量高達 4200 mAh/g（約 10 倍），但硅充放電時體積膨脹達 300%，容易粉化。硅碳負極通過納米硅-碳複合結構，以硅之膨脹問題控制於可接受範圍，使電池能量密度提升 15%-30%。
- **”請**：2024-2025 年採用硅碳負極之旗艦機（如小米 14 Ultra、榮耀 Magic 6 至臻版、vivo X100 Pro 等），於 7-8mm 機身內塞進 5000-6000 mAh 電池，比上代多出約 800-1000 mAh，重度使用可撐一天半。
- **百瓦速充配合**：硅碳負極對快充兼容性良好，100W/120W 快充 17 分鐘充滿已非新聞。2025 年部分機型支持 150W-240W，10 分鐘內充至 80%。
- **選購宜**：若汝乃重度用戶，優先選採用”硅碳負極 + 百瓦快充”組合之 2024 年後機型，續航焦慮將大幅緩解。

七、本章速覽：一句話切記此前沿

- AI 手機三標準：端側大模型、NPU 算力 TOPS、AI Agent 跨 App 場景。
- 衛星通信：北斗短報文下放中端，天通通話仍屬旗艦。
- 摺疊屏：硬件成熟、App 適配七成、懸停/分屏乃獨有體驗。
- 6G 還於路上，Wi-Fi 7 已落地，藍牙 LE Audio 開始普及。
- 硅碳負極 + 百瓦快充：續航與充電之雙突破。

來試試：賜汝之下一臺手機作一份”技術需求清單”

拿出一張紙，按“必備/加分/不須”三檔，列出汝對 AI、衛星、摺疊屏、Wi-Fi 7、快充、電池容量之需求。此樣下次去線下店時，導購推薦之“賣點”汝能迅速判斷乃真需求抑爲推銷。例如：

- 必備：5000mAh 上電池、80W 上速充、NFC、IP68 防水。
- 加分：端側 AI 摘要、Wi-Fi 7、北斗短報文。
- 不須：摺疊屏（吾習慣直板）、衛星通話（不出遠門）、240W 極限快充。

本章之要

- AI 手機三大判斷標準：端側大模型、NPU TOPS、AI Agent 場景
- 各廠商 AI 戰略：蘋果重隱私整合、谷歌端側開源、華爲/小米/OPPO/vivo 各有所長
- 衛星通信：北斗短報文下放中端之機，天通之話仍屬旗艦所專
- 摺疊屏：硬件已成熟，App 適配約 70%，懸停態與分屏乃獨有體驗
- 6G 標準 2027 年制定，Wi-Fi 7 已普及，藍牙 LE Audio 始落地
- 硅碳負極電池使 5000-6000mAh 成爲旗艦標配，續航焦慮大幅緩解

五、實戰篇（第 27-32 章）

第二十七章：父母亦能輕鬆用——適老化手機完全指南

曰句實話，寫此一章之時，吾欲至了我爸媽。我媽每次手機彈出一個“系統更新”，她皆不敢點，怕將手機弄壞矣。我爸之微信字體比螞蟻尚小，但他不知道何以調大，遂此麼眯着眼睛觀了兩年。此非彼等“笨”，乃手機默認以所有人皆當成矣二十多歲之年輕人。此一章，吾用最直白則，以長輩等手機使用中最頭疼之問題一個個解開。

零、先以心態擺正：此非“老人用不來”，乃“手機沒作好”

於作任何設置之前，有一個觀念要先曰清楚：**而今之智能手機界面，自設計之初遂沒何以考慮老年用戶**。彈窗小得觀不清、操作手勢須精準觸碰、圖標長得皆差不多——此非用戶之問題，乃產品之問題。故帶長輩用手機，第一件事非教，乃幫彼等**以環境調好**。環境調好了，大半之問題遂消失矣。

帶長者以手機有三條鐵律：(1) **勿催**——汝用矣十年之智能機，彼等或方用矣三個月。(2) **用類較**——“Wi-Fi 遂像家裏之水管，開矣水龍頭方有水”，此種比喻比專業術語好理解一萬倍。(3) **寫下來**——口頭教一遍彼等或忘矣，汝拿張紙將關鍵步驟寫下來貼於他手機殼背面，比何皆管用。

一、首步：開“簡易模式”——手機變清爽矣

而今之手機基本皆內置矣一個“簡易模式”（有之叫“關懷模式”“老年模式”“長輩模式”），打開之後，整個手機皆將大變樣：

- **字體與圖標變得甚大**——再亦不用戴老花鏡觀手機矣。
- **桌面圖標大幅精簡**——唯保留電話、短信、相機、相冊、微信等最常用之幾個 App，其他全收起來。
- **操作變簡單矣**——甚多花裏胡哨之功能（負一屏、智能助手、浮窗等）默認關閉，界面主打一個清爽。
- **聲音變大矣**——來電鈴聲、通知音量、通話聲音皆自動調至最大。

各品牌簡易模式開啓方法（直接照着操作遂行）：

品牌	模式名稱	開法
iPhone	輔助用（無單獨簡易模式，但可逐項調大）	設置 > 顯示與亮度 > 文字大小，拉至最大。再進”輔助功能 > 顯示與文字大小”開”粗體文本”“按鈕形狀”
電荷泵	簡易模式	設置 > 系統與更新 > 簡易模式 > 開啓
小米/紅米	至簡模式	設 > 益多設 > 至簡模式 > 開啓
OPPO/一加	簡易模式	設 > 特色能 > 簡易模式
vivo/iQOO	簡易桌面	設 > 桌面、鎖屏與壁紙 > 簡易桌面
三星	簡易模式	設 > 顯 > 簡易模式

若汝用之乃其他品牌，於手機“設置”頂部之搜索框裏直接搜“簡易”或“長輩”兩個字，一般皆能尋到。尋不至則，按下面之“手動調大”方案一條條設置，效果亦一樣。

二、若不開簡易模式，手調亦可——“大字 + 大圖標”五步走

有些長輩不欲用簡易模式，覺得界面太“老氣”。那遂手動調以下五項，效果一樣好：

1. **字體調至最大：**設置 > 顯示 > 字體大小，拖至最右邊。順便以”顯示大小”（有之叫”顯示縮放”）亦調至最大，此樣所有界面元素皆與着變大。
2. **圖標變大：**於主屏幕雙指捏合（像縮小照片那樣）進入桌面設置 > 桌面佈局（或圖標大小），選格子最少之佈局（比如 3×5），圖標自然遂大矣。
3. **開”高對較度文字”：**使文字邊緣更銳利，於”輔助功能”裏可尋到此開關。
4. **關閉不必要之通知：**設置 > 通知，以除矣電話、短信、微信之外之 App 通知全關掉。此樣手機未能一將兒響一下，未能使人覺得煩。
5. **以常用聯繫人放至桌面：**長按桌面空白處 > 添加小工具 > 聯繫人，以老伴、兒女、醫生、緊急聯繫人之快捷撥號放至桌面上。點一下遂撥出去，不用翻通訊錄。

動手：助長者”裝修”一下手機

1. 打開簡易模式（按上面之品牌表格操作）。
2. 以電量數字顯示打開（設置 > 電池 > 電量百分比），使彼等隨時知道尚剩幾何電。
3. 調出通知欄，以藍牙、NFC、飛行模式此長輩用不着之快捷開關移除或藏起來。
4. 於桌面上放四個快捷撥號：老伴、兒女、家庭醫生（若有）、社區/物業。

三、防詐——此章請必帶着長者共觀

老年人乃電信詐騙之重災區。非彼等警惕性低，而乃騙子則術經過矣精心設計，專門針對老年人之心理弱點。以下乃吾整理之最常見騙局類型與應對方法，建議汝直接念賜長輩聽，或打印出來貼於電話旁邊。

六大常見局（聞之矣即刻掛電話）：

- **(1) ” 汝孫子出事矣，趕緊匯錢”**——騙子將曰” 汝孫子/孫女於外面闖禍矣，爲警察抓矣，須保證金”。記住：**任何要求轉賬才能” 放人” 之電話皆乃詐騙**。公檢法未能電話要錢。
- **(2) ” 汝之醫保卡爲停用矣”**——短信裏曰醫保卡異常，使汝點鏈接” 激活”。記住：**醫保局、社保局未能發短信使汝點鏈接**。遇至此種情況直接打 12333（社保官方熱線）覈實。
- **(3) ” 高益理財，穩賺不賠”**——微信羣、短信裏薦各種” 養舊理財”” 國家扶持項目”，承諾高額報。切記：**承諾年化收益超過 5% 尚” 保本” 之，99.9% 乃騙局**。
- **(4) ” 吾乃汝領導/老朋友，換號碼矣”**——冒充熟人借。切記：**打電話覈實比何皆管用**，曰” 不方便接電話” 之八成乃騙子。
- **(5) ” 免費領雞蛋/保健品講座”**——線下免費活動誘導老人聽” 健康講座”，而後賣天價保健品。記住：**天上不掉餡餅，免費活動背後皆乃生意**。
- **(6) ” 汝之手機中毒矣，須遠程修復”**——騙子使汝下載一個遠程控制 App（如向日葵、TeamViewer），而後遠程操控汝之手機轉走錢。記住：**絕對勿使任何人遠程操控汝之手機！**手機廠商、通信運營商、銀行皆未能賜汝打電話使汝裝遠程控制軟件。

三道防線，立好之後能擋掉 90% 之詐騙：

1. **首道：手機自帶攔截**——安卓手機打開” 騷擾攔截”（於撥號 App 或手機管家 App 裏），iPhone 打開” 靜音未知來電”（設置 > 電話 > 靜音未知來電）。
2. **次道：運營商攔截**——發送短信”KTFSR” 至 10086，免費開通移動之高頻騷擾電話攔截；電信聯通也有類似服務。打運營商客服電話使人工幫汝開通遂行。
3. **三道：國家反詐中心 App**——於應用商店下載” 國家反詐中心”App，註冊並開啓來電預警。此 App 與公安之數據庫聯動，騙子號碼一撥過來遂能識別。

賜長輩之” 防騙口訣”（打印出來貼電話旁邊）

- **陌生電話要轉賬掛掉！**
- **短信鏈接要汝點刪掉！**
- **曰高收益保本之拉黑！**
- **使汝裝遠程控制軟件直接掛！**
- **真警察未能電話辦案更未能要錢！**

- **拿不準之事，先賜兒女打個電話問問。**

四、遠程協助——汝身於異地亦能助父母修手機

有時候長輩遇至之問題，汝於電話裏描述半天彼等亦尋不至對應之按鈕。此時候“遠程協助”功能遂派上用場矣——汝可直接於自己之手機上看到彼等手機之屏幕，幫彼等操作。各品牌之方案不一樣：

- **電荷泵**：自帶”暢連”App，裏面有”屏幕共享”功能。汝與爸媽皆裝暢連，發起視頻通話後點擊”共享屏幕”，汝就可於彼等屏幕上畫圈標註，告訴汝媽媽”點此裏”。
- **小米/紅米**：小米視頻通話 App 裏有”遠程協助”，支屏幕共享 + 語音通話同步。
- **OPPO/vivo**：內置”家庭空間”或”遠程守護”，於設置裏搜索”遠程”遂能尋到。
- **通用方案（所有品牌皆可用）**：下載”向日葵遠程控制”App（免費版夠用），汝等各裝一個。註冊同一個賬號或加好友，使爸媽將手機上顯示之驗證碼念賜汝，汝遂能遠程看到與操作彼等之屏幕矣。
- **iPhone 之用戶**：FaceTime 通話中支持”共享屏幕”。於 FaceTime 通話時，點擊屏幕共享按鈕，對方遂能看到汝之屏幕。但 iOS 安全限制無法遠程操作，只能”指揮式教學”。

五、微信之“適舊改”——與長者曰此幾句遂夠矣

微信乃長輩等用得最多之 App，但界面太密太雜。以下乃幾個立竿見影之調整：

- **調大微信自己之字體**：微信 > 吾 > 設置 > 通用 > 字體大小，拖至最大。注意，此唯影響微信內部之字體，不影響手機系統字體。
- **開”關懷模式”**：微信 > 吾 > 設置 > 關懷模式 > 開啓。打開後字體與按鈕皆將變大，色彩對比度加強，還能一鍵”聽文字消息”（點一下文字消息遂能朗讀出來）。
- **幫彼等備註好重要聯繫人**：以兒女、親戚之名字於通訊錄裏改成容易認之——比如”大兒子-小明”“二女兒-小紅”，使彼等一打開微信遂知道誰乃誰。
- **教彼等用”語音輸入”而非打字**：聊天框旁邊之鍵盤上，按住空格鍵（安卓）或點麥克風圖標（iPhone），直接說話遂能變成文字發送。識別準確率已甚高矣，對未能拼音打字之老人來曰乃”質變級”之便利。
- **關閉”發現頁”裏不用之功能**：微信 > 吾 > 設置 > 通用 > 發現頁管理，以”朋友圈”“視頻號”“直播”“購物”“遊戲”裏不用之全關掉，減少幹擾。

六、手機丟矣何以辦？助長者作個“應急預案”

預則立，不預則廢。趁手機尚沒丟，先以此幾件事作好：

1. **確認”查尋吾之手機”已打開**：iPhone 於設置 > 汝之名字 > 查尋 > 查尋吾之 iPhone；安卓於設置搜”查尋設備”。務必確認此開關乃亮着之。
2. **以重要之密碼與賬號記於一張紙上**：Apple ID / 華爲賬號 / 小米賬號之密碼，鎖於家裏抽屜裏一份，賜信得過之兒女一份。丟矣手機之後要登錄此賬號來遠程鎖定與定位，若忘矣密碼遂麻煩矣。
3. **教長者一句話：”手機丟矣，第一時間賜家人打電話。”**家人可立刻登錄 iCloud 或安卓之查尋設備網頁，遠程標記爲”丟失之模”，此樣撿至之人亦打不開、用不了。

七、無線耳機與助聽器兼容——爲忽視之好能

甚多長輩有聽力問題，但不知道手機可與助聽器協同工作。iPhone 與部分安卓手機支持 MFi（Made for iPhone）助聽器，可直接以電話聲音傳至助聽器裏，比舉着手機聽筒清楚太多矣。

- **連法**：iPhone 設置 > 輔助功能 > 助聽設備，打開藍牙，按照助聽器之配對說明操作。
- **若不用助聽器**：於”輔助功能 > 音頻/視覺”裏打開”電話降噪”與”單聲道之音”，能改善通話清晰度。
- **安卓**：於”輔助功能 > 聽力增強”裏有類似設置，各品牌位置略有差異，直接搜”助聽”兩個字。

八、健康相關能——手機不僅乃通訊工具

- **SOS 緊急求助**：連續按電源鍵五次（大多數手機），手機會自動撥打緊急電話並賜預設之緊急聯繫人發送定位。幫長輩提前設置好——iPhone：”設置 > 緊急 SOS”，安卓：”設置 > 安全 > 緊急求助”。教長輩用之前先演習一遍，免得到時候緊張按不準。
- **跌倒檢測**：Apple Watch 有此功能，檢測至汝摔倒且一段時間不動，將自動撥打急救電話並通知家人。有預算則賜長輩配一個，關鍵時刻能救命。
- **用藥提醒**：iPhone 之”健康”App 內置用藥提醒功能，設好藥名、時間與劑量，到點遂彈出提醒。安卓可用”定時鬧鐘”或下載”喫藥提醒”類 App。

本章之要

- 先開簡易模式或手動調大字體/圖標，環境調好大半問題遂沒矣
- 帶長輩用手機三鐵律：勿催、用類比、寫下來

- 六大常見局要屢提醒，三道防線立好能擋 90% 詐
- 遠程協助使汝人於外地也能幫爸媽修手機
- 微信開” 關懷模式”+” 語音輸入” 乃長輩最實用之兩個功能
- SOS 緊急求助與跌倒檢測，關鍵時刻救命之配置值得花五分鐘設好

第二十八章：花對錢買至心儀機——手機選購終極指南

買手機此件事，甚多人將提前觀一週評測、比一個月價格，結果終抑稀裹糊塗花五六千買矣臺自己實則用不上之旗艦。此章不列參數表——彼汝於官網皆能看到。吾欲作之，乃賜汝一套“選機思維”，以後無論市場上出矣何新手機，汝皆能自己判斷它適不適合汝。

零、三步選機法：需預算機型

甚多人於網上問“三千塊買何手機好”，此問題實則沒法回答——蓋因不知道汝買手機主要用來幹何。買手機之正確思考順序宜乃：

選機三板斧

1. **先欲清楚汝最於乎何**：拍照？打遊戲？續航？屏幕？系統流暢？抑唯須能打電話發微信遂行？以需求排個序。
2. **復定預算上限**：非”約幾何錢”，乃”最多能接受花幾何錢”。定一個硬上限，選機之時未能越觀越貴。
3. **終方挑機型**：拿着汝之需求排序與預算，去匹配機型。

一、核心參數何以觀——不用全懂，觀懂此五項遂夠矣

參數表動輒幾十項數據，實則普通用戶唯須關注此五個維度：

維度	何以觀	一句話總
處器（SoC）	決定了手機能流暢用幾年。2026 年之標準：驍龍 8 Elite Gen5/天璣 9500 以上 = 頂級；驍龍 8s Gen4/天璣 8400= 中高端；驍龍 7+ Gen4/天璣 7350= 夠用三年。低於驍龍 6 系之建議慎重。	買新不買舊，SoC 決定了汝手機能用幾年不卡
屏幕	OLED 比 LCD 色彩好、更省電；刷新率 120Hz 比 60Hz 滑屏流暢甚多；分辨率至少 1080P 以上。高頻 PWM 調光（1920Hz 以上）對眼睛更友好。	OLED+120Hz+ 高頻調光 = 好屏幕之及格線
攝像頭	**不觀像素數觀傳感器尺寸**。底大一級壓死人，傳感器尺寸比像素重要得多。超廣角實用；長焦（3x 以上光學變焦）拍遠處之東西；微距鏡頭多半乃湊數之。	大底主攝 + 實用長焦 > 多顆湊數鏡頭
電池與充電	4500mAh 乃基礎及格線，5000mAh 以上一天一充甚穩。快充觀瓦數：60W 以上算快，100W 以上飛快。無線充電方便但慢。	5000mAh+60W 速充 = 不焦慮組合
系統更新承諾	決定了手機能安全用多久。蘋果一般賜 5-6 年系統更新；三星旗艦賜 4 年系統 +5 年安全更新；谷歌 Pixel 賜 7 年；國產中端機更新承諾普遍偏短（2-3 年）。	欲用久一點，選系統更新承諾長之品牌

二、按預算分檔薦（以 2026 年中市場爲基準）

以下推薦基於 2026 年中國內地市場行情，價格爲官方起售價。具體價格將有浮動，以汝入手時之實際價格爲準。

1000 元以下：能打電話，能刷微信，夠用遂行

適誰：學生、備用機、對手機要求不高、主要用以打電話與微信。

能買至何：Redmi Note 系列、榮耀 X 系列、realme 數字系列之中低配。處理器約驍龍 6

系或天璣 6000 系列，屏幕乃 LCD 爲主，拍照能拍清楚但對焦慢，系統流暢度一般。

選購宜：此價位觀兩點：電池夠不夠大（選 5000mAh 以上之），內存夠不夠（選 6GB+128GB 起步，勿買 4GB 之將卡）。紅米於此價位優勢最明顯。

1000-2000 元：性價比最高，能用三四年不卡之甜點區

適誰：學生、預算有限但欲用得久、對拍照與遊戲無極致追求。

代表機型：Redmi Turbo 系列、iQOO Z 系列、榮耀 X 系列高配、一加 Ace 系列入門。

選購宜：選搭載驍龍 7+ Gen3 或天璣 7300 以上處理器之機型，配 8GB+256GB 內存。此配置日常使用 + 偶爾打王者榮耀完全夠用，正常能用 3-4 年不卡。

2000-3500 元：中堅力量，性能與拍照皆有明顯提升

適誰：想要不錯之拍照、打遊戲不卡、希望用得久之普通用戶。此乃最多人買之價格區間。

代表機型：小米數系列（如小米 17 標準版降價後）、vivo S 系列高配、OPPO Reno 系列、榮耀數系列、Redmi K 系列。

選購宜：此價位最大之特點乃“水桶機”——無明顯短板。比起 2000 元以下之機子，主要提升於：相機明顯變好（有光學防抖矣）、屏幕上矣高素質 OLED、快充更快（80W 以上常見）、作工質感亦更精緻。選 12GB+256GB 或 12GB+512GB，用四五年沒問題。

3500-5000 元：準旗艦，除矣極客玩家其他人完全夠用

適誰：對拍照有追求之用戶、欲體驗頂級質感之用戶。

代表機型：小米數系列標準版、vivo X 系列、OPPO Find X 系列標準版、iPhone 舊款旗艦（如 iPhone 16）。

選購宜：此價位已能買至去年之旗艦機型。吾之建議乃：**買去年之旗艦，不買今年之中端**。去年旗艦之用料、作工、攝像頭硬件通常比今年之中端好一個檔次。前提乃汝能接受它非“最新款”。

5000 元上：旗艦區，不差財或對某項體驗有極致求

適誰：攝影發燒友、重度遊戲玩家、追求頂級屏幕與工藝之用戶、用手機拍視頻作內容之創作者。

代表機型：iPhone 17 Pro/Pro Max、三星 S26 Ultra、華爲 Pura 80 Ultra、小米 17 Ultra、vivo X300 Pro。

選購宜：此價位各家之差距越來越小，實則主要觀汝喜歡何系統。喜歡拍照選 vivo 華爲，喜歡拍視頻選 iPhone，喜歡屏幕與 SPen 選三星，喜歡均衡選小米。說實話至了此價位，**汝可任性，選汝觀着順眼之那臺遂行**。

三、按場景選機——汝之需乃何？

以下乃幾個最常見之“使用畫像”，對號入座遂行：

使用之場	重點觀何	薦
拍照達人	大底主攝（1 英寸最佳）、光學防抖、長焦（3x 上光學變焦）、影像芯片	vivo X 系列、華爲 Pura/Mate 系列、小米 Ultra 系列
遊戲玩家	頂級處器、高觸控採樣率、散熱系統、大電池 + 速充	iQOO 數系列、ROG Phone、紅魔、Redmi K 系列
續航優先	5500mAh 以上電池、能效好之處理器、LTPO 屏幕	榮耀 Magic 系列、一加、Redmi Note 大電池版
長輩使用	簡易模式、大字體、大音量、大電池、售後網點多	華爲暢享、榮耀 Play、OPPO A 系列、Redmi Note
學生用機	性價較高、耐用、電池大、有家長控制能	Redmi Note/Turbo、realme GT、iQOO Neo
蘋果生態用戶	已有 Mac/iPad/AirPods，買 iPhone 乃唯一解	iPhone 17 標準版（性價較）或 Pro 版（滿血體驗）
出國旅行	eSIM 支、全球頻段覆蓋全、Google 服	iPhone、三星 S 系列、Pixel、一加

四、五個容易踩之坑——買之前對照觀一眼

1. **坑一：四攝五攝不等拍照好**——廠商喜歡於參數上堆鏡頭數量，裏面屢有兩顆乃 200 萬像素之” 湊數鏡頭”（微距 + 景深），實際完全沒用。觀手機拍照能力，**只觀主攝 + 超廣角 + 長焦**此三顆遂夠。其他多出來之可當它不存在。

2. **坑二：勿爲 1 億像素唬住**——像素高不代表畫質好。傳感器尺寸、光圈、算法調教比像素重要得多。甚多旗艦機用 5000 萬像素之大底傳感器，拍出來之照片比某些 1 億像素之千元機好太多。

3. **坑三：勿買運營商定製機**——” 存話費送手機” 看起來甚劃算，但運營商定製機系統更新慢、或有刪除不了之預裝 App、且通常乃運營商鎖定之（只能用此家運營商之卡）。看似省矣錢，實則” 鎖定” 矣汝。

4. **坑四：” 曲面屏甚高級”——乃營銷，非真理**——曲面屏曾經乃旗艦機之標緻，但而今越來越多人發現它之問題：貼膜難、邊緣誤觸、碎矣維修貴。2026 年之趨勢乃迴歸” 微曲” 甚至” 純直屏”。勿蓋因” 曲面之即高級之” 而選曲面屏。

5. **坑五：非所有”AI 手機” 皆真之有用**——2026 年所有廠商皆於喊”AI 手機”，但甚多隻為以已有之語音助手換矣個名字。真正有用之 AI 功能乃：實時翻譯、AI 消除（拍照以人去掉）、AI 修圖、端側大模型推理、本地化之智能摘要。其他之花哨功能基本皆乃噱頭。

五、何時買最劃算？

- **618（5 月底至 6 月中旬）**：一年中手機降價幅度最大之時段。甚多手機廠商爲了沖銷量，旗艦機能降價 800-1500 元。乃上半年買手機之最佳窗口。
- **雙 11（10 月底至 11 月中旬）**：下半年之大促，力度與 618 差不多。此時間段甚多品牌新一代旗艦剛發佈，上一代旗艦降價幅度最大。
- **新機發布後一個月**：剛發佈時原價，一個月後渠道價（第三方店鋪）一般將便宜 200-500。
- **春節前兩週至元宵節**：線下實體店清庫存之”年貨節”，可與店員談價。
- **不推薦之時間**：新品發佈將後當天衝首發——除非汝真之特別想要當第一批用戶，否則首發價乃最貴之。

六、渠何以選——於哪買靠譜？

- **品牌官方商城/天貓旗艦店/京東自營**：最保險，價格沒優勢但不用擔心翻新與假貨，售後最好。
- **拼多多”百億補”**：價格最低，但須選標註矣”百億補貼”之店鋪。沒標註之勿碰。收到貨記住錄開箱視頻，有問題能退貨。
- **線下實體店**：可親手摸、試手感。注意：導購將推利潤高之機型（而非最適合汝之），建議進店之前先於網上鎖定 2-3 款目標機型，去矣遂曰”吾只欲觀此幾款”。
- **二手平臺（閒魚/轉轉）**：便宜但有風險。若要買二手，記住三條：要求提供購買發票或訂單截圖、要求提供 IMEI 號查驗、**唯走平臺交易，絕不私下轉賬**。

動手：用此套法選一臺手機

1. 拿一張紙，寫下汝之兩個核心需求（比如：續航第一、拍照第二）。
2. 定一個預算上限（較如 3500 元）。
3. 開京東或天貓，按價篩，圈出 3-5 款備選機型。
4. 去 B 站搜”XX 機型一個月使用體驗”（非第一天評測，乃用矣至少一個月之後之真實感受），觀兩三個視頻。
5. 確渠靠譜後下單，收貨時錄開箱視頻。

本章之要

- 選機三板斧：先欲需求再定預算終挑機型
- 核心參數只觀五項：處理器、屏幕、攝像頭、電池充電、系統更新承諾
- 1500-3500 元乃性價比最高、最推薦之甜點區間

- 買去年之旗艦比買今年之中端更劃算
- 五大坑：多攝局、高像素陷阱、運營商定製、曲面屏迷信、AI 噱頭
- 618 與雙 11 乃一年中最適合買手機之兩個時間窗口

第二十九章：數字健康——與手機建立健康關係

汝有無此樣之體驗：本來只欲刷 5 分鐘短視頻，結果一抬頭已過去矣兩小時？或晚上躺於牀上刷手機，越刷越精神，終凌晨兩點方睡着？手機改變矣吾等之生活，但亦於悄悄吞噬吾等之時間與健康。此一章非使汝“戒掉手機”，而乃幫汝重新奪回對手機之主動權。

一、先觀汝之“屏幕時”——數使人清醒

於改變之前，先搞清楚現狀。手機裏有一個甚多人自未打開過之功能，叫“屏幕用時”（iPhone）或“數字健康”（安卓），它能告訴汝每天到底於手機上花矣幾何時間，以及此時間花於矣何 App 上。

- **iPhone**：設置 > 屏幕使用時間 > 查看所有活動。汝將看到每天之解鎖次數、拿起手機之次數、每個 App 之使用時長。
- **安卓**：設置 > 數字健康與家長控制。華爲叫” 健康使用手機”，小米叫” 屏幕時間管理”。功能類似，皆能看到每日使用數據。

第一次觀此數據之人，80% 將爲嚇至。吾有個朋友觀完之後發現自己每天於抖音上花矣 4 個多小時，一週即 28 小時——相當於一份兼職之時間。此種“數字衝擊”乃改變之第一步。

二、賜 App 設“用限額”——賜己裝個限速器

知道問題於哪之後，就可有針對性地設置限制。不須一刀切以 App 刪矣，唯須賜它設個“每日額度”：

1. **iPhone**：設置 > 屏幕使用時間 > App 限額 > 添加限額。選中汝要限制之 App（比如抖音），設置每天 30 分鐘。至時間後 App 圖標將變灰，打開須額外輸入密碼。
2. **安卓**：設置 > 數字健康 > 應用計時器。選中 App 後設置每日使用上限，至時間後 App 圖標將變灰。
3. **小術**：以限額密碼設成一個汝不常用之、不太好記之密碼。太容易輸則，汝將忍不住每次皆” 續命”，限額遂形同虛設矣。

三、“勿擾之模”之正用法——非關掉告，乃關掉焦慮

甚多人之手機一天響幾十次通知，每響一次注意力遂爲打斷一次。汝以爲汝只為“觀一眼通知”，實際上每次打斷後重新集中注意力須 15-20 分鐘。勿擾模式非使汝“失聯”，而乃幫汝重新掌控注意力。

- **定時開啓：**設置勿擾模式於晚上 10 點至早上 7 點自動開啓。此樣睡覺時手機未能蓋因一條垃圾短信以汝吵醒。
- **許要聯繫穿透：**於勿擾模式設置裏，以” 允許來電” 設爲” 藏聯繫人” 或” 重複來電”（3 分鐘內第二次來電將正常響鈴）。此樣汝未能錯過家人之緊急電話，但亦未能爲推銷電話打擾。
- **工作專注模式：**iPhone 之” 專注模式” 可設自定義場景——比如” 工作” 模式下唯允許工作相關 App 通知，” 閱讀” 模式下所有通知全靜音。

四、護眼不靠 App 靠習慣——20-20-20 法則

長時間觀手機屏幕，眼睛乾澀、酸脹乃必然之。護眼模式（藍光過濾）有一定幫助，但真正有效之乃改變用眼習慣：

20-20-20 法則

每觀屏幕 **20 分**，以視線移至 **20 英尺（約 6 米）**以外之處，保持 **20 秒**。此簡單之動作能有效緩解睫狀肌疲勞。

- **開啓護眼模式：**設置 > 顯示 > 護眼模式（或夜覽模式），開啓後屏幕色溫變暖，減少藍光。建議設爲日落至日出自動開啓。
- **調屏幕亮度：**環境暗時屏幕亦要暗。甚多人於黑暗中觀手機亮度調太高，此乃傷眼最嚴重之行爲。開啓” 自動之亮” 並確保它正常工作。
- **持距離：**手機距離眼睛至少 30 釐米。甚多人喜歡將手機拿得特別近觀，此樣睫狀肌始終於緊張收縮狀態，近視加深甚快。

五、“低頭輩”之頸椎自救指南

低頭觀手機時，汝之脖子承受之壓力乃正常站立時之 5-6 倍。長期低頭將導致頸椎曲度變直、骨質增生、頭暈手麻。以下乃幾個簡單之改善方法：

- **把手機舉高：**觀手機時儘量將手機舉至與眼睛平齊之高度，而非低頭觀放於腿上之手機。雖手臂將累，但脖子將感謝汝。
- **每 30 分活動一下脖子：**慢慢向左轉頭至極限，停 3 秒，回正；再向右轉，停 3 秒。重複 5 次。而後作” 米字操”——用下巴於空中寫一個” 米” 字。

- **靠牆站立 3 分**：後腦勺、肩胛骨、臀部、腳後與四點貼牆，每天站 3 分鐘，能有效糾正含胸低頭之體態。

六、睡眠與手機——臥室裏最該改之習慣

手機屏幕之藍光將抑制褪黑素分泌，使汝“越觀越精神”。甚多人覺得“吾刷着刷着遂困矣”，實際上乃爲藍光欺騙矣大腦，等汝放下手機反而不容易入睡。以下乃改善建議：

- **睡前 1 小時不觀手機**：此乃最有效亦最難作至之一條。若作不至，至少開” 夜覽/護眼模式” 以藍光降至最低。
- **手機放於伸手夠不至之處**：以充電器放於臥室另一頭，而非牀頭櫃。此樣汝既未能忍不住拿起來刷，鬧鐘響矣汝亦得起牀才能關掉——一舉兩得。
- **開啓” 睡眠模式”**：iPhone 之” 睡眠” 專注模式可自動於睡眠時段鎖定屏幕、簡化鎖屏；安卓各品牌也有類似功能。
- **用鬧鐘代替手機**：買一個十幾塊錢之實體鬧鐘，不用手機當鬧鐘，汝將發現少矣” 觀一眼時間而後忍不住刷一下” 之陷阱。

七、信息焦慮症——學會“選擇性失明”

我等每天接收之信息量，乃古人一輩子接收之量。但人之大腦非爲處理此麼多信息設計之，信息過載將導致焦慮、注意力渙散、決策疲勞。解決方法非“獲取更多信息”，而乃“主動屏蔽”：

- **取消關注**：打開汝關注之所有公衆號、UP 主、博主列表，問自己一個問題：” 過去一個月吾從中獲得過實際價值嗎？” 若無，取消關注。信息渠道貴精不貴多。
- **閉非必要 App 推**：除矣微信、電話、短信、日曆之外，其他 App 之通知全關掉。新聞不須推送——汝主動去觀之時自然將看到。
- **設立” 信息齋戒日”**：每週選一天（比如週日），全天不觀手機新聞、不刷短視頻。汝將發現天未能塌下來，且週一精神狀態明顯更好。

動手：一週數健康挑戰

1. 今天：打開屏幕使用時間/數字健康，查看汝上週之使用數據。
2. 明天：賜耗時最長之娛樂 App 設置每日限額（建議 30-60 分鐘）。
3. 後天：設勿擾模式定時開啓（22:00-07:00），許藏聯繫人來電。
4. 第四天：睡前 1 小時不觀手機，手機充電器放至臥室另一頭。
5. 第五天：取消關注 5 個汝不再觀之公衆號或博主。
6. 第六天：嘗試一整天之” 信息齋戒”——不觀新聞不刷短視頻。

7. 第七天：查看本週之屏幕使用時間報告，與上週對比。

本章之要

- 先觀屏幕使用時間數據，認清現狀乃改變之第一步
- 賜娛樂 App 設限額，賜勿擾模式設定時，重新掌控注意力
- 20-20-20 法則乃最簡單有效之護眼習慣
- 低頭觀手機時脖子承重 5-6 倍，舉高手機 + 定期活動脖子
- 睡前 1 小時不觀手機，充電器放至伸手夠不至之處
- 信息焦慮之解藥乃”主動屏蔽”，非”獲取更多”

第三十章：投屏、演示與車載互聯

手機屏幕再大亦遂 *7* 英寸，但汝之電視有 *55* 英寸、汝之投影儀有 *100* 英寸。將手機畫面“扔”至大屏幕上，體驗乃完全不同維度之。此一章講清楚投屏、演示與車載互聯此三個高頻場景，幫汝將手機之能力延伸至更大之屏幕上。

一、無線投屏三大陣營——勿搞混矣

無線投屏看起來皆乃“手機畫面至電視上”，但底層技術有三種，互不通用：

技術	誰用	最高速率
AirPlay	iPhone/iPad/Mac	蘋果生態，體驗最流暢。須 Apple TV 或支持 AirPlay 之電視
Miracast	安卓手機	Wi-Fi Direct 直連，不須路由器。安卓手機” 無線投屏” 默認用此
DLNA	凡平臺	唯推送媒體文件（視頻/音樂/圖片），不鏡像屏幕。適合推視頻至電視上播放

各平臺操法：

- **iPhone**：控制中心 > 屏幕鏡像 > 選擇汝之電視/Apple TV。若電視不支持 AirPlay，可買一個 Apple TV 盒子或支持 AirPlay 之電視棒。
- **安卓**：下拉通知欄 > 尋到” 無線投屏”/” 智能投屏”/” 多屏互動” 按鈕 > 選擇電視。須手機與電視連同一個 Wi-Fi。
- **萬能策**：若電視太老不支持任何無線投屏，買一個 Chromecast 或小米盒子/華爲盒子插至電視 HDMI 口上，遂能接收投屏矣。

二、有線投屏——更穩、零延遲

無線投屏方便但有延遲，玩遊戲與作演示時體驗不好。有線投屏能解決此問題：

- **iPhone**：須一根 Lightning/USB-C 轉 HDMI 之轉換線（約 100-200 元）。插上後手機畫面直接輸出至電視/投影儀之 HDMI 口。
- **安卓**：部分手機支持 USB-C 轉 HDMI 直連（須手機支持 DP Alt Mode，大部分旗艦皆支持）。買一根 USB-C to HDMI 線即可。
- **會議室場景**：若汝屢須用手機作演示，建議於包裏常備一根轉 HDMI 線。會議室之電視/投影儀幾皆有 HDMI 口，插上遂能用，比折騰無線投屏靠譜多矣。

三、手機變電腦——桌面模式

部分安卓手機支持“桌面模式”——連上顯示器後，手機會輸出一個類似電腦桌面之界面，可用鼠標鍵盤操作：

- **Samsung DeX**：三星旗艦之招牌功能。連接顯示器後彈出桌面界面，可多窗口操作，體驗最接近真正之電腦。配合藍牙鍵鼠，出差可不帶筆記本。
- **華爲桌面模式**：華爲 Mate 系列/Pura 系列連顯器後亦有類似之能。支多窗口、文管、辦公 App。
- **摩託羅拉 Ready For**：Moto 旗艦亦支持，功能類似 DeX。
- **適用場景**：輕辦公（回郵件、改文檔）、演示 PPT、大屏觀視頻。不適合重度 Excel 操作或專業設計。

四、車載互聯——開車以手機之正法

開車時操作手機乃違法之，但導航、聽音樂、接電話又乃剛需。車載互聯繫統使汝於車機屏幕上使用手機之核心功能，手機可放於一邊不碰：

- **Apple CarPlay**：iPhone 用戶之首選。通過 USB 或無線連接車機，車機屏幕上出現 iPhone 之精簡界面——導航（高德/百度）、音樂、電話、信息、播客。Siri 語音控制全程免手。
- **Android Auto**：安卓用戶對策。能與 CarPlay 似，支 Google Maps、Spotify 等。國內因 Google 服限，部能不可用。
- **華爲 HiCar**：華爲自研方案，國內車型支持率高。除矣導航音樂，還能用華爲視頻通話、泊車影像融合等特色功能。
- **百度 CarLife**：跨平臺方案，iOS 與安卓皆能用。功能較少但兼容性廣，甚多老車型唯支持 CarLife。

安全提醒

無論哪種車載互聯方案，開車時皆應儘量使用語音控制。導航提前設好目的地，音樂提前選好歌單，勿於行駛中操作車機屏幕。

五、會議室投屏實戰——不復手忙腳亂

開將乃投屏最高頻之場景，亦乃最容易出問題之場景。以下乃幾條實戰經驗：

- **提前至場測試：**永遠勿等至會議開始方連投屏。提前 10 分鐘到場，測試好連接方式。Wi-Fi 投屏失敗之概率比汝欲之高得多。
- **準備有線方案作 Plan B：**於包裏放一根轉 HDMI 線。無線投屏連不上之時，有線即救命稻草。
- **閉告預覽：**投屏時汝之微信消息將顯示於會議室大屏上。投屏前於設置裏關閉” 鎖屏通知內容預覽”，或開啓” 演示模式”（部分手機支持，投屏時自動隱藏通知）。
- **用” 演示文檔” 而非” 操作手機”：**若汝要展示 PPT，直接以 PPT 存到手機裏用 WPS 打開全屏播放，而非投屏整個手機桌面。此樣更專業、更流暢。

動手：完成一次手機至電視之投屏

1. 確保手機與電視連同一 Wi-Fi。
2. 以手機開一張照片或一個視頻。
3. 試無線投屏（iPhone 用屏幕鏡像，安卓用無線投屏之鍵）。
4. 若失敗，檢查電視為否支持對應協議，或嘗試 DLNA 推送（於相冊/視頻 App 裏點” 投屏” 按鈕）。
5. 投屏成功後，試試於手機上作其他操作，觀大屏為否同步顯示。

本章之要

- 無線投屏三陣營：AirPlay（蘋果）、Miracast（安卓）、DLNA（跨平臺推媒體）
- 有線投屏更穩零延遲，會議室場景必備轉 HDMI 線
- Samsung DeX/華爲桌面模式使手機變身輕辦公電腦
- 車載互聯：CarPlay/Android Auto/HiCar/CarLife 四大策
- 會議室投屏鐵律：提前測試、準備 Plan B、閉告預覽

第三十一章：短視頻拍攝與內容創作入門

2026 年，短視頻已非“年輕人之玩具”矣，它變成矣所有人獲取信息與表達自我之基礎方式。無論汝乃欲記錄生活、分享知識，抑作點副業，手機即汝最好之創作工具——不須買相機、不須學 *Premiere*，一部手機遂能開始。此一章幫汝自零起步。

一、拍前先欲清楚：汝要拍賜誰觀？

甚多人拍視頻之第一個錯誤即“拿起手機遂拍”，拍完不知道賜誰觀。於開拍之前，先回答三個問題：

1. **吾之觀衆乃誰？**——拍賜孩子觀、拍賜同齡人觀、拍賜長輩觀，內容風格完全不同。
2. **吾能提供何價值？**——使人笑（娛樂）、使人學（知識）、使人饞（美食）、使人暖（情感）。至少佔一樣。
3. **吾能於哪個平臺持續更新？**——抖音/快手（泛娛樂）、小紅書（生活方式）、B 站（深度內容）、視頻號（社交圈）。選一個主陣地深耕，勿一上來遂全平臺分發。

二、手機攝基設——開拍前必作之 5 事

1. **分辨率設 1080P，幀率設 30fps：**此乃短視頻平臺之標準。4K 文件太大上傳慢，60fps 於社交平臺上無優勢反而文件更大。
2. **擦乾淨鏡頭：**第一條亦乃最重要之一條。拍之前用衣服擦一下鏡頭，畫面立刻清晰一個檔次。
3. **鎖定對焦與曝光：**於取景框長按畫面主體 2 秒，將出現”AE/AF 鎖定” 提示。此樣拍之時未能蓋因移動導致畫面忽明忽暗。
4. **橫拍抑豎拍？**——發抖音/速手/視頻號/小紅書**豎着拍**（9:16），發 B 站/YouTube **橫着拍**（16:9）。欲清楚平臺再決定方向。
5. **開網格線：**設置 > 相機 > 網格線。三分法構圖之基礎，幫汝快速尋到畫面重心。

三、運鏡五招——使畫面“動”起來

新手拍視頻最大之問題乃畫面太“靜”——全程站着不動拍。學會下面五個基本運鏡，畫面立刻有專業感：

- **推（Push）**：手機慢慢向爲拍對象靠近。製造”靠近”之緊張感或親密感。
- **拉（Pull）**：手機慢慢遠離爲拍對象。自細節拉至全景，交代環境。
- **搖（Pan）**：手機處不動，左右轉動。適拍風景、展示空間。
- **移（Track）**：手機平行移動，與着爲拍對象走。須手持穩定，最好配合手機穩定器。
- **與（Follow）**：手機與於爲拍對象後面或前面走。Vlog 最常用之運鏡。

穩乃第一要務：手抖之畫面看起來甚不專業。雙手持機比單手穩；走路時膝蓋微屈，用“忍者步”減少上下顛簸；若有預算，買一個手機穩定器（大疆 OM 系列約 500-800 元），效果質變。

四、剪輯入門——剪映遂夠矣

剪映（CapCut 之國內版）乃目前最適合新手之手機剪輯工具，免費、功能全、模板多。以下乃基礎工作流：

1. **導入素材**：打開剪映 > 開始創作 > 選擇拍好之視頻片段導入。
2. **裁**：拖動時間軸，以每個片段開頭結尾之廢片剪掉。原則：每一段唯保留有信息量之部分，寧短勿長。
3. **加轉場**：兩段視頻之間點”轉場”按鈕，選一個簡單之轉場效果。新手建議用”疊化”或”黑場”，勿用花哨之 3D 轉場。
4. **添字幕**：剪映有”識別字幕”能，一鍵自動生字幕。準率約 90%，手校對一遍即可。
5. **配音樂**：剪映音樂庫裏有大量版權免費之音樂。選一首節奏與內容匹配之 BGM，音量調至視頻原聲之 1/3 左右。
6. **導出**：1080P 導出，幀率 30fps，碼率選”薦”即可。

五、封面之要性——70% 之人僅唯封面決觀不觀

短視頻之封面即“門面”，決定別人要勿點進來。幾個作封面之要點：

- **封面要有一句話**：用大字寫明視頻之核心內容，比如”3 分鐘學會手機修圖”。字體要大、顏色要亮、與背景有對比。

- **封面與內容要一致**：勿作” 標題黨”——封面曰一件事，點進去乃另一回事，觀衆將立刻劃走，算法將降權。
- **人物表情要生動**：若封面有人，選一個表情生動或誇張之瞬間。人臉比文字更能吸引點擊。
- **剪映作封面**：於剪映裏選一幀最好之畫面，加文字疊加，導出爲圖片作爲封面。

六、內容創作之三條心法

- **前 3 秒定生死**：短視頻平臺，觀衆 3 秒內決定拂去抑繼續觀。前 3 秒須賜出” 鉤子”——一個問題、一個懸念、一個意外畫面。
- **一條視頻唯講一件事**：新手最容易犯之錯乃一條視頻塞太多內容。一條視頻唯解決一個問題，講透比講全重要。
- **堅持較天賦要**：算法推薦須” 養號”。前 20 條視頻幾未能有播放量，此乃正常之。堅持每週更新 2-3 條，使算法認識汝之內容方向，流量將慢慢起來。

動手：拍一條 30 秒之短視頻

1. 選一個汝擅長之小知識（比如” 何以將手機字體調大”）。
2. 以手機豎屏攝 3-5 段素材，每段 5-10 秒。
3. 用剪映綴、裁、添字幕、配 BGM。
4. 作一張帶大字之封面圖。
5. 導出後先發予朋友觀，收集反饋再發布。

本章小結

- 拍之前欲清楚：觀衆乃誰、提供何價值、於哪個平臺發
- 開拍 5 事：1080P/30fps、擦鏡頭、鎖對焦曝光、確橫豎、開網格線
- 運鏡五招：推、拉、搖、移、與——穩乃第一要務
- 剪輯用剪映：裁轉場字幕 BGM 導出，六步走
- 前 3 秒定生死，一條視頻唯講一件事，堅持比天賦重要

第三十二章：手機安全急救手冊

此章乃一份「應急預案」。望爾永遠用不上，然果遇手機丟失、被盜、被詐騙之情事，此手冊能助爾將損失降至最低；地震、暴雨等災害來臨時，手機同為接收預警、定位求救、保持聯絡之工具。建議爾今即通讀一遍，然後將關鍵步驟截圖保存，或打印出來置於家中——蓋出事之時爾之手機或已不於手上，災時則可能沒電或沒信號。

場景零：地震 / 極端天氣——手機何以助爾保命

所防者人身之安，非帳號資金。預警設置詳見第十三章「天氣預警、自然災害預警與地震預警」；此處只講警報響起之後該何以做。

地震警報響了：先躲後撤，勿只顧拍視頻

1. 就近避險：趴下、掩護、抓牢（桌下、承重牆角、堅固傢具旁），護住頭頸；遠離玻璃、外牆、吊燈。
2. 勿盲目奔樓：主震往往只有十幾秒至幾十秒，樓梯間可能更危險；電梯絕對不能用。
3. 震停後再有序撤離：走樓梯、留意餘震與掉落物；到開闊地，遠離電線桿、廣告牌、危房。
4. 手機用於緊要處：予緊急聯絡人發簡短消息（「我於 XX，安全 / 受傷，須幫助」）較打長電話更省電；能發短信 / 微信時優先文字。
5. 沒信號時：見第七章衛星 SOS / 衛星消息（機型支持時）；或到高處、開闊地再試；部分場景下無網藍牙對講亦有助。
6. 關閉非必要之功：應急時啟飛行模式，定時開關；罷藍牙搜尋，降其亮度，可多支數時；留末格之電，以報平安與救援之訊。

暴雨 / 颱風 / 山洪：預警之期，即當行動

- 見紅色預警：宜少外出，遠離地下之室、下穿之橋、河道、陡坡。
- 困於戶外：登堅固屋宇之高處，勿強涉水（積水或失井蓋、或漏電）；呼 119/110 時，詳陳地標與水位。
- 未行之先：以天氣 App 與當地預警之公號核其險；長途自駕，避山區臨水臨崖之路。
- 家人相報：約曰「若失聯，以最後微信、短信之定位為準」，免各自狂撥而耗盡其電。

平素即須做之四事

1. 啟系統地震預警 / 正規預警 App，許其破靜音而出聲。
2. 於通訊錄置頂二三人為「緊急聯絡人」，鎖屏之界面可顯之（iOS、安卓之醫療急救卡或鎖屏之訊）。
3. 家中行囊，常備充電寶；遠行之前，滿其電而攜其線。
4. 默識至少一號（家人之手機號），雖無通訊錄亦可撥者，書之於紙，藏之錢囊。

場景一：手機遺失或被竊——黃金三十分鐘

手機既失，首三十分鐘乃「黃金之時」，處之得宜，則財貨與隱私可保。依下序而行，一步毋越：

失機後三十分鐘標準操作之流程

1. 借他人之機撥己之號（第 1 分鐘）：或止墜於近處，善人拾而接之，即可取回。
2. 遠程鎖機（第 2-5 分鐘）：借他人之機或電腦登入搜尋之服務：iPhone：iCloud.com > 尋尋 > 擇爾之設備 > 標為遺失
3. 安卓諸牌：華為 cloud.huawei.com / 小米 i.mi.com / OPPO cloud.oppo.com 等
4. 掛失 SIM 卡（第 5-10 分鐘）：撥運營商客服之電話（移動 10086 / 聯通 10010 / 電信 10000），報手機已失，請停 SIM 卡之務。此步最要——騙者得爾之 SIM 卡，即可收驗證之碼，盡改爾之諸密。
5. 凍結支付之帳（第 10-20 分鐘）：支付寶：95188 掛失，或於他人之機登支付寶 > 我之 > 設定 > 安全中心 > 緊急凍結
6. 微信：撥 110 報警後，聯微信客服，或於微信安全中心 > 凍結帳號
7. 銀行 App：撥銀行卡背面之客服電話，請凍網銀
8. 告親友（第 20-25 分鐘）：發於朋友圈，或託家人傳訊，告以手機已失，防騙者假爾之名以欺人。
9. 報警（第 25-30 分鐘）：撥 110，或詣最近之派出所報案，取其回執。他日保險之賠、銀行之爭，皆需此執。

場景二：為人所騙——七十二小時黃金處理之期

既覺受騙，要於鎮定。慌亂而誤決，必致再損。依下步而行：

1. 立止一切轉帳之事：無論騙者何言，勿復轉一錢。「再轉一筆保金，前錢可返」者，二次行騙之常語也。

2. 盡存其證：聊天之錄勿刪！通話之錄勿刪！截圖以存聊天之錄、轉帳之錄、對方帳號之訊。此皆報案追資之要證也。

3. 急撥 96110：此全國反詐之專線。接線員將導爾緊急止付——若轉帳未久（常於二十四小時內），銀行或可凍對方之帳而截其資。

4. 撥 110 報警：攜手機、身份證、轉帳之憑，詣最近之派出所錄供。

5. 聯銀行、支付之臺：告以受騙之情，請凍涉案之帳。若信用卡為盜刷，請銀行啟拒付之程。

6. 盡改諸密：於另一安全之器，改爾 Apple ID、品牌之帳、微信、支付寶、銀行 App 之密，並啟雙重認證。

場景三：手機染毒或木馬——奈何？

若爾之機現此諸症——廣告無故自出、奇 App 自裝、話費暴增、流量異耗——則中惡意軟件之疑甚大：

- 第一步：入安全模式——安卓之機長按電源鍵以關機，復長按以開機；開機既震，長按音量減鍵勿釋，至入安全模式而止（左下角現「安全模式」之字）。安全模式中，第三方 App 皆禁，便於排查。

- 第二步：卸可疑之 App——安全模式中，入設定 > 應用管理，以時間序之，得最近所裝陌生之 App 而卸之。

- 第三步：裝安全之 App 以掃之——下「騰訊手機管家」或「360 手機衛士」，全盤一掃。

- 第四步：復原廠之設（終極之方）——若上法皆無效，先備要數據，乃復原廠。慎之：復原廠則諸數據盡刪，照片、通訊錄諸要檔，必先備之。

- iPhone 幾不染毒——iOS 沙盒之制，惡意軟件難存。若爾之 iPhone 有異，多為某 App 之 bug 或設定之失，非毒也。復原廠之設，可解九成九之患。

場景四：帳號被竊——速復掌控之權

1. 急改其密：若尚能登其帳，第一時改密。新密須與舊密全異。

2. 踢去他器：於帳號安全之設中得「登錄設備管理」，凡陌生之器，盡踢下線。

3. 查登錄之記：覽近七日登錄之記，若有異地之登，則帳果被竊矣。

4. 啟雙重認證：若未嘗啟，今即啟之。此防帳再竊之至效者也。

5. 查所綁之訊：視帳號所綁之手機號、郵箱有無被改。若已改，以帳號申訴之功復之。

6. 查支付之設：若被竊者為微信、支付寶，則視免密支付、自動扣款有無被篡，銀行卡有無見加於陌生之帳。

場景五：得勒索之訊——勿懼，勿付

勒索之術有二：(1)「吾有爾不雅之影片、照片，不予錢則盡發爾通訊錄之人」；(2)「爾機已為黑客所制，不予錢則鎖死」。遇此：

- 九成九皆詐也：彼實無爾絲毫之私。以同辭羣發數萬人，總有懼而付錢者。
- 勿復，勿付：一復則告彼曰「此號有人用」，爾遂被標為「可勒索之之」，其擾滋多。
- 截圖存證：遂拉黑其人。
- 改其密：若有憂，改爾主要帳號之密，以自安。
- 若果有洩私之虞：報警。警方辦此類之案，素有經驗。

場景六：AI 換臉、AI 擬聲之詐——眼見不復為實

自 2024 年，AI 換臉、擬聲之技，已能使騙者於視訊之中化為爾家人、長上、朋友之面與聲。「娘，兒出事矣，速轉錢來」——視中之人，貌與聲皆肖爾子，實則偽也。此當今至難防亦至危之詐術之一也。

第一步：識其破綻——AI 偽造四常見之漏

- 眨眼之數異：真人每分鐘眨眼十五至二十，AI 換臉者多眨之過少，或其節僵。
- 口型與聲不諧：諦視口型與所聞之聲，有零點幾秒之差。
- 面際模糊：臉之輪廓、耳畔、髮際，易現「毛邊」或閃爍。
- 光影之向不侔：面上高光、陰影與背景照光之向不合，或側面時五官隨轉失其度。

第二步：三術驗其真偽

- 令作特定之動：使彼轉頭、掩面、以手障口而復開合。AI 換臉於障蔽與大角度轉頭之際最易露餡——面或變形，或追之不及。
- 問惟彼知之事：設一虛問，如「前日吾家之犬送何醫院視之？」——若果騙者，則不能對，或支吾其詞。
- 回撥已知之號：斷視訊，以爾通訊錄所存之號徑撥之。騙者不能制真之機號，一撥即敗。

第三步：既覺受騙，報警之流程

1. 立止轉帳：雖急亦姑止，凡催爾「速轉帳、毋掛斷」者，殆皆騙也。
2. 盡存其證：錄屏、截圖，存視訊之畫、聊天之錄、對方之帳號、轉帳之憑——此技術鑑定之要材也。
3. 撥 96110 反詐專線：告以「AI 換臉、擬聲」之詐，接線員將啟緊急止付。

4. 撥 110 報警：攜證詣派出所錄供，特陳其用 AI 偽造之技，以便警方立案定性。
5. 告親友：騙者或以偽爾之視訊欺人，先於朋友圈、家族之羣發一提醒。

場景七：SIM swap 之攻（SIM 劫持）——爾號為人所「竊」

此攻人多未聞，而其害至巨。SIM swap 者（SIM 卡劫持），謂攻者偽造爾身份之訊，冒爾之名詣營業廳補辦新 SIM 卡。補辦既成，爾機之信號驟失，而攻者之機插彼新卡，盡收爾短信驗證之碼——支付寶、銀行、微信，門戶盡開矣。

何以覺之？SIM swap 三預警之兆

- 機忽示「無服務」、「無 SIM 卡」，而爾未嘗動卡，重啟亦無效——此多為被劫之頃。
- 得運營商「補卡既成」或「業務既辦」之短信——而爾未嘗辦也。
- 驗證之碼驟不得，他短信亦不得——以其短信被劫至攻者之卡也。

防護之術——並鎖爾之 SIM 卡

- 設 SIM 卡之 PIN 碼：其徑設定 > 電話 > SIM 卡 PIN 碼。既啟，每重啟或換卡，必輸 PIN 而後能用此卡。攻者雖得卡，亦無所用之。
- 啟運營商再驗之制：登中國移動、聯通、電信之 App，得「安全中心」或「業務辦理驗證」，啟非本人補卡之短信提醒與業務辦理再驗之密。如為他人補爾之卡，營業之員必求身驗之加焉。
- 要帳勿徒恃短信之驗：銀行、Apple ID 諸高值之帳，宜先用認證器 App（Google Authenticator、微軟 Authenticator）或硬體之鑰，毋用短信驗證之碼。
- 身份證影本加印水印：辦務所需之影本，明書「僅供某務之用」——攻者多賴洩漏之身份訊以冒爾。

被劫之後應急之處——唯速為要

1. 急撥運營商客服掛失：移動 10086 / 聯通 10010 / 電信 10000，請立停機並凍補卡之務，以阻攻者續用爾號。
2. 以他器登要帳：用電腦或家人之機，急改銀行、支付寶、微信、Apple ID 之密，並踢陌生之器。
3. 凍結支付之帳：支付寶 95188 掛失，微信安全中心凍之，銀行客服凍網銀。
4. 持身份證詣營業廳：本人攜身份證至自營之廳補辦新卡，復其號之用。
5. 報警：SIM swap 已構「侵犯公民個人信息之罪」，攜證（停機時間之記錄、資金之損失、補卡之記錄）以報。
6. 遍察之：號既復，逐一視近期登錄之記與資金之流水，驗有無被冒用之跡。

防勝於治——日用五事

1. 鎖屏之密與雙重認證，二者皆啟
2. 「尋尋吾之機」務必啟
3. SIM 卡設 PIN 碼（設置 > 電話 > SIM 卡 PIN 碼），丟了手機他人不能用汝卡
4. 不裝來歷不明之 App，不點陌生之鏈
5. 定期備份要數據（照片、通訊錄、聊天之錄）

來動手：製爾「應急聯絡之卡」

1. 取紙一張，書下列諸號：運營商客服（移動 10086 / 聯通 10010 / 電信 10000）、銀行客服、96110 反詐專線、110 報警之電話、家人之電話。
2. 書爾 Apple ID、品牌之帳所註冊之郵箱，與備用聯絡之法。
3. 置此紙於錢囊，或貼於冰箱——毋存於機，蓋事起之時，機或不於側也。
4. 今即查之：「尋尋吾之機」已啟乎？SIM 卡 PIN 碼已設乎？

本章小結

- 失機黃金三十分鐘：借機撥號遠程鎖機掛失 SIM 卡凍結支付告親友報警
- 被騙七十二小時：止轉帳存其證撥 96110 報警聯銀行改其密
- 機染毒：安全模式卸可疑 App 安全之掃復原廠（終極之方）
- 帳被竊：改密踢器查登錄啟雙重認證查所綁查支付
- 勒索之訊九成九皆詐，勿復勿付，截圖而拉黑
- AI 換臉、擬聲之詐：觀眨眼、口型、邊際、光影之破綻，令彼轉頭掩面，問私密之事，回撥已知之號
- SIM swap 之劫：機忽「無服務」即警；設 SIM 卡 PIN、啟運營商再驗、要帳改用認證器 App
- 日用防患五事：鎖屏 +2FA、尋尋手機、SIM 卡 PIN、不裝來歷不明之 App、定期備份

六、展望篇與車聯篇（第 33-34 章）

第三十三章：未來展望——手機之下一程

手機未能一夜之間消失，但它之“獨霸地位”或爲新一代設備分流。此一章我等聊三件事：下一代設備長何樣、手機何以“隱身”爲新身份基石、以及彼我等須提前思考之倫理議題。

一、AR/VR 與空間算：手機外之“次桌面”

下一代設備之競爭已打響，最有或分流手機時間之，乃 AR/VR 與空間計算設備：

- **Vision Pro 後續**：蘋果 Vision Pro 之發佈標誌着” 空間計算” 概念之正式落地，但第一代產品價格（29999 元起）與重量仍乃門檻。預計 2026-2027 年將推出更輕、更便宜之平價版 Vision，使空間計算自” 開發者玩具” 走向” 早期大衆”。屆時 iPhone 或成爲 Vision 眼鏡之算力伴侶與數據中樞。
- **AR 眼鏡**：賜汝之眼前開個” 第二桌面”，數字信息變得像空氣一樣無處不於。Meta 與 Ray-Ban 合作之智能眼鏡已賣出數百萬副，國內 Rokid、XREAL、INMO 等廠商亦於推出消費級 AR 眼鏡。2025-2026 年之 AR 眼鏡已能作實時翻譯、導航 HUD、AI 助手對話，但仍受限於光波導顯示與續航，距離” 取代手機” 還有 3-5 年。
- **智能戒指與可穿戴**：Samsung Galaxy Ring、Oura Ring 等智能戒指開始流行，作爲手機之” 貼身傳感器” 補充健康監測。手機之角色正自” 萬能終端” 轉向” 中樞 + 多個輕量化配件” 之組合。

二、腦機接口：思欲成爲交互媒介

思想本身將成爲交互之媒介，但此條路比想象中漫長：

- **Neuralink 最新進展**：馬斯克之 Neuralink 已於 2024 年完成首例人體植入，首位受試者 Noland Arbaugh 通過意念控制電腦鼠標下國際象棋。目前植入式 BCI（腦機接口）主要面向癱瘓患者輔助場景，目標乃幫助彼等重新” 操作” 數字世界。
- **非侵入式 BCI**：多家公司開發非侵入式頭環（如 Kernel、OpenBCI），無需手術即可讀取腦電信號，但精度遠低於侵入式。距離” 用意念打字” 尚有相當距離。
- **消費級展望**：腦機接口距離消費級手機交互還有 10-20 年，但短期或先用以無障礙輔助、專注力監測等場景。倫理與隱私問題（” 吾之腦電波算誰之隱私”）將先於技術成熟成爲爭論焦點。

三、手機之“隱身”與新身份基石

手機之另一個未來進化方向或許乃“隱身”——化身爲無處不於之“幕後推手”，成爲汝數字身份之唯一基石：

- **雲端雙因素認證**：瞬間完成” 汝乃汝” 之認證，登錄任何服務無需手動輸驗證碼。
- **數字遺產成爲出廠設置**：新機激活時預設數字資產繼承方案，使” 身後事” 成爲出廠默認。
- **自” 設備中心” 至” 身份中心”**：汝之雲端個人數據保險櫃方乃中心，硬件只為訪問入口。換機、換設備，身份與數據始終與着汝。

四、倫理思考：吾等，抑手機之主人嗎？

有一終極之問恆存：吾等乃受技術賦能之“神”，抑爲算法所餵養之“缸中之腦”？

- **AI 倫理**：當端側 AI 比汝自己更懂汝之偏好，汝之” 選擇” 乃自主意志抑爲引導之結果？AI Agent 代汝下單、回覆消息時，責任與邊界於何處？
- **數據主權**：汝之健康數據、腦電數據、生物特徵數據，乃屬於汝、屬於設備廠商抑屬於雲服務商？《個人信息保護法》與歐盟 GDPR 只為起點，” 數據可攜帶權” 與” 爲遺忘權” 之真正落地還有甚長之路。
- **數字鴻溝**：AI 手機、摺疊屏、衛星通信皆於拉大” 用得起” 與” 用不起” 之間之差距。數字基礎設施之公平接入，將成爲新之社會議題——遂像 20 年前” 將未能上網” 一樣關鍵。
- 當算法比汝更懂汝欲觀何物，汝之” 喜歡” 有幾分出於自發？
- 當汝每隔數分鐘便不自覺點亮屏幕，此乃汝之習慣，抑被人設計而出之” 上癮行爲”？

五、持連，亦持獨——汝之數生活哲學

無論技術何以進化，下面此幾條數字生活哲學，10 年後依然適用：

1. 爲資源之分配者，而非被推送之接收者。
2. 令自動化成爲汝忠實之” 副駕駛”。
3. 築汝穩固之” 數字後方”。
4. 定期行” 數字斷捨離” 與” 社交齋戒”。
5. 恆以安全與隱私爲一切便利之前提。

來試之：制定汝之《個人數字設備使用服務條款》

爲汝與汝之手機（以及未來之 AR 眼鏡、AI 助手）寫一份簡單之“服務條款”，來定義汝等之關係。例如：

- 第 1 條：吾之注意力時間屬於吾。吾工作 / 陪伴家人時，汝當主動進入” 勿擾” 之狀態。
- 第 2 條：吾之個人數據乃吾之財產。汝之首務乃安全加密與便捷同步。
- 第 3 條：吾等之共同目標乃提升生活效率與體驗。若汝以誘導性設計令吾沉迷，吾有權將汝卸載。

本章小結

- Vision Pro 與 AR 眼鏡乃手機之外之新計算入口，但短期難以取代手機
- 腦機接口距離消費級交互尚有 10-20 年，倫理題先於技術成熟
- 手機正向” 隱身” 方向進化，自設備中心走向身份中心
- AI 倫理、數據主權、數字鴻溝乃未來 10 年之核心議題
- 保持連接亦保持獨立，乃數字生活不變之哲學

第三十四章：車機互聯深指南——手機與汽車之融合

汽車方自“四個輪子加沙發”進化爲“四個輪子加智能手機”。無論汝開之乃燃油車抑新能源，手機與車機之關係皆決定了汝之駕駛體驗。此一章帶汝釐清車機互聯之三大路線、各派系車機系統之特點，以及彼使開車更順手之小技巧。

一、車機互聯之三大路線

當前手機與汽車“聯網”主要有三種思路，理解矣彼等遂理解矣整個生態：

- **手機投屏至車機**：車機唯當一塊” 顯示器” 與” 音箱”，運算與地圖數據皆來自手機。代表方案：Apple CarPlay、Google Android Auto、華爲 HiCar、百度 CarLife。優點乃手機生態直接搬至車上，不用重新裝 App；缺點乃車機本身之硬件能力爲閒置。

- **手機與車機深融合**：車機乃獨立系統，但與手機共享賬號、應用、數據，甚至能” 接力” 任務。代表方案：鴻蒙座艙（問界/智界）、比亞迪 DiLink、蔚來 NIO OS、理想車機系統。優點乃體驗一體化；缺點乃通常綁定同一品牌生態。

- **CarPlay 2.0 深集成版**：2025 年蘋果發佈之新一代 CarPlay，不再只為” 投屏”，而乃接管車機之儀表盤、空調控制、座椅調節等核心功能。法拉利、阿斯頓 • 馬丁、保時捷等品牌率先搭載，標誌車機系統進入” 手機廠商主導” 之新階段。

二、CarPlay / Android Auto / HiCar 對較

三大主流投屏方案各有側重，選車或選手機時可對照下表：

方案	支手機系統	支車型	能豐度	國內可用性
Apple CarPlay	僅 iPhone	極廣，絕大多數車型支	高，地圖/音樂/電話/消息均可	好，原生可用
Android Auto	Android（國內手機需裝服框架）	較廣，合資/進口車居多	高，與 CarPlay 相當	通常，需科學上網激活
華爲 HiCar	華爲手機爲主，部支其他安卓	國產車與部合資車型	較高，可與鴻蒙座艙聯動	甚好，本土生態
百度 CarLife	iOS 與 Android 均可	較廣，舊車型後裝常見	中，更新較慢	甚好，本土生態

三、鴻蒙座艙：手機與車機之無縫流轉

若曰 CarPlay 乃“手機投至車機”，鴻蒙座艙則乃“手機與車機本即一家”。華爲手機與問界/智界車機之協同，代表矣當前國內車機互聯之最高水準：

- **超級桌面**：車機直接調用手機上之 App，無需於車機重新安裝。手機裏於聽之音樂、於導航之路線，上車自動續接至車機屏幕，下車再回到手機，全程無縫。
- **應用接力**：手機上設好之導航，關車門時車機自動接管；到達停車場，下車後手機繼續步行導航至最終目的地。” 終一公里” 不再斷檔。
- **手機車機共用賬號**：華爲賬號一登，車機自動同步汝之音樂會員、地圖藏、日程、聯繫人。車機與手機乃同一身份之不同終端，而非兩套獨立系統。

四、造車新勢力之車機系統

國產新能源車之車機已自“導航 + 音樂”進化爲移動智能終端，各家之玩法各有特色：

- **蔚來 NIO OS**：NOMI 語音助手乃行業標杆，能識別多指令、上下文連續對話，尚帶情感表情。NIO Phone 則將手機作成” 車鑰匙 + 第二屏”，深度綁定蔚來生態。
- **理欲車機**：多屏交互乃亮點，副駕屏可獨立觀劇、孩子屏可觀動畫片。” 任務大師” 使汝自定義場景，比如” 開門自動暫停音樂、關門繼續播放”。
- **小米澎湃車機**：與小米手機、手錶、家電深度聯動。手機上設之鬧鐘將於車機提醒；車上導航回家，家裏空調提前開啓。HyperOS 打通矣” 人車家全生態”。
- **較亞迪 DiLink**：標誌性之旋轉屏，豎屏觀短視頻、橫屏觀地圖，自適應 App 方向。自帶應用市場，生態豐富度於傳統車企中領先。

五、手機當車機——能用嗎？

若汝之車機太老或太難用，能否直接用手機“頂上”？答案乃：可，但要觀場景。

- **摩託羅拉 Ready For / 桌面模式：**部摩託羅拉、三星、華爲手機支”桌面模式”，連上顯器便乃一臺簡易電腦。理上接車機屏亦能用，但實際兼容性參差不齊，且缺汽車專屬交互（如方向盤按鍵）。
- **手機支架 vs 原車車機：**若原車車機卡頓、地圖不更新，一個穩固之手機支架 + 手機導航乃務實之選。但要注意：(1) 手機屏幕小，駕駛時觀不清；(2) 夏天暴曬將使手機過熱降頻甚至自動關機；(3) 長時間導航耗電快，需配車載充電器。
- **建議：**短途或臨時用車，手機支架夠用；日常通勤長途，建議升級支持 CarPlay/HiCar 之車機或加裝後裝車機，體驗與安全皆更好。

安全提醒

無論用哪種方案，駕駛時操作手機皆違反交規。手機支架只能用以“觀導航”，不可用以“操作手機”。設置目的地、切換歌曲等操作，請務必停車後再作，或使用語音控制。

六、實用術

- **無線 CarPlay 連不穩：**(1) 關閉車機藍牙再重新配對；(2) 刪除車機端已配對設備記錄後重連；(3) 檢查手機為否同時連矣多個藍牙設備造成衝突；(4) 升級車機固件與手機系統至最新版；(5) 實於不行，改用有線 CarPlay，更穩定。
- **車機流量用完矣：**(1) 用手機開熱點，車機連手機熱點上網；(2) 於車機設置中以”高耗流量”應用（如視頻、於線音樂）切至 Wi-Fi；(3) 定期清理車機離線地圖緩存。
- **駕駛模式自動觸發：**(1)iOS”專注模式”設定”連接車載藍牙時自動開啓”，自動屏蔽非緊急通知；(2) 安卓”駕駛模式”或第三方 App（如 Tasker）可實現連藍牙自動靜音、自動回覆消息；(3) 華爲手機可設置”智感支付”，上車自動切至車機播放音樂。

選車時之小建議

若於乎車機體驗，選車時務必去 4S 店實測：(1) 帶自己之手機試 CarPlay/HiCar 連接為否穩定；(2) 試語音助手之反應速度與指令識別率；(3) 觀應用市場有無汝常用之 App；(4) 關注後續 OTA 升級頻率——好車機乃用軟件迭代出來之。

動手：優化汝之車機互聯體驗

1. 確認汝之手機與車機支持之互聯方案（CarPlay / HiCar / Android Auto / CarLife）。
2. 清理一次車機之藍牙配對列表，唯保留常用手機，減少連接衝突。
3. 設一個”駕駛專注模式”，連車機藍牙時自動開啓。
4. 下載車機品牌 App（如鴻蒙智行、理想汽車、蔚來 App），觀為否能遠程控制空調、查

看車況。

本章小結

- 車機互聯三大路線：投屏（CarPlay 等）、深融合（鴻蒙座艙等）、CarPlay 2.0 深集成
- CarPlay 生態最廣；HiCar 國內體驗好；Android Auto 國內需折騰；CarLife 適舊車後裝
- 鴻蒙座艙之超級桌面、應用接力、共用賬號代表矣深度融合方向
- 蔚來 NOMI、理欲多屏、小米人車家、較亞迪旋轉屏各有特色
- 手機當車機僅適臨時場景，久宜升或後裝車機
- 實用術：CarPlay 重連、手機熱點補流量、藍牙觸發駕駛模式

兒童與手機專題（第 35 章）

第三十五章：兒童與手機——數字時代之教育指南

本章不講“手機乃洪水猛獸”，亦不鼓吹“越早接觸越聰明”。我等按年齡分段賜設備、設規則、教素養、護健康，一步步來。

一、幾歲賜孩子手機？

無標準答案，但有年齡基線。過早賜手機，孩子大腦前額葉尚未發育成熟，自控力跟不上算法之“誘惑設計”；過晚賜，又或使他於同伴中“社交脫節”。下面乃分齡建議：

- **3-6 歲**：儘量不碰手機。如需使用，單次不超過 15 分鐘，內容以家長精選之教育動畫爲主，且須**陪同觀看**。此一階段屏幕對視力與留意力發育影響最大。
- **7-9 歲**：可於家長設備上”借用”，用以網課、聽故事、查資料。開始建立”使用前申請、用完歸還”之規則，不配置專屬手機。
- **10-12 歲**：可考慮賜一臺”功能機”或兒童手錶，解決上下學聯絡。如確需智能機，應開啓嚴格之家長控制。
- **13 歲以上**：進入初中，社交需求增強，可賜智能機，但需配合”使用公約”與定期覆盤。

> **”夠用遂好”原則**
>
> 孩子之第一臺手機不須旗艦。處理器之“過剩性能”往往爲用來跑遊戲。選一臺電池夠大、性能中等、屏幕護眼之千元機或官方翻新機，反而更利於管控。預算參考：1500-2500 元區間即可滿足絕大多數學生需求。

智能手錶作爲過渡方案

若只欲解決“放學後能否聯繫上”之問題，兒童手錶比手機更合適。其只能打電話、發語音、定位，無瀏覽器與短視頻，天然限制矣“沉迷入口”：

- **小天才電話手錶**：生態最完整，同齡人社交”碰一碰加好友”乃其核心賣點，但專屬 App 生態較封閉。

- **華爲兒童手錶**：與華爲手機聯動好，定位精準，支持視頻通話，性價比高。
- **小米兒童手錶**：價格親民，基礎功能齊全，適合預算有限之家庭。

買手錶時留意：選擇支持 **4G 全網通**之型號（2G/3G 正逐步退網），並確認運營商兒童套餐資費。

二、家長控制系統全攻略

設備賜至孩子之前，第一件事非開機，而乃設置家長控制。兩大平臺皆有原生方案：

iOS：屏幕使用時間

- **App 限額**：設置 > 屏幕使用時間 > App 限額，可爲每個 App 類別設定每日可用時長（如遊戲 30 分鐘）。
- **停用時間**：設定每日”熄燈時段”（如 22:00-7:00），期間凡 App 變灰禁用，僅電話可用。
- **內容限制**：可按年齡段過濾影視、網頁、App 安裝，並禁止 App 內購買。
- **家庭共享**：用家長之 iPhone 遠程管理孩子之設備，批准下載請求、查看使用報告。

安卓：Google Family Link / 品牌家長控制

- **Google Family Link**：谷歌官方方案，可遠程管理設備使用時長、鎖定設備、查看活動報告。國內安卓需視機型可用性。
- **小米兒童空間**：設置 > 兒童空間，可設定使用時長、白名單 App，退出需家長密碼。
- **華爲學生模式**：設置 > 健康使用手機 > 學生模式，限制使用時段與 App，支持遠程管理。
- **OPPO/vivo 兒童模式**：於”設置”或專屬”兒童空間”App 中開啓，原理類似。

功能	iOS 屏幕使用時間	安卓 Family Link / 品牌方案
App 限額	支持，按類別	支持，按 App
停用時間	支持	支持（品牌方案有差異）
內容過濾	系統級，較徹底	依賴品牌，參差不齊
遠程管理	家庭共享	Family Link / 品牌雲
安裝 App 審批	需家長批准	Family Link 支持

家長控制非”一勞永逸”

孩子總能尋到繞過之方法（網上搜“何以破解屏幕使用時間”之教程一大把）。技術手段只爲底線，真正之防線乃親子溝通與信任。建議每月與孩子一起復盤使用報告，而非單方面監控。

三、青少年模式

各大內容平臺皆內置矣“青少年模式”，開啓後將限制使用時長、過濾不良內容、關閉充值打賞。此乃國家法規要求之強制功能，但**須家長主動開啓並設置密碼**：

- **抖音/快手青少年模式**：每天限 40 分鐘（22:00-6:00 禁用），內容唯展示精選科普、教育類視頻，無法直播、無法打賞、無法充值。
- **微信青少年模式**：限制微信支付（需家長授權額度），限制公衆號、視頻號內容，搖一搖、附近之人等功能默認關閉。
- **遊戲防沉迷系統**：國家層面之實名認證 + 時間限制。未成年人僅可於週五、週六、週日及法定節假日之 20:00-21:00 登錄遊戲，每日 1 小時。

實名認證勿用家長身份證“借號”

甚多孩子用家長身份證註冊遊戲賬號繞過防沉迷，此既違反用戶協議，亦使孩子失去保護。家長應保管好自己之支付密碼與身份信息，勿爲圖省事“借號”賜孩子。

四、網絡素養教育

技術能攔截 90% 之風險，剩下 10% 靠孩子之判斷力。比起“不許上網”，更重要之乃教彼等“何以上網”：

- **隱私保護**：不向陌生人透露真實姓名、學校、家庭住址；不發送個人照片、身份證、家庭合影；定位功能默認關閉，僅於使用時開啓。
- **網絡欺凌識別與應對**：被辱罵、孤立、惡意 P 圖皆屬於欺凌。告訴孩子遇到時勿對罵、勿刪除證據、第一時間告訴家長。微信可長按消息投訴，平臺將處理施暴賬號。
- **內容批判**：教孩子分辨” 事實” 與” 觀點”，識別標題黨、震驚體；理解算法唯將推汝欲觀之，不代表世界之全貌；遇至” 天上掉餡餅” 之廣告一律不信。
- **網絡安全**：不點擊陌生鏈接、不掃碼來歷不明之二維碼、不下載非官方商店之 App；密碼不分享賜任何人（包括” 好朋友”）；遇至索要驗證碼之一律視爲騙子。

動手：與孩子一起籤一份《手機使用公約》

以規則寫下來、雙方簽字貼於顯眼處，比口頭約定有效得多。建議包含：

- 使用時段：如” 作業完成後 19:30-20:30 可用”。
- 禁用之域：餐桌上、臥室臨睡前、課堂之上，手機一概收之。
- 內容邊界：不安裝未告知家長之 App，不與陌生人私聊。
- 違約後果：如” 超時一次，次日用時長減半”。
- 家長承諾：不於孩子面前” 刷手機” 超過約定時長，以身作則。

五、健康使用習慣

孩子之眼睛、頸椎、睡眠皆於發育期，手機對身體之傷害比成年人更大：

- **屏幕時間建議**：3-6 歲每日不逾 1 小時；6-12 歲學習類不限制，娛樂類不逾 1 小時；12 歲以上與孩子協商，但娛樂類不逾 2 小時。
- **護眼習慣**：遵守”20-20-20 法則”——每觀屏幕 20 分鐘，抬頭觀 20 英尺（約 6 米）外之遠處 20 秒；睡前 1 小時不觀手機，藍光將抑制褪黑素影響睡眠。
- **姿勢管理**：手機屏幕與眼睛平齊，勿低頭含胸；連續使用不超過 30 分鐘遂要起身活動。
- **親子共同規則**：餐桌、臥室、書房設” 無手機區”；全家一起觀電影、戶外活動時手機統一收起，使” 放下手機” 成爲家庭文化而非對孩子之懲罰。

家長乃最好之” 防火牆”

研究表明，父母自身手機使用時長與孩子呈強正相關。要求孩子不觀手機，自己卻刷短視頻至深夜，規則遂將失效。改變孩子，先自放下自己之手機開始。

六、數字遺產與兒童

此一節容易被忽略，但甚重要。孩子之數字資產（賬號、照片、充值餘額、遊戲裝備）同樣面臨繼承與隱私問題：

- **兒童數字資產之繼承**：孩子不幸離世時，其社交賬號、遊戲賬號、雲盤照片之繼承流程與成年人相同，需提供死亡證明、親屬關係證明。建議家長平時遂爲孩子之重要賬號作” 遺產聯繫人” 或” 緊急聯繫人” 設置（如蘋果遺產聯繫人、微信賬號繼承）。
- **未成年人之數字隱私**：《個人信息保護法》規定，處理不滿 14 週歲未成年人個人信息需取得監護人同意，且應制定專門之隱私規則。家長應定期審查孩子常用 App 之隱私政策，關閉” 個性化推薦” 與” 數據共享” 開關。
- **賬號註冊建議**：爲孩子註冊之賬號應使用家長可控之郵箱/手機號，便於未成年階段統一管理；孩子成年後可再作賬號移交。

警惕” 兒童數字遺產” 之另一面

孩子自出生起便在被“記錄”——家長曬之成長照、幼兒園發之視頻、智能手錶之健康數據。此數據一旦上傳遂難以徹底刪除。發佈前想一想：十年後孩子會否因此張照片感到尷尬？尊重孩子之數字人格，自剋制“曬娃”開始。

本章小結

- 賜手機之年齡無標準答案，但 3-6 歲儘量不碰、13 歲前用兒童手錶或功能機過渡
- ” 夠用遂好”：第一臺手機選千元機而非旗艦，性能過剩等於誘惑過剩

- iOS 屏幕使用時間與安卓 Family Link/品牌家長控制乃兩套技術防線，配合使用更穩
- 青少年模式與遊戲防沉迷乃法規要求，但需家長主動開啓並守護密碼
- 網絡素養較技術攔截更要：教孩子識別欺凌、護隱私、批判內容
- 健康習慣要自家長作起——汝放下手機，孩子方將放下
- 兒童數字遺產與隱私乃新興議題，平時遂應作”遺產聯繫人”設置並剋制曬娃

第三十六章：手機與智能家居——使家生智

手機尚能任一日益要之事——為舉家之「遙控」。自一燈、一插座至一門鎖，智能家居方自「極客之玩」變為「家常之設」。此章為爾理清協議、生態、中樞與入門之套件，終以安全清單收之，使爾不為「智能」所反噬。

一、智能家居之協議：所擇至要

購智能之器，先觀其支持何協議，此定其能為爾之機與生態所使與否。今主流之協議有四，其性各異如下：

- Matter 之協議：2022 年由 CSA（連接標準聯盟）所頒，蘋果、谷歌、亞馬遜、三星諸巨擘合於其後。其志惟一——使諸品牌之智能器同其語言。2023 至 2026 年，正驟為一統之標準，支持之器可同接 Apple HomeKit、Google Home、Amazon Alexa 與米家，不復為一生態所縛。

- Thread 之協議：Matter 底層網絡之「管道」。耗電少、自組網（一節斷電，信號自繞他路），應之速而不滯。購帶 Thread 邊界路由器之器（如 HomePod mini、Apple TV 4K、小米中樞網關），其驗顯然更穩。

- Wi-Fi：帶寬大，可徑連路由器，然耗電多，器眾則易擠佔信道，宜攝像頭、掃地機之屬需傳大數據者。

- 藍牙（BLE）：配網簡、省電，然距短、不能自組網，宜門鎖、溫濕度計之屬小數據之器。

- Zigbee：老牌低耗自組網之協議，須專用之網關，器廉而穩，然跨品牌互通者劣，方為 Matter+Thread 所漸代。

一言之則

2026 年購智能之器，先視有無「Matter over Thread」之標——此當今最經得起時驗之所擇也。舊器若僅支持 Wi-Fi 或藍牙，亦毋急棄，且用之，新購乃趨 Matter。

二、三大生態之比

協議者，「語言」也；生態者，「圈」也。雖皆支持 Matter，日用仍須擇一生態為主，以搭自動化與聲控。三大主流生態之比如下：

生態	所長	短板	適用之人
Apple HomeKit	隱私至強（以本地處理為主）、與 iPhone、Siri、快捷指令相聯甚順	設備偏少偏貴、須 HomePod 或 Apple TV 作中樞	舉家蘋果之器、重隱私與體驗之用者
米家（小米）	器至富、性價比至高、小愛同學易用	生態較封閉、數據多上雲、廣告偏多	預算有限、欲一處配齊全屋之初學者
華為智聯（鴻蒙）	與鴻蒙之機、平板、手錶無縫流轉、器數驟增	海外之器支持者少、生態尚於追趕	華為機之用者、重「超級終端」多器協同者

擇生態之決策建議

觀機以擇生態：iPhone 之用者先 HomeKit，華為之用者先智聯，餘安卓之用者米家最省心。恃 Matter 之協議，異生態之器亦可為一機所制，不必一次「站隊」。

三、以機為家居之中樞

多數入門之境，機即爾「家居之中樞」——以 App 制燈光、空調、窗簾、安防，且可搭自動化之境，使家「自能運轉」：

- 制器：啟 HomeKit、米家、智慧生活之 App，可遠程或本地以開關燈、調色溫、定空調之溫、啟閉窗簾、觀攝像頭之畫。
- 自動化之境：合「條件動作」而為之，常用三大之境：回家之式：定位至家 + 天暮啟玄關之燈 + 啟空調 + 落窗簾 + 閉門鎖。
- 離家之式：末者出門（定位離家或門鎖上鎖）熄燈 + 關空調 + 閉插座 + 啟攝像頭 + 佈防。
- 就寢之式：每夜 23:00 熄全屋主燈 + 落窗簾 + 空調切至 26 度 + 啟夜燈。

NFC 一碰：貼 NFC 標籤於門，機一觸即發其應之境（一觸啟門燈，再觸切就寢）。iPhone 以「快捷指令」為自動化，安卓以「錢包」或廠家之 App 皆可寫入。

試之：搭一「回家模式」

於機智能家居 App 之「自動化」中新建一條：發之之條件擇「至某地」（家），動作依次加「啟玄關之燈」「空調設 26 度」「窗簾閉」。既存，出門一遊，歸時觀其燈自明否——初成之快，勝於新購一器。

四、入門套件推薦

不必一次配齊，自此四件始，可蓋日用之需八成：

- 智能音箱：小愛同學（米家生態，性價比高）/ HomePod mini（蘋果生態，兼為 Thread 中樞）/ Alexa（海外生態廣）。慎之：音箱定爾聲控之生態，先定生態而後購音箱。
- 智能燈泡：Yeelight（米家、Matter，色溫亮度可調，廉）/ Philips Hue（色佳而穩，然須 Hue 網關，稍貴）。慎之：購前核燈座之口徑（E27、B22）與需調色否。
- 智能插座：小米智能插座（廉、計電量）/ TP-Link Kasa（海外 Matter 支持佳）。慎之：用於大功率之器（熱水器、電暖器），必觀額定之功率，毋過載。
- 智能門鎖：鹿客、小米（性價比、米家相聯）/ 凱迪仕（傳統鎖具之大廠、機械之構堅實）。慎之：核其支持 Matter 或爾之生態否，留心電池之續航與應急鑰孔之所於。

選購順序建議

先購智能音箱（定生態）次購智能燈泡與插座（門檻低、效驗立見）終購門鎖與攝像頭（繫乎安全，慎擇之）。一次惟增一類之器，既熟乃廣，免「購而不能用」之患。

五、安全注意事項

家之智能器愈眾，受攻之面愈廣。下列四條，底線也，請逐條對照：

1. 改其默認之密碼：攝像頭、路由器、門鎖出廠之默碼，網上盡公開矣。既得，首事即改為「長且不重複」之強碼。
2. 及時更新固件：廠家所推之固件，多補安全之漏。於 App 中啟「自動更新」，或至少每月手查一次。
3. 網絡之隔：置 IoT 之器於一 Wi-Fi（或訪客之網），與機、電腦之主網分之。萬一某器被破，亦難橫向脅及爾之要數據。
4. 護其私：攝像頭啟「私域」之蔽（如寢室、廁所不攝），閉雲端二十四小時之錄影；家中攝像頭之畫面，毋長期開予第三方 App。

此數坑毋蹈

- 毋以門鎖「遠程開鎖」之功予陌生人——宜用臨時之碼，用畢即廢。
- 毋以攝像頭徑對牀榻、對窗之外——所洩者，爾己之私也。
- 毋貪廉購「白牌」無名之器——多不更新固件，為猶於家置一後門也。

本章小結

- Matter+Thread 者 2023 至 2026 年一統之向也，購新器先觀此標
- 三大生態各有所長：蘋果重隱私與體驗、米家重性價比、華為重多器協同
- 機可為中樞，搭好「回家、離家、就寢」三大之境，家遂能自運矣
- 入門四件：智能音箱、燈泡、插座、門鎖，依次第漸置之
- 改默碼、新固件、隔網絡、蔽其私——安全底線四條，不可省也

七、AI 編程實戰篇（第 37-47 章）

第三十七章：範式之革——自「作碼」至「指 AI 作碼」

賀爾，展此書者，正立於職業範式之革之中央。為革之起，乃 2025 年一程序員於社交之媒所發之一言。聞之似近戲劇乎？然事實固如為也。

一、Vibe Coding 之誕生：一言所起之運動

2025 年二月，Andrej Karpathy——其名請先識之，後尚數見之——於社交之媒發一推文，後為人所屢引：「有一新編程之法，曰 Vibe Coding，爾全憑感（vibes）而行，略碼之細節，使 AI 包攬一切。爾見何言何，複製而粘之，大抵能用即止。」

此言似脫口而出，實精名一已悄然生之範式之移：人不復逐行作碼，而以自然之語述其意，使 AI 成其實。Karpathy 之措辭微含戲謔，然其後有深察焉：AI 生碼之力既逾一閾，編程之隘，不復於「此行吾能作否」，而於「此事吾思之審否」。

> **小識：Vibe 何謂也？**
>
> Vibe 直譯為「氛、感」。Vibe Coding 者，「隨感而指 AI 編程」也——爾述所欲，AI 任其實。識此詞，為革之「出生之證」也。

自彼時始，「編碼」之義始分：舊之編碼，譯設計為語法；新之編碼，譯意為令。前者試指，後者試腦。

Vibe Coding 之詞，驟自戲謔而為嚴肅工程之念。至 2026 年之中，已自社交之媒一梗，演為一套完備之方法論——自需求之表、上下文之管、迭代之驗至質量之保，全流程悉涵焉。人亦悟：「憑感」者，入門之態也；真高效之 AI 編程，須系統之思維。而此思維之系，即此書所欲授爾者也。

二、2026 年之勢：Agentic Engineering 之世

至 2026 年八月，AI 編程之域已歷數要範式之迭代。Vibe Coding 者初火也，然眾旋覺：徒「憑感」而與 AI 語，所生之碼，質至不穩——時而驚豔，時而傾覆。於為範式始進：自簡之對話式編程，趨結構化之 Agent 工程。

又 Karpathy 也，2026 年之初，倡 Agentic Engineering 之念。異乎 Vibe Coding「言出即

生」者，Agentic Engineering 所重者：予 AI 一之，使自規其步、調其器、驗其果、迭而正之。AI 不復徒生碼之器，乃能感境、能決策、能行事之智能體（Agent）也。

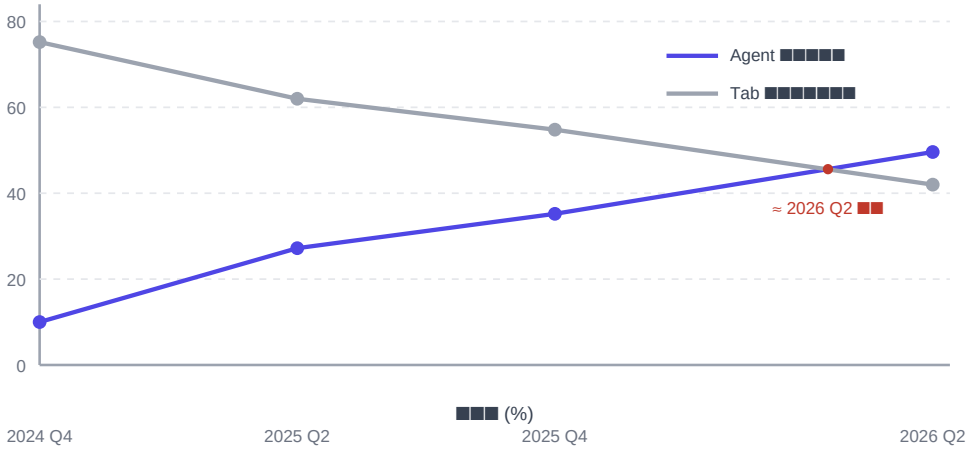
此變有三要焉：

1. 交感之式變：自「一問一答」變為「既設其之而授權」。

2. 工之所產變：自「一段之碼」變為「一完整之項目」。

3. 人之所任變：自「逐行審碼」變為「定其界、驗其果」。

數不誣也：Cursor 於 2026 年四月所頒之 Cursor 3 中，Agent 模式之使用率首超 Tab 補全之式，為開發者已大規模趨此工法之徵；而 Claude Code 者，終端原生 AI 編程之器也，至 2026 年之中，年化之入已達二十五億美元之規模。下之趨勢圖，可使爾親睹此交會之時：



圖一：AI 編程交感模式使用率之趨勢（據行業調研之數估算，2026 Q2 Agent 模式超 Tab 補全）

然實言之：器與模型之變，表象也。無論爾用 Cursor 抑 Claude Code，無論底層之模型為 GPT-5 抑 Claude 4.5，成之者皆循同一底層之方法論。此本指南所欲授爾者也。

三、核心之悟之變：自 Coder 至 Conductor

舊之編程之教，授爾為 Coder——解語法、記 API、調邏輯之誤。AI 編程之世，此任方被重定。爾不復須親作每行之碼，然須為「指揮官」（Conductor）：定戰場、立規則、分其任、驗其成。

此變之核，不於「爾能作碼否」，而於「爾能思審所欲為否」。良將不必為至勇之卒，然必最審戰之之、地之形、兵之布。同理，善 AI 編程者不必通每種編程語言之細，然必最審項目之之、架構之限與質量之準。

此表宜爾再觀——為變之全義，盡括於斯：

維度	舊之 Coder	AI 之指揮官（Conductor）
核心之技	記語法、調 API、除誤	定需求、管上下文、斷驗收
工之方式	逐行作碼	述其意，審其產
質量之控	己所作者，己心有數	以測試與審查驗 AI 之產
所需之知	深通特定之語言、框架	廣識技術之全貌，深解業務之邏輯
隘之所於	編碼之速與語法之記	思之審與達意之能
誤之所自	語法之誤、邏輯之漏	需求之晦、上下文之乏

舊 Coder 與 AI 指揮官之能維之比

此非謂編程之知不足重也。正相反：能解 AI 所作之碼，方能審而正之；解數據之構與算法，方能斷 AI 之策合理否。知之所積，定爾「指揮之上限」——然知之所任，自「執行之器」變為「決斷之據」矣。

四、方法論何以重於工具

器三月一迭代，方法論三年一進化。習一器之操作，三月後或已陳；得一思維之法，三年後猶可用。

多人初學 AI 編程，首問「何器至善」「何模型至強」。此問非不重要，然所問之序誤矣。2026 年八月之器局，與 2025 年八月迥異——Trae、Qoder 諸器興衰起伏，Cursor 自插件而為獨立之 IDE，復進至 Cursor 3；Claude Code 自實驗之案而為終端編程之主流。然劇變之後，成者所循思維之則，幾無所變。

為故此書不惜費篇幅於某器之細操——此等之內容，甫成即陳舊。反之，全書之重，盡於一套已驗之思維方法論：何以思需求、何以與 AI 通、何以驗其產、何以續而迭代。得此法，則易任何之器皆可速上手；徒學器之操，一易其器，則自零始矣。

慰零基之讀者

爾若零基，毋為「方法論」三字所懾。方法論者，非玄之哲也，乃「遇事當何以思」之操作之冊也。譬之學庖：可循譜而步（器之操），然大庖所解者，「何以如此調味」「火候何以制」（方法論）也。此書自至基之念始，每一則皆配對比之案例，使爾非惟解「何謂」，更能得「何以為」。

來動手：為今之爾攝一「角色之影」

出紙一張，書三問之答：(1) 爾今何以用 AI 助爾作碼（若曾用之）？(2) 爾肖上表之「Coder」抑「指揮官」？(3) 爾望三月後之己，何以述其工法？書畢夾之書中，讀竟第四十七章復觀之——爾將見其變之速，逾於所想。

本章小結

- 2025 年 Karpathy 以「Vibe Coding」名 AI 編程之革：自逐行作碼，變為以自然之語述意

- 2026 年進為 Agentic Engineering：與之之、授之權，AI 自規劃而行，人任驗收
- Cursor 3 之 Agent 模式使用率首超 Tab 補全；Claude Code 年化之入達二十五億美元
- 爾之新任為 Conductor（指揮官）：定戰場、立規則、分其任、驗其成
- 工具每三月一迭代，方法論每三年一進化——學方法論，勿徒學操作

第三十八章：AI 編程五大思維之則

若此書惟可使爾記一章，則必此章也。五大思維之則者，全書之心也，後之諸章——上下文、需求、對話、質量——皆此五則於異境之展也。毋憂，每一則皆非難解，然每一則皆值爾以數項目「長入肌理」。

事之起頗可觀：Andrej Karpathy 開源其項目時，附一僅 70 行之 CLAUDE.md 之檔，括與 AI 協作之四核之則。此檔於 GitHub 得 Star 七萬餘，為天下 AI 編程實踐者至廣引之則。此四則非虛號，乃自大量實踐中提煉、可操之思維之習也。本書於其上增第五則——迭代驗證——以成完備五則之系。

70 行之檔，何以值錢若此？

Star 七萬，何謂也？GitHub 之上，項目之絕大多數終身不能至斯數。一 70 行之文檔而得之，適證一事：AI 之世，至稀缺者非碼，乃「思審」之能也。此 70 行，「思審」之濃縮也。

一、則一：Think Before Coding（先思而後碼）

此五則中之至反直覺者也。AI 既可助吾作碼，何必「先思」？答至簡：AI 產之質，與爾入之質正相係。爾自未思審所欲為，則 AI 之答徒使爾陷「改而復改」無已之環——俗謂「AI 與爾偕迷」也。

「先思」者，謂未下令於 AI 之先，爾須答三問，合成「思之三角」，缺一不可：

1. 吾欲解何事？（其之）
2. 吾所望之果何若？（驗收之準）
3. 有何約束之條當守？（其界）

思之維	晦之入（劣）	審之入（佳）
事之定	助吾作一網站	為吾作一個人工具之站，陳三 AI 小工具入口之卡
所望之果	稍美觀可矣	響應式佈局，暗色之題，卡有懸停之動，移動之端可用
約束之條	（未言）	不用框架，純 HTML+CSS+JS，單檔部署，無須後端

同一需求晦入與審入之比

慎之：先思 ≠ 先作完備之文檔

「先思」非謂「先作至善之 PRD 而後始」。乃速而有序之思也——費三十秒思審上之三問，遠勝費三十分鐘於對話中屢改。小任三問可短；大項目可廣之為完備需求之文（詳第四十章）。

二、則二：Simplicity First（簡為至上）

AI 編程至大之阱，非「作不成」，乃「作之過多」。爾謂 AI「作一用戶管理之系」，彼或生認證、權限、角色、審計日誌、數據導出諸功能一整套——而爾今所需，或惟一簡之登入之頁。生之過度，AI 碼膨脹之主因也。

「簡為至上」者，令爾每下令皆循「至小可行」之則：以至寡之碼成核心之功，以至簡之策解事，以至寡之能驗其念。Karpathy 有至精之喻：「爾欲算一折價，無須一電商之引擎。」

行之於實，「簡為至上」者，數具體之操作之習也：

- 每次惟使 AI 成一功能之點，毋一氣而述全系；
- 先擇標準之庫與簡之策，毋引繁複之依賴；
- 可 50 行碼解之事，毋使 AI 作 500 行；
- AI 之產逾所望之域，則勇告之曰「削其贅」。

情境	過度之策（宜避）	簡之策（宜取）
須存數據	架 MySQL 之庫 +ORM	先用 JSON 之檔或 SQLite
須用戶登入	OAuth2.0 + JWT + 角色之權	先作簡之密碼之驗
須前端之框架	React + Redux + Router	先試純 HTML + 原生 JS
須部署而上線	Docker + CI/CD + K8s	先於本地行通而後論

常境過度之策與簡策之比

三、則三：Surgical Changes（改之必精）

爾迭代既有之項目，每改皆當如外科之術之精——惟動當動之處，毋觸不當觸者。此於 AI 編程尤要，蓋 AI 有「順手一改」之習：爾使修一 bug，彼或因而重構半檔。

「改之必精」之核要：每與 AI 之令，明指所改之域與界。告之曰「惟改 calculate_discount 之函」「毋動他部」「存今碼之風」。AI 之改逾爾所求之域，則立令退其贅。

Karpathy 舉 diff（差異之比）一例以明之。設爾使 AI 於購物車折算之函增「會員價」之邏輯：善之產者，惟改此函，增數行之碼而已；惡之產者，重構全檔——新碼雖或更「雅」，然爾不能定其有無新 bug。AI 編程之中，「觀之更佳」終不得為違令之辭。

劣之令（其域晦）

```
助吾優此項目之 折扣計算之邏輯
```

佳之令（精指其域）

```
惟改 cart.py 中之 calculate\_discount 函： 1. 於現有折扣之上增會員價之邏輯 2. 會員另
    享 10\% 之折 3. 毋改函之署與他函 4. 存今變數命名之風
```

四、則四：Goal-Driven Execution（以之馭行）

此則答一要問：爾當下令於 AI 以「何以為」，抑以「作何事」？Karpathy 之答，後者也。與其告 AI「先立一類，次增其法，後作其測」，毋寧告之「吾須一器，能析 CSV 之檔而計每列獨一之值之數」。使 AI 自擇實現之徑，爾任驗其果。

「以之馭行」之利有二。於 AI，得更大自主之域，可擇至優之徑，不為爾中間之步所縛；於爾，移其注自「何以為」至「所作當否」——此正指揮官之所當為也。

慎之：以之馭行 ≠ 放而不問

為須爾於 AI 行後嚴驗之：果足需求乎？邊界之況處乎？有顯然過度之設乎？果不副所望，則予具體之饋——非曰「重作」，乃曰「此之弊於空列表之入則報 IndexError，請增空值之檢」。

令之風	其例	果之性
命令之式（何以為）	「先立一 Python 之類，名曰 DataParser，增一 parse 之法，受 filename 之參，以 csv 之模讀之……」	AI 遂為繕寫之員，爾中間之步或非至優之徑
以之之式（作何事）	「吾須一器：讀 CSV 之檔，計每列獨一之值之數，出以 JSON 之式。請畀完備之實。」	AI 自擇至優之徑，爾任驗其果

命令之令與以之之令之比

五、則五：Iterate and Verify（迭代而驗）

此本書於 Karpathy 四則之上所增之第五則也。其核之信：AI 初產，幾終非定本。有效之 AI 編程，乃「生—驗—正」迭代之環，非一次之「令—產」也。

「迭代而驗」非被動之「不副則再試」，乃有結構之驗之流程也。AI 每有所產，循三層而驗：

- 一層・功能之驗：其能行乎？入出當乎？
- 二層・邊界之驗：空之入、過長之入、異常之入，其將何如？

- 三層・架構之驗：碼之構當乎？有硬編碼乎？便於後續之擴乎？

定義目標

AI 生成代碼

功能驗證

通過

邊界驗證

通過

架構審查

通過，進入下一步

不通過

提供具體反饋

重新生成

圖二：AI 編程迭代驗證之環

此環之要，於「予具體之饋」一步。驗之不過，勿曰「不對，重作」，當曰「此函於空列表之入則報 IndexError，請於第 15 行增空值之檢」。饋之具體，AI 乃能精正；饋之晦，徒使其復測爾意而已。

六、Harness Engineering：Agent = Model + Harness

2026 年之中，有所謂 Harness Engineering 之念，始行於 AI 編程之社羣。其核之式至簡：

Agent 之能力 = 模型能力 × Harness 質量

模型之能，AI 腦智之上限也；Harness 者，繞模型而搭之「工之境」——上下文之管、工具之集成、流程之編排、記憶之系之屬皆於其中。

此念解一常惑：同用 GPT-4 或 Claude，何以有者能作驚豔之項目，有者徒得平庸之碼？異不於模型，而於 Harness。善之 Harness，含：明澈之項目規範之檔、合理之工具集成、有序之對話上下文、完備之驗證流程。模型定智之上限，Harness 定爾能盡此上限幾何。

天才實習生之喻

爾可想像模型為一天才實習生——智至高，然不畀項目背景之檔、不告以用何器、不為設期限與驗收之準，亦不能作善物。Harness Engineering 者，「何以為天才實習生搭善工境」之工程也。

此書後之諸章，實皆授爾構善之 Harness：第三十九章授爾管上下文（項目記憶之系），第四十章授爾定需求（PRD），第四十三章授爾對話之術（Prompt 分層），第四十五章授爾質量之保（驗證之流程）。合而為之，即爾之 Harness 也。

來動手：以「思之三角」析爾下一之念

擇一爾久欲為而未動手之小物（如一倒數日之網頁、一飲水提醒之腳本）。費 3 分鐘以答：(1) 其所解何事？(2) 作成何若？(3) 有何約束（時、技、境）？乃以答成一段之文。賀爾，爾已畢人生首次「Think Before Coding」——此段之文，即一合格高質之令也。

本章小結

- 五則源自 Karpathy 70 行之 CLAUDE.md（70K+ Star），本書增第五則「迭代驗證」
- Think Before Coding：言之先答三問——作何事？果何若？有何約束？
- Simplicity First：AI 至大之阱於「作之過多」——算折價無須電商之引擎
- Surgical Changes：改如外科之術之精，拒 AI「順手一改」
- Goal-Driven Execution：言「作何事」，不言「何以為」，使 AI 自擇其徑
- Iterate and Verify：三層之驗（功能、邊界、架構）+ 具體之饋，循環而迭代
- Harness Engineering：能 = 模型 × Harness，視 AI 為「天才實習生」而理其境

第三十九章：上下文之工——使 AI 解爾之意

爾有此驗乎：易一新對話之窗，*AI* 如失憶然，盡忘爾項目之概。此非 *AI* 故慍爾，乃其天性「健忘」。此章授爾為 *AI* 裝一套「長憶之系」，且識 *2026* 年至火開發範式之一——規範驅動之開發。

一、項目記憶之檔：AI 之「長憶」

AI 之模型有一根本之限：其憶乃「短時」者。每對話之始，AI 於爾項目所知為零。爾須於對話中屢告之曰「吾儕用 React」「後端乃 Node.js」「數據之庫乃 PostgreSQL」「碼之風用雙引號」。此非惟低效，且易有遺——如日日為新至之同僚復作入職之訓。

解之之方，乃「項目記憶之檔」——置於項目根目錄之純文字之檔，AI 之器每於對話之始自讀之。異器異其命名之約：

其器	檔之名
Cursor	`.cursorrules`
Claude Code	`CLAUDE.md`
GitHub Copilot	`copilot-instructions.md`
通用之方	`AGENTS.md`

諸器項目記憶之檔命名之約

名雖異，理則一：以爾欲 AI「恆記」之訊，書於此檔。善之項目記憶檔當含：項目之概（此何項目）、技術之棧（用何技）、碼之規（命名、縮進、註之風）、目錄之構（碼何以組織）、開發之約（分支之策、提交之規、測試之求）。Karpathy 之 CLAUDE.md 惟 70 行，然要訊悉備——簡而完也。

```
# 項目：個人記賬應用
## 概述
此為一個全棧個人記賬應用，助用戶記日常收支。
## 技術棧
- 前端：React + TypeScript + Tailwind CSS
- 後端：Node.js + Express + TypeScript
- 數據庫：SQLite（開發）/ PostgreSQL（生產）
```

```
- 測試：Vitest
## 代碼規範
- 用雙引號
- 縮進2個空格
- 函數名用camelCase
- 類型定義用PascalCase
- 每個函數必須有JSDoc註釋
## 目錄結構
/src
  /frontend  - 前端代碼
  /backend   - 後端代碼
  /shared    - 前後端共享之類型
  /tests     - 測試文件
## 開發約定
- 提交信息用中文
- 新功能必須先寫測試
- API返回統一用 { success, data, error } 格式
```

慎之：毋作「第二份需求之文」

留心此檔之長——毋作數十頁需求之文。項目記憶檔之旨，於「使 AI 速立背景之知」，非「詳述每功」。宜簡、宜有序、惟書「AI 所當知」。

二、規範驅動之開發（SDD）

Spec-Driven Development（SDD，規範驅動之開發）者，2026 年漸熟之開發範式也，其核之念一言：先作規範，後作碼。異乎舊 TDD（測試驅動之開發）者，SDD 之「規範」非惟測試之例，並含需求之述、接口之定、數據之模之屬。AI 據此規範以生碼，人之職，作與守規範也。

SDD 工作之流程，五步而行：

1. 以自然之語述需求之規範（書於一.md 之檔）；
2. 定接口與數據之構（書於規範之檔）；
3. 使 AI 據規範生碼以實現之；
4. 行測試以驗所產副規範否；
5. 不副，則正其規範，或使 AI 正其碼。

此流程之中，規範之檔乃「唯一真理之源」——碼可復生，規範必先正。

編寫規範文件

審查規範為否完備？

完備

AI 根據規範生成代碼

運行測試驗證產出

不完備 / 測試不過

代碼問題修正代碼；規範問題修正規範

回到驗證

全部通過交付

圖三：規範驅動之開發（SDD）工作之流程

SDD 之長：規範之檔較碼為穩。底層模型或器有變，爾之規範檔依然有效，惟使新 AI 之器復生其碼可矣。此亦本書故重「方法論重於器」——規範之檔，即爾方法論之具載也。

三、上下文之窗之管理

雖有項目記憶之檔，爾仍將於對話中遇「上下文不足」之患。AI 模型上下文之窗（能同時理之訊量）有限。對話過長、所引之檔過眾，AI 則「忘」其早訊，產之質遂降——如與友語六小時之久，彼早忘爾始言之首事矣。

理上下文之窗之核策，乃「分層而引」：

上下文之層	其內	管理之法
一層：項目之憶	技術之棧、規範、目錄之構	書於 CLAUDE.md 諸檔，自載入，毋須復述
二層：檔之引	今務所涉之碼檔	以 @ 之符精引之，毋粘全文於對話
三層：對話之上下文	今務之對話之史	一務既畢，開新對話，免上下文之膨脹
四層：即時之令	此一步之具體之求	明、簡、有之，免冗長之述

上下文之窗分層管理之策

四、上下文之工實踐之要

上下文之工，非一時之設，乃續而優之之程也。下數端，實踐中屢驗之要：

1. 記憶之檔，隨項目之演進而偕新。引新技或改架構，第一時新其記憶之檔，毋於對話中臨時言之。

2. 善用 @ 引，毋粘全文。AI 編程之器多支持 @file 或 @folder 之語法，會於幕後將檔之內容注入 AI 之上下文，較手粘尤效，且不穢對話。

3. 對話宜「短小精悍」。一對話惟理一務或一功能之點。畢則開新對話以理其次。長對話非惟耗上下文之窗，且使 AI「分心」。

4. 使 AI「觀」其果，毋徒「述」其果。遇 bug，則以誤之日誌徑以 @ 引予 AI，毋以己語述「若有某誤」。AI 觀原訊，勝於觀爾之轉述也。

5. 定期清而歸檔。既畢之對話與臨時之檔，佔工之空間。定期清無關之內容，保項目目錄之潔，亦助 AI 更效以解項目之構。

來動手：為爾項目作一 70 行之記憶檔

擇一爾方為（或欲為）之項目，作一 CLAUDE.md：項目之概（2-3 語）、技術之棧（列之）、碼之規（3-5 條）、目錄之構（畫一樹）、開發之約（2-3 條）。限 70 行以內——既成，爾將覺於項目之解，明瞭多矣。

本章小結

- AI 之憶乃「短時」者：每新對話，於項目所知為零
- 項目記憶之檔（CLAUDE.md 之屬）乃 AI 之「長憶」，每對話自載入
- 善之記憶檔 = 概述 + 技術之棧 + 規 + 目錄 + 約，70 行足矣
- SDD 規範驅動之開發：先規後碼，規範乃「唯一真理之源」，較碼尤穩
- 上下文分四層而理：項目之憶 @ 檔之引對話之史即時之令
- 實踐五要：偕時而新、@ 引、短對話、予原訊、定期而清

第四十章：需求之工——化念為 AI 可行之令

若上下文之工所解者「使 AI 知其背景」，則需求之工所解者「使 AI 知此次所作何事」。此 AI 編程至顯人我之差之節也——同器同模，有者能作令人驚豔之項目，有者徒得平庸之玩。其異往往不於技之能，而於述需之能。

一、自晦之念至結構化之需求

每項目皆始於一晦之念——「吾欲作一記帳之 App」「吾欲作一個人網站」「吾欲作一待辦清單之器」。此念自無過，然爾徑擲與 AI，則產之質，全繫乎運。需求之工之之，即化此晦之念為 AI 能精行之結構化之需求也。

可喜者：結構化之需求，毋須爾作數十頁之 PRD。惟須爾答五問，吾儕謂之「需求五要」：

其要	其問	其例：記帳 App
Who	誰用之？	自用之，記日用之收支
What	作何事？	一能記每筆之收支、按月而總、可視之應用
Why	何為而作？	欲知己消費之構，得其不必之支出
How	大致何以為？	前端用 React，數據存瀏覽器之本地，姑不作後端與登入
Constraint	有何限制？	一人為之，2 週而畢，毋須上線部署

需求五要與其例

此五要毋須作正式之文，即於腦中疾過一遍亦足矣。要於養「先思審而後言」之習。爾將見，能明答此五問，則爾述需自結構化矣。

二、需求拆解分層之藝

項目之大者，一述需不足也。須拆大需為小而可獨驗之子需。此程謂之「需求之拆解」，其核之則：每子需求皆當能獨成、獨驗、獨付。

以「作一個人記帳 App」為例，合理之拆解如次：

1. 成基礎收支記錄之功（能增、觀、刪其錄）；
2. 增分類之功（予每筆之錄以飲食、交通等籤）；
3. 增月度彙總之功能（按月統計收支之況）；
4. 增資料可視化（以圖表展示消費結構）。

凡每一步，皆獨立可運行可驗證之功能增量也。

需求拆解之金律：每步既畢，項目皆當為" 可用之物"——縱功能至陋。慎勿使項目滯於" 事事半而無一可用" 之境。

此拆解之法有二利。其一，使君恆能見進展——每成一步，君之 App 即多一功能，此正回饋於長期專案至要（人皆需成就感也）。其二，使 AI 每次任務皆足夠小，以降「AI 理解偏差」之風險——任務愈小，AI 理解愈準確，產出之質愈高。

三、PRD 撰寫模板與實踐

較大之專案（如一有前後端、多功能模組之全棧應用），僅憑需求五要素或不足。此時君須撰一份更正式之專案需求文檔（PRD）。一易誤解之點：PRD 非寫予 AI 觀——乃寫予君自己，助君理清思路，而後君方能予 AI 更精確之指令。

一份善 PRD 不必甚長，然當含六部：專案概述（一段話說清所為）、功能列表（每功能點為何）、技術方案（用何技術棧、大致架構）、資料模型（核心資料結構）、界面描述（主要頁面之貌）、開發計畫（分幾步而成）。

```
# 個人記賬應用 PRD
## 項目概述
一輕量個人記賬之應用，備收支記錄、分類、月度匯總與數據可視化。
## 功能列表
1. 記錄收支：金額、類型(收入/支出)、分類、備註、日期
2. 分類管理：預設餐飲、交通、購物、娛樂、其他
3. 月度彙總：按月計總收入、總支出、結餘
4. 消費可視化：餅圖展示各分類佔比
5. 數據導出：導出爲CSV
## 技術方案
- 前端：React + TypeScript + Tailwind CSS
- 數據存儲：localStorage（無需後端）
- 圖表：使用Chart.js
- 構建：Vite
## 數據模型
Record {
  id: string
  amount: number
  type: 'income' | 'expense'
```

```
  category: string
  note: string
  date: string  // YYYY-MM-DD
}
## 開發計劃
Step 1: 基礎記錄功能 (CRUD)
Step 2: 分類管理
Step 3: 月度彙總
Step 4: 數據可視化
Step 5: 數據導出
```

注意此 PRD 之撰寫風格：用簡潔之列表與結構化格式，而非長篇大論。每功能點皆有明確定義，資料模型以簡單型別描述。此風格之 PRD 不惟君自覽清楚，君予之於 AI 時，AI 亦能精確理解每需求之點。

四、實戰：從想法到方案之完整推演

徒紙上談兵無味，吾儕以一完整之例遍走需求工程之全程。設君之念曰：「吾欲作一學習追蹤器，能追吾每日所學者何」。

第一步，遍核需求五要素。

- Who：個人使用。
- What：一器也，能記每日學習之內容、追學習之時長、顧學習之進度。
- Why：欲養持續學習之習，見己之所積。
- How：先作網頁版，資料存於瀏覽器本地。
- Constraint：零基礎開發，期一週內完成核心功能。

次步者，析其所求也。

1. Step 1：實現「記錄學習條目」之功能（日期、內容、時長）；
2. Step 2：增「學習時長統計」（今日、本週累計）；
3. Step 3：實現「學習日曆」視圖（如 GitHub 貢獻圖之展示）；
4. Step 4：增「學習回顧」之功能（隨機展示往日某天之學習記錄，以助複習）。

第三步，撰一份簡短 PRD。不必長——上之拆解本身即構成 PRD 之主體。整理之為.md 之檔，置於專案根目錄，以為君與 AI 之共同參考。

第四步，始第一步之開發。予 AI 之指令可如為：

```
我須一個學習追蹤器之第一個功能——記錄學習條目。
用戶可添加一條學習記錄，包含日期、學習內容、學習時長（分鐘）。
數據存在localStorage。
前端用React + TypeScript。
請予出完整實現。
```

注意此指令之特點：惟述當前此步之目標（記錄功能），不摻他功能；明技術約束（React、TypeScript、localStorage）；予足夠信息使 AI 知「何所為」與「何以致之」，而具體實現由 AI 自決。此即經需求工程加工之後高品質之 AI 編程指令也。

來動手：為君之「心頭好」撰一份五要素卡

擇一君醞釀已久之小專案，費 5 分鐘填一卡：Who / What / Why / How / Constraint，各一句話。乃拆之為 3-5 個「每一步皆能用」之步驟。君將見：念既書之於紙，即自「白日夢」化為「待辦清單」。

本章小結

- 需求五要素：Who / What / Why / How / Constraint——思慮審明而後啟口
- 需求拆解黃金法則：每步既成，專案皆「能用之物」
- PRD 寫予自己：概述、功能列表、技術方案、資料模型、界面、計畫
- 善指令 = 惟述當前一步 + 明確約束 + 言「何所為」不言「何以致之」
- 完整推演模板：五要素拆解 PRD 予指令，四步而行

第四十一章：2026 年 AI 編程工具全景

先為君打一預防針：本章乃 *2026* 年 *8* 月工具生態之一「快照」。半年之後即將過時——此正本章欲教君之第一課。本章之旨非教君「用何工具」，乃助君立「如何擇工具」之判斷框架。君既解不同工具之設計哲學，則任何時皆能為適己之擇。

一、三大流派與選擇框架

2026 年之 AI 編程工具大別為三大流派，每派之後皆有一不同之設計哲學。先觀此全景圖：

流派	代表工具	設計哲學	適合場景
IDE 內置 AI	Cursor 3, Trae, Windsurf, CodeBuddy	深度集成 AI 之能於編輯器	日常開發、代碼審查、漸進式增強
終端 Agent	Claude Code, Codex CLI, Aider	以終端為界面，AI 直接操作檔案系統與命令	專案級操作、批量修改、CI/CD 集成
雲端 Builder	v0, Bolt, Replit Agent	於瀏覽器中從零創建完整專案	快速原型、零配置上手、即時預覽

AI 編程工具之三大流派

擇具之要，非「孰最善」，乃「孰最宜君當前之場景」。三實用之判斷維度：

- 君編程基礎何如？零基礎擇雲端 Builder，有基礎擇 IDE 內置 AI，高階用戶擇終端 Agent。
- 君專案於何階段？從零始擇雲端 Builder，迭代開發擇 IDE 內置 AI，維護重構擇終端 Agent。
- 君部署之需為何？速驗擇雲端 Builder，正經之業擇 IDE 內置 AI，自動化流程擇終端 Agent。

二、IDE 內置 AI 流派

IDE 內置 AI 為最主流之流派：嵌 AI 之能於君之代碼編輯器，存傳統編程體驗之同時疊加 AI 增強。2026 年此流派之代表為 Cursor 3、Trae、Windsurf 與 CodeBuddy。

Cursor 3

Cursor 為今全球使用最廣之 AI 編程 IDE。2026 年 4 月發布之 Cursor 3，標誌 AI 編程工具之一重要轉折：Agent 模式之使用率首次逾 Tab 補全模式——開發者自「AI 助我補全代碼」轉為「AI 代我完成任務」。Cursor 3 之核心特性有三：多檔編輯（AI 可同時改多檔以成任務）、上下文感知（自動解專案結構與依賴之關）、與終端深度集成（AI 可於編輯器內徑行命令而觀其輸出）。

Trae

Trae 者，字節跳動所出之 AI 編程 IDE 也，於國內市場用戶基數甚大。其 SOLO 模式（亦稱 Builder 模式）乃典型「一語創建專案」之能——君述其需，AI 自主完成自專案初始化至代碼生成之全流程。Trae 之長，於中文之援與國內網絡環境之適配，宜國內用戶快速上手。

其他工具

Windsurf（原 Codeium）以補全代碼流暢之驗著稱，宜求「打字如飛」之驗之開發者。騰訊雲 CodeBuddy 為 VS Code 之插件，長於與雲生態集成，宜擬部署於騰訊雲或開發微信小程序之專案。阿里通義靈碼與百度 Comate 亦供相類之 IDE 集成之驗，各於中文場景優化有其特色。

三、終端 Agent 流派

終端 Agent 者，2026 年增長最速之流派也。核心理念：AI 不當徒為編輯器之一插件，當能如開發者然操作整個開發環境——讀寫檔案、運行命令、查看輸出、安裝依賴。Claude Code 即此流派之代表。

Claude Code

Claude Code 者，Anthropic 所出終端 AI 編程之具也，至 2026 年中期，年化收入已達 25 億美元之規模。其運作與傳統 IDE 全然不同：君於終端以自然語言述任務，Claude Code 則自主讀相關之檔、改代碼、運行測試、查看結果、迭代修正——整個過程不需君啟編輯器。

Claude Code 之長於「全專案感知」：不為編輯器之窗所限，可自由遍覽整個專案目錄，解檔間依賴之關。亦支援 CLAUDE.md 專案記憶之檔，使 AI 於每會話之始即悉專案背景。至若大型專案之重構、批量修改、測試編寫諸務，終端 Agent 較 IDE 插件更高效。

Codex CLI 與 Aider

OpenAI 之 Codex CLI 供相類之終端編程之驗，且支援 /import 之功能——可導入既有代碼庫使 AI 解之。Aider 者開源之終端 AI 編程工具也，支援多種模型（GPT、Claude、DeepSeek 等），宜欲靈活切換模型之用者。諸工具各有特色，然核心理念一也：使 AI 於終端握完整開發環境操作之能。

四、雲端 Builder 流派

雲端 Builder 者，最宜零基礎用戶之流派也。君不必安裝任何軟體，惟啟瀏覽器，述君之需，AI 即於雲端創建完整專案，且供即時預覽。

- v0（Vercel 出品）：長於生成前端 UI 組件與頁面，宜快速製作網頁原型。
- Bolt：支援從一語創建全棧應用，含前端、後端與資料庫。
- Replit Agent：運作於 Replit 之雲端 IDE，非惟創建專案，亦可徑於雲端運行與部署。

諸工具共同之優勢於「零配置上手」——啟瀏覽器即可始。其侷限於：專案既趨複雜，雲端編輯環境之靈活不如本地 IDE。故其最宜「從 0 到 1」之原型階段：以雲端 Builder 速驗想法，驗證通過後乃遷於本地 IDE 深度開發。

五、工具屢變，方法論不易

若明日 AI 編程之器盡亡，汝所當守者三習：先思而後動、每作一步、作畢即驗。器者放大之器也，方法論者基也。器可代，基不可搖。

回顧 2024 至 2026 年工具之變遷：2024 年 GitHub Copilot 為王者，2025 年 Cursor 取而代之，2026 年 Claude Code 異軍突起。每一新工具出現時，皆有人宣稱「此次不同」，然終究發現：善用新工具者，正方法論紮實之人。彼換工具只需數日適應期，而僅學工具操作者一換即須重新學起。

故本書建議之學習路徑：先以一工具（任一皆可）入門，於實踐中體會五大思維原則；方法論既足紮實，乃試不同流派之工具，感受其設計哲學之異；最終君將形成己之工具組合——如以 Cursor 為日常開發，以 Claude Code 為重構與測試，以 v0 為快速原型。工具組合因人而異，然方法論共通。

來動手：為君自配一「兵器之組合」

以三判斷維度（編程基礎、專案階段、部署需求）為己定位：君若零基礎試以 Bolt 或 v0 作一頁面；有基礎以 Cursor 或 Trae 作一迭代任務；欲挑戰於終端跑一次 Claude Code。然後答之：何種流派最合君當前之階段？

本章小結

- 三大流派：IDE 內置 AI（日常開發）、終端 Agent（專案級操作）、雲端 Builder（零基礎原型）
- 擇具觀三問：編程基礎？專案階段？部署需求？
- Cursor 3 之 Agent 模式首次反超 Tab 補全；Claude Code 年化收入 25 億美元
- 雲端 Builder 宜「從 0 到 1」，驗後遷回本地 IDE 深度開發
- 工具每三月一迭代：學方法論者換工具只需數日適應期

第四十二章：大模型全景與 API 實戰

前數章所言皆「如何與 *AI* 協作」，本章解一更基之問：「用何 *AI*？如何用之？」*2026* 年 *8* 月大模型之格局已極豐——國際有 *GPT*、*Claude*、*Gemini* 三強爭霸，國內有 *DeepSeek*、通義千問、*GLM*、*Kimi*、豆包百花齊放。每模型各有其性與長：擇之得當則事半功倍，擇之失宜則事倍功半。本章非惟導君識此諸模型，且教君如何以 *API* 接之入己之應用，及如何依己之需擇合宜之 *Token Plan*（用量套餐）。

一、國際主流大模型：三強格局

2026 年 8 月國際大模型之市場，已形成 OpenAI（GPT 系列）、Anthropic（Claude 系列）、Google（Gemini 系列）三強鼎立之局。每家各有鮮明之特色——吾儕逐一識之。

OpenAI GPT-5.6 系列：全能均衡之六邊形戰士

OpenAI 者，大模型浪潮之開啟者也，GPT 系列至今猶為全球使用最廣之模型。2026 年 7 月發布之 GPT-5.6 系列用三檔定位之策：旗艦 GPT-5.6 Sol 面向高端推理之需，均衡 GPT-5.6 Terra 面向日常開發，輕量 GPT-5.6 Luna 面向高併發低成本之場景。三款模型共享 105 萬 Token 之上下文窗口，皆支援圖像輸入、函數調用與結構化輸出。

型號	定位	輸入（/百萬 Token）	輸出（/百萬 Token）	上下文窗口
GPT-5.6 Sol	旗艦推理	US$5.00（約 36 元）	US$30.00（約 217 元）	105 萬 Token
GPT-5.6 Terra	均衡日常	US$2.50（約 18 元）	US$15.00（約 109 元）	105 萬 Token
GPT-5.6 Luna	輕量低成本	US$1.00（約 7 元）	US$6.00（約 44 元）	105 萬 Token

OpenAI GPT-5.6 系列定價（2026 年 8 月）

GPT 系列之核心優勢於「均衡」——推理、編程、寫作、多模態，無明顯短板。加以 OpenAI 龐大之插件生態與開發者社區，適配文檔與工具鏈最為完善。於「不知所擇」之場景，GPT 乃「閉目擇之亦不誤」之默認答案。然其短於：國內訪問需特殊網絡，中文表達偶有「翻譯腔」，API 價格居中高之列。

好消息

OpenAI 於 2026 年 8 月宣佈免費用戶默認模型升為 GPT-5.6 Luna，且開放無限量文本聊天。為則君可零成本體驗 GPT 之基礎能力，惟需更強推理或 API 調用時方須付費。

Anthropic Claude 系列：編程與嚴謹之天花板

若 GPT 乃全能之選手，則 Claude 為編程領域之「天花板」。Anthropic 之 Claude 系列於 SWE-bench（軟體工程基準測試）長期斷層領先，代碼生成之質與工程級專案落地之能業界第一。尤難得者，Claude 之幻覺率（生成虛假信息之概率）全網最低，僅約 1%——故特宜學術、法務諸高精度之場景。

Claude 之另一殺手鐧乃超長上下文：支援 100 萬 Token 之上下文窗口，能一次「讀完」數百頁之合同或整個代碼庫而不漏細節。此於代碼重構、長文檔分析諸場景幾無可替代。Claude Code 為終端 AI 編程之具，至 2026 年中期年化收入已達 25 億美元——此乃 Claude 編程之能最受認可之證也。

型號	定位	輸入（/百萬 Token）	輸出（/百萬 Token）	上下文窗口
Claude Fable 5	旗艦款	US$10.00（約 72 元）	US$50.00（約 362 元）	100 萬 Token
Claude Opus 4.8	專業款	US$5.00（約 36 元）	US$25.00（約 181 元）	100 萬 Token
Claude Sonnet 5	性價比款	US$2.00（約 14 元）	US$10.00（約 72 元）	100 萬 Token

Anthropic Claude 系列定價（2026 年 8 月，Sonnet 5 為優惠價）

Claude 之短於價格——Fable 5 之輸出價為所有模型中至高者，每百萬 Token 需 50 美元。多模態之能（圖像、影片理解）亦較弱。於預算有限之個人開發者，Sonnet 5 優惠期內之性價比甚為突出。

Google Gemini 系列：多模態之王

Google 之 Gemini 系列至 2026 年已迭代至 3.1 Pro 版。Gemini 至大之殺手鐧乃原生多模態——圖文解析精準、OCR 表格辨識無誤、影片幀擷取高效，且支援音訊影片深度解析。其與 Google 生態（搜索、郵箱、雲服務）深度綁定，於實時信息檢索與跨模態理解表現突出。Gemini 亦支援 200 萬 Token 超長上下文之輸入，為今上下文窗口至大之模型。

Gemini 定位為「科學推理與多模態場景之首選」。若君專案涉大量圖像分析、影片理解或科學計算，Gemini 幾無對手。然其於純文本代碼生成稍遜於 Claude，中文表達亦不如國產模型自然。

對比項	GPT-5.6	Claude Fable 5	Gemini 3.1 Pro
綜合得分	94.8	95.0	92.1
編程能力（SWE-bench）	74.9%	64.3% (Pro)	中等
上下文窗口	105 萬 Token	100 萬 Token	200 萬 Token
多模態能力	均衡	較弱	最強
幻覺率	較低	最低（約 1%）	較低
國內可用性	需特殊網絡	需特殊網絡	需特殊網絡
API 生態	最完善	成長中	綁於 Google 生態

國際三大模型橫向對比（2026 年 8 月）

注意：國內使用之現實門檻

於國內開發者，GPT、Claude、Gemini 核心之障於網絡訪問。雖可經 Azure 代理、API 聚合平臺諸法間接用之，然穩定性與合規性須自行評估。若君專案於資料安全有嚴格之求，宜優先考慮國產模型。

二、國產主流大模型：百花齊放

2026 年國產大模型已自「追趕者」化為「並跑者」，某些領域甚且實現「領跑」。異於 2023 年「套殼」之爭，2026 年國產旗艦模型於邏輯推理、代碼生成、中文理解諸端已與國際一線模型旗鼓相當，差距縮至 5% 以內。尤要者，國產模型於中文場景、價格成本與合規部署有不可替代之優勢。

DeepSeek V4：性價比之王

DeepSeek（深度求索）者，國產大模型中現象級之存在也。2025 年初，DeepSeek-V3 以「較 GPT 廉十之一而能力相當」之定位震撼市場，業內稱之「通用 AI 大模型斬殺線」——廉於彼者不及其好用，貴於彼者不及其量大管飽。至 2026 年 8 月，DeepSeek V4-Pro 正式版發布：1.6 萬億參數 MoE 架構，激活約 49B，支援 100 萬無損上下文，代碼性價比世所廣認為全球天花板。

DeepSeek V4 於 Agent 能力大躍升——DeepSWE 基準自 12.8 躍至 62.7（+390%），NL2Repo 自 38.5 升至 61.5。API 新增 Responses API 與 Anthropic API 雙協議支援，已兼容 Codex 與 Claude Code 等主流編程工具。為則君可 DeepSeek 之 API 代 Claude 之 API，用於 Claude Code——成本降數倍，體驗幾乎一致。

型號	定位	輸入（/百萬 Token）	輸出（/百萬 Token）	上下文窗口
DeepSeek V4 Pro	旗艦推理	3 元（約 US$0.43）	6 元（約 US$0.87）	100 萬 Token
DeepSeek V4 Flash	輕量快速	1 元（約 US$0.14）	2 元（約 US$0.28）	100 萬 Token

DeepSeek V4 系列定價（2026 年 8 月，促銷價）

價格對比：同為 100 萬 Token 之輸入 + 100 萬 Token 之輸出，DeepSeek V4 Flash 花費約 0.42 美元，GPT-5.6 Sol 花費約 35 美元，Claude Fable 5 花費約 60 美元。差距高至 80-140 倍。

避坑提醒：DeepSeek 將漲價

DeepSeek 官方已於 2026 年 8 月 6 日公告「計劃近期大幅上調 API 定價」，促銷價恐難持久。部分媒體傳聞漲幅或達 30 倍，至 8 月 14 日官方尚未公佈最終調價方案。此外，DeepSeek V4 正式版引入峯谷定價之制——高峯時段調用價格徑倍。宜留意官方公告以取最新定價。

通義千問 Qwen3.7 Max：阿里雲企業級之選

阿里雲之通義千問（Qwen）系列倚阿里雲生態，於企業級部署有天然優勢。2026 年 7 月之 Qwen3.7 Max 為當前旗艦：235B 參數稠密模型，256K 原生上下文，可擴展至 1M。Qwen 核心優勢於：開源生態成熟（自 7B 至 235B 全系列開源）、中文場景理解精準、與阿里雲基礎設施深度集成。

型號	輸出價格（/百萬 Token）	適用場景
qwen-turbo	0.6 元	極簡問答、文本分類、高併發
qwen-long	2.0 元	10M 超長上下文處理
qwen3.7-flash	7.2 元	通用對話、內容生成
qwen3.7-plus	12.0 元	多模態（圖片 + 影片）、中文創作
qwen3.7-max	36.0 元	旗艦中文能力、複雜任務

通義千問 Qwen 系列階梯定價（2026 年 8 月）

Qwen 定價用階梯之策——自極低成本之 qwen-turbo（輸出 0.6 元/百萬 Token）至旗艦 qwen3.7-max（輸出 36 元/百萬 Token），蓋自輕量問答至複雜推理全場景之需。已用阿里雲之企業，Qwen 乃至順手之集成選擇。

智譜 GLM-5.1：國產 SWE-bench 第一

智譜 AI 之 GLM 系列 2026 年表現亮眼。GLM-5.1 於 SWE-bench 摘國產第一之績：744B MoE 稠密混合架構，支援 100 萬上下文。其長於中文語義理解精準，支援國內合規私有化部署——此於資料安全要求高之金融、政務場景至要。GLM-5.1 定價輸入 6 元、輸出 24 元每百萬 Token，居國產模型中遊之列。

Kimi K3：超長文本之王

月之暗面之 Kimi 系列以超長文本處理之能著稱。2026 年 7 月發布之 Kimi K3 有 2.8 萬億參數之規模，於多項複雜任務評測入全球第一梯隊。Kimi 核心場景乃長代碼庫分析與長文檔檢索增強——能處理整本書之文本或整個專案之代碼，於「大海撈針」式檢索任務表現卓越。Kimi 定價較高（輸出約 12-20 元/百萬 Token），然於長文本場景之性價比無人能及。

豆包 Doubao：字節跳動全能之選手

字節跳動之豆包系列 2026 年表現穩定。Doubao Seed 2.1 Pro 為火山引擎旗艦模型，256K 上下文，實測中綜合發揮穩定，中文場景體驗突出。豆包之長於：註冊即贈大量免費 Token，火山引擎生態集成完善，定價居國內最低梯隊之一。初學者欲速體驗 AI 編程者，豆包乃零門檻入門之選。

對比項	DeepSeek V4	Qwen3.7 Max	GLM-5.1	Kimi K3	豆包 2.1 Pro
綜合得分	87.5	89.7	90.5	88+	85+
編程能力	極強	強	國產 SWE 第一	強	中上
中文理解	優秀	優秀	精準	優秀	優秀
上下文窗口	100 萬	256K-1M	100 萬	超長	256K
輸出價格（元/M）	6	36	24	12-20	30
開源	MIT 協議	全系列開源	部分開源	閉源	閉源
私有化部署	支援	支援	支持	不支援	不支援

國產五大模型橫向對比（2026 年 8 月）

一重要趨勢：國產模型與 GPT/Claude 之「代差感」已消。於邏輯推理與數學競賽，國產旗艦與 GPT-5 之分差大抵於個位數。於系統化架構思維與複雜工程推理，Claude 仍領先——其架構方案更有「靈氣」，國產模型時顯「教條」。然於中文場景、成本控制與合規部署，國產模型有壓倒性優勢。

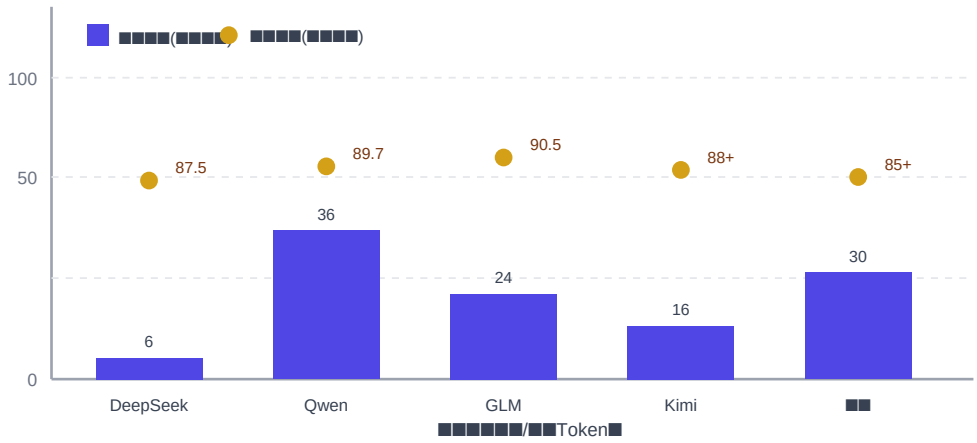


圖 4：五款模型之價格與性能對比（價愈低愈好，分愈高愈好）

三、模型選擇決策框架

既識諸模型之特點，如何為己之專案擇模型？此授君一套「四問決策法」——答四問，即得鎖定至宜君之模型。

汝之項目於國內抑國際部署？

國內選國產模型

|

國際可選海外模型

核心任務為編程抑其他？

編程 Claude 或 DeepSeek

|

其他繼續判斷

須超長上下文 (>50 萬 Token)？

須 Gemini 或 Kimi

|

不須繼續判斷

預算敏感嗎？

敏感 DeepSeek 或 Qwen-turbo

|

不敏感 GPT-5.6 或 Claude

圖 5：大模型選擇四問決策法

小貼士：組合策略

此決策法非教條——諸多場景需組合用多模型。一常見組合策略：以 DeepSeek 或 Qwen 為日常開發（成本低、中文佳），以 Claude 為關鍵架構設計（推理強、幻覺低），以 Gemini 為多模態分析（圖片影片處理）。按需切換模型，而非「一模型用至終」。

四、API 調用入門：從零接入大模型

諸多 AI 編程教程惟教對話界面之用，然真「AI 編程」——使君之應用調用大模型——須經 API。API（Application Programming Interface，應用程序編程接口）者，程序之間通信之協議也。經 API，君之代碼可如調用一函數然，調用大模型之能。

何謂 Token

未解 API 之先，必先明「Token」之概念。Token 者，大模型計費之基本單位也，可解為「AI 閱讀與寫作之單位」。一 Token 約當：英文 0.75 詞（4 字符），中文 0.5-1 漢字。1000 字中文之文本，約耗 1500-2000 Token。

Token 非字數，亦非字符數。大模型用專門分詞器（Tokenizer）切文本為 Token。同詞於不同模型 Token 之數或異——如「人工智能」於 GPT 或為 3 Token，於 DeepSeek 或為 2。多數模型之 API 返實際消耗 Token 之數，君可於 API 回應中察之。

Token 成本速估

100 萬字中文約當 150 萬 Token。以 DeepSeek V4 Flash 處理 100 萬字輸入 + 100 萬字輸出，成本約 0.42 美元（約 3 元人民幣）；以 Claude Fable 5 處理同量，成本約 60 美元（約 434 元人民幣）。此故模型之擇直接影響君專案之成本也。

獲取 API Key

用大模型 API 之首步乃獲 API Key——驗證身份之密鑰也。每模型提供商各有其 API 平臺，君須註冊帳號並創建 API Key。以下乃主流模型 API 平臺入口與獲取流程：

模型	API 平臺	獲取流程	免費額度
DeepSeek	platform.deepseek.com	註冊實名認證創建 API Key	新用戶贈額度
通義千問	bailian.aliyun.com（百煉平臺）	阿里雲帳號登入開通服務創建 Key	新用戶免費試用
智譜 GLM	open.bigmodel.cn	註冊實名認證創建 API Key	新用戶贈額度
Kimi	platform.moonshot.cn	註冊實名認證創建 API Key	免費額度有限
豆包	volcengine.com（火山引擎）	註冊開通方舟創建 API Key	註冊贈大量 Token
OpenAI GPT	platform.openai.com	註冊綁定信用卡創建 API Key	無免費額度（須付費）
Anthropic Claude	console.anthropic.com	註冊綁卡創建 API Key	新用戶少量額度

主流模型 API 平臺及 Key 獲取之法（2026 年 8 月）

安全提示：API Key = 錢囊密碼

API Key 同於君錢囊之密碼——凡得 Key 者皆可君之額度調用模型。故：永勿書 API Key 於代碼中（以環境變數管理之）；永勿以含 Key 之代碼提交於公開之 Git 倉庫；永勿於社羣媒體分享含 Key 之截圖。若 Key 洩露，即於平臺刪之而重新創建。

首次 API 調用

既獲 API Key，君可任何編程語言調用大模型 API。下以 Python 示至簡之調用——向 DeepSeek 發送一訊並取其覆。

```
# 安裝SDK：pip install openai
# DeepSeek兼容OpenAI SDK格式，可直接用
from openai import OpenAI
import os
```

```
# 1. 建客戶端（API Key從環境變量讀取）
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

# 2. 發送請求
response = client.chat.completions.create(
    model="deepseek-chat",  # 指定模型
    messages=[
        {"role": "system", "content": "汝乃Python編程之助。"},
        {"role": "user", "content": "寫一個快速排序函數。"}
    ]
)

# 3. 取回覆
print(response.choices[0].message.content)

# 4. 觀Token之耗
print(f"輸入Token: {response.usage.prompt_tokens}")
print(f"輸出Token: {response.usage.completion_tokens}")
print(f"總Token: {response.usage.total_tokens}")
```

此代碼核心邏輯：創建客戶端（攜 API Key 與模型之址）構造訊息列表（system 設角色，user 為用戶輸入）發送請求獲取回覆。此式幾遍用於兼容 OpenAI 格式之模型——只需換 api_key 與 base_url 即可切換模型。此故諸多國產模型擇兼容 OpenAI SDK 格式：開發者只需改兩行代碼即切換模型，遷移成本至低。

下乃切換至他模型之代碼之異——注意只需改兩行：

```
# DeepSeek
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)
# model="deepseek-chat"

# 通義千問（百鍊平臺）
client = OpenAI(
    api_key=os.environ.get("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)
# model="qwen-plus"

# 智譜GLM
client = OpenAI(
    api_key=os.environ.get("GLM_API_KEY"),
```

```
    base_url="https://open.bigmodel.cn/api/paas/v4"
)
# model="glm-4"

# OpenAI GPT
client = OpenAI(
    api_key=os.environ.get("OPENAI_API_KEY"),
    # base_url用默認值即可
)
# model="gpt-5.6-terra"
```

流式回應處理

實用之中，君常不欲待 AI 寫完所有內容方一次示之——為令用戶待之過久。流式回應（Streaming）可使君於 AI 生成之同時逐字顯示，大增用戶體驗。如打字機然，字一一蹦出，用戶之感全然不同也。

```
# 流式之應：逐字而輸
stream = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "user", "content": "著Python冒泡排序之教程"}
    ],
    stream=True  # 開啓流式響應
)

# 逐塊接收並打印
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
# 輸出如打字機然，逐字而顯
```

五、Token Plan 購買指南

多數模型提供商有二計費之法：按量計費（Pay-as-you-go）與套餐訂閱（Token Plan / Subscription）。明此二法之別，可助君大降使用之成本。

按量計費與套餐訂閱

維度	按量計費（Pay-as-you-go）	套餐訂閱（Subscription）
計費方式	用幾許 Token 付幾許錢	預付固定費用，獲得額度
適合場景	使用量不穩、偶爾調用	使用量穩、高頻調用
價格優勢	標準價	較按量低 20%-50%
靈活性	隨時可用，無須承諾	須預付，額度過期或作廢
超額處理	自動按量計費	超出額度後轉為按量計費
代表平臺	各模型官方 API 平臺	OpenAI Plus/Pro, Claude Pro, Cursor Pro

二計費方式之對比

初學者宜自按量計費始——不必預付費用，用幾許花幾許。月使用量既穩於一水平（如每月逾 50 元），乃考慮切換至套餐訂閱以得更低單價。

主流訂閱套餐一覽

套餐	月費	所含內容	適合人羣
ChatGPT Free	免費	GPT-5.6 Luna 無限文本對話	零基礎體驗
ChatGPT Plus	US$20/月	GPT-5.6 Sol、圖像生成、資料分析、插件訪問	中度使用戶
ChatGPT Pro	US$200/月	無限制用所有模型、優先訪新功能	重度專業用戶
Claude Free	免費	有限次數 Claude 對話	零基礎體驗
Claude Pro	US$20/月	無限制 Claude 使用、優先訪新模型	編程開發者
Cursor Free	免費	有限次數 AI 補全與對話	初學者體驗
Cursor Pro	US$20/月	無限 AI 補全、500 次/月高級請求	日常開發者
Cursor Business	US$40/月/人	團隊協作、管理後臺、隱私模式	團隊使用

主流 AI 工具訂閱套餐（2026 年 8 月）

國內模型訂閱之方案略異。DeepSeek 主按量計費，無月度訂閱套餐——君儲值後按實際使用量扣費。通義千問於百煉平臺供預付費資源包（購一定量之 Token 額度，享折扣價）。豆包於火山引擎亦有相類預付費方案。於國內開發者，至經濟之組合通常為：DeepSeek 按量計費（日常開發）+ 豆包免費額度（體驗測試）。

API 聚合平臺

若君需同時用多模型，API 聚合平臺乃省心之選。聚合平臺供一統一之 API Key，君即得調用全球數十種模型，毋須於各平臺分別註冊。

平臺	特點	價格優勢	適合場景
OpenRouter (openrouter.ai)	海外聚合平臺，一 Key 調用全球模型	價格透明，支援實時比價	需靈活切換多個海外模型
SiliconFlow (siliconflow.cn)	國內聚合平臺，供多種國產模型 API	較官方低 20%-30%	國內開發者，多模型比價
Azure OpenAI（微軟代理）	微軟 Azure 代理之 OpenAI API	Batch API 可打 5 折	企業級 OpenAI 調用，大批量異步任務
百度千帆（qianfan.baidubce.com）	百度 AI 雲平臺，供文心等模型	新用戶贈額度	百度生態用戶

主流 API 聚合平臺（2026 年 8 月）

避坑提醒：勿購「共享 Key」

勿於淘寶、閒魚等平臺購所謂「共享 API Key」——此等 Key 通常來路不明，或隨時被封號，且有資料洩露之險。恆經官方平臺或正規聚合平臺獲取 API Key。

成本優化策略

擇模型得當僅為首步，如何「用得省錢」亦同要。下乃數條經實踐驗證之成本優化策略：

1. 善用緩存命中。諸多模型供「緩存」功能——君之請求若含之前已處理之內容（如系統提示詞），此部分僅以至低價格計費。DeepSeek 緩存命中之價僅為常價之 1/60。置固定之系統提示於訊息列表之首，可極大化緩存命中。

2. 分級路由——以廉價模型處理簡單任務，貴模型處理複雜任務。如以 DeepSeek V4 Flash 為文本分類（輸出 2 元/百萬 Token），以 Claude 為架構設計（輸出 181 元/百萬 Token）。一專案中 80% 之調用可用廉價模型。

3. 限制輸出長度。API 按輸出 Token 計費，輸出愈長愈貴。於不需長篇大論之場景，設 max_tokens 參數以限輸出長度，免 AI「廢話過多」。

4. 以 Batch API 處理非實時任務。若君有大量不需即時回應之任務（如批量資料標註、文檔摘要），用 Batch API 可享 50% 折扣。OpenAI 與 Azure 皆供此功能。

5. 監察 Token 用量。多數 API 平臺供用量統計面板。定期察用量之分佈，尋耗 Token 至多之調用，因之優化。一小小 Prompt 之調整（如改「請詳解」為「請以 3 句解之」）可省 50% 之 Token。

優化策略	節省幅度	施行難度	說明
緩存命中	至高 98%	低	置固定內容於訊息之首
分級路由	50%-80%	中	簡單任務用廉價模型
限制輸出長度	30%-60%	低	設 max_tokens 參數
Batch API	50%	低	非實時任務用批處理
Prompt 優化	20%-50%	中	精簡 Prompt，減少冗餘

成本優化策略及效果（2026 年 8 月）

六、實戰：於專案中接入大模型 API

請以一完整實戰終本章——創一「AI 摘要生成器」：用戶輸入一段長文本，調用大模型 API 生成摘要。此實戰將示 API 調用之完整流程：環境配置、API 調用、錯誤處理、成本控制。

```
# ai_summarizer.py - AI摘要生成器
import os
from openai import OpenAI

# 1. 配置（API Key從環境變量讀取，絕不硬編碼）
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

def summarize(text: str, max_length: int = 200) -> dict:
    """生成文本摘要

    Args:
        text: 要摘要之原文
        max_length: 摘要最大長度（Token數）

    Returns:
        dict: 包含摘要與Token消耗信息
    """
    try:
        response = client.chat.completions.create(
            model="deepseek-chat",
            messages=[
                {
                    "role": "system",
                    "content": "汝為一個摘要生成器。"
                               "請用簡潔之中文總結用戶提供之文本。"
                               "摘要不超過3句話。"
```

```
                },
                {
                    "role": "user",
                    "content": f"請摘要以下文本:\n\n{text}"
                }
            ],
            max_tokens=max_length,  # 限制輸出長度
            temperature=0.3  # 低溫度=更穩定
        )

        return {
            "success": True,
            "summary": response.choices[0].message.content,
            "input_tokens": response.usage.prompt_tokens,
            "output_tokens": response.usage.completion_tokens,
            "total_tokens": response.usage.total_tokens
        }

    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }

# 2. 使用示例
if __name__ == "__main__":
    long_text = """
    （粘貼一段長文本...）
    """

    result = summarize(long_text)

    if result["success"]:
        print(f"摘要: {result['summary']}")
        print(f"消耗Token: {result['total_tokens']}")
        # 估算成本：DeepSeek V4 Flash
        cost = result['total_tokens'] * 0.000004  # 約0.4分/千Token
        print(f"預估成本: {cost:.4f}元")
    else:
        print(f"出錯了: {result['error']}")
```

此代碼現 API 調用數項最佳實踐：API Key 經環境變數讀取，絕不硬編於代碼；函數有明晰文檔與型別提示；設 max_tokens 限輸出長度；temperature 設低值（0.3）以得更穩輸出；有完整錯誤處理；返 Token 消耗之訊以追成本。君以 AI 作此代碼時，可告以此求——彼全然能助君實現。

本章之終極啟示：模型者，”彈藥”也；方法論者，”戰術”也。彈藥之良者可使戰鬥愈順，

然戰術不善，彈藥雖美亦不能勝。既悉各模型之性與 API 之法，汝便握一充足之” 彈藥庫”——然此仗如何打，終賴前數章所授之思維方法論。

來動手：跑通君首次 API 調用

註冊一國產模型平臺（薦 DeepSeek 或智譜，皆贈新用戶額度），創建 API Key，書之於系統環境變數（而非代碼中！），乃運行上「首次 API 調用」之代碼。見 AI 回覆打印而出之刻，君已跨「用 AI」與「造 AI 應用」之界線矣。

本章小結

- 國際三強：GPT-5.6（均衡全能）、Claude（編程天花板 + 幻覺率僅 1%）、Gemini（多模態之王 +200 萬上下文）
- 國產五強：DeepSeek V4（性價比之王，留意漲價公告）、Qwen3.7（企業級）、GLM-5.1（SWE 國產第一）、Kimi K3（超長文本）、豆包（免費入門）
- 四問決策法：部署何地？編程抑其他？超長上下文？預算敏感乎？
- Token 為計費單位：中文 1 字約 1.5-2 Token；100 萬字約 150 萬 Token
- API Key = 錢囊密碼：環境變數管理，絕不硬編、不上傳 Git
- 國產模型普遍兼容 OpenAI SDK：換模型只需改 api_key 與 base_url 兩行
- 省錢五招：緩存命中、分級路由、限制輸出、Batch API、監察用量

第四十三章：對話工程——與 AI 高效協作之對話技術

對話乃 AI 編程核心之交互方式。然請注意：「對話」不等於「聊天」——高效之 AI 編程對話乃一種有結構之溝通技術，吾儕謂之「對話工程」。其所研究者：如何以最少之對話輪次，獲得最高質量之代碼產出。直言之，此章即教汝如何與 AI「將話說清楚」。

一、Prompt 分層策略

與 AI 之對話可分為三個層次，每層承擔不同之功能。理解此三個層次，汝便能如搭積木一般，組合出高效之 Prompt。

層次	功能	示例	頻率
系統層	設定 AI 之角色與全局約束	「汝乃 React 專家，代碼必須用 TypeScript」	項目開始時設定一次
任務層	描述當前所欲作之具體任務	「實現一個表單組件，包含用戶名與密碼字段」	每個任務開始時予出
反饋層	對 AI 產出進行評價與修正	「密碼字段缺少顯示／隱藏切換，請添加」	根據產出質量予出

對話 Prompt 之三個層次

系統層之 Prompt 通常寫於項目記憶檔案中（如 CLAUDE.md），不須每次對話重複。任務層乃汝於對話中最常用者——它應包含「做何」「用何技術」「有何約束」三個要素。反饋層最考驗功力——好之反饋能使 AI 精準修正，壞之反饋只會使其重新猜測。

小訣：反饋三板斧

一個實用之反饋模板乃「指出問題＋說明期望＋提供上下文」。譬如：「第 25 行之錯誤處理只用了 console.log（指出問題），生產環境當用統一之錯誤處理器並返回用戶友好提示（說明期望），項目裡有一個 handleError 函數可用（提供上下文）」。此種反饋比「改一下錯誤處理」有效十倍。

二、調試對話：如何描述錯誤使 AI 快速定位

調試乃 AI 編程中最頻繁之場景之一。汝遇 bug 時，如何向 AI 描述問題，直接決定其能否快速助汝定位與修復。以下為調試對話之四個要點：

1. 予 AI「看」原始信息，而非汝之轉述。將錯誤日誌、異常堆棧、相關代碼直接以 @ 引用予 AI，勿用「好像有個何報錯」來描述。AI 看到原始信息比看到汝之理解更準確。

2. 描述「期望行為」與「實際行為」之差異。「吾期望點擊按鈕後跳轉至首頁（期望），但實際點擊後頁面無反應（實際）」——此種描述比「按鈕壞了」有用得多。

3. 提供復現步驟。「先輸入用戶名，然後點擊登錄按鈕，就會出現此錯誤」——復現步驟使 AI 能模擬汝操作之路徑，更容易定位問題。

4. 說清楚汝已嘗試了何。「吾已檢查了網絡請求，返回 200，數據正常」——此能助 AI 排除已驗證之方向，不浪費時間重複排查。

差之調試描述

```
按鈕點擊沒反應，助吾看看
```

好之調試描述

```
點擊登錄按鈕後頁面無反應。 期望行為：點擊後跳轉至/home頁面 實際行為：點擊後無任何反應，
    控制臺無報錯 復現步驟： 1. 於登錄頁輸入用戶名admin與密碼123456 2. 點擊「登錄」按鈕
    3. 頁面無反應 已排查： - 網絡請求正常，返回200 - onClick事件已綁定 - 已檢查
    LoginButton組件之代碼 相關文件：@src/components/LoginButton.tsx @src/pages/Login
    .tsx
```

三、迭代策略：小步快跑對大步重構

AI 編程中之迭代有兩種基本策略，適用於不同場景。理解它等之區別，能助汝於不同情況下作出正確選擇。

「小步快跑」策略適用於功能開發與 bug 修復。其核心乃：每次使 AI 只作一個小之修改，驗證通過後再作下一個。優勢乃每一步皆可驗證、可回退，風險可控；劣勢乃速度較慢，須頻繁之對話交互。適合：新功能開發、不確定最優方案時之探索、對 AI 產出不太放心時之謹慎迭代。

「大步重構」策略適用於明確知曉目標之大規模修改。其核心乃：一次性描述完整之目標，使 AI 自主規劃並執行整個修改。優勢乃速度快，AI 能看到全局圖景作出更優之架構決策；劣勢乃風險高，若 AI 理解有偏差，修正成本大。適合：明確之架構升級、大規模代碼遷移、有充分測試保障之重構。

維度	小步快跑	大步重構
每次修改量	一個功能點或一個 bug	一個模組或一個架構層
驗證頻率	每次修改後立即驗證	全部修改完成後統一驗證
對話輪次	多（每功能 5-10 輪）	少（每模組 1-3 輪）
風險	低，每步可回退	其弊也，偏差既現，修正之費甚鉅
適用場景	新功能、探索性開發	目標明確之重構、遷移
前提條件	無特殊要求	須有測試覆蓋

兩種迭代策略之對比

實踐中，兩種策略常常組合使用。譬如重構一個模組時，先用「大步重構」使 AI 予出整體方案，審查方案後，再用「小步快跑」逐步執行。如此既保全局之視角，又控執行之風險。

四、多輪對話之上下文管理

隨著對話輪次增加，汝須主動管理對話上下文，避免「AI 忘記了早期信息」之問題。以下為幾個實用之上下文管理技巧。

第一，「錨定」關鍵信息。對話開始時，將關鍵約束寫清楚，並於後續對話中適時重申。譬如「記住，吾儕之數據存在 localStorage，不能用數據庫」——於相關之修改請求中重申此約束，防止 AI 遺忘。

第二，善用「總結—續接」技巧。當對話變得甚長時，可使 AI「總結一下目前之進展與待辦事項」，然後開一個新對話，將此總結作為起點。如此既保留了關鍵上下文，又避免了長對話之退化。

第三，「一個對話一個任務」。此乃最簡單亦最有效之上下文管理策略。每個對話只處理一個任務或一個功能點，完成後開新對話處理下一個。此天然限制了對話長度，保持上下文清晰。

第四，利用工具之上下文管理功能。大多數 AI 編程工具皆有「清除上下文」或「新會話」按鈕——當汝感覺 AI 之回答質量下降時，勿猶豫，開一個新會話。丟失一些對話歷史，勝過繼續於退化之上下文中掙扎多矣。

動手一試：為汝之 AI 當一次「病人」

故意於瀏覽器裡作一個小頁面，製造一個 bug（譬如按鈕沒反應）。然後按本章「好之調試描述」之格式，向 AI 完整描述一次：原始信息、期望對實際、復現步驟、已排查項。感受一下 AI 定位問題速度之差異——汝便會理解為何「描述問題」本身即門手藝。

本章小結

- Prompt 三層：系統層（寫於記憶檔案）、任務層（做何＋技術＋約束）、反饋層（指出問題＋期望＋上下文）
- 調試四要點：予原始信息、期望對實際、復現步驟、已排查項
- 小步快跑：低風險高輪次，適合新功能；大步重構：高風險低輪次，須測試保障

- 上下文管理四招：錨定關鍵信息、總結—續接、一個對話一個任務、果斷開新會話

第四十四章：自 0 到 1 之實戰——完整項目全流程

理論學完了，而今為動手之時。本章不為簡單之「照著敲」教程，乃一套「何以想、何以問、何以驗」之實戰方法論。吾儕會用三個不同難度之項目，展示從需求到成品之完整思考過程。無論汝用哪個工具，此套流程皆適用。

一、項目一：極簡個人工具站（入門級）

需求思考

目標：創建一個個人工具站網頁，展示 3-4 個實用小工具之入口。此乃一個純前端項目，不須後端與數據庫。重點不為工具本身有多複雜，乃體驗「從需求到成品」之完整流程。

按照需求五要素：Who——個人使用。What——一個展示工具入口之網頁。Why——將常用工具集中於一個頁面方便訪問。How——單檔案 HTML+CSS+JS，不使用框架。Constraint——30 分鐘內完成，不須部署。

下達指令

基於 Think Before Coding 原則，汝之指令應包含足夠之信息使 AI 理解目標，但不須指定每個實現細節：

```
請創建一個個人工具站網頁，要求：
1. 單文件HTML（內聯CSS與JS），不使用框架
2. 暗色主題，現代極簡風格
3. 展示4個工具入口卡片：
   - 文本摘要工具
   - 字數統計工具
   - 顏色選擇器
   - 二維碼生成器
4. 每個卡片有圖標、標題、簡短描述
5. 卡片有懸停動畫效果
6. 響應式佈局，手機上可用
7. 頂部有一個標題欄
請直接予出完整之HTML文件。
```

驗收與迭代

AI 予出代碼後，按照迭代驗證之三層檢查進行驗收。第一層功能驗證：檔案能否於瀏覽器中打開？4 個卡片為否皆顯示了？懸停動畫為否正常？第二層邊界驗證：於手機屏幕尺寸下佈局為否正常？窗口甚窄時卡片為否會換行？第三層架構驗證：代碼為否可讀？CSS 為否內聯於一個 style 標籤中？有無明顯之冗餘？

若某個工具卡片之功能也想實現（而不只為入口），可逐步添加——每次添加一個工具之功能，驗證後再添加下一個。此即「小步快跑」策略之實際應用。

二、項目二：全棧記賬系統（進階級）

需求思考與 PRD

目標：一個有前端、後端與數據庫之全棧記賬應用。此項目比第一個複雜得多，須用到第四十章之需求工程方法。

先寫 PRD。按照第四十章第三節之模板，寫出項目概述、功能列表、技術方案、數據模型與開發計劃。技術方案之選擇要考慮 Simplicity First 原則——前端用 React 但不引入 Redux 等複雜狀態管理，後端用 Node.js 但不引入微服務，數據庫用 SQLite 而非 PostgreSQL（開發階段）。

開發計劃拆解為 5 步：Step 1 後端 API 骨架（Express 服務器＋ SQLite ＋ CRUD 接口）；Step 2 前端基礎頁面（記錄列表＋添加表單）；Step 3 前後端聯調（API 調用＋數據展示）；Step 4 分類與篩選功能；Step 5 月度彙總與簡單圖表。每步完成後皆為「能用之東西」。

設置項目記憶檔案

於開始編碼之前，先創建項目記憶檔案。此乃上下文工程之核心實踐——使 AI 於每次對話開始時皆瞭解項目背景。

```
# 項目：全棧記賬應用
## 概述
個人記賬應用，支持收支記錄、分類、月度彙總。
## 技術棧
- 前端：React + TypeScript + Vite
- 後端：Node.js + Express + TypeScript
- 數據庫：SQLite (通過 better-sqlite3)
- 前後端共享類型定義
## API約定
- 基礎路徑：/api
- 返回格式：{ success: boolean, data: any, error: string }
- RESTful風格
## 數據模型
Record { id, amount, type, category, note, date }
## 代碼規範
- TypeScript嚴格模式
```

```
- 縮進2空格
- 函數有JSDoc註釋
- 中文註釋與提交信息
```

逐步實現

按照開發計劃，每一步用一個獨立之對話來完成。以下為第一步之指令示例：

```
吾需後端API之骨。
要求：
1. Express服務器，端口3001
2. SQLite數據庫，表名records
3. records表字段：
   id (TEXT PRIMARY KEY)
   amount (REAL)
   type (TEXT: 'income'|'expense')
   category (TEXT)
   note (TEXT)
   date (TEXT: YYYY-MM-DD)
4. CRUD接口：
   GET /api/records - 取所有記錄
   POST /api/records - 新增記錄
   PUT /api/records/:id - 更新記錄
   DELETE /api/records/:id - 刪除記錄
5. 錯誤處理中間件
請予完整之後端代碼結構。
```

注意此指令之特點：它只描述當前此一步（後端骨架），不摻雜前端需求；它明確列出了字段與接口（予 AI 足夠之約束），但具體實現由 AI 決定；它遵循了 Goal-Driven Execution 原則——說「做何」而非「何以做」。

AI 予出代碼後，立即驗證：運行服務器，用 curl 或 Postman 測試每個接口為否正常工作。確認無誤後，開新對話處理 Step 2（前端基礎頁面）。每一步皆遵循「指令—驗證—新對話」之循環。

三、項目三：AI 增強應用（高級）

項目構思

目標：一個接入大模型 API 之應用——譬如一個「AI 學習助手」，用戶輸入學習內容，AI 生成摘要、關聯知識點、推薦學習資源。此項目之特殊之處於於：汝之應用本身就於調用 AI，汝須處理 API 調用、prompt 工程、錯誤處理等新問題。

此項目教會汝之最重要一課乃為：當汝「用 AI 來寫調用 AI 之代碼」時，汝須格外注意 prompt 工程。汝之應用調用大模型 API 時所用之 prompt，也須遵循本指南所討論之原則——清晰、簡潔、目標驅動。

關鍵技術點

此項目涉及幾個汝於前兩個項目中沒遇到之技術點：

- API 密鑰管理：密鑰絕不能硬編碼於代碼中，用環境變量管理；
- 流式響應處理：大模型之回覆通常為流式之，須逐字顯示；
- Token 用量控制：大模型按 token 計費，須於應用層面控制用量；
- Prompt 模板管理：不同功能用不同之 prompt 模板，須結構化管理。

此諸技術點皆可使 AI 助汝實現，但汝須理解每個技術點之「為何」——為何密鑰不能硬編碼？為何須流式響應？為何 token 用量須控制？理解了「為何」，汝才能判斷 AI 之方案為否合理。

四、通用實戰流程清單

無論項目大小，以下為一個通用之 AI 編程實戰流程清單。每次開始新項目時，過一遍此清單，確保無遺漏關鍵步驟。

階段	做何	對應原則
1. 需求思考	過一遍需求五要素，想清楚要做何	Think Before Coding
2. 需求拆解	將大需求拆成可獨立驗證之小步驟	Simplicity First
3. 寫 PRD	整理成結構化文檔，作為參考	Goal-Driven
4. 設置記憶檔案	創建 CLAUDE.md 或同類檔案	Harness Engineering
5. 逐步實現	每步一個對話，指令—驗證—下一步	Surgical Changes
6. 測試驗證	每步完成後立即測試功能之正確性	Iterate & Verify
7. 代碼審查	檢查架構合理性、安全性、可維護性	Iterate & Verify
8. 迭代優化	根據測試與審查結果迭代改進	Iterate & Verify

AI 編程通用實戰流程清單

動手一試：本週完成「項目一」

將第四十四章第一節之「個人工具站」當成汝之第一個 AI 編程作業：花 5 分鐘過五要素，將指令發予 AI（任何工具皆行），再按三層驗證驗收。30 分鐘內完成它——汝之第一個 AI

編程作品就誕生了。做完記得將它存好，此乃汝從「讀者」變成「實踐者」之紀念品。

本章小結

- 項目一（入門）：純前端工具站——體驗完整流程，30 分鐘能完成
- 項目二（進階）：全棧記賬系統——PRD ＋記憶檔案＋ 5 步逐步實現，每步「能用」
- 項目三（高級）：AI 學習助手——用 AI 寫調用 AI 之代碼，注意 prompt 工程
- 通用流程八步：需求思考拆解 PRD 記憶檔案逐步實現測試審查迭代
- 每一步皆遵循「指令—驗證—新對話」循環

第四十五章：質量保障——使 AI 所寫之代碼可靠

AI 所寫之代碼有一個特點：它通常「看起來甚對」——變量命名規範、註釋齊全、邏輯清晰。但「看起來對」不等於「真之對」。*AI* 代碼中隱藏之問題往往比手寫代碼更難發現，蓋其表面質量太高了，容易使人放鬆警惕。本章教汝如何為 *AI* 代碼建立有效之質量保障體系——用測試、審查、版本控制三道防線，將「看起來對」變成「真之對」。

一、測試驅動：使 AI 寫測試

測試乃 AI 編程質量保障之第一道防線。好消息為：使 AI 寫測試比使它寫功能代碼更容易、更可靠——蓋測試之預期行為為明確之，不太容易產生「看起來對但實際不對」之代碼。

測試驅動之 AI 編程流程乃為：先使 AI 寫功能代碼，驗證基本功能通過後，立即使它寫測試用例。測試用例應覆蓋正常路徑、邊界情況與異常情況。然後再使 AI 根據測試反饋修正功能代碼。此流程之好處乃為：測試一旦寫好，即持續之驗證工具——後續任何修改皆可透過運行測試來確認無引入回歸問題。

```
請爲 calculate_discount 函數編寫單元測試。
測試用例應覆蓋：
1. 正常路徑：
   - 100元商品打8折 -> 80
   - 200元商品打5折 -> 100
2. 邊界情況：
   - 0元商品打8折 -> 0
   - 100元商品打0折 -> 100（不打折）
   - 100元商品打10折 -> 100（全價）
3. 異常情況：
   - 負數價格 -> 應拋出錯誤
   - 折扣率 > 1 -> 應拋出錯誤
   - 非數字輸入 -> 應拋出錯誤
使用Vitest框架，予出完整測試文件。
```

注意此指令之特點：它明確列出了三類測試用例（正常、邊界、異常），不予 AI 自由發揮之空間去「選擇性測試」。AI 有時候會只寫正常路徑之測試來「快速交差」，明確列出所有類別可避免此問題。

二、代碼審查技巧

即使有測試覆蓋，汝仍然須對 AI 產出之代碼進行人工審查。代碼審查不為逐行閱讀——那太耗時也太依賴汝之編程水平。有效之 AI 代碼審查當關注以下幾個關鍵維度：

審查維度	關注何	常見問題
安全性	輸入驗證、密鑰處理、SQL 注入防護	AI 經常忘記驗證輸入，密鑰硬編碼於代碼中
錯誤處理	異常為否被捕獲，錯誤信息為否合理	AI 喜歡用空 catch 塊或只 console.log
性能	為否有明顯之性能問題，如 N+1 查詢、無限循環	AI 有時會寫出 $O(n^2)$ 之算法
可維護性	代碼結構為否清晰，命名為否有意義	AI 偶爾會生成過度抽象之代碼
依賴管理	引入之依賴為否必要，版本為否明確	AI 傾向於引入不必要之庫

AI 代碼審查之關鍵維度

審查時不必自己逐一檢查——可使 AI 助汝做第一輪審查。指令如：「請審查此段代碼之安全性、錯誤處理與性能，指出潛於問題。」AI 之自我審查能力於不斷提升，雖然不能完全替代人工審查，但能助汝過濾掉大部分明顯問題。

三、版本控制：Git 於 AI 編程中之角色

Git 於 AI 編程中之角色比於傳統編程中更加重要。蓋 AI 之每次修改皆可能引入不可預知之變化，汝須一個可靠之「後悔藥」——版本控制即那個後悔藥。

AI 編程中 Git 之最佳實踐乃為「頻繁提交、小步快進」。每完成一個可驗證之功能點，就提交一次。如此當 AI 之某次修改出了問題時，汝可精準回退到上一個可用狀態，而不須推翻所有工作。

另一個實用技巧乃為「分支隔離」。對於結果不確定之 AI 修改（譬如大步重構），先創建一個新分支來做。若結果滿意，合併回主分支；若不滿意，直接刪除分支，不影響主分支。此比於主分支上反覆修改安全得多。

1. 每次 AI 修改前，確保工作區為乾淨之（無未提交之更改）。如此若 AI 之修改不滿意，可用 git checkout . 快速回退。

2. 每次 AI 修改後、驗證通過後，立即提交。提交信息描述此次修改做了何，譬如「feat: 添加用戶註冊功能」。

3. 對於大步重構，先創建分支：git checkout -b refactor/auth-system。於分支上完成修改，測試通過後再合併。

4. 養成查看 diff 之習慣。每次 AI 修改後，用 git diff 查看具體改了何。此不僅能助汝發現問題，還能助汝學習代碼。

四、常見陷阱與避坑指南

AI 編程中有幾個反覆出現之陷阱，幾乎所有初學者皆會踩。提前知道它等，能助汝少走甚多彎路。

陷阱一：信任陷阱

「AI 所寫之代碼看起來甚專業，一定沒問題吧？」——此乃最危險之陷阱。AI 生成之代碼於表面上甚整潔——命名規範、註釋齊全、結構清晰——此種「表面專業感」會使人放鬆警惕。但表面整潔不代表邏輯正確。解藥：永遠透過測試驗證，而非透過「看起來對不對」判斷。

陷阱二：膨脹陷阱

「既然 AI 能助吾寫代碼，那不如將功能做得全面一點？」——此會導致代碼膨脹。AI 甚擅長「多予」：汝使它做一個登錄頁面，它會予汝一套完整之用戶管理系統。多餘之功能不僅增加複雜度，還增加了出 bug 之概率。解藥：嚴格遵循 Simplicity First 原則，明確告訴 AI「只做此功能，勿額外添加」。

陷阱三：理解陷阱

「AI 當理解吾之意思吧？」——不一定。AI 之理解能力雖然甚強，但對含混之需求仍然會「猜」。當它猜錯了，產出之代碼即錯之。更麻煩者，它猜錯之代碼通常「看起來甚合理」，使汝誤以為它理解了。解藥：於指令中提供足夠之具體細節，減少 AI 須「猜」之空間。

陷阱四：上下文陷阱

「吾於之前之對話裡說過了，AI 當還記得吧？」——甚可能不記得。隨著對話變長或開新會話，AI 會丟失早期上下文。若某個關鍵約束無被寫入項目記憶檔案，它甚可能於後續修改中被忽略。解藥：關鍵約束寫入項目記憶檔案，勿依賴對話記憶。

四大陷阱速記

信任陷阱——勿信「看起來對」；膨脹陷阱——AI 愛「多予」；理解陷阱——含混需求它會「猜」；上下文陷阱——對話記憶會「忘」。對應之解藥分別為：測試驗證、砍掉多餘、予足細節、寫入記憶檔案。

動手一試：為汝之項目建三道防線

若汝已經開始做項目一（工具站）：(1) 使 AI 予核心函數寫 3 類測試用例；(2) 按五個維度審查一遍代碼（安全性 / 錯誤處理 / 性能 / 可維護性 / 依賴）；(3) 用 git 初始化項目，做第一次提交。三道防線齊了，汝之項目才真正屬於汝。

本章小結

- AI 代碼「看起來對」≠「真之對」——表面質量太高反而容易放鬆警惕
- 使 AI 寫測試：明確列出正常 / 邊界 / 異常三類用例，防止「選擇性測試」
- 審查五維度：安全性、錯誤處理、性能、可維護性、依賴管理
- Git 乃「後悔藥」：頻繁提交、小步快進、分支隔離、常看 diff
- 四大陷阱：信任（測試驗證）、膨脹（砍掉多餘）、理解（予足細節）、上下文（寫入記憶檔案）

第四十六章：進階之路——從使用者到架構師

當汝掌握了第三十七至四十四章之方法論，能嫻熟地用 *AI* 完成項目開發後，下一步為何？本章探討從「*AI* 工具使用者」到「*AI* 系統架構師」之進階之路。此階段之關注點不再為「何以使 *AI* 寫好代碼」，乃「何以使 *AI* 更好地融入整個開發流程」。

一、MCP 協議與工具擴展

MCP（Model Context Protocol）乃 2025 年出現、2026 年逐漸普及之一個開放協議，它定義了 AI 模型與外部工具之間通信之標準。簡言之，MCP 使 AI 能夠「使用工具」——不只生成代碼，還能執行命令、查詢數據庫、調用 API、操作文件系統。

於 MCP 之前，每個 AI 工具皆有自己之工具集成方式，互不兼容。MCP 之出現意味著：一個符合 MCP 標準之工具可被任何支持 MCP 之 AI 工具使用。此大大降低了工具集成之成本，也使 AI 之能力邊界從「生成文本」擴展到了「執行動作」。

對於進階用戶來說，MCP 之意義於於：汝可為 AI 構建定製化之工具能力。譬如，若汝之團隊有一套內部之部署系統，汝可寫一個 MCP 工具使 AI 直接調用部署系統，而不須人工切換到終端操作。此種「AI 調用工具」之能力，乃 Agentic Engineering 之基礎設施。

二、自定義工作流與 Skills

當汝反覆執行某些固定之 AI 編程流程時，可將它等固化成「自定義工作流」或「Skills」。譬如，汝之團隊可能有一套標準之代碼審查流程——先檢查安全性，再檢查性能，再檢查可維護性。汝可將此流程寫成一個 Skill，使 AI 於每次代碼審查時自動按此流程執行。

自定義工作流之核心價值乃為「一致性」——它確保每次執行皆遵循同樣之標準，不會因為對話長短或上下文不同而產生偏差。此對於團隊協作尤其重要：每個成員使用同樣之工作流，產出之代碼風格與質量標準就會趨於一致。

構建自定義工作流之方法因工具而異。Cursor 支持 Rules 與 Custom Instructions，Claude Code 支持自定義命令，Replit 支持 Runme。但核心思路一樣：將汝之最佳實踐「編碼」成 AI 可執行之指令模板。

三、多 Agent 協作模式

到 2026 年，多 Agent 協作開始成為一個熱門話題。其核心理念乃為：不只用一個 AI 來完成任務，乃用多個分工不同之 AI 來協作。譬如，一個 AI 負責寫代碼，一個 AI 負責寫測試，一個 AI 負責代碼審查——它等各司其職，互相檢查。

多 Agent 協作之優勢於於「分工」與「交叉驗證」：每個 Agent 專注於自己之領域，產出質量更高；Agent 之間之交叉檢查可發現自己可能忽略之問題。但劣勢也甚明顯：協調成本高，通信開銷大，調試更複雜。

小訣：一個人也能玩「多 Agent」

對於個人開發者來說，汝不須搭建複雜之多 Agent 系統——汝可「扮演」不同之角色來模擬多 Agent 協作。譬如：先用一個對話使 AI 寫功能代碼；然後開一個新對話（提供 CLAUDE.md 上下文），使 AI「作為代碼審查者審查此段代碼」；最後再開一個對話使 AI「根據審查意見修正代碼」。此種「一人多角」之方式，能獲得多 Agent 協作之大部分好處，而不須複雜之基礎設施。

四、持續學習路徑

AI 編程乃一個快速變化之領域。今日之最佳實踐，明天可能就須更新。以下為幾條持續學習之建議：

第一，關注源頭而非二手解讀。Andrej Karpathy 之原始推文與 CLAUDE.md、Anthropic 與 OpenAI 之官方文檔、各大工具之更新日誌——此些乃第一手信息。社區文章與教程乃為對此些信息之解讀與總結，有價值但可能失真。養成讀原始資料之習慣。

第二，實踐驅動學習。勿「學完再做」，而要「邊做邊學」。每次遇到新之問題類型（譬如第一次處理文件上傳、第一次接入第三方 API），就以此為機會學習相關知識。實踐中之學習效率遠高於脫離場景之理論學習。

第三，保持工具敏感度但勿過度追新。每隔一兩個月，花半天時間瞭解最新之工具與趨勢——但勿每次有新工具就切換。判斷標準乃為：新工具為否能顯著提升汝之某個瓶頸？若不能，當前之組合就夠了。

第四，構建個人知識庫。將汝每次解決複雜問題之對話、每次踩坑之教訓、每次有效之方法記錄下來。不須公開發布——此乃汝自己之「方法論筆記」。隨著時間積累，它會成為汝最寶貴之資產。

動手一試：開一個「踩坑筆記本」

今天就建一個文檔（筆記軟件或.md 檔案皆行），寫下汝到目前為止最想記住之三條 AI 編程心得——哪怕為「記得先想清楚再問」。以後每次踩坑、每次學到新招，皆往裡面記一條。半年後回看，汝會感謝今天之自己。

本章小結

- MCP 乃 AI 與工具之間之通信標準：使 AI 能執行命令、查數據庫、調 API
- 自定義工作流/Skills 之核心價值乃為「一致性」：將最佳實踐編碼成指令模板
- 多 Agent ＝分工＋交叉驗證；個人開發者可用「一人多角」模擬
- 持續學習四招：讀源頭、邊做邊學、勿過度追新、建個人知識庫

第四十七章：速查手冊與資源

本章乃一份可獨立使用之速查手冊。當汝記不清某個原則或須快速決策時，直接翻到此裡。建議汝將它當成「口袋卡片」：五大原則速查、工具決策樹、常見問題、學習資源，一頁不落。

一、思維模型速查卡

原則	一句話	操作要點
Think Before Coding	先想清楚再開口	回答三問:做何?結果長何樣?有何約束?
Simplicity First	用最少代碼解決問題	每次只做一個功能,砍掉不必要之複雜度
Surgical Changes	精準修改不越界	明確指定修改範圍，拒絕 AI 之「順手改一下」
Goal-Driven Execution	說「做何」不說「何以做」	予目標不予步驟，使 AI 自主選擇路徑
Iterate & Verify	生成—驗證—修正循環	三層驗證：功能、邊界、架構

五大思維原則速查卡

二、工具選擇決策樹

當汝須選擇 AI 編程工具時，按照以下決策路徑選擇。注意：此不為「哪個最好」之排名，乃「哪個最適合汝」之判斷框架。

汝要做何？

零基礎，想快速體驗選雲端 Builder (v0/Bolt/Replit)

日常開發，須代碼補全選 IDE 內置 AI (Cursor/Trae)

大型項目，須批量操作選終端 Agent (Claude Code)

快速原型驗證想法選雲端 Builder 或 Trae SOLO

圖 6：AI 編程工具選擇決策樹

三、常見問題 FAQ

Q1：吾完全不會編程，能用 AI 編程嗎？

能。2026 年之 AI 編程工具已經足夠強大，汝完全可用自然語言描述需求，使 AI 生成代碼。但有兩點建議：第一，從雲端 Builder 工具（如 v0、Bolt）開始，它等不須配置開發環境；第二，邊用邊學基礎概念——不須精通語法，但汝須能「看懂」AI 所寫代碼之大意，如此纔能有效驗收。

Q2：吾當學哪個編程語言？

取決於汝要做何。做網站前端——JavaScript/TypeScript；做數據處理或 AI——Python；做企業級後端——Java 或 Go。但更重要之為：於 AI 編程時代，「語言」之重要性於下降。AI 能助汝處理語法細節，汝須理解者乃「程序之結構」——何為函數、何為循環、何為數據結構。此諸概念乃跨語言者，一旦理解，換何語言皆快。

Q3：AI 所寫之代碼有版權問題嗎？

截至 2026 年 8 月，此仍然為一個有爭議之法律問題。一般來說，AI 生成之代碼於大多數司法管轄區不被視為有明確之版權所有者。但若汝用於商業項目，建議：(1) 瞭解汝所用 AI 工具之服務條款中關於知識產權之條款；(2) 對關鍵代碼進行人工審查與修改，使其具有汝之創造性投入；(3) 避免使 AI 直接複製可能有版權之開源代碼。

Q4：AI 會取代程序員嗎？

短期內不會。AI 所取代者不為「程序員」此職業，乃「逐行編碼」此工作。就像計算器無取代數學家，乃使數學家從繁瑣之手算中解放出來一樣。AI 編程工具使程序員從「寫代碼」轉向「想問題」——此實際上提高了程序員之價值上限。未來之競爭力不於於「能寫多少行代碼」，而於於「能解決多複雜之問題」。

Q5：如何判斷 AI 之產出為否正確？

三層驗證法。第一層：功能驗證——它能不能跑？輸入輸出對不對？此乃最基本者。第二層：邊界驗證——空輸入、超長輸入、異常輸入會怎樣？此能發現隱藏之問題。第三層：架構驗證——代碼結構為否合理？有無硬編碼？為否便於擴展？此須汝有一定之技術判斷力。若汝無法判斷第三層，至少做到前兩層——能跑、邊界正常，就已比大多數初學者強了。

四、推薦學習資源

以下為截至 2026 年 8 月之推薦學習資源。請注意，互聯網資源連結可能變化，若某個連結失效，搜索其標題通常能尋到最新版本。

類型	資源	推薦理由
原始文獻	Andrej Karpathy 之 CLAUDE.md	五大原則之源頭，70 行但字字珠璣
原始文獻	Anthropic 官方文檔（Claude Code）	終端 Agent 之最佳實踐指南
教程	Vibe Coding 社區整理之 Skills 合集	社區實踐者總結之實戰技巧集合
教程	Cursor 官方文檔與 YouTube 頻道	IDE 內置 AI 之權威使用指南
社區	GitHub 上之 awesome-claude-code	Claude Code 之最佳實踐與插件集
社區	Reddit 之 r/cursor 社區	全球 Cursor 用戶之經驗分享與討論
基礎	MDN Web Docs (developer.mozilla.org)	前端開發最權威之參考文檔
基礎	Python 官方教程（docs.python.org）	Python 學習之官方起點

推薦學習資源清單

五、結語：工具於手，方法論於心

回到本書之核心信念：工具每三個月迭代一次，方法論每三年才進化一次。於 2026 年 8 月之今日，AI 編程工具之能力已經強大到令人驚歎——汝能用一句話創建一個網站，用一段描述生成一個全棧應用，用一條指令完成一個複雜重構。但工具越強大，方法論之重要性反而越突出——蓋工具之強大放大了汝之每一個決策。

若汝之需求清晰，AI 能助汝高效實現；若汝之需求含混，AI 也會高效地做出錯誤之東西。若汝遵循簡單至上，AI 能助汝快速驗證想法；若汝貪多求全，AI 也會助汝快速堆積技術債。若汝精準修改，AI 能助汝外科手術般地優化代碼；若汝指令含混，AI 也會助汝大刀闊斧地引入 bug。

此乃本書故花大部分篇幅講方法論而非工具操作之故。不為因為工具不重要，乃因為方法論乃工具之「乘數」——方法論越好，工具之價值越大；方法論為零，工具再強也只為零乘以任何數。

汝不復獨自編程。汝乃統帥，AI 乃汝之軍團。汝思之精粗，定軍團之戰力。自今日始，以方法論武其思，以工具張其能——此 AI 編程之全部祕密也。

參考文獻

1. Andrej Karpathy. (2025). Vibe Coding. Social media post introducing the concept of vibe-driven programming with AI.

2. Andrej Karpathy. (2025). CLAUDE.md - Project guidelines for AI-assisted development. GitHub repository, 70+K stars.

3. Anthropic. (2026). Claude Code: Terminal-native AI programming. Official documentation and best practices.

4. Cursor Team. (2026). Cursor 3: The next generation of AI-first IDE. Release notes and documentation.

5. OpenAI. (2026). Codex CLI: Bringing AI to the terminal. Technical documentation.

6. Model Context Protocol (MCP) Specification. (2025-2026). Open protocol for AI-tool integration.

7. Vibe Coding Community. (2026). Community-curated skills and best practices for AI-assisted programming.

8. Spec-Driven Development (SDD) methodology. (2026). Emerging practices in specification-first AI programming.

八、安卓開發實戰篇（第 48-58 章）

第四十八章：認識安卓——從操作系統到開發生態

恭喜汝做出此決定——從「玩手機之人」變成「做手機應用之人」。於動手之前，吾儕先將地圖攤開：安卓從哪裡來？為何全世界之開發者皆於為它寫應用？一個 *App* 從汝之手指敲鍵盤到用戶手機上，中間皆發生了何？

一、安卓之身世：從「小機器人」到「全球最大系統」

安卓（Android）之故事始於 2003 年——一個名叫 Andy Rubin 之工程師創辦了 Android 公司，目標甚樸素：做一套「更懂相機」之智能操作系統。後來谷歌於 2005 年收購了它，2007 年聯合幾十家廠商成立開放手機聯盟（OHA），2008 年 9 月，第一款安卓手機 HTC Dream（G1）正式發售。

後面之故事大家皆知道了：到 2026 年，安卓乃全球市場份額最大之行動操作系統，活躍設備數以十億計。汝之手機、電視盒子、車載大屏、智能手錶，甚可能皆於跑安卓。此意味著何？意味著汝所寫之每一行代碼，皆有機會被世界上最多之人用到。

小訣：安卓之開源基因

安卓之核心代碼（AOSP，Android Open Source Project）乃開源之——任何廠商皆能拿去修改定製。故小米有 MIUI（而今叫澎湃 OS）、華為有鴻蒙兼容層、三星有 One UI。此對開發者乃好事：汝學之乃「安卓開發」，不為「某個品牌之開發」，一套功夫，全網通喫。

版本號之進化也甚有意思：從 1.5 之紙杯蛋糕（Cupcake）開始，每個大版本皆以甜點命名——甜甜圈、閃電泡芙、凍酸奶、薑餅……一直到 2019 年之安卓 10 才改成了純數字。到 2026 年 8 月，最新正式版乃 Android 17，而開發時汝真正須關心之「最低支持版本」，主流 App 一般定於 Android 8（API 26）到 Android 10（API 29）之間——支持到幾年前之設備，汝之用戶盤子才夠大。

版本	代號 / 年份	對開發者重要之事
Android 8 (API 26)	Oreo • 2017	通知渠道、後臺限制——甚多老機型之最低線
Android 10 (API 29)	2019	分區存儲登場，文件訪問規矩大改
Android 12 (API 31)	2021	Material You 動態取色，界面觀感大升級
Android 13 (API 33)	2022	運行時通知權限，彈窗問汝「準不準發通知」
Android 14-15 (API 34-35)	2023-2024	預測性返回、16KB 頁、隱私進一步收緊
Android 16-17 (API 36-37)	2025-2026	更快之小版本節奏、AI 能力整合（如 Gemini 擴展）

安卓版本簡史（截至 2026 年 8 月，開發重點關注項）

二、用何語言寫：Java 抑 Kotlin？

安卓開發歷史上出現過兩種主流語言。Java 乃「開國元老」——安卓誕生之初就用它；Kotlin 乃 2017 年穀歌欽點之「一等公民」，2019 年起官方明確「Kotlin 優先」（Kotlin-first），到 2026 年，新項目幾乎全部用 Kotlin，舊項目也大多於陸續遷移。

說句實於話：若汝乃完全零基礎，直接學 Kotlin，勿回頭碰 Java。原因有三：一為 Kotlin 代碼量比 Java 少 30%-40%（同樣之功能，Kotlin 更短更清晰）；二為它天生防「空指針」此種新手噩夢；三為所有現代教程、官方文檔、AI 編程工具（還記得吾儕之 AI 編程書嗎？）皆於以 Kotlin 為準。Java 只於維護老項目、讀老代碼時會遇到。

Java 風格（囉嗦版）

```
public class MainActivity extends Activity \{
@Override
protected void onCreate(Bundle b) \{
super.onCreate(b);
setContentView(R.layout.activity);
\}
\}
```

Kotlin 風格（清爽版）

```
class MainActivity : ComponentActivity() \{
override fun onCreate(b: Bundle?) \{
super.onCreate(b)
setContent \{ App() \}
\}
\}
```

同樣一件事，Kotlin 少了一堆花括號與樣板。第 50 章吾儕會正式學它，而今汝只須記住：安卓開發之「普通話」叫 Kotlin。

三、一個 App 為何以誕生之？

甚多人以為「開發 App」即打開電腦狂寫代碼。實際上，一個 App 從想法到用戶手機上，大致要經過如此一條流水線：

想法

設計（界面 + 功能）

寫代碼

編譯打包（APK/AAB）

測試（模擬器 + 真機）

發佈（商店/安裝包）

圖 1：一個安卓 App 從想法到上架之全流程

流程裡有兩個概念汝而今就要認識：

- APK：安卓應用之安裝包格式（.apk），雙擊就能裝到手機上，就像 Windows 之.exe。平時自己測試、予朋友傳著玩，用它。

- AAB：Android App Bundle（.aab），上架 Google Play 用之「高級打包格式」。商店會根據用戶之手機型號動態裁剪，使每個人下載之包更小。自己裝不了，只能交予商店。

另外汝還會不斷聽到一個詞：SDK（Software Development Kit，軟件開發工具包）。它即安卓官方提供之一整套「開發裝備」——包括各版本之 API 庫、調試工具、模擬器鏡像等。汝寫代碼時調用之 Activity、Button 此些類，皆來自 SDK。

四、安卓技術棧全景：汝之「武器清單」

正式學習前，先認識一下 2026 年主流安卓項目之標準配置——此張清單汝而今看不懂沒關係，全書學完回來看，會豁然開朗：

層次	主流選擇（2026）	所用為何
語言	Kotlin 2.x	寫邏輯之語言
界面	Jetpack Compose + Material 3	用代碼畫界面（本書主線）
架構	MVVM + ViewModel	將界面與邏輯分開，代碼不亂
數據存儲	Room（數據庫）+ DataStore（配置）	將數據存進手機
網絡	Retrofit + OkHttp + Kotlin 協程	與服務器對話
構建工具	Gradle + Kotlin DSL	將代碼變成安裝包
開發環境	Android Studio	谷歌官方之「全家桶」IDE

2026 年安卓主流技術棧速覽

勿被此一串英文嚇到。它等就像廚房裡之鍋碗瓢盆——汝而今只須知道「做飯要進廚房」，具體每樣工具何以用，後面章節逐個拆予汝看。而且好消息為：Android Studio 創建新項目時會自動幫汝將此一整套配置好，汝只管炒菜。

五、安卓 vs iOS：開發者之第一堂「選邊課」

汝手裡可能正同時有安卓與蘋果設備。作為開發者，兩邊之體驗差異值得提前瞭解——不為「哪個更好」之站隊，乃「哪邊更適合汝起步」之現實賬：

維度	安卓開發	iOS 開發
開發電腦	Windows / Mac / Linux 皆行	必須 Mac（Xcode 獨佔）
開發工具	Android Studio（免費）	Xcode（免費）+上架年費 99 美元
上架審核	寬鬆，幾小時至幾天	嚴格，1-7 天乃常態
裝機試玩	APK 隨便發、隨便裝	分發受限，測試須簽名 / TestFlight
收入環境	廣告 / 國內渠道為主	App Store 內購付費習慣更好
學習成本	概念較多（四大組件等）	生態封閉但體系統一、文檔精良

安卓與 iOS 開發路徑對比（2026 年 8 月）

說句實於話：若汝之電腦不為 Mac，那此題根本不用選——安卓乃唯一選項。若汝有 Mac，本書第八、九兩篇可對照著讀，編程思路為相通之，學完一邊再學另一邊會快甚多。

動手一試：看看汝手機裡之「安卓」

打開手機之［設定］［關於手機］（或「關於本機」），尋到「安卓版本」或「Android 版本」那一欄——連續點它幾下，還能觸發系統之彩蛋小遊戲。記下汝之版本號，對照上面之版本表看看它屬於哪一代。汝之手機，即汝之第一個「開發測試機」。

本章小結

- 安卓 2008 年誕生，2026 年乃全球市場份額最大之行動系統，活躍設備數十億
- 最新正式版 Android 17；主流 App 最低支持 Android 8-10，用戶盤子才夠大
- 新項目一律用 Kotlin：代碼少 30% 以上、防空指針、官方欽定
- APK 自己裝，AAB 上商店；SDK 乃官方開發裝備包
- 2026 標準技術棧：Kotlin ＋ Compose ＋ MVVM ＋ Room ＋ Retrofit ＋ Gradle ＋ Android Studio

第四十九章：搭建開發環境——Android Studio 與第一個項目

工欲善其事，必先利其器。此章吾儕將「廚房」搭好：裝好 *Android Studio*、跑通模擬器、創建人生第一個安卓項目，再將工程結構拆開看個明白。裝好環境之後，汝會第一次看到自己寫之代碼變成一個能點之按鈕——此一瞬間，甚多人會記一輩子。

一、安裝 Android Studio

Android Studio 乃谷歌官方之集成開發環境（IDE），免費，功能全家桶：寫代碼、看界面預覽、調試、打包全於其中。安裝四步：

1. 打開官網 developer.android.com/studio，下載最新穩定版（2026 年 8 月乃 Narwhal 之後之版本，Windows/Mac/Linux 皆有）；

2. 運行安裝包，一路「下一步」——安裝嚮導會自動帶上 Android SDK 與模擬器組件；

3. 首次啟動時按提示下載 SDK 組件（默認勾選就好，可能須幾分鐘與幾個 GB 之空間）；

4. 完成後汝會看到歡迎頁——恭喜，「廚房」搭好了。

注意：硬件要求心裡有數

Android Studio 乃一個「喫記憶體」之傢夥：建議 16GB 記憶體起步（8GB 也能跑，但模擬器一開則比較卡）、磁盤預留 20GB 以上、CPU 支持虛擬化（BIOS 裡開啟 VT-x/AMD-V）。若電腦確實老舊，可用「真機調試」代替模擬器（本章第四節會講），體驗反而更流暢。

二、創建汝之第一個項目

於歡迎頁點 New Project，選 Empty Activity 模板（最乾淨之新手模板），然後填三樣東西：

配置項	填何	說明
Name	My First App	應用名字，顯示於手機桌面上
Package name	com.example.myfirstapp	全球唯一標識，習慣乃「域名反寫」，先按默認
Minimum SDK	API 26（Android 8.0）	最低支持版本，覆蓋 95% 以上設備

創建項目要填之三件事

點 Finish 後，Android Studio 會用 Gradle 自動下載依賴、構建項目（第一次要等幾分鐘，之後每次會快甚多）。構建完成後，汝就有了一個能跑之 App——雖然它還只會顯示一行「Hello Android!」。

小訣：包名之門道

包名（package name）就像 App 之「身份證號」，全球不能重複，上架後就不能改了。個人練習用 com.example.xxx 就行；正式產品要換成汝自己之域名反寫（譬如 com. 汝之名. 應用名）。勿用 com.example 上架——商店會拒收。

三、跑起來：模擬器與真機

代碼寫好了，得尋個「手機」來跑。汝有兩個選擇：

模擬器（Emulator）

模擬器者，於電腦中虛擬安卓手機一部也。啟 Android Studio，開 Device Manager Create Device，擇一機（宜 Pixel 系列）選系統影像（取帶 Google APIs 者）下載啟動。繼點工具欄綠色三角 Run，俄而數秒，「手機」之屏遂現汝之 App。

真機調試（尤所薦）

真機者，至真之試煉之境也——滑動之手感、真網、真感應之器，模擬器皆莫能及。安卓真機調試甚簡：

1. 於手機啟［設定］[關於手機]，連點「版本號」七次，則「開發者選項」啟矣；
2. 於［開發者選項］中啟 USB 調試；
3. 以數據線連電腦，手機彈窗，擇「允許 USB 調試」；
4. 返 Android Studio，工具欄設備下拉之列遂現汝機，點 Run。

避坑之警：USB 調試勿恆開

USB 調試者，猶為電腦開「後門」也。凡公共之所充電、接不可信之電腦，務閉 USB 調試，以防他人由之竊讀汝之數據。平居開發，用畢即閉，習以為常。

四、工程結構拆解：諸文件何所司

初啟項目，左側文件之樹或令汝目眩。勿慌，日常開發，汝百分之九十惟與下列數者相交接：



圖 2：新手必曉之工程結構（口訣：惟觸 app/ 目錄）

- MainActivity.kt：App 之「大門」也。用戶點圖標而啟，首被喚醒者即之。今模板中，其司顯「Hello Android!」之職。
- AndroidManifest.xml：App 之「戶口簿」也——聲明此 App 何名、何圖標、有何頁面、需何權限（相機、定位之屬）。
- res/ 目錄：資源之倉也，藏圖片、圖標、字符串、顏色。文字置於資源之文件而不硬寫於代碼，乃安卓開發之善習（易語言、易主題皆便）。
- build.gradle.kts：構建之方也——聲明最低支持之版本、所倚之庫（如 Compose）。

五、小試牛刀：易「Hello」為汝名

啟 MainActivity.kt，則見 Compose 模板所生之代碼。得 Text(“Hello Android!”) 一行，易引號中之字為汝名，如 Text(“汝好，張三！此吾之首個 App”)，繼按 Ctrl+R（Mac 則 Cmd+R）復行。數秒之後，模擬器遂顯汝名矣。

```
setContent {
    MyFirstAppTheme {
        Surface(modifier = Modifier.fillMaxSize()) {
            // ∨ 改此為汝之文字
            Text("汝好，張三！此乃吾之首App")
        }
    }
}
```

賀矣，汝已竟一全環：「改代碼編譯安裝運行」。此環者，異日日日開發之常也。乘熱打鐵，試更易他者：增字之號、易其色——次章將明此諸「魔法」所自來。

六、常見報錯急救：新手前兩週之「急診室」

環境初建、首次構建之際，汝十有八九將遇下列三種報錯。勿慌，皆有解藥：

報錯之象	常見之因	解藥
Gradle sync failed / 依賴下載遲緩	網絡通倉庫不暢	為 Gradle 配國內鏡像倉庫（阿里雲/騰訊雲 maven 鏡像），網上搜「gradle 國內鏡像」依而改之
SDK 組件缺失	安裝時跳過 SDK 之下載	Settings SDK Manager 補裝所缺之平臺與工具
模擬器啟動敗 / 黑屏	虛擬化未啟、影像損壞	BIOS 啟 VT-x/AMD-V；刪模擬器而重建；或逕用真機調試

新手環境報錯急救之表

小識：搜報錯之法

報錯之首行最有值——原樣抄入搜索引擎，加以「android studio」之關鍵詞。勿搜「吾環境裝不上奈何」之泛問，當搜「Gradle sync failed: Could not resolve com.android.tools.build」之具體錯。問也精，答方精。

來動手：竟汝「開發儀式感」之單

(1) 裝 Android Studio 且通模擬器；(2) 以真機行己之 App 一遭，截圖以存；(3) 易 Hello 之文為汝名；(4) 壓縮項目而存檔。四步既畢，即可於朋友圈宣曰：吾亦安卓開發者矣（截圖為證）。

本章小結

- Android Studio = 官家免費之 IDE；宜 16GB 記憶體 + 20GB 磁盤 + 啟虛擬化
- 新建項目三要素：名、包名（舉世唯一，com.example 勿上架）、最低 SDK（自 API 26 始）
- 運行之道二：模擬器（電腦中虛擬之機）與真機（啟 USB 調試，用畢記閉）
- 工程口訣：惟觸 app/ 目錄——MainActivity.kt 為大門，Manifest 為戶口簿，res/ 為資源之倉
- 開發日常 = 改代碼 Run 觀其效，循環往復

第五十章：Kotlin 速成——安卓開發之首語

此章吾儕始正學其語。勿怯——*Kotlin* 素以待新手甚善稱，且汝毋須背語法之書：先通其百分之二十之語法，則能曉且書日常代碼之百分之八十。其餘者，且作項目且查之可也。記吾儕為學之則：曉之 > 記之，能用 > 精熟。

一、變量與類型：為數據命名

程序者，「數據 + 邏輯」也。變量者，與數據命一名而儲之也。Kotlin 有二聲明之法：

```
val name = "張三"          // val：唯讀變量（宜以為默認）
var age = 18               // var：可變變量（後尚可改）

age = 19                   // <i class="fas fa-check-circle mr-1"></i> var 可改
// name = "李四"           // <i class="fas fa-times-circle mr-1"></i> val 不能改，編譯會報錯

val score: Int = 100       // 亦可明書其型
val pi = 3.14              // 不書亦自推之
```

常用之類不過數種，一表可記：

類型	所儲何物	示例
Int	整數	42
Double	小數	3.14
String	文字	「汝好」
Boolean	真/偽	true / false
List< T>	一組數據	listOf(1, 2, 3)

Kotlin 常用基礎類型

小識：val 為先之則

Kotlin 社區之鐵律：常則用 val，誠有必要方用 var。可用 val 處而用 var，IDE 將以黃字警汝。此習能免汝一大類「變量遭意外改易」之 bug——代碼愈少變，愈易推演。

二、函數：束邏輯為「按鈕」

函數者，有名之邏輯一段也，猶遙控器上之鈕——按之（調用），則行一串之動。Kotlin 以 fun 聲明：

```
fun greet(name: String): String {
    return "汝好，$name！"     // $ 符插變量於字串
}

println(greet("張三"))        // 輸出：汝好，張三！

// 單行函數可簡寫成表達式
fun add(a: Int, b: Int) = a + b

println(add(3, 5))            // 輸出：8
```

數要點：參數書於括中（名於前、類於後，以冒號隔之）；返回之類書於括後；字符串模板 $ 變量名者，Kotlin 招牌之便也——不必如 Java 之「字符串 + 變量」輾轉相拼矣。

三、條件與循環：使程序能「思」

程序之智出於二事：能判斷（if/when）與能重複（for/while）。

```
// 判斷：if / else
val age = 17
if (age >= 18) {
    println("成年人")
} else {
    println("未成年")
}

// 多分支：when（比 switch 好用一百倍）
val score = 85
val level = when {
    score >= 90 -> "優秀"
    score >= 60 -> "及格"
    else -> "再接再厲"
}

// 重複：for 循環
val fruits = listOf("蘋果", "香蕉", "橙子")
for (fruit in fruits) {
    println("吾好食$fruit")
}
```

when 者，Kotlin 最常用之多分支寫法也，安卓開發之中汝將日日用之（譬如依狀態而顯不

同之界面）。

四、空安全：Kotlin 之「保命符」

Java 之世，至著名之 bug 曰 NullPointerException（空指針之異）——程序觸一「空無所有」之物，遂崩。稽諸統計，其恆居安卓崩潰之榜首。Kotlin 自語言之層決之：常則不許空值。

```
var name: String = "張三"    // 普通之型：決不容為 null
// name = null                 // <i class="fas fa-times-circle mr-1"></i> 編譯之誤，徑阻汝

var nickname: String? = null // 加問號：許為 null
nickname = "三兒"

// 用可空之型，必安處之：
println(nickname?.length)    // 安全調用：null 則返 null，不崩
val len = nickname?.length ?: 0   // Elvis 運算符：null 時予默認值
println(len)
```

慎之：? 與?: 勿混

String? 者，「許其為空」也；a ?: b 者，「a 空則用 b」也（形似貓王之髮，故號 Elvis 操作符）。初書代碼時，見編譯器報「null 之誤」，十有八九乃汝忘處可空之類——此正語言之護汝也。

五、類與對象：造「零件」之模

類者，對象之「模」也；對象者，類之「成品」也。安卓中萬物——頁面、按鈕、數據——皆對象也。Kotlin 有神器曰 data class（數據類），專以盛數據，一行可抵 Java 數十行：

```
// 普通類：模具
class Dog(val name: String) {
    fun bark() = println("$name 汪汪！")
}

val dog = Dog("旺財")       // 造物件
dog.bark()                  // 旺財 汪汪！

// data class：盛數據之盒（自動帶 equals/copy 等）
data class User(val id: Int, val name: String, val email: String)

val me = User(1, "張三", "zhangsan@example.com")
val twin = me.copy(name = "張三豐")   // 複製並改一個字段
```

他日汝作記賬之冊，一筆之賬即一 data class：data class Record(val amount: Double, val category: String, val date: String)。觀之，汝已能書真實項目代碼之構矣。

六、lambda 與高階函數：Kotlin 之「語法糖」

安卓代碼之中，汝將日日見 { ... } 此類「無名之函」，為曰 lambda（匿名函數）。Button 之 onClick = { count++ } 即一 lambda——束「點而後何為」為一團，以授於鈕。

```
// 普通寫法：定義一個有名函數再傳入
fun double(x: Int) = x * 2
val result = listOf(1, 2, 3).map(::double)

// lambda 寫法：就地定義，輕快直接
val result2 = listOf(1, 2, 3).map { it * 2 }   // it = 當前元素

// 篩選 + 變換連招（真實項目裏極常見）
val bills = listOf(12.5, 88.0, 5.0, 200.0)
val bigBills = bills.filter { it > 50 }.sorted()
// [88.0, 200.0]——"逾 50 之賬單，序之"
```

高頻「集合連招」數者宜記：filter（篩選）、map（變換）、sorted（排序）、sum（求與）。汝異日列表之理，百分之九十皆此四招之合。Kotlin 之哲曰：可一語明之之邏輯，決不書五語。

七、貽零基者之語法速查卡

勿欲一日盡背語法。藏此卡，書代碼時對而查之，二旬之後自熟矣：

欲為何事	寫法
定唯讀之變量	`val x = 1`
定可變之變量	`var x = 1`
定函數	fun 名 (參數): 返回類型 { ... }
判斷	if (條件) {...} else {...}
多分支	when { 條件 -> 結果}
遍歷	for (x in 列表) { ... }
字符串拼接	「汝好，$name」
處可空者	`x?.length ?: 0`
定數據類	`data class X(val a: Int, val b: String)`

Kotlin 語法速查卡（新手之版）

來動手：不裝任何軟件，先玩 Kotlin 十分鐘

以瀏覽器啟 play.kotlinlang.org（Kotlin 官家線上演武之場），粘本章每段代碼而運之一遭，易其數與文，觀所出如何變。此「先玩後學」之十分鐘，將使汝至第五十一章上手之時，其易倍之。

本章小結

- val 為先，var 慎用；類可自動推之，冒號後顯而聲明亦可
- 函數 = 束成之邏輯；字符串模板 $ 為招牌之便
- if/when 以判，for 以循；when 者，安卓開發之「多面手」也
- 空安全者，Kotlin 之保命符也：常類拒 null，可空之類必以? / ?: 處之
- data class 者，盛數據之匣也，一字段一行，真實項目日日用之
- 百分之二十之語法，成百分之八十之功：先用之，後精之

第五十一章：界面開發——Jetpack Compose 入門

界面者，用戶所見之全也。*Compose* 未興之前，安卓界面以 *XML* 書之——繁瑣如填空之題；*2021* 年穀歌出 *Jetpack Compose*：逕以 *Kotlin* 代碼「畫」界面，所見即所得，改一行而預覽立新。至 *2026* 年，*Compose* 已為新項目之絕對主流。此章吾儕學「畫」之道。

一、Compose 之思：界面即函數

Compose 核心之思，一言蔽之：界面 = 函數。凡可顯於屏者（文字、按鈕、圖片）皆一「可組合函數」（Composable），以 @Composable 誌之。嵌套之如積木，則界面成焉。

```
@Composable
fun Greeting(name: String) {
    Text(text = "汝好，$name！")     // 一"文字之積木"
}

// 嵌套而組：大積木套小積木
@Composable
fun HomeScreen() {
    Column {                          // 豎而列之
        Greeting(name = "張三")
        Greeting(name = "李四")
        Button(onClick = { /* 點之何為 */ }) {
            Text("點我")
        }
    }
}
```

慎視 @Composable 此「冠」——所以告編譯器曰：此乃界面積木也。積木可無限相套，此即 Compose 之全祕。

二、基礎組件：文字、按鈕、輸入框、圖片

Compose 之組件即「積木之塊」，至常見者四：

組件	所為何事	必備參數
Text	顯文字	text（所欲顯之字）
Button	按鈕	onClick（點擊之回調）
TextField	輸入框	value、onValueChange（輸入變化之回調）
Image	顯圖片	painter（圖片之資源）

Compose 四大基礎組件

```
@Composable
fun BasicWidgets() {
    Column {
        Text(text = "此乃一段文字")
        Button(onClick = { println("被點擊了") }) {
            Text("按鈕")
        }
        var input by remember { mutableStateOf("") }
        TextField(
            value = input,
            onValueChange = { input = it },
            label = { Text("輸入若干") }
        )
    }
}
```

輸入框彼兩行中現 remember { mutableStateOf(“”) }——此即 Compose「狀態」之魔法，本章第五節詳言之。今姑識其面焉。

三、布局三件：Column、Row、Box

組件既有，何以擺置？記三「盒子」：

- Column：縱而列之（如列隊而取食）；
- Row：橫而列之（如並肩而坐）；
- Box：疊而置之（如撲克之牌相壘，宜為角標、遮罩）。

```
Column {
    Text("第一行")
    Row {
        Text("左邊")
        Text("右邊")
    }
    Box {
```

```
        Text("底層")
        Text("疊於上之角標")
    }
}
```

欲布局愈美乎？與組件加 Modifier（修飾符）——猶為積木「上妝」也：

```
Text(
    text = "美之文字",
    modifier = Modifier
        .padding(16.dp)           // 四周留白
        .fillMaxWidth()           // 滿其寬
        .background(Color.Blue)   // 背景色
        .clickable { /* 點擊 */ }// 可點擊
)
// 鏈式調用：相繼而行，序有其義（先 padding 後背景，背景不含邊距）
```

慎之：Modifier 之序有考

padding(16.dp).background(Blue) 與 background(Blue).padding(16.dp) 其效不同——前者「先留白後塗色」（背景不含邊距），後者「先塗色後留白」（背景含邊距）。新手最易於此畫出「怪異之白邊」。未定之時，易序而試，預覽立告汝以答。

四、狀態：使界面「活」

靜態之界面無味也。欲鈕點而數增一、輸入框可打字，則須狀態（State）。Compose 之則：界面恆忠以映狀態，狀態一變，界面自重畫焉。

```
@Composable
fun Counter() {
    // remember + mutableStateOf：記此將變之值
    var count by remember { mutableStateOf(0) }

    Column {
        Text(text = "汝點了 $count 次")
        Button(onClick = { count++ }) {   // 改狀態
            Text("點我 +1")                // 界面自動刷新
        }
    }
}
```

三行代碼，得一活之計數器。此 Compose 勝舊式寫法之處：毋庸手動「刷新界面」，改數據足矣。記二關鍵函數：remember（使值於界面重組時不被清空）與 mutableStateOf（告 Compose 曰：此值變則當重畫）。

用戶點擊

狀態改變（count++）

Compose 感知到變化

自動重畫界面（重組）

圖 3：Compose 之「狀態界面」單向數據流

五、主題與深色模式：一配而全局生效

模板項目中之 MyFirstAppTheme { ... } 即主題也。其本 Material 3 設計之範，助汝一統理顏色、字體、形狀。欲支深色模式，僅需三步：

1. 於主題中定淺色/深色兩套之配色（模板已自生 ColorScheme）；
2. 系統切至深色模式時，isSystemInDarkTheme() 自判之；
3. 用主題所供之色（如 MaterialTheme.colorScheme.primary），勿硬寫色值。

小識：Material 3 動態取色

自 Android 12 起，「Material You」能使 App 自汝壁紙取色。Compose 模板默認支之（dynamicColor）。汝之 App 於不同用戶之機上將自易其裝，此現代安卓之浪漫也。

六、預覽、動畫與實時編輯：Compose 之「外掛」

Compose 開發體驗之「外掛」有三，一用則不可復返：

- @Preview 預覽：與 Composable 加 @Preview 之註，Android Studio 右側立渲染界面——改碼所見即所得，不必每番裝於機上；
- 實時編輯（Live Edit）：2026 年之 Android Studio 支於模擬器/真機上「熱更新」部分之改，若易文字顏色之屬小調，機上秒變；
- 動畫 API：animateFloatAsState、AnimatedVisibility 數行代碼即令諸元平滑而過——「高級之感」往往僅欠此數十行耳。

```
@Composable
fun AnimatedBox() {
    var big by remember { mutableStateOf(false) }
    // 值變則自平滑至標之
    val size by animateDpAsState(if (big) 120.dp else 60.dp)

    Box(Modifier
        .size(size)
        .background(MaterialTheme.colorScheme.primary)
        .clickable { big = !big }
    )
}
```

小識：先功能後動畫

新手常犯之病，乃「界面尚未做對，遽糾結於動畫」。先使功能通焉，終乃一統加動效——此亦「簡單至上」之則：動畫者錦上添花也，功能者雪中送炭也。

來動手：作一「心情按鈕」

易模板項目為心情切換之器：一鈕，點之則於喜哀怒之間循環而切，文與背景之色隨之而變。提示：以 when 依狀態擇文案與色。既成以遺友玩之，可獲第一波「哇汝能作 App 矣」。

本章小結

- Compose 之思：界面 = @Composable 函數，積木式相套
- 四大組件：Text / Button / TextField / Image
- 布局三件：Column 縱列、Row 橫列、Box 疊置
- Modifier 連環「上妝」：padding、fillMaxWidth、background、clickable——其序有意
- 狀態者魂也：remember + mutableStateOf，數據一變界面自重畫
- 主題用 Material 3 + 動態取色，深色模式一配而全局生效

第五十二章：四大組件與 Activity 生命之期

若謂 *Compose* 為 *App* 之「顏」，則此章所言者，*App* 之「骸」也。安卓系統有四大基本組件，凡 *App* 皆由之拼成；而其中至要者曰 *Activity*，有一「生命之期圖」，凡安卓開發者皆須銘之於心。此章既學，則「何切後臺而返，吾 *App* 之狀態遂失」此類千古之謎，汝能解之矣。

一、四大組件：安卓之「四梁八柱」

安卓官家定四種基本組件，各司一類之事：

組件	職守	設譬
Activity	可見之界面	店鋪之門面——用戶可見、可交互
Service	後臺任事之「隱者」	倉庫之工——奏樂、下載文件，毋須界面
BroadcastReceiver	受系統之廣播	傳達之室——「電池無電矣」「網斷矣」之喇叭
ContentProvider	跨 App 共享數據	公家資料之櫃——譬如以聯絡人共享於他 App

安卓四大組件速覽

新手之階，汝百分之九十五之光陰皆與 Activity 相交接；Service 與 BroadcastReceiver 者，作音樂之播、消息之推時用之；ContentProvider 用之至寡（系統已封常用之數據矣）。又當慎之：四大組件皆須於 AndroidManifest.xml 中登記戶口，不登則系統莫能尋之。

二、Activity 生命之期：安卓開發之首張「藏寶圖」

機上之 App 隨時可被斷：電話至矣、用戶切而回微信、系統記憶體不足而殺汝……Activity 之一生，即於此圖上循環而遊：

onCreate（出生：初始化）

onStart（可見了）

onResume（前臺交互中）

onPause（部分被遮住）

onStop（完全看不見）

onDestroy（銷燬）

onRestart / onStart / onResume：返前臺時循原路而返

圖 4：Activity 生命之期（行之方：上排生下排離，返則原路而折）

回調	何時而發	典型之用
onCreate	Activity 創建之時（一遭）	載入界面、初化數據
onStart	界面轉而可見	始行輕量之邏輯
onResume	得焦點、可交互	始動畫、復其播
onPause	見遮其半/切走之頃	停動畫、存其稿
onStop	全不可見	釋重資、停下載
onDestroy	毀之前	終局之清理

六生命之期回調之典型用

避坑之警：轉屏將「殺」Activity

新手至經典之覆車：App 轉屏一遭，輸入框中之字盡失。其故：轉屏則常毀而重建 Activity（歷 onDestroy onCreate 一遭）。解之之法：用 rememberSaveable 存其狀態（Compose 中代 remember 者），重建之時自尋回其值。第六、七章將屢用之。

三、Intent：App 內外通行之「車票」

Intent（意）者，組件之間通信之「車票」也。汝欲自一頁跳於他頁，或啟瀏覽器、起相機，皆須「買票登車」：

```
// 方式一：顯式 Intent——明指所往之頁
val intent = Intent(this, DetailActivity::class.java)
intent.putExtra("name", "張三")      // 車上附"行李"（數據）
startActivity(intent)

// 方式二：隱式 Intent——但言所為，系統自擇其人
val web = Intent(Intent.ACTION_VIEW, Uri.parse("https://developer.android.com"))
startActivity(web)   // 系統啟瀏覽器/或使汝擇
```

二法各有其用：顯式者用於己 App 內頁之跳轉；隱式者用於調系統或他 App 之能（啟網頁、撥電話、分享）。Compose 項目之中，於 Composable 內以 LocalContext.current 得 Context，即可 startActivity。

四、多頁與導航

真實之 App 皆有多頁。Compose 官家所薦之導航方略，曰 Navigation Compose（依賴之庫 androidx.navigation）：

```
val navController = rememberNavController()

NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen(navController) }
    composable("detail/{name}") { entry ->
        val name = entry.arguments?.getString("name")
        DetailScreen(name ?: "未知")
    }
}

// 於頁面中跳轉：
navController.navigate("detail/張三")      // 帶參數
navController.popBackStack()               // 返回上一頁
```

小識：頁多則奈何？

項目既長，宜集諸頁之路由於一文件（譬如 AppNavGraph.kt），統而理之，毋使「跳而何往」之邏輯散於諸頁。此自「能行」趨「可維護」之第一步也。

五、順而識之：Fragment 為何物（及本書何以不教）

檢舊之教程，汝必遇 Fragment——其為 Activity 中「可復用界面之片段」，於舊式多頁架構中為絕對之主角。然 Compose 之世，頁面逕以 Composable 函數 + Navigation 路由足矣，官家亦已弱 Fragment 之位。本書故不列之於主線。

貽汝之議：新項目勿主動用 Fragment；然讀舊碼、面試見問之時，知其為「Activity 中所嵌之界面片段」足矣。為學之資，恆先擇 2023 年以後者——安卓技術之棧，其「老黃曆」之新，速於所想。

來動手：為計數器裝「記憶」

改第五十一章之計數器：(1) 易 remember 為 rememberSaveable，轉屏而驗其數尚在；(2) 加「關於」一鈕，以 Navigation 跳於第二頁而顯版本之號；(3) 於兩頁之 onCreate/onPause 中印日誌（Log.d），以 Logcat 觀生命之期調用之序。三步既畢，生命之期遂非紙上談兵矣。

本章小結

- 四大組件：Activity（門面）、Service（後臺）、BroadcastReceiver（喇叭）、ContentProvider（資料櫃）
- 生命之期六回調：onCreate onStart onResume onPause onStop onDestroy

- 轉屏則毀而重建 Activity：用 rememberSaveable 保其狀態
- Intent 者「車票」也：顯式跳己頁，隱式調系統之能
- 多頁用 Navigation Compose：NavHost + composable 路由

第五十三章：數據之儲與網絡之請

App 之僅可「觀」者，死物也；能儲數據、能連網絡者，方為活物。此章識安卓數據層之「三件」：*DataStore*（儲小配置）、*Room*（儲結構之數據）、*Retrofit*（與服務器言），外益其「總調度」——協程。此章既學，汝遂能書真 *App* 矣，能記用戶之習、能實時更新其內容者也。

一、儲存之方何擇：先觀此表

方案	宜儲何物	特點	本書何處用之
DataStore	零散配置（主題、開關）	鍵值之對，簡而安	第五十四章記賬冊之設置
Room	結構之數據（賬目、用戶）	真 SQLite 數據庫之封裝，可查詢	第五十四章賬目之記錄
Retrofit	（不儲，聯網而取）服務器之數據	HTTP 請求庫之事實標準	第五十五章天氣之數據

數據三件之分工

記一口訣：「小配置入 DataStore，成批之數據入 Room，外之數據行 Retrofit」。舊教程所常見之 SharedPreferences，已為 DataStore 所代；SQLite 手書之 SQL，亦已為 Room 所代——逕學其新，毋蹈舊轍。

二、DataStore：儲少許小配置

欲記「用戶前所擇之幣種」「深色模式啟否」此類零散之值，用 DataStore（Preferences 之版）：

```
val Context.dataStore by preferencesDataStore(name = "settings")

// 寫：存一個值
suspend fun saveTheme(isDark: Boolean) {
    context.dataStore.edit { prefs ->
        prefs[THEME_KEY] = isDark
    }
}
```

```
// 讀：取之（Flow 自告其變）
val themeFlow: Flow<Boolean> = context.dataStore.data
    .map { it[THEME_KEY] ?: false }
```

慎之，所返者 Flow 也——能流之數據流也。值一變，凡觀之之處自受新值。此概念（響應式）乃現代安卓之基石，第五十四章將實戰焉。

三、Room：為數據安「真家」

記賬之冊有賬成百上千，且欲依類、依月查而計之——此須數據庫也。Room 者，SQLite 官家之封裝，使「建表、查數據」如書 Kotlin 之自然。有三物焉：

```
// 1. Entity：表結構（一張表 = 一個 data class）
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val amount: Double,
    val category: String,
    val note: String,
    val date: String
)

// 2. Dao：操作接口（增刪改查）
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT SUM(amount) FROM records WHERE category = :c")
    suspend fun sumByCategory(c: String): Double
}

// 3. Database：總裝
@Database(entities = [Record::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun recordDao(): RecordDao
}
```

此「三件」之模板可原樣搬入汝之任何項目——易其字段之名足矣。查詢所返者 Flow，界

面隨數據自新，妙不可言也。

四、Retrofit：與服務器言

天氣、新聞、匯率……此諸數據不在機中，在他人之服務器上。Retrofit 使「向服務器索數據」易如調函數。三步而行：

```
// 1. 定接口：狀 HTTP 之請如函數
interface WeatherApi {
    @GET("v1/weather")
    suspend fun getWeather(@Query("city") city: String): WeatherResponse
}

// 2. 建實例
val api = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(WeatherApi::class.java)

// 3. 如調函數然（於協程中）
val result = api.getWeather("北京")
```

避坑之警：網絡鐵律二條

(1) 網絡之請決不可行於主線程——將卡死界面乃至崩潰（NetworkOnMainThreadException）。凡網絡之調必置於協程（viewModelScope.launch）之中。(2) 明文之 HTTP，默則被禁：自安卓 9 始須 HTTPS。測試之時若欲用 http:// 本地之址，須於 Manifest 顯聲 usesCleartextTraffic——然正式之品必全行 HTTPS。

五、JSON 之析：為網絡數據作「譯」

服務器所返之數據多為 JSON——一串如「嵌套字典」之文。Retrofit 須一「譯官」化之為 Kotlin 對象。2026 年兩大主流之方：

方案	特點	所薦之境
Gson / Moshi	老牌反射之方，配置無費	速於起步、字段無多之小項目
kotlinx.serialization	官家編譯期之方，愈速愈安	新項目之首選（2026 年主流）

JSON 析之兩大方

```
// 予數據類加 @Serializable 註解
@Serializable
data class WeatherResponse(
    val weather: List<Weather>,
    val main: Main,
```

```
    val name: String
)

// Retrofit 掛對應之 Converter（以 Retrofit 2.11+ 官方支持為例）：
// .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
```

無論何方，鐵律皆曰：先觀真返之 JSON，後定字段。多「析之敗」之 bug，皆因字段名書誤、嵌套之層猜誤——印原 JSON 而對之，十秒可定。

六、協程：安卓並發之「通語」

八、協程：安卓並發之「通語」

上文屢現之 suspend 與 launch，出於 Kotlin 協程（Coroutine）——安卓官家並發之方也。汝可粗而解之曰：使「須待片時」之操作（網絡、數據庫）不塞界面。

```
// 於 ViewModel 中啟協程
viewModelScope.launch {
    val records = dao.getAll().first()  // suspend：自掛起而待
    // 更新界面狀態……
}

// 關鍵詞速記：
// suspend   —— "吾欲待，毋阻他人"
// launch    —— "開工！"
// viewModelScope —— 隨頁面之"工坊"，頁毀則自收工
```

零基者但記其用法可也，其後「掛起與復還」之理可後而深究之。日常開發，汝所書者百分之九十皆此模板也。

來動手：作「每日一句」之小應用

覓一免費名言之 API（譬如 hitokoto 之屬），以 Retrofit 拉「每日一句」，顯於屏之中央，加「易一句」之鈕（協程中復請之）。復以 Room 存所喜之句於「收藏之夾」。此一習，DataStore/Room/Retrofit/協程盡嘗用之——既成之，則第七、八章如小菜一碟矣。

本章小結

- 口訣：小配置入 DataStore，成批數據入 Room，外之數據行 Retrofit
- DataStore 返 Flow：值一變，觀者自受
- Room 三件：@Entity（表）+ @Dao（操作）+ @Database（總裝），模板可逕復用
- Retrofit 三步：定接口創實例協程中調之
- 鐵律二條：網絡不入主線程；正式之品必 HTTPS

- 協程 = 不塞界面之待；viewModelScope 者，頁面專屬之作坊也

第五十四章：項目一——吾之記賬冊（本地版）

喜迎汝首個「正經之項目」。此記賬冊雖不連網，然五臟俱全：能記一筆之賬、能觀流水、能依月而計、能刪能改。既成之，汝遂全歷「數據邏輯界面」之全程。勿遽抄代碼——先思之明而後動，此吾儕恆守之則也。

一、需求之思：先問五問

動手之前，以「需求五要素」理其念使明：

要素	所答
Who（誰用之）	己用之，家人朋友亦可
What（為何事）	記收支、觀流水、依月而彙、依類而計
Why（何以為之）	練手 + 明錢費於何所
How（以何術為之）	Kotlin + Compose + Room + MVVM，純本地無後端
Constraint（所限）	一週而成；最低支持 Android 8；雖不上架，亦先依上架之準而書

記賬冊需求五要素

繼而拆為「步步可用」之小節：(1) 建 Room 數據之層（能儲一賬）；(2) 列表之頁（能觀流水）；(3) 錄入之頁（能加賬）；(4) 刪與編；(5) 月彙之卡。每竟一步，App 皆一「能用之物」也。

二、數據之層：使「賬」有家

先定一賬之形——字段須足用而毋貪多：

```
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val type: String,        // "income" 收入 / "expense" 支出
    val amount: Double,      // 金額（元）
```

```
    val category: String,     // 分類：餐飲、交通、薪俸……
    val note: String = "",    // 備註（可選）
    val date: String          // "2026-08-15"
)
```

Dao 中加二「統計用」之查詢——此記賬冊之魂也：

```
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC, id DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='income' AND
    date LIKE :month || '%'")
    suspend fun monthIncome(month: String): Double   // month 形如 "2026-08"

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='expense' AND
    date LIKE :month || '%'")
    suspend fun monthExpense(month: String): Double
}
```

小識：金額何以用 Double？

謹嚴之金融 App 當用 BigDecimal 或「以分為單位之 Int」以免浮點之差（0.1+0.2≠0.3 之舊題）。練手之項目用 Double 足矣；他日若作真金融之品，記易「分」為單位——此無數先輩踐而後知之坑也。

三、MVVM：界面與邏輯「分家」

逕使界面操作數據庫乎？能行，然界面之碼將冗長可憎。MVVM 之構使各司其職：View（界面）惟司畫，ViewModel（邏輯之樞）司數據，Model（Room）司儲。

View（Compose 界面）

ViewModel（狀態與邏輯）

Model（Room 數據層）

圖 5：MVVM 三層之構——數據單向而流，層層不相踰

```
class RecordViewModel(private val dao: RecordDao) : ViewModel() {
```

```
    // 將 Flow 化為界面可觀之狀態
    val records = dao.getAll()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(), emptyList())

    // 用戶操作：調用 suspend 函數
    fun addRecord(type: String, amount: Double, category: String, note: String,
    date: String) {
        viewModelScope.launch {
            dao.insert(Record(type = type, amount = amount,
                category = category, note = note, date = date))
        }
    }

    fun deleteRecord(record: Record) {
        viewModelScope.launch { dao.delete(record) }
    }
}
```

界面（Compose）中如是用之：val vm: RecordViewModel = viewModel()，繼 val list by vm.records.collectAsState()——數據一變，列表自新，汝無所事。此 MVVM + Flow + Compose 三劍合璧之威也。

四、界面：列表之頁 + 錄入之頁

列表之頁用 LazyColumn（懶載之列）——數據雖多不卡：

```
@Composable
fun RecordListScreen(vm: RecordViewModel) {
    val records by vm.records.collectAsState()

    Column {
        // 頂部匯總卡片（收入-支出=結餘）
        SummaryCard(records)

        // 流水列表
        LazyColumn {
            items(records, key = { it.id }) { record ->
                RecordRow(record) { vm.deleteRecord(record) }
            }
        }

        // 懸浮"記一筆"按鈕  彈錄入對話框
        FloatingActionButton(onClick = { showAddDialog = true }) {
            Text("+")
        }
    }
```

```
}

@Composable
fun RecordRow(record: Record, onDelete: () -> Unit) {
    Row(Modifier.fillMaxWidth().padding(12.dp)) {
        Text(record.category, fontWeight = FontWeight.Bold)
        Spacer(Modifier.weight(1f))
        Text(if (record.type == "income") "+${record.amount}" else "-${record.
    amount}",
             color = if (record.type == "income") Color(0xFF27AE60) else Color(0
    xFFC0392B))
    }
}
```

錄入用 AlertDialog（對話之框）至省事：數輸入框 + 收入/支出之切換 + 分類之下拉，點「確定」則調 vm.addRecord(...)。

慎之：輸入之校驗勿惰

金額之輸入框必驗之：空值、負數、「abc」此類非法之入皆須攔之（以 toDoubleOrNull() 判之）。App 之崩，至常之因一即「以用戶之入為乖孺」。記之：凡用戶所入，皆當敵而審之。

五、編輯與刪除：使數據「能進能出」

惟加而莫刪之記賬冊，用之三日即廢。補二操作：

```
// Dao 中補二條：
@Query("UPDATE records SET amount=:amount, category=:category, note=:note WHERE id
    =:id")
suspend fun update(id: Long, amount: Double, category: String, note: String)

@Query("DELETE FROM records WHERE id=:id")
suspend fun deleteById(id: Long)

// 界面上：長按列表項彈"編輯/刪除"之單（Compose 用 DropdownMenu 或 AlertDialog）
// 刪前宜彈確認之框——"誤刪"者記賬類 App 用戶之所最深恨也
```

交互之細，經驗二條：刪必再確認之（防手之滑），編宜預填原值（易一字較重輸省事）。毋輕此二細——產品之感，即此諸細所堆成也。

六、驗收：三層之檢 + 二進階之挑戰

八、驗收：三層之檢 + 二進階之挑戰

項目既成，依「功能邊界架構」三層而驗：能記否、能刪否、重啟 App 數據尚在不（功能）；金額入 0 或負數崩否、轉屏失數否（邊界）；界面與數據有無相混、字段有無硬編（架構）。

基礎之版既過驗收，為己加二道「進階之菜」：

1. 月度之篩選：頂加月份之切換（譬如 2026-08 v），列表惟顯當月之流水；
2. 分類之餅圖：以 Compose 之 Canvas 或引入輕量圖表之庫，繪「錢費何處」為餅圖——既成此，記賬冊遂頗有「產品之樣」矣。

來動手：本週竟之

依「數據層 ViewModel 列表頁錄入頁彙總卡片」之序而動手，日竟一步。困則以報錯原樣擲於搜索引擎或 AI（猶憶第七篇《AI 編程實戰篇》乎？述問題時明言「期望之行、實際之行、復現之步」）。既成以之記賬一週——自作之 App 自用之，成就之感將推汝前行不輟。

本章小結

- 需求五要素拆為「步步能用」之小節，而後動手
- 數據層：Record 實體 + Dao（含月度統計之 SQL）+ Database
- MVVM 之分：View 惟司畫，ViewModel 司邏輯，Model 司儲
- Flow + stateIn + collectAsState：數據一變界面自新
- LazyColumn 懶載之列；AlertDialog 以錄入
- 三層之驗（功能/邊界/架構）+ 進階之挑戰（月度之篩選、分類之餅圖）

第五十五章：項目二——天氣查詢 App（網絡版）

記賬冊者「練內功」也，天氣 *App* 者「見世面」也——其將與互聯網相交接。此項目汝將學：請免費之 *API*、析 *JSON*、理「載中/成/敗」三種界面之態，及何以善藏 *API* 之鑰。既成之，汝遂具作任何「連網 *App*」之能——新聞、匯率、翻譯，其術一也。

一、申請免費天氣之 API

個人練手，薦 OpenWeatherMap 或國內之和風天氣（QWeather）——皆有免費之額，註冊即用。以 OpenWeatherMap 為例：

1. 註冊一賬於 API keys 之頁生一 Key（apikey）；
2. 免費版可調 Current Weather（當前天氣）之接口：api.openweathermap.org/data/2.5/weather?q=beijing&appid= 汝之 Key&units=metric；
3. 先於瀏覽器試一遭，觀所返 JSON 之形——此接任何 API 前必為之「預習」也。

避坑之警：密鑰決不可硬寫於代碼

API Key 者，猶汝「錢囊之密碼」也，一書入代碼而提交於 GitHub，數時之內即為爬蟲掃而盜用。正法：置於 local.properties（此文件已為 gitignore 所忽），復由 Gradle 之 buildConfig 讀入代碼。搜「Android 隱藏 API Key 之最佳實踐」依而改之可也。

二、網絡之層：Retrofit + JSON 之析

先依 API 所返之 JSON 定「數據之模」，字段不相合者一切忽之：

```
// JSON: {"weather":[{"description":"晴"}],"main":{"temp":26.5,"humidity":40},"name":"北京"}
data class WeatherResponse(
    val weather: List<Weather>,
    val main: Main,
    val name: String
)
```

```
data class Weather(val description: String)
data class Main(val temp: Double, val humidity: Int)

interface WeatherApi {
    @GET("data/2.5/weather")
    suspend fun getWeather(
        @Query("q") city: String,
        @Query("appid") key: String,
        @Query("units") metric: String = "metric",
        @Query("lang") lang: String = "zh_cn"
    ): WeatherResponse
}
```

Gson 自以 JSON 塞入此諸 data class——字段之名相合，魔法自生。

三、三種界面之態：載中 / 成 / 敗

連網 App 之魂在「狀態之理」。網遲、網斷、城名書誤……每一情形，界面皆須白於用戶。以一 sealed class（密封之類）雅而表之：

```
sealed class UiState {
    object Loading : UiState()                      // 旋轉中
    data class Success(val data: WeatherResponse) : UiState()  // 成功
    data class Error(val message: String) : UiState()          // 失敗
}

// ViewModel 中：
fun search(city: String) {
    viewModelScope.launch {
        _state.value = UiState.Loading
        _state.value = try {
            UiState.Success(api.getWeather(city))
        } catch (e: Exception) {
            UiState.Error("查無此城，易其名再試之？")
        }
    }
}

// 界面中：
when (val s = state) {
    is UiState.Loading -> CircularProgressIndicator()   // 旋轉
    is UiState.Success -> WeatherContent(s.data)        // 展示
    is UiState.Error -> Text(s.message)                 // 報錯
}
```

此 Loading/Success/Error 三段式者，連網界面之「萬金油模板」也，先背之，後凡連網之

App 依樣畫葫蘆可也。

四、實戰界面之碼：繪天氣以出

狀態既有，界面如斯（關鍵之片段）：

```
@Composable
fun WeatherContent(data: WeatherResponse) {
    Column(
        Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(data.name, fontSize = 28.sp, fontWeight = FontWeight.Bold)
        Text(
            "${data.main.temp}°C",
            fontSize = 72.sp,
            fontWeight = FontWeight.Light
        )
        Text(
            data.weather.firstOrNull()?.description ?: "未知",
            fontSize = 20.sp,
            color = Color.Gray
        )
        Spacer(Modifier.height(16.dp))
        Row {
            Text(" 溼度 ${data.main.humidity}%")
            Spacer(Modifier.width(24.dp))
            Text(" 體感 ${data.main.temp}°C")
        }
    }
}
```

小細數者可學：firstOrNull() 安取其首元（列表或空），?: 「未知」以為兜底之文——空安全之則於界面亦然。溫度以 72.sp 大字之號壯其場面，「輕」字重（Light）使數顯雅，此天氣類 App 之經典之方也。

五、界面與交互之細

七、界面與交互之細

- 搜索之框：頂置 TextField，書城名而點搜（鍵上設「搜索」之動，回車即搜）；
- 天氣之卡：大字之溫（26.5°）、天氣之述（晴）、濕度、城名；
- 收藏之城：常觀之城存於 DataStore，次啟自查之——便而溫習第六章；

- 下拉而新：Compose 中用 PullToRefreshBox（Material 3 所供），手感頓「專業」矣。

小識：圖片使界面活

每種天氣配一 emoji 或圖標（），以 when(description) 映之——三行代碼，界面溫情倍之。亦可載第三方天氣圖標之庫，然練手之階 emoji 足矣。

來動手：天氣 App 驗收之單

(1) 循常搜索而得天氣；(2) 斷網，App 雅而報錯不崩（啟飛行之模式以測！）；(3) 入不存之城，其提示也善；(4) 藏二城，重啟 App 尚在；(5) 轉屏不失其態。五條皆過，此項目方算卒業。

本章小結

- 先於瀏覽器試 API、曉 JSON，而後書碼——「預習」恆無虧
- API Key 藏於 local.properties，決不入代碼之庫
- Retrofit 接口 + data class：字段相合，JSON 自析
- Loading/Success/Error 三段之態，連網界面之萬金油也
- 網斷、空入、城之不存——理錯者，連網 App 及格之線也

第五十六章：琢磨與發佈——測試、調試與上架 Google Play

App 既成，繼之奈何？此章答「自能行」至「能交付」之三事：何以高效調試而得 *bug*、何以書測試以防回歸、何以送 *App* 上 *Google Play* 而使天下皆得下載之。上架非徒儀式之感——乃使一作化為一產之分水嶺也。

一、調試三板斧：Logcat、斷點、UI 檢查器

工具	所為何事	何以用之
Logcat	觀系統/自書之日誌	代碼中 Log.d(”TAG”,” 值 =$x”)，面板搜 TAG 以濾
斷點調試	使程序停於某行，觀變量之值	行號旁一點而出紅點，Debug 模式而運，逐步行之
Layout Inspector	觀運行中界面之構	菜單 Tools Layout Inspector，實時觀組件之樹

調試三板斧

小識：報錯何以讀？

崩之後，Logcat 中有赤色堆棧（Stacktrace），自下而上覓首行以己包名起者（譬如 com.example.yourapp），彼行基本即案發之場。先觀「Exception 之類」（空指針乎？類型之轉乎？），復觀「at ... 汝之文件: 行號」。此招能解崩潰之百分之八十。

二、測試：為代碼上保險

手測不足恃——改此功能，彼者潛壞。測試者，「悔之藥」而兼「防彈之衣」也。安卓之測分三層，新手先精究其第一層：

層級	所測何物	置於何所	遲速
單元測試	純邏輯（計算、校驗）	app/src/test/	秒級
儀器測試	真機之上界面之行	app/src/androidTest/	分鐘級
手動測試	真人之感、體驗	——	恆須之

測試三層

```
// 將"金額字符串轉數字"此種邏輯抽成純函數
fun parseAmount(input: String): Double? {
    val v = input.trim().toDoubleOrNull()
    return if (v != null && v > 0) v else null
}

// 測試它（JUnit4）
@Test
fun emptyInputReturnsNull() {
    assertEquals(null, parseAmount(""))
}
@Test
fun negativeReturnsNull() {
    assertEquals(null, parseAmount("-5"))
}
@Test
fun normalValueParses() {
    assertEquals(12.5, parseAmount("12.5"))
}
```

記賬冊中金額之校驗、統計之算，皆當有單元測試罩之。行測試：右鍵 test 目錄 Run。綠而皆過，則可安然而改碼矣。

三、簽名打包：為 App 蓋「印」

安卓要求凡安裝之包必簽之——猶蓋公印，以證「此包吾所出，未遭改竄」。調試時 Android Studio 自用 debug 之簽；發佈須用己之 release 簽。其序：

1. 菜單 Build Generate Signed Bundle / APK；
2. 創 keystore 之文件（密鑰之庫）——設其密碼而妥藏之；
3. 擇 release + AAB 之式，構建乃成。

避坑之警：keystore 失則 App 失

上架之後更新 App，必用同一之簽，否則店將拒其新。keystore 之文件與密碼一失，汝惟易新應用而重上架——舊用戶皆無由接焉。務以 keystore 與密碼備於安隱之所（雲盤加密 + 記事之冊），此安卓開發者至昂之教訓之一也。

四、上架 Google Play 之全程

註冊開發者賬號

準備商店素材

上傳 AAB

填數據安全表

審覈（數小時 ~ 數天）

上架

圖 6：Google Play 上架六步

- 註冊：一次 25 美元之註冊費，須可用之 Google 賬號；
- 素材：應用之圖標（512×512）、截圖（至少二張）、功能之述、私隱政策之鏈（收數據者必填）；
- 數據安全之表：如實申報所收何數據——不連網之記賬冊可「零收集」；
- 審核：新賬號首番上架恆為人工所審，數時至數日不等，見駁則依議而改。

國內之用戶慎之：Google Play 須特殊網絡之境，且中國大陸之分發常行國內之店（華為、小米、應用寶之屬）。上架國內之店，其程相類，然各店自有其審之好——同一 AAB/APK 可分發於多店。以為練手，先通一家即勝矣。

五、商店素材何由製之：第一印象即繫下載之數

商店頁者，用戶遊店之時所見之櫥窗也。三材定其第一印象：

- 圖標：512×512，勿以截圖充數。若無設計之能，可以 AI 之器生簡潔幾何之形，縮於手機桌面之上猶能一望而識者，方為合格；
- 截圖：至少二張，以四至六張為佳，以彰最核心之功能場景；每張配一淺白之語（「一秒記一筆」勝「高效記帳之方」十倍）；
- 描述：首句當明「此為何物、予誰用之」，勿堆「最專、最強」之詞——商店搜索之權重，主視關鍵詞與下載轉化，誠實勝於誇張。

小訣：以 A/B 之法觀同行

上架之前，遍觀商店同類 App 之前十名，取其截圖比而較之：其圖標何風？其截圖售何賣點？其標題何書？模仿非抄襲也——學其套路而易以己之內容，此凡產品冷啟動至速之一課也。

五、性能優化與崩潰上報小清單

App 上架之前，此二事宜各費半小時為之：

一、性能自察四條

1. 列表當用 LazyColumn：長列切莫以 Column 強排之——懶載者，安卓性能之第一課也；
2. 圖勿硬塞原圖：當用 Coil 圖庫自動縮放而緩存之，一張 5MB 之原圖足以令低階之機滯塞；
3. 勿於主線程任重事：數據庫、網絡、大文件皆當於協程中為之；
4. 當以 Profiler 量之：Android Studio 自帶之性能分析器能觀記憶體、CPU、耗電——先測而後優，勿憑感覺。

二、崩潰上報

爾測試之時不崩，非謂用戶之機不崩也。上架之前，當接 Firebase Crashlytics（谷歌官方之物，免費）：配置數行，用戶崩潰則自動上報堆棧，爾於後台即可見「何機型、何版本、崩於何行」。無崩潰監控而上架者，猶閉目而駕車也。

來動手：將記帳本「交付」一次

(1) 為 parseAmount 諸邏輯補單元測試三則；(2) 以 Logcat 親捕自造之崩潰一處而修之；(3) 生 release 簽名之 APK 一枚，致諸友朋安裝之（此乃「作品」之儀也）；(4) 力有餘則註冊開發者之號，以記帳本真呈上架。前三條今夕可竟。

本章小結

- 調試之三技：Logcat 觀日誌、斷點觀變量、Layout Inspector 觀界面
- 讀崩潰：自堆棧之下而上，覓己包名之首行
- 測試三層：單元（秒級，先學此）/儀器/手動
- 發布必簽名：keystore 與密碼宜善備份之，失之則 App「易主」矣
- 上架六步：註冊素材上傳 AAB 數據安全表審核上架

第五十七章：進階之路——權限、通知與現代架構

能作 *App* 者，僅其始也。此章論自「會作」至「作善」之數關：何以雅然請權限、何以發通知而不擾人、何以再抬架構一層，以及——於 *2026* 年，何以與 *AI* 相伴而作安卓之碼。凡此諸條，皆爾於真實項目中頃刻將遇者也。

一、運行時權限：先敲門，再進門

相機、定位、通訊錄、通知……此皆「敏感之地」，安卓令 App 於運行之時請於用戶（自 Android 6 始）。其規：先聲明（Manifest），再申請（代碼），用戶拒之則宜雅然降級。Compose 之中以官方庫 Accompanist 之 rememberPermissionState 為最省心：

```
// Manifest 中先聲明：
// <uses-permission android:name="android.permission.CAMERA" />

@Composable
fun CameraButton() {
    val permission = rememberPermissionState(
        permission = android.Manifest.permission.CAMERA
    )
    Button(onClick = {
        when {
            permission.status.isGranted -> openCamera()
            permission.status.shouldShowRationale -> showExplainDialog()
            else -> permission.launchPermissionRequest()
        }
    }) { Text("拍照") }
}
```

注意：遭永拒則奈何？

用戶既勾「不再詢問」，則 launchPermissionRequest 不復彈窗。此時當引用戶往系統設定手動啟之（以 Intent 跳至應用設定之頁），且於界面明言「此權限何所用也」。態度稍善，則用戶啟權限之意必高——申請權限之文案，宜認真書之。

二、通知：於正確之時說有用之言

通知者，App 達用戶之途也，亦最易為用戶所「拒」之功能也。發通知有三戒：惟發有用者、令用戶能關之、勿半夜擾之。其基礎之法：

```
// Android 13+ 必須先申請 POST_NOTIFICATIONS 權限（見上一節）

// 1. 建通知渠道（Android 8+ 必須）
val channel = NotificationChannel(
    "reminder", "提醒", NotificationManager.IMPORTANCE_DEFAULT
)
val manager = getSystemService(NotificationManager::class.java)
manager.createNotificationChannel(channel)

// 2. 發通知
val notification = NotificationCompat.Builder(this, "reminder")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("記賬提醒")
    .setContentText("今日尚未記賬也")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .build()
if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.POST_NOTIFICATIONS) == PackageManager.
    PERMISSION_GRANTED
) {
    manager.notify(1, notification)
}
```

定時之務（如「每日 20:00 提點記帳」）當用 WorkManager——此官方所薦「後台調度之管家」也，能保任務於合宜之時而行，不必自慮電池之優化。鬧鐘之屬用之必無誤。

三、架構進階：依賴注入與模塊化

項目既大，二「甘美之擾」將至：

- 對象之造紊亂：每一頁皆自 new 數據庫、new API——當用 Hilt（依賴注入之框架）統而管之，「誰欲何物，啟口即有人奉之」；
- 代碼積之如山：單模塊之項目愈改愈緩——當拆為多模塊（app / core / feature-xxx），諸功能各自開發、各自測試。

小訣：勿過度設計

Hilt 與模塊化皆良器也，然獨人之項目勿遽用之。記帳本單模塊加手創依賴，綽綽有餘。待代碼逾數萬行、編譯始緩、多人共作之時，再因需引入。器者所以達的也，非以誇示也——此亦吾儕 AI 編程之書「簡單至上」之旨也。

四、2026 年之新玩法：AI 輔助安卓開發

2026 年，AI 已深嵌安卓開發之流程。爾若欲提速，此組合之法甚成熟：

環節	AI 何以助	爾之責
作代碼	Android Studio 內置 Gemini 及第三方 AI 之插件以補全與生成	明述需求，審其產出
查資料	以官方文檔與報錯之訊付 AI 解之	關鍵之結論，復核於官方文檔
作測試	令 AI 依「正常/邊界/異常」三類生用例	確用例合乎業務之義
改 bug	貼堆棧與相關代碼，AI 定位而修之	明其改動，行測試以驗之

AI 佐安卓開發之四場景

其具體之法——何以令 AI、何以驗收、何以避坑——凡十一章全備於本書第七篇《AI 編程實戰篇》（第 37-47 章），力薦並讀之：安卓之語法付諸本篇，AI 協作之心法付諸第七篇。

五、Compose 性能優化與調試工具

Compose 書之甚暢，然「重組過多」亦滯。三實用之技：

- 穩其參數：數據之類加 @Immutable，或用 derivedStateOf，以免無關狀態之變觸發廣域之重組；
- 切記用 key：items(list, key = { it.id }) 令列表增刪之時惟動當動之項——不加 key，刪一項或致全表重繪；
- Layout Inspector 之 Compose 模式：實時觀每 Composable 重組之次數，孰頻繁重組，一覽了然，循其熱點而優之。

又當開 Compose 編譯器之報告（Metrics），可生「何組件不穩」之報——進階之路，數據終較感覺可靠。

來動手：給記帳本加「三件套」

(1) 給記帳本加「每日 20:00 記帳之提點」（WorkManager 加通知之渠道加 Android 13 通知之權限）；(2) 以 Hilt 注入改數據庫與 API 之創建；(3) 試以 AI 之器令「月度篩選」之功速成，而後依三層驗收之法察其產出。三件既畢，爾之項目遂有「正規軍」之貌矣。

本章小結

- 敏感權限：Manifest 聲明加運行時申請加被拒之後雅然引導
- 通知三戒：惟發有用者、能闢、不擾；渠道與權限缺一不可
- 定時之務用 WorkManager，勿自造輪
- Hilt 管對象、模塊化管代碼——獨人小項目勿遽用之
- AI 佐助四場景：作碼、查資料、作測試、改 bug；其法見第七篇（第 37-47 章）

第五十八章：速查之冊與資源

此章者，可獨用之速查冊也。爾若忘某寫法或欲速決，徑翻至此。宜視之如「囊中之卡」：常用代碼之速查、常見之問、學習之資源，一頁不遺。

一、常用代碼速查之卡

欲何為	關鍵代碼
記一可變之狀態	var x by remember { mutableStateOf(初值) }
轉屏不失其狀	rememberSaveable { mutableStateOf(初值) }
縱排/橫排之佈局	`Column { }/Row { }`
懶載之列表	LazyColumn { items(列表) { ... } }
頁面之跳轉	navController.navigate(「路由」)
於協程任久待之事	`viewModelScope.launch { ...}`
觀數據之流	`val list by flow.collectAsState()`
存少量配置	context.dataStore.edit { it[KEY] = 值}
存結構化之數據	Room 三件套：@Entity / @Dao / @Database
聯網取數據	Retrofit 之接口加 suspend 之函
發通知	NotificationChannel + NotificationCompat.Builder
書日誌	`Log.d("TAG", "message")`

安卓開發速查卡（2026 年 8 月版）

二、常見問題 FAQ

Q1：吾全然不解編程，學安卓開發須時幾何？

循本書之路而行：第 1-2 週裝環境、學 Kotlin 之基；第 3-4 週學 Compose 以成界面；第 5-6 週竟記帳本之項目；第 7-8 週畢天氣之項目。兩月之間，自零至能獨作小 App。其要在日投 1-2 小時，且親敲代碼，非徒觀書也。

Q2：Java 尚須學乎？

不須也。新項目、新教程、新職缺，率以 Kotlin 為準。惟「守十年之舊項目」時方遇眾多 Java。既學 Kotlin，若工作所需，再補 Java 甚速——二者底層同一虛擬機之生態也。

Q3：模擬器乎，真機乎？

開發調試以模擬器為便（有快照、多機型），然驗收必過真機——觸感、性能、真網、傳感之器，模擬器莫能代也。力若能及，購中端真機一台以為測試之機，乃至合算之投資也。

Q4：安卓開發易覓職乎？

2026 年之市況：徒「會作界面」之初級之職，競爭甚烈；然「通 Kotlin 加善架構加有上線之作」之開發者，猶為人所爭。學習之期，爾最佳之履歷，即商店中爾自作之 App 也。

Q5：遇報錯而卡住，奈何？

黃金三步：(1) 以報錯之首行原文搜 StackOverflow 或搜索引擎；(2) 查官方文檔對應之頁；(3) 猶不解，則以「期望之行加實際之象加報錯全文加相關代碼」打包致 AI 或社區而問之。問題之大半，第一步已解矣。

三、推薦學習資源

類別	資源	所薦之由
官方	developer.android.com	安卓官方之文檔，Compose 與 Kotlin 之教程皆在其中，第一權威
官方	Kotlin 官網 (kotlinlang.org)	語言之官方文檔加在線演練之場
官方	Google Codelabs	手把手分步實驗之課，於新手至為友善
社區	Stack Overflow (android 標籤)	報錯求救之首站
社區	掘金 / CSDN 之安卓專區	中文實戰之文眾多，宜辨新舊（先觀日期）
視頻	Google Developers 官方之頻道	每歲開發者大會講解新技術
配套	第七篇《AI 編程實戰篇》（第 37-47 章）	以 AI 佐安卓開發之全副心法

安卓開發學習資源清單（2026 年 8 月）

四、Gradle 常用命令速查

平時點按鈕足矣，然命令行有時更高效（尤以配合 AI 之器或 CI 流水線之時）：

命令	功用
`./gradlew assembleDebug`	造一調試版之 APK（Windows 則去./）
`./gradlew assembleRelease`	造發布版（須先配簽名）
`./gradlew test`	行全部單元之測試
`./gradlew lint`	靜態之查：得潛藏之 bug 與不規之寫法
`./gradlew clean`	清構建之產物（「重啟大法」之編譯版）
`./gradlew :app: dependencies`	觀依賴之樹（排查版本之衝突，利器也）

Gradle 常用命令速查

小訣：構建遇事先 clean

「昨日可行今日不可行」之怪事，十之六七乃構建緩存之亂也。行./gradlew clean 而後重構，可愈玄妙之問題大半。猶不解則再啟 Android Studio——此法宜與 File Invalidate Caches 並用之。

五、結語：自第一個 App 至第 N 個 App

七、結語：自第一個 App 至第 N 個 App

回顧之，爾自「不知 Activity 為何物」，至裝成環境、學會 Kotlin、以 Compose 繪出界面、為 App 接上數據庫與網絡，乃至測試、打包、上架——此路之上，爾已竟自零至一至難之一段矣。

技術之世變甚速：框架必升，工具必代，AI 將接「敲鍵盤」之事日多。然有一物不過時：爾以一模糊之念，拆為清明之步，而後步步成之之能也。器者放大之器也，思者地基也。放大之器可換，地基不可搖。

汝不復徒為" 玩手機之人"。汝能以胸中之念，化為三億安卓之器上實運行之 App。自今日始，以鍵盤創物——此安卓開發者之浪漫也。

參考文獻

1. Android Developers. (2026). Android Developer Guides and Jetpack Compose Documentation. developer.android.com.
2. Kotlin Foundation. (2026). Kotlin Language Documentation. kotlinlang.org.
3. Android Open Source Project. (2026). Android Platform Version History and API Levels.
4. Google Play. (2026). Play Console Help: Publishing and App Signing.
5. AndroidX Release Notes. (2026). Room / DataStore / Navigation / WorkManager releases.
6. Square. (2026). Retrofit Documentation. github.com/square/retrofit.

九、iOS 開發實戰篇（第 59-69 章）

第五十九章：識 iOS——蘋果生態與開發生態

賀爾成此決——自「用 *iPhone* 之人」而為「為 *iPhone* 作應用之人」。動手之前，吾儕且先攤開輿圖：*iOS* 何自而來？蘋果「圍牆之園」於開發者何謂？一 *App* 自爾指敲鍵盤至用戶之機，其中何事之有？

一、iOS 之身世：一機改一行

2007 年正月，喬布斯於發布會出首代之 iPhone。其時尚不容第三方之應用——至 2008 年，蘋果出 App Store 與 iPhone SDK，許天下開發者為 iPhone 作軟件。當是時，「應用經濟」生焉：至今 App Store 之上有應用數百萬，為開發者累分成逾數千億美元。

系統亦自進化：初名 iPhone OS，2010 年改曰 iOS；版本號行至 iOS 17（2023），至 2025 年，蘋果改命名以隨年份——iOS 26 於 2025 年發布，2026 年 8 月今之版乃 iOS 27。命名雖變，然一事未嘗變：iOS 用戶付費之習與軟件質量之譽，移動生態之中公認至善——同一之 App，iOS 端之入往往顯高於安卓端。

節點	年份	於開發者至要之事
App Store 上線	2008	第三方開發之生態啟
Swift 發布	2014	現代 iOS 開發之語登場
SwiftUI 發布	2019	聲明式界面之框架，本書主線
iOS 26（改名之元年）	2025	命名隨年份；Apple Intelligence 全面整合
iOS 27	2026	今之最新版（本書基準）

iOS 開發大事記（截至 2026 年 8 月）

二、圍牆之園：蘋果生態之「規」與「利」

蘋果生態常被謂為「圍牆之園」（Walled Garden）——規多而管嚴：App 必經審核方得上架、開發必用 Mac 與 Xcode、分發多走 App Store。聞之豈不憋悶？然於開發者，此套規矩換得真金白銀之利：

- 設備碎片之化至微：iPhone 之型不過數款，屏之尺寸、系統之版高度劃一——爾適 5 型之機，即覆 95% 之用戶（安卓或須適數十款）；
- 系統版本更新之率至高：新系統發布數週之內，七八成用戶已升，新 API 可大膽用之，不必顧「五年前之舊系統」；
- 用戶付費之習善：App Store 之用戶更樂為應用與內購出資；
- 審核雖嚴，然商店潔：山寨流氓之軟件少，佳品易為人所見。

注意：先明其成本

iOS 開發有硬檻：必有 Mac 一台（Xcode 為 macOS 所獨佔），上架須 99 美元一年之開發者帳號（個人開發者免費之帳可於本地真機調試，然不能上架）。思此投入而後啟程——然可喜者，學習之期用免費之帳綽綽有餘，第 60 章將言之。

三、以何語言書寫：Objective-C 抑或 Swift？

iOS 開發之史亦有二「主角」。Objective-C 者開國之元老也——自 2008 年用至 2014 年；Swift 者蘋果 2014 年所發之新語也，其後進化至 Swift 6（2024 年發布，引入嚴格並發之檢查），至 2026 年，新項目幾盡用 Swift。

贈零基之爾一句實話：徑學 Swift，Objective-C 並觀亦不必——除非爾就職之後須守十年前之遺產代碼。Swift 之利：語法清爽（無 C 語之世分號與頭文件之累）、可選值（Optional）之制自語言之層滅「空指針崩潰」之業中惡夢、與 SwiftUI 無縫相合、且 iPad 之上 Swift Playgrounds 可且玩且學。

Objective-C 之風（上古之遺跡）

```
NSString *name = @"張三"; [name stringByAppendingString:@"你好"]; // 方括號 + 指針 + 分號三件套
```

Swift 之風（今世之清爽）

```
var name = "張三" name += "你好" // 直白如自然之言
```

四、一個 App 何以誕生？

與諸平臺同，iOS App 之生亦有一條流水之線。然蘋果之版多兩道「安檢」：

想法

設計（界面 + 功能）

寫代碼（Swift + SwiftUI）

編譯簽名

測試（模擬器 + 真機）

蘋果審覈

上架 App Store

圖一：一 iOS App 自念頭至上架之全流程（注意多出之「審核」環節）

流程中之關鍵概念：

- 簽名（Signing）：每 iOS App 皆必「簽名」方能安裝——以證「此包出於爾，未被篡改」。學習之期 Xcode 以爾之 Apple ID 自動簽之；上架則用開發者之證書。第 67 章詳之。
- 模擬器（Simulator）：Mac 上虛擬而出之 iPhone，開發調試之主戰場也。其所行者 Mac 之 CPU，故特流暢，較安卓之模擬器體驗勝一大截。
- TestFlight：蘋果官方「內測分發」之器——上架之前以測試之版致至多 10000 名測試之用戶，彼等以 TestFlight 之應用安裝而試之。此 iOS 生態獨有之雅也。

五、iOS 技術棧全景：爾之「武器清單」

層次	主流之擇（2026）	何所用
語言	Swift 6	作邏輯之語言
界面	SwiftUI 加蘋果設計之規範（HIG）	以代碼繪界面（本書主線）
架構	MVVM + @Observable	分界面與邏輯
數據儲存	SwiftData（數據庫） 加 UserDefaults（配置）	存數據於手機之中
網絡	URLSession + async/await + Codable	與服務器對話
構建之器	Xcode Build System / xcodebuild	化代碼為安裝之包
開發環境	Xcode（macOS 獨佔）	蘋果官方之 IDE

2026 年 iOS 主流技術棧速覽

覺之否：此單較安卓者短一截——以蘋果多事皆「包辦」矣。封閉生態之價，自由度也；其償，省心也：不必糾結擇何網絡之庫、何數據庫，官方所予往往即至優之解。

來動手：看看爾手機之「蘋果戶口」

開 iPhone 之［設定］［一般］［關於本機］，記三事：iOS 之版本、機型之名、可用之容量。再往 App Store「今日」之頁隨意遊之，留意精品 App 之截圖與文案——自今日始，爾遊商店之目，當自「用戶」而易為「開發者」矣。

本章小結

- 2008 年 App Store 上線，「應用經濟」生焉；2026 年今之版乃 iOS 27
- 圍牆之園之帳：規矩多，然設備少、版本新、用戶肯出資、商店潔

- 硬檻：必有 Mac；上架須 99 美元一年之開發者帳號；學習之期免費之帳足矣
- 語言徑學 Swift；Objective-C 惟屬遺產之代碼
- 流水線較安卓多一環：蘋果審核；TestFlight 乃內測分發之利器
- 2026 標準技術棧：Swift 6 加 SwiftUI 加 MVVM 加 SwiftData 加 URLSession 加 Xcode

第六十章：築開發之環境——Xcode 與第一個項目

工欲善其事，必先利其器。此章吾儕先築「庖廚」：裝成 Xcode、跑通模擬器、創人生第一個 iOS 項目，再拆工程之結構而觀之明白。環境既成，爾將初見己所書之代碼化為可點之鈕——此一瞬，多人記之終身。

一、備 Mac 一台

先言其檻：iOS 開發必於 macOS 之上行之。爾若無 Mac，其擇有三：購一台（縱二手之 Mac mini 或 MacBook Air 亦足用）、短期租雲上之 Mac、或先讀第八篇《安卓開發實戰篇》（Windows 即可為之）。

配置之項	至低之要求	寬裕之線（所薦）
晶片	Apple Silicon（M1 起）	M2 及更新
記憶體	8GB	16GB
磁碟	預留 40GB 以上（Xcode 本體甚巨）	256GB 起步
系統	最新之 macOS	常保自動更新

Mac 開發機配置參考（2026 年 8 月）

小訣：無 Mac 亦能先學語言

iPad 之上免費之 Swift Playgrounds 可學 Swift 語言之基（不必 Mac）；即 iPhone 之上亦可裝之。先於 Playgrounds 之中玩熟第 61 章語言之知識，待得 Mac 則徑起飛矣。磨刀不誤砍柴之工，此之謂也。

二、安裝 Xcode

Xcode 者，蘋果官方集成開發之環境也，免費。三步裝成：

1. 於 Mac 開 App Store，搜「Xcode」，點而安裝之——注意其有好幾 GB 之巨，擇網速善之時段；

2. 裝畢首次啟動，允其協議、裝畢組件（自動而成）；
3. 菜單 Xcode Settings Locations，確認 Command Line Tools 已選——命令行之器，乃多腳本與第三方之器之前提也。

注意：版本與系統之「匹配遊戲」

新 Xcode 往往須最新之 macOS，舊 Mac 則滯於舊版。上架 App Store 之時，蘋果必求以較新之 Xcode 構建（通常予一寬限之期）。故系統之更新勿拖太久——iOS 開發者乃「追新版」至勤之一群也。

三、創爾之第一個項目

開 Xcode Create New Project 擇 iOS App 之範本，而後填之：

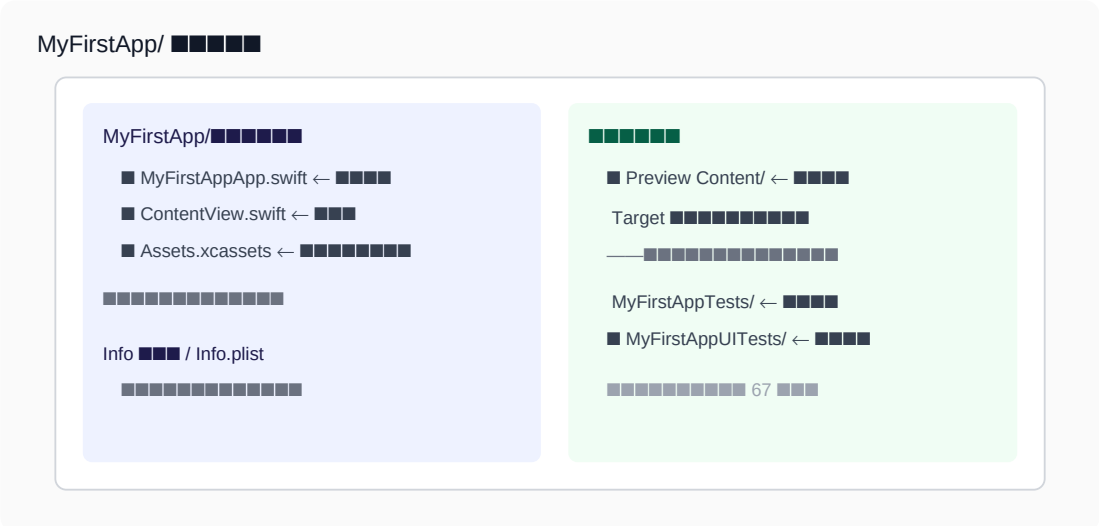
配置之項	填何物	說明
Product Name	MyFirstApp	應用之名
Team	爾之 Apple ID（免費）	簽名所用，個人帳號即可
Organization Identifier	com.example	與 Product Name 併為包之標識
Interface	SwiftUI	本書主線之框架
Language	Swift	毋庸贅釋

創建項目所填之五事

點 Create 之後，Xcode 將生一可徑行之項目。左上角擇模擬器一台（如 iPhone 17），點 > 而運行——數秒之後，模擬器之屏現一書「Hello, world!」之窗。爾之第一個 iOS App 誕生矣。

四、拆工程之結構：每文件何為

左側乃工程導航之器，新手但識此數者可也：



圖二：新手必懂之 Xcode 工程結構

- MyFirstAppApp.swift：程序之入口，SwiftUI 之中惟數行——「啟之，而後示主界面」；
- ContentView.swift：主界面之視圖，爾今所見皆在此；
- Assets.xcassets：資源之倉——圖標、圖、色皆置於此；
- Target 之設置：點左側藍色工程之圖標而入，版本之號、至低 iOS 之版本、簽名皆在其中。App 顯示之名、圖標亦於此配置。

五、真機調試：裝 App 於爾之 iPhone

五、Xcode 效率之技：新手之五「快捷鍵外掛」

熟用此數者，爾將顯得「甚專業」：

操作	快捷鍵	功用
運行	CmdR	編譯並跑之
開/切畫布之預覽	OptionCmd	所見即所得之開關
格式化縮進	CtrlI	自整所選代碼之縮進
快速開文件	CmdO	輸入文件之名而秒開（至常用！）
註釋/取消註釋	Cmd/	調試之時隨手開關代碼

Xcode 高頻快捷鍵（新手五件套）

又有兩個隱藏之福利：多開模擬器（同時跑 iPhone 與 iPad 以比其佈局）與代碼片段之庫（以常用之數行拖入右側 Snippets 之面板，下次一鍵插入）——效率之器先用之，手感自來矣。

六、真機調試：裝 App 於爾之 iPhone

模擬器雖善，真機方為終點。以免費之 Apple ID 亦可真機調試：

1. 以數據之線連 iPhone（或同 Wi-Fi 無線調試）；
2. Xcode 頂部設備之列表擇爾之機，首次將提示登 Apple ID 而自動配置簽名；
3. iPhone 之上［設定］[一般］[VPN 與設備管理］信爾開發者之證書；
4. 點 >，App 入真機矣。

注意：免費帳號之「七日限制」

免費 Apple ID 簽名之 App，七日之後失效（再運行一次即可續之）。此蘋果防濫用之制也，學習之期足用；付費開發者帳號（99 美元一年）則無此限，第 67 章詳說之。

七、小試其刃加常見報錯之急救

九、小試其刃加常見報錯之急救

開 ContentView.swift，以爾之名易 Text(“Hello, world!”) 之字，按 CmdR 再運行——數秒之後模擬器之上現爾之名。賀爾，爾已竟首次完整之「改代碼編譯安裝運行」之循環矣。

報錯之象	常見之因	解之之方
Signing 之報錯	Team 未選、證書過期	Target Signing 之中勾爾之帳號；免費之帳再跑一次
模擬器黑屏/不能開	鏡像已壞	菜單 Device Erase All Content；或刪之而重加模擬器
Xcode 下載緩/不能裝	App Store 網絡之波動	換時段；或至蘋果開發者之網站下載.xip 離線之包

新手環境報錯急救表

來動手：竟爾「開發儀式感」之清單

(1) 裝成 Xcode 並跑通模擬器；(2) 以真機跑己之 App 一次而截圖留念；(3) 易 Hello 之字為爾之名；(4) 至 Target 之設置易 App 顯示之名為「吾之第一個 App」。四步既畢，爾乃有作品之人矣。

本章小結

- 硬檻：Mac 加 Xcode；無 Mac 者先以 iPad 之 Swift Playgrounds 學語言
- Xcode 自 App Store 裝之；新版本要求新系統，更新勿拖
- 新建項目五要：名、Team、組織標識、SwiftUI、Swift
- 工程之訣：代碼皆在 MyFirstApp/ 之目錄；Target 之設置管版本與簽名
- 免費 Apple ID 真機調試有七日簽名之限；付費之帳無此限
- 開發之日常 = 改代碼 CmdR 觀其效，循環往復

第六十一章：Swift 速成——蘋果開發之第一語言

此章吾儕正式學其語言。勿懼——*Swift* 素以「善待新手」著稱，且爾不必背語法之書：先掌握 *20%* 之語法，即可看懂且寫出 *80%* 日常之代碼。其餘者，且作項目且查之。記吾儕學習之則：看懂勝於強記，會用勝於精通。

一、變量與類型：為數據起名

程序者，「數據加邏輯」也。變量者，為數據起一名而儲之也。Swift 以 let 與 var 聲明之：

```
let name = "張三"          // let：常量（宜以為默認）
var age = 18               // var：變量（後尚可改）

age = 19                   // <i class="fas fa-check-circle mr-1"></i> var 可改
// name = "李四"           // <i class="fas fa-times-circle mr-1"></i> let 不可改，編譯報錯

let score: Int = 100       // 亦可明書其型
let pi = 3.14              // 不書亦自推之

print("汝好，\(name)")     // 字符串插值：反斜槓 + 括號
```

常用類型一表記之：

類型	儲何物	示例
Int	整數	42
Double	小數	3.14
String	文字	「你好」
Bool	真/假	true / false
[Int]（陣列）	一組數據	[1, 2, 3]

Swift 常用基礎類型

小訣：let 優先之原則

Swift 社區之鐵律：默認用 let，確有所須方用 var。可用 let 之處而用 var，Xcode 將以黃字警爾。此習能助爾少卻一大類「變量遭意外之改」之 bug——代碼之變愈少，推理愈易。

二、可選值（Optional）：Swift「防崩護身之符」

此 Swift 至要之概念，無出其右。一言以蔽之：一值或「不存」之時，Swift 強爾理之——自語言之層滅「空指針崩潰」此業中之惡夢。

```
var name: String = "張三"   // 普通之型：決不容為 nil
// name = nil                // <i class="fas fa-times-circle mr-1"></i> 編譯之誤，徑阻之

var nickname: String? = nil  // 加問號：許為 nil（可選值）
nickname = "三兒"

// 用可選值，必"解包"：
if let n = nickname {        // if let：安全解包
    print("汝好，\(n)")
} else {
    print("尚無暱稱")
}

let len = nickname?.count ?? 0   // ?? 予默認值（空合運算符）
print(len)
```

注意：！乃「高危操作」

代碼中書 nickname!（強制解包）猶謂編譯器曰「吾保其非 nil」——一猜而誤，App 當場即崩。新手之期其則曰：務用 if let / ??，勿用!。Xcode 補全代碼之時偶爾順手加!，記而易之。

三、函數與閉包：打包邏輯成「鈕」

函數者，一段有名之邏輯也，若遙控之鈕。閉包（Closure）者，「無名之函」也，可隨手遞於人：

```
// 函數：名字於前，參數與返回類型於後
func greet(name: String) -> String {
    return "汝好，\(name)！"
}
print(greet(name: "張三"))      // 調用時要帶參數名

// 閉包：就地定義，輕快直接
let numbers = [1, 2, 3, 4]
```

```
let doubled = numbers.map { $0 * 2 }   // $0 = 當前元素
// [2, 4, 6, 8]

let big = numbers.filter { $0 > 2 }    // 篩選
// [3, 4]
```

SwiftUI 之中閉包無所不在：按鈕 Button(「點我」) { ... } 之後大括號即一閉包也。map、filter、sorted 此數「集合連招」請牢記之——來日 90% 列表之處理皆其組合也。

四、結構體與協議：Swift「積木」之體系

Swift 官方之好甚明：先用結構體（struct），須繼承之時方用類（class）。結構體者「值類型」也——複製之時各自為之，天然少一類之 bug：

```
// 結構體：盛數據之盒
struct Record {
    var id = UUID()
    var amount: Double
    var category: String
    var date: Date

    // 結構體自帶"逐一成員構造器"：可徑建
}
let r = Record(amount: 25.5, category: "餐飲", date: Date())

// 協議：約"必具何能"
protocol Identifiable {
    var id: UUID { get }
}
struct Record: Identifiable { ... }   // 遵守協議 = 承諾實現其要求
```

SwiftUI 之中幾乎皆 struct（View 即協議也）。此言今不解無妨，第 62 章將日日相見——Swift 之世界，先擁抱 struct 可也。

五、貽零基者之語法速查卡

欲何為	寫法
定常量/變量	`let x = 1`/`var x = 1`
字符串插值	「你好，\(name)」
定函數	func 名 (參數: 類型) -> 返回類型 { ... }
判斷	if 條件 { ... } else { ... }
遍歷	for x in 陣列 { ... }
安全解包	if let x = 可選值 { ... }
默認值	x ?? 默認值
定結構體	`struct X { var a: Int; var b: String }`
閉包	`{ $0 * 2 }`

Swift 語法速查卡（新手版）

來動手：於 Playgrounds 玩 15 分鐘之 Swift

有 Mac 則開 Xcode Playground；無 Mac 則用 iPad 上之 Swift Playgrounds 或網頁版之演練場。以本章每一段之代碼皆跑一遍，改其數字與文字，故意造數誤以觀 Xcode 何以提示之。此「先玩後學」之 15 分鐘，將令爾第 62 章上手之時輕鬆一倍。

本章小結

- let 為先，var 慎用；類型自動推斷，冒號之後明言亦可
- 可選值者 Swift 之魂也：或空必理之，if let 與?? 乃護身之符，！乃高危操作
- 函數之調用須帶參數之名；閉包者無名之函也，SwiftUI 之中無所不在
- 先用 struct，守協議（Protocol）；SwiftUI 之中幾乎皆 struct
- 20% 之語法為 80% 之事：先會用之，再精通之

第六十二章：界面之開發——SwiftUI 入門

界面者，用戶所見之一切也。*SwiftUI* 者，蘋果 *2019* 年所推出聲明式界面之框架也——爾惟「聲明」界面之狀貌，系統自任渲染、適配深色之模式、適配諸般之屏。至 *2026* 年，*SwiftUI* 已為新項目絕對之主流。此章，吾儕學「繪畫」。

一、SwiftUI 之思想：聲明式，非命令式

舊式界面之開發乃「命令式」：先告系統「創按鈕置於 (20,30) 設其色」，一步一步指揮之。SwiftUI 乃「聲明式」：爾徑述「界面之上有一赤色按鈕書『點我』」，如何繪、何時重繪，系統自任之。

```
import SwiftUI

struct ContentView: View {
    var body: some View {      // body：狀"界面之形"
        VStack {               // 豎而列之
            Text("汝好，世界！")
                .font(.largeTitle)      // 修飾符：大標題
            Button("點我") {
                print("被點擊了")
            }
            .buttonStyle(.borderedProminent)  // 藍色實心按鈕
        }
        .padding()
    }
}
```

關鍵之點：視圖者，守 View 協議之結構體也，body 之屬性述界面，. 修飾符 () 鏈式「上妝」。此即 SwiftUI 全部之基本盤也。

二、基礎視圖：文字、按鈕、輸入框、圖

視圖	何為	示例
Text	顯示文字	Text(「你好」)
Button	按鈕	Button(「點我」) { 動作}
TextField	輸入框	TextField(「提示」, text: $文本)
Image	顯示圖片	`Image(systemName: "star.fill")`

SwiftUI 四大基礎視圖

注意 Image(systemName:)——蘋果內置 SF Symbols 圖標之庫（免費圖標數千），「star.fill」此等之名即可徑示一實心之星。此 iOS 開發隱藏之福利：作界面幾乎不必覓圖標之素材。

三、佈局三件套：VStack、HStack、ZStack

- VStack：縱而列之（若列隊取餐）；
- HStack：橫而列之（若並肩而坐）；
- ZStack：疊而置之（若撲克之牌相壘，宜作角標、遮罩）。

```
VStack(alignment: .leading, spacing: 12) {
    HStack {
        Text("賬單")
        Spacer()                    // 彈性之空：推右至盡
        Text("-25.5 元").foregroundColor(.red)
    }
    Divider()                       // 分割線
    Text("備註：午餐")
}
.padding(16)                        // 四周留白
.background(Color(.systemGray6)) // 淺灰底
.cornerRadius(12)                   // 圓角

// 修飾符之序有義：先 padding 後 background，背景不含邊距
```

按：修飾符次序有講究

padding(16).background(灰) 與 background(灰).padding(16)，其效迥異——前者先留白而後塗色（背景不含邊距），後者先塗色而後留白（背景兼含邊距）。新手所見怪異之白邊，多源於此。倘有疑，互易其序，預覽立示其答。

四、狀態與綁定：使界面「活」起來

靜態界面殊無意味。SwiftUI 之規則：界面恆忠實反映狀態，狀態一變，界面自重繪焉。

```
struct CounterView: View {
    @State private var count = 0   // @State：此值變則重畫

    var body: some View {
        VStack {
            Text("汝點了 \(count) 次")
            Button("點我 +1") {
                count += 1          // 改狀態，界面自動刷新
            }
        }
    }
}
```

輸入框須用「雙向綁定」——$ 符號將「值與修改之權」一併傳入：

```
struct InputView: View {
    @State private var text = ""

    var body: some View {
        TextField("輸入若干", text: $text)  // $ 雙向綁定
            .textFieldStyle(.roundedBorder)
        Text("汝輸入了：\(text)")   // 輸入則實時更新
    }
}
```

用戶操作

狀態改變（count += 1）

SwiftUI 感知到變化

自動重畫界面（body 重新計算）

圖 3：SwiftUI「狀態界面」單向數據流

五、預覽、主題與動畫：SwiftUI 之「外掛」

- #Preview 宏：於檔案之末加 #Preview { ContentView() }，右側畫布即時渲染——改碼所見即所得；
- 深色模式免費：用系統顏色（Color(.systemGray6)、.primary）而非寫死之色，深色模式自能相配，不多書一行；
- 動畫一行可成：withAnimation { count += 1 }，或予視圖加.animation(.default, value: count)——過渡絲滑。

```
struct AnimatedHeartView: View {
    @State private var liked = false

    var body: some View {
        Button {
            withAnimation(.spring(duration: 0.4)) {
                liked.toggle()
            }
        } label: {
            Image(systemName: liked ? "heart.fill" : "heart")
                .font(.system(size: 48))
                .foregroundColor(liked ? .red : .gray)
                .scaleEffect(liked ? 1.3 : 1.0)
        }
    }
}
```

小貼士：先功能後動畫

新手常病：界面未正，而先糾結於動畫。宜先令功能通達，終乃一體加諸動效——動畫者錦上添花也，功能者雪中送炭也。此亦吾儕一貫「簡單至上」之旨。

來動手：做一「心情按鈕」

六、常用修飾符速查：一頁在手，界面無憂

欲作何事	修飾符
留白	`.padding(16)`
圓角	`.cornerRadius(12)`
背景色	`.background(Color(.systemGray6))`
字體與字號	`.font(.title.bold())`/`.font(.system(size: 20))`
前景色（文字/圖標）	`.foregroundColor(.red)`
撐滿寬度	`.frame(maxWidth: .infinity)`
固定尺寸	`.frame(width: 100, height: 44)`
對齊之法	`VStack(alignment: .leading)`
透明度	`.opacity(0.6)`
陰影	`.shadow(radius: 5)`
點擊手勢	`.onTapGesture { ...}`
鍵盤類型	`.keyboardType(.decimalPad)`

SwiftUI 常用修飾符速查

記之不住亦無妨——此頁正為查檢而設。用之最多者，實惟前四：padding、圓角、背景、字體。

改模板之項為心情輪換器：一按鈕，點之則於開心難過生氣之間循環遞換，文字與背景之色隨之而變。提示：以陣列貯三組（文字 + 顏色），@State 貯其下標，點時用 (index + 1) % 3。成則示諸友朋，可得第一聲「哇，汝能作 App 矣」之歎。

本章小結

- SwiftUI 為聲明式：述其「所長之形」，系統任描繪重繪之責
- 視圖 = 遵 View 協議之結構體；body 述界面；修飾符連綴如「上妝」
- 四大基礎視圖：Text / Button / TextField / Image；SF Symbols 圖標庫免費而用之不竭
- 佈局三件：VStack 豎列、HStack 橫列、ZStack 疊置；Spacer 者彈性之空也
- @State 使界面活起來；$ 雙向綁定以饋輸入框
- #Preview 所見即所得；系統顏色自配深色模式；withAnimation 一行成動效

第六十三章：視圖、狀態與導航

單頁之玩物既已能為。此章解二「成長」之惑：如何於眾頁之間往來跳轉（導航），及數據如何在頁間流動（狀態管理）。學畢此章，則能搭「列表詳情編輯」經典多頁 *App* 之骨架矣。

一、狀態管理之「數據流全景」

SwiftUI 狀態之具，非惟 @State 一端。先觀此全景之圖，明「何時用何者」：

工具	司何事	孰用之	典型場景
@State	頁面私有之暫態	當前視圖	計數器、輸入框文字
@Binding	借他人之狀態	子視圖改父視圖	自定義組件
@StateObject / @Observable	跨頁共用之邏輯	視圖模型（ViewModel）	數據載入、業務邏輯
@EnvironmentObject / .environment	全局共用之狀態	整個 App	用戶登入之態、主題設置

SwiftUI 狀態之具全景（2026 年，@Observable 已為主流）

```
// iOS 17+ 之新法：@Observable 宏（新項目宜用）
import Observation

@Observable
class CounterModel {                    // 視圖模型
    var count = 0
    func increment() { count += 1 }
}

struct CounterView: View {
    @State private var model = CounterModel()   // 持有模型

    var body: some View {
        VStack {
            Text("汝點了 \(model.count) 次")
            Button("點我") { model.increment() }
        }
```

```
    }
}
```

予新手直截之勸：頁面私有者用 @State，跨頁或有邏輯者用 @Observable 類。舊籍之 @StateObject/@ObservableObject 乃前代寫法，能讀懂即可，新碼悉遵上式。

二、NavigationStack：頁面跳轉之「棧」

iOS 16 以降，多頁導航用 NavigationStack——若一疊盤，推一入一，返則逐一彈出：

```
struct HomeView: View {
    var body: some View {
        NavigationStack {
            List {
                NavigationLink("張三之賬本") {
                    DetailView(name: "張三之賬本")   // 目標頁面
                }
                NavigationLink("李四之賬本") {
                    DetailView(name: "李四之賬本")
                }
            }
            .navigationTitle("賬本列表")   // 頂部大標題
        }
    }
}

struct DetailView: View {
    let name: String
    var body: some View {
        Text("\(name) 之詳情頁")
            .navigationTitle(name)
            .navigationBarTitleDisplayMode(.inline)
    }
}
```

NavigationLink 即「可點之跳轉連結」；navigationTitle 自生導航欄之題與返鈕。欲程式化跳轉（如點「保存」而自返）者，用 @Environment(\.dismiss) 以闔當前之頁——二法相濟，九成導航之需備矣。

三、彈窗與底部標籤：sheet 與 TabView

除「推頁」外，尚有二種經典之交互：

```
// 1. 底部彈出半屏/全屏（錄入表單最愛）
struct HomeView: View {
    @State private var showAdd = false
```

```
    var body: some View {
        Button("記一筆") { showAdd = true }
            .sheet(isPresented: $showAdd) {
                AddRecordView()      // 彈出之錄入頁
            }
    }
}

// 2. 底部標籤欄（App 之"骨架"）
struct MainTabView: View {
    var body: some View {
        TabView {
            RecordListView()
                .tabItem { Label("賬本", systemImage: "book") }
            StatsView()
                .tabItem { Label("統計", systemImage: "chart.pie") }
            SettingsView()
                .tabItem { Label("設置", systemImage: "gear") }
        }
    }
}
```

小貼士：何時用何種？

經驗之法：主流程以 NavigationStack 推頁（列表詳情），暫時之操作以 sheet 彈窗（新增、編輯、確認），並列數大塊以 TabView（底部標籤）。三者相合，即絕大多數 App 之骨架。

四、List：iOS 之「國民列表」

iOS 最常用而最具「蘋果味」之組件即 List——天生帶分割線、滑動而刪、點而高亮：

```
struct RecordListView: View {
    @State private var records = ["午餐 25.5", "地鐵 6.0", "薪俸 +8000"]

    var body: some View {
        List {
            ForEach(records, id: \.self) { r in
                Text(r)
            }
            .onDelete { indexSet in          // 左滑而刪
                records.remove(atOffsets: indexSet)
            }
        }
    }
}
```

列表 + NavigationLink + sheet + TabView——合此四者，汝已能搭「賬本列表點開詳情點 + 彈錄入」完整 App 之骨架矣。次章記賬本之項，即如是搭成。

來動手：搭一「三頁骨架」

五、程序生命之期：前台與後台

與安卓 Activity 生命週期相類，iOS App 亦有「生、前台、後台、亡」狀態之變。SwiftUI 中以 scenePhase 環境值察之：

```
@main
struct MyApp: App {
    @Environment(\.scenePhase) private var phase

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .onChange(of: phase) { _, newPhase in
            switch newPhase {
            case .active:    print("返前臺，復刷新")
            case .inactive:  print("過渡中（來電/切後臺瞬間）")
            case .background: print("入後臺，存草稿")
            @unknown default: break
            }
        }
    }
}
```

典型之用：返前台則刷數據，入後台則存草稿。iOS 於後台之運行管束甚嚴——常 App 入後台不久即被懸置（凍結），勿冀於後台潛作重務；誠欲定時之務，用第 68 章本地通知，而非「後台常駐」。

作一假數據版記賬本之骨架：TabView 三標籤（賬本/統計/設置）賬本頁以 List 陳假數據、點行跳詳情（NavigationStack）右上「+」鈕彈錄入 sheet。不貯數據、不連網，純練骨架——骨架既成，第 65 章填以真數，如水到而渠成。

本章小結

- 狀態四具：@State（頁私有）、@Binding（借用）、@Observable（邏輯）、.environment（全局）
- 新項以 @Observable 宏作視圖模型；舊籍之 @ObservableObject 能讀即可
- NavigationStack 推頁；@Environment(\.dismiss) 程式化而返
- 主流程推頁、暫事彈 sheet、並列大塊用 TabView
- List 者「國民列表」也：分割線、滑動而刪，啟箱即用

第六十四章：數據存貯與網絡之請

惟「觀」而不能他為之 *App*，死物也；能存數據、能連網絡者，方為活物。此章識 *iOS* 數據層之「三件」：*UserDefaults*（貯小配置）、*SwiftData*（貯結構化數據）、*URLSession*（與伺服器言），外加解 *JSON* 之 *Codable* 與現代之 *async/await*。學畢此章，則能作能記用戶之習、能實時更其內容之真 *App* 矣。

一、存貯之方何以擇：先觀此表

方案	宜貯何物	特點	本書何處用之
UserDefaults	零散配置（開關、偏好）	鍵值之對，簡而直	第 65 章設置、第 66 章所藏之城
SwiftData	結構化數據（賬目、記錄）	官方數據庫之封裝，@Model 一行而聲明	第 65 章賬目之記錄
URLSession	（不貯，連網而取）伺服器數據	系統自帶網絡之框，無第三方之倚	第 66 章天氣數據

數據三件之分工

記一口訣：「小配置入 UserDefaults，成批數據入 SwiftData，外來數據行 URLSession」。舊籍之 Core Data 猶可用（SwiftData 即其今之馬甲），手書 SQLite 則大抵不須觸——徑學其新，勿履舊轍。

二、UserDefaults：貯些小配置

```
// 寫：存一個值
UserDefaults.standard.set(true, forKey: "isDarkMode")
UserDefaults.standard.set("北京", forKey: "favoriteCity")

// 讀：取之（無則返默認值）
let isDark = UserDefaults.standard.bool(forKey: "isDarkMode")
let city = UserDefaults.standard.string(forKey: "favoriteCity") ?? "北京"

// 刪：移除一個鍵
UserDefaults.standard.removeObject(forKey: "isDarkMode")
```

惟其僅宜零散之小配置——數十上百之記錄請付 SwiftData，勿以 UserDefaults 為數據庫而用（將緩啟動、且不便查詢）。

三、SwiftData：為數據安「真家」

SwiftData 者，iOS 17 以降官方數據庫之方也，使「建表、增刪改查」簡至其極——一 @Model 之註，數據類遂為數據庫之表：

```
import SwiftData

// 1. @Model：此類自化為"表"
@Model
class Record {
    var amount: Double
    var category: String
    var note: String
    var date: Date

    init(amount: Double, category: String, note: String, date: Date) {
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}

// 2. App 入口掛容器
@main
struct MyApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .modelContainer(for: Record.self)   // 數據庫就緒
    }
}

// 3. 視圖中用 @Query 讀、@Environment 寫
struct RecordListView: View {
    @Query(sort: \Record.date, order: .reverse) var records: [Record]
    @Environment(\.modelContext) private var context

    var body: some View {
        List(records) { r in
            Text("\(r.category) \(r.amount) 元")
        }
```

```
    }

    func add(amount: Double, category: String) {
        let r = Record(amount: amount, category: category, note: "", date: Date())
        context.insert(r)     // 存
        try? context.save()   // 落盤
    }
}
```

此「@Model + modelContainer + @Query」之模板，可原式移入汝任何之項——易其字段之名即可。數據一變，@Query 自刷界面，妙不可言。

四、Codable：為 JSON「譯」

伺服器之數多為 JSON 之文。Swift 之 Codable 協議，使「JSON 結構體」之轉換自發——字段名相合，魔法即效：

```
// JSON: {"name":"北京","main":{"temp":26.5},"weather":[{"description":"晴"}]}
struct WeatherResponse: Codable {
    let name: String
    let main: Main
    let weather: [Weather]
}
struct Main: Codable { let temp: Double }
struct Weather: Codable { let description: String }

// JSON 字串  結構體
let decoder = JSONDecoder()
let data = jsonString.data(using: .utf8)!
let result = try decoder.decode(WeatherResponse.self, from: data)
print(result.weather[0].description)   // "晴"
```

鐵律：先觀實返之 JSON，後定字段。多「解析不遂」之 bug，皆字段名書誤、類型猜誤（伺服器發“26.5”之字串，汝聲為 Double）——印原 JSON 相對照，十秒可定。

五、URLSession + async/await：與伺服器言

iOS 連網用系統自帶之 URLSession，合 Swift 之 async/await（異步之函），「候網」如「候速遞」之自然：

```
struct WeatherService {
    // 異步函數：async 標記"我要等一會兒"，throws 標記"可能失敗"
    func fetchWeather(city: String) async throws -> WeatherResponse {
        let url = URL(string:
            "https://api.example.com/weather?city=\(city)")!
        let (data, _) = try await URLSession.shared.data(from: url)
```

```
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}

// 調用之處（於 .task 中）：
.task {
    do {
        let weather = try await service.fetchWeather(city: "北京")
        // 更新界面……
    } catch {
        print("有誤：\(error)")
    }
}
```

避坑之警：網絡二鐵律

(1) 連網必 async/await，絕不阻主線程——界面滯塞，崩潰級之驗也。Swift 結構化之並發使之自然，然勿忘 try await 與 do/catch。(2) ATS 默禁明文 HTTP：iOS 責以 HTTPS。試本地 http:// 之服時，須於 Info.plist 暫開例外（NSAppTransportSecurity），然正式之品必全行 HTTPS——審核亦監此。

來動手：作一「每日一句」小應用

六、網絡之誤亦宜「分級」：勿以凡咎盡擲於用戶

斷網、逾時、404、伺服器 500……咎各不同，用戶所當見之言亦異：

```
func friendlyMessage(_ error: Error) -> String {
    if let urlError = error as? URLError {
        switch urlError.code {
        case .notConnectedToInternet: return "網絡斷矣，試查連接？"
        case .timedOut:               return "請求逾時，稍後再試？"
        case .cannotFindHost:         return "尋不見服務器"
        default:                      return "網絡有疾：\(urlError.code)"
        }
    }
    if let d = error as? DecodingError {
        return "數據之式有誤（或接口已變）"
    }
    return "有誤：\(error.localizedDescription)"
}
```

予用戶以「人言」，予日誌以「原言」（print 完整 error）。勿以 error.localizedDescription 之英文天書直擲於用戶——產品之感，正藏於此等細節。

覓一免費名言之 API（如 hitokoto 之屬），以 URLSession + async/await 取「每日一句」，陳於屏心，加「換一句」之鈕。復以 SwiftData 貯所愛之句於「收藏之夾」。此習遍用 UserDefaults/SwiftData/Codable/URLSession——成之，則第 7、8 章直小菜一碟耳。

本章小結

- 口訣：小配置入 UserDefaults，成批數據入 SwiftData，外來數據行 URLSession
- SwiftData 三斧：@Model 聲其表、modelContainer 掛其器、@Query 讀而 context.insert 寫
- Codable 自「譯」JSON：字段相合，魔法即效
- URLSession + async/await：連網如候速遞；do/catch 處其不遂
- 二鐵律：不阻主線程；正式之品必 HTTPS（ATS）

第六十五章：項目一——吾之記賬本（本地版）

迎汝入首個「正經之項」。此記賬本雖不連網，然五臟悉備：能記一筆之賬、能觀流水、能按月統計、能刪改。成之，則「數據邏輯界面」之全流程已遍歷矣。勿遽抄代碼——先思之審而後動手，此吾儕一貫之則也。

一、思其所需：先問五事

要素	答案
Who（為誰而作）	自用，家人朋友亦可
What（作何事）	記收支、觀流水、按月彙總、按類統計
Why（何為）	練手 + 明錢之所往
How（何以為之）	Swift + SwiftUI + SwiftData + MVVM，純本地而無後端
Constraint（約束）	一週內成之；最低須支 iOS 17（SwiftData 之檻）

記賬本所需五要素

繼而析為「步步可用」之小程：(1) 建 SwiftData 數據層（能貯一賬）；(2) 列表頁（能觀流水）；(3) 錄入彈窗（能增賬）；(4) 刪與編；(5) 月度彙總之卡。每成一程，App 皆「可用之物」也。

二、數據層：使「賬」有其所

先定一賬之形——字段足用即可，勿貪多：

```
import SwiftData

@Model
class Record {
    var type: String      // "income" 收入 / "expense" 支出
    var amount: Double    // 金額（元）
```

```
    var category: String  // 分類：餐飲、交通、薪俸……
    var note: String      // 備註（可選）
    var date: Date        // 記賬日期

    init(type: String, amount: Double, category: String, note: String = "", date:
    Date = Date()) {
        self.type = type
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}
```

小貼士：金額何以用 Double？

嚴謹之金融 App 用 Decimal（精確十進制）以避浮點之差（0.1+0.2≠0.3 之舊惑）。練手之項用 Double 足矣；他日若作真金融之品，宜易為 Decimal 或「以分為單位之 Int」——此無數先輩所踐之坑也。

三、MVVM：界面與邏輯「分家」

徑使界面操數據庫乎？可行，然界面之碼將冗長可憎。MVVM 之構使各司其職：View（界面）惟司繪，ViewModel（邏輯之樞）司數據，Model（SwiftData）司存貯。

View（SwiftUI 界面）

ViewModel（@Observable 邏輯中樞）

Model（SwiftData 數據層）

圖 4：MVVM 三層之構——數據單向而流，各層互不越界

```
import Observation

@Observable
class RecordViewModel {
    var records: [Record] = []
    private let context: ModelContext

    init(context: ModelContext) { self.context = context }

    func load() {
        // SwiftData 之 #Predicate 篩選 + 排序
        let descriptor = FetchDescriptor<Record>(
            sortBy: [SortDescriptor(\.date, order: .reverse)]
        )
        records = (try? context.fetch(descriptor)) ?? []
```

```
    }

    func add(type: String, amount: Double, category: String, note: String) {
        let r = Record(type: type, amount: amount, category: category, note: note)
        context.insert(r)
        try? context.save()
        load()                          // 重載，界面自新
    }

    func delete(_ record: Record) {
        context.delete(record)
        try? context.save()
        load()
    }
}
```

界面中 @State private var vm = RecordViewModel(...)，遂徑以 vm.records 渲列表——數據一變，界面自刷。

四、界面：列表頁 + 錄入彈窗

```
struct RecordListView: View {
    @State private var vm: RecordViewModel
    @State private var showAdd = false

    var body: some View {
        NavigationStack {
            List {
                Section("本月結餘") {           // 匯總卡片
                    SummaryRow(records: vm.records)
                }
                Section("流水") {
                    ForEach(vm.records) { r in
                        RecordRow(record: r)
                    }
                    .onDelete { idx in
                        idx.forEach { vm.delete(vm.records[$0]) }
                    }
                }
            }
            .navigationTitle("吾之記賬本")
            .toolbar {
                Button { showAdd = true } label: {
                    Image(systemName: "plus")
                }
            }
```

```
            .sheet(isPresented: $showAdd) {
                AddRecordView(vm: vm)          // 錄入彈窗
            }
        }
    }
}
```

錄入彈窗之要：金額輸入框之鍵盤用.keyboardType(.decimalPad)（數字鍵盤），收入/支出用分段之控件（Picker），分類以菜單擇之，點「保存」則調 vm.add(...) 並 dismiss() 以闔窗。

按：輸入之驗勿懈

金額輸入框必驗之：空值、負數、“abc”之類非法輸入皆當攔（以 Double(text) 審之，不遂則返 nil）。App 之崩，其常因之一，即「視用戶之輸為馴良」。謹記：凡用戶所輸，皆當如敵而察之。

五、編、刪與彙總

- 刪：List 之.onDelete 左滑而刪 + 確認彈窗（誤刪者，記賬類 App 用戶之所深憾也）；
- 編：點行入編輯彈窗，預填舊值（易一字較重輸為省）；
- 彙總：以 filter + reduce 算「本月收入 / 支出 / 結餘」，頂部之卡陳之：

```
func monthSummary(_ records: [Record]) -> (income: Double, expense: Double) {
    let cal = Calendar.current
    let monthRecords = records.filter {
        cal.isDate($0.date, equalTo: Date(), toGranularity: .month)
    }
    let income = monthRecords
        .filter { $0.type == "income" }
        .reduce(0) { $0 + $1.amount }
    let expense = monthRecords
        .filter { $0.type == "expense" }
        .reduce(0) { $0 + $1.amount }
    return (income, expense)
}
```

六、驗收：三層之檢 + 二進階之挑戰

項既成，按「功能邊界構架」三層驗之：能記否、能刪否、重啟 App 而數據存否（功能）；金額輸 0 或負數，崩否（邊界）；界面與數據有無相混、字段有無硬編（構架）。

基礎版驗過，則自加二道「進階之饌」：

1. 分類統計：增「統計」標籤頁，以 Swift Charts 繪分類支出為餅圖/柱狀圖——蘋果官方圖表之庫，數行碼而圖成；

2. iCloud 同步：SwiftData 開 iCloud 同步，惟須配數項設置——賬本於 iPhone/iPad 之間自同步，頓覺「高級」矣。

來動手：本週成之

按「數據層 ViewModel 列表頁錄入彈窗彙總卡片」之序而動手，日進一程。困則以報錯原文擲於搜索引擎或 AI（尚憶第七篇《AI 編程實戰篇》乎？述問題時，明言「期望之行、實際之行、復現之步」）。成則以之記賬一週——自作之 App 自用之，成就之感將推汝前行。

本章小結

- 所需五要素析為「步步能用」之小程，而後動手
- 數據層：@Model 之模 + modelContainer 之器 + FetchDescriptor 之詢
- MVVM 分職：View 惟司繪，@Observable 之 ViewModel 司邏輯，SwiftData 司存貯
- 界面：List + Section + toolbar + sheet 錄入彈窗
- 三層驗收（功能/邊界/構架）+ 進階挑戰（Swift Charts、iCloud 同步）

第六十六章：項目二——天氣查詢 App（網絡版）

記賬本者「練內功」也，天氣 *App* 者「廣見聞」也——將與互聯網相周旋。此項汝將學：請免費之 *API*、解 *JSON*、處「載入中/成/敗」三種界面之態，及藏 *API* 密鑰之法。成之，則作任何「連網 *App*」之能具矣——新聞、匯率、翻譯，其術如一。

一、請免費天氣之 API

個人練手，薦 OpenWeatherMap 或國內之和風天氣（QWeather）——皆有免費之額，註冊即用。以 OpenWeatherMap 為例：

1. 註冊賬號於 API keys 頁生一 Key（apikey）；
2. 免費版可調 Current Weather 之接口：api.openweathermap.org/data/2.5/weather?q=beijing&appid= 汝之 Key&units=metric；
3. 先於瀏覽器一試，觀其返 JSON 之狀——此接任何 API 前必為之「預習」。

避坑之警：密鑰絕不可寫死於碼中

API Key 猶汝「錢囊之密碼」，一入碼而提交於 GitHub，數時之內即為爬蟲所掃而盜刷。iOS 之正法：納於 Config.xcconfig（配置之檔，已為 gitignore 所忽），由 Info.plist 讀入碼中。搜「iOS xcconfig 管理密鑰」依而改之即可。

二、網絡層：URLSession + Codable

先依 API 所返之 JSON 定其模，後書其服：

```
// JSON: {"weather":[{"description":"晴"}],"main":{"temp":26.5,"humidity":40},"name":"北京"}
struct WeatherResponse: Codable {
    let weather: [Weather]
    let main: Main
    let name: String
}
```

```
struct Weather: Codable { let description: String }
struct Main: Codable { let temp: Double; let humidity: Int }

struct WeatherService {
    let key: String   // 從 xcconfig 讀入

    func fetchWeather(city: String) async throws -> WeatherResponse {
        var comps = URLComponents(string: "https://api.openweathermap.org/data/2.5/
    weather")!
        comps.queryItems = [
            URLQueryItem(name: "q", value: city),
            URLQueryItem(name: "appid", value: key),
            URLQueryItem(name: "units", value: "metric"),
            URLQueryItem(name: "lang", value: "zh_cn")
        ]
        let (data, _) = try await URLSession.shared.data(from: comps.url!)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}
```

三、界面三態：載入中 / 成 / 敗

連網 App 之魂在「狀態管理」。網緩、斷網、城名誤書……凡此諸況，界面皆當予用戶以交代。以枚舉 + switch 雅而表之：

```
enum LoadState {
    case loading                  // 旋轉中
    case success(WeatherResponse) // 成功
    case error(String)            // 失敗
}

@Observable
class WeatherViewModel {
    var state: LoadState = .loading
    private let service = WeatherService(key: Config.apiKey)

    func search(city: String) async {
        state = .loading
        do {
            let data = try await service.fetchWeather(city: city)
            state = .success(data)
        } catch {
            state = .error("查無此城，易其名再試之？")
        }
    }
}
```

```
// 界面中：
switch vm.state {
case .loading: ProgressView("載入中…")
case .success(let d): WeatherContent(data: d)
case .error(let msg): Text(msg).foregroundColor(.secondary)
}
```

此 loading/success/error 三段式者，連網界面之「萬金油模板」也，先誦之，後凡連網 App 皆可依樣畫瓢。

四、實戰界面之碼：繪天氣而出

```
struct WeatherContent: View {
    let data: WeatherResponse

    var body: some View {
        VStack(spacing: 12) {
            Text(data.name)
                .font(.title.bold())
            Text("\(Int(data.main.temp))°C")
                .font(.system(size: 76, weight: .thin))
            Text(data.weather.first?.description ?? "未知")
                .font(.title3)
                .foregroundColor(.secondary)
            HStack(spacing: 24) {
                Label("溼度 \(data.main.humidity)%", systemImage: "humidity")
                Label("體感 \(Int(data.main.temp))°C", systemImage: "thermometer")
            }
        }
        .padding()
    }
}
```

數細節可學：first? 安全取首元（列表或空），?? “未知”以為備底之文；76 號細字重之溫度數字，天氣類 App 之經典配方也；Label 者「圖標 + 文字」官方之組件，較手拼 HStack 更為規範。

五、交互之細與所藏之城

- 搜索之框：頂部.searchable 修飾符，系統自帶搜索之交互，提交時調 vm.search；
- 藏城：以常觀之城貯於 UserDefaults，下次啟則自動查之；
- 下拉而刷：.refreshable { await vm.search(...) } 一行而成；

- 天氣之圖標：以 Image(systemName:) 依所述而映（sun.max / cloud.rain / snowflake），界面立活。

小貼士：中文城名之「梗」

OpenWeatherMap 於中文城名之助平平，搜「北京」或無所得而“beijing”自若。產品化之法，乃內置「城名英文/坐標」映射之表，或接國內天氣之服。練手之期，先支拼音/英文城名即可，勿與之較。

來動手：天氣 App 驗收之單

六、網絡之請何以調試：印之、抓包與擬緩網

連網 App 有疾，十居八九在「請未發正」或「應未解正」。調試之術三：

1. 印原始數據：以 String(data: data, encoding: .utf8) 印而觀原 JSON——字段名、類型一目了然，較猜速十倍；
2. 抓包之具：Mac 上免費之 Proxyman / 開源之 Charles 能截 App 諸請與應，如 X 光之透視網絡層；
3. 擬弱網：擬器菜單 Features Network Link Conditioner（須自開發者之站載具），能擬 3G、高遲滯——以驗「旋轉半晌」之疾。

小貼士：請未過 ATS 乎？

若請甫發即報“App Transport Security”之誤，什九為用 http:// 明文之址所致。正式之接口必 HTTPS；本地調試暫開例外（Info.plist 增 NSAllowsLocalNetworking），上架前務滌除之。

(1) 常搜而得天氣；(2) 斷網，App 雅報其誤而不崩（於擬器中關 Mac 之網以試之！）；(3) 輸不存之城，提示宜人；(4) 藏二城，重啟 App 猶在；(5) 深色模式下界面不刺目。五者悉過，此項方為卒業。

本章小結

- 先於瀏覽器試 API、讀懂 JSON，而後書碼——「預習」終無所虧
- API Key 藏於 xcconfig，絕不入碼庫
- Codable 之模 + URLSession async/await：字段相合，JSON 自解
- loading/success/error 三段之態，連網界面之萬金油也
- 斷網、空輸、城之不存——誤之處置，連網 App 及格之線也

第六十七章：琢磨與發布——測試、調試與上架 App Store

App 既成，繼之何為？此章答「自能行」至「能交付」之三事：何以高效調試而覓 *bug*、何以書測試以防回歸、何以獻 *App* 於 *App Store* 而使天下載之。上架非徒儀式——乃使作品化為產品之分水嶺也。

一、調試三斧：print、斷點、預覽

工具	司何事	何以用之
print + 控制臺	觀運行之日誌	碼中 print(" 值 =\(x)")，Xcode 底部控制臺觀其出
斷點調試	使程序止於某行，觀變量之值	行號旁一點而出藍標，行至彼而暫止，逐步歷之
SwiftUI 預覽 + 調試器	實時觀界面與視圖之層級	#Preview 畫布 + 運行時之 View Hierarchy 檢查器

調試三斧

小貼士：崩而何以讀之？

Swift 崩時，控制臺印“Fatal error”及其因，Xcode 止於出誤之行而高亮。真機之舊崩，可於 Xcode Window Organizer Crashes 觀之（須付費之賬）。關鍵之詞，先搜「誤因首句 + swiftui」——問精則答精。

二、測試：為碼投保

手測不足恃——改此一能，彼一能潛壞。測試者，悔之藥也，兼防彈之衣。iOS 用 XCTest 之框，分二種：

層級	測何物	Target	速度
單元測試	純邏輯（計算、校驗）	MyFirstAppTests	秒級
UI 測試	真界面之行	MyFirstAppUITests	分鐘級
手動測試	真人之感、體驗	——	恆所需也

iOS 測試層級（模板之項已預置二 Test Target）

```
import XCTest

// 將"金額字符串轉數字"此種邏輯抽成純函數
func parseAmount(_ input: String) -> Double? {
    guard let v = Double(input.trimmingCharacters(in: .whitespaces)),
          v > 0 else { return nil }
    return v
}

final class AmountTests: XCTestCase {
    func testEmptyInputReturnsNil() {
        XCTAssertNil(parseAmount(""))
    }
    func testNegativeReturnsNil() {
        XCTAssertNil(parseAmount("-5"))
    }
    func testNormalValueParses() {
        XCTAssertEqual(parseAmount("12.5"), 12.5)
    }
}
```

行測試：CmdU。既全綠而後改碼，心中乃有據。記賬本中金額之驗、彙總之算，皆當有單元測試護之。

三、署名與打包：為 App 鈐「印」

凡 iOS App 必署名而後可安——以證「此包出於汝，未遭竄改」。層級之係：

- 學習之期：免費 Apple ID + Xcode 自動署名（Personal Team），真機可行，七日一續；
- 分發之期：付費開發者之賬（99 美元/年）簽發證書與描述之檔，而後可上架與久安。

打包：Xcode 菜單 Product Archive（歸檔），構建既成，於 Organizer 中 Distribute App 擇 App Store Connect 上傳。歸檔必以 Generic iOS Device 為的（非擬器），此新手所常忘之一程也。

避坑之警：證書亡則更新絕

署名之證與私鑰一亡，則舊 App 不得復更新——用戶惟可安「全新」之應用。善備證書（鑰匙串導出.p12）而謹記其密碼。幸蘋果有「自動署名」與雲證書之管（Xcode 中勾 Automatically manage signing），能省手工諸坑之大半。

四、上架 App Store 之全程

註冊開發者賬號

準備商店素材
上傳構建包
填隱私與審覈信息
蘋果審覈（1-7 天）
上架

圖 5：App Store 上架六步

- 註冊：developer.apple.com，個人賬號 99 美元/年；
- 素材：圖標（1024×1024，勿帶圓角——蘋果自裁之）、截圖（iPhone 各尺寸）、描述、隱私政策之連結；
- 隱私之問卷：如實申報所收何數據——不連網之記賬本可「不收任何數據」；
- 審核：首提常候一至七日，見拒則依「審核之註」逐條改之——蘋果拒之由，常書之甚詳，循而行之即可；
- TestFlight：上架之先，以測試版予友內測（至多 10000 人），集一輪之復而後正提，專業之範十足。

五、商店素材何以為之：第一印象即下載之量

商店之頁，用戶「逛店」所見之櫥窗也。三素材定第一印象：

- 圖標：1024×1024，勿以截圖苟且。無設計之能，則以 AI 之具生一簡潔之形，於桌面縮之而猶能一望而識，方為合格；
- 截圖：至少二套（iPhone 主流之尺寸），陳其最核之功能之景；每張配一句白話之文（「一秒記一筆」較「高效記賬之方」善十倍）；
- 描述：首句當明「此何物 + 為誰而作」，勿堆「最專、最強」——誠較誇為效。

來動手：以記賬本「交付」一過

六、崩潰之上報與性能優化小單

App 上架之先，此二事宜各費三十分鐘：

崩潰監控

汝測時不崩，非謂用戶之機不崩。蘋果於 Xcode Organizer 中予免費之崩報（付費之賬可觀真機崩潰之日誌）；更省心者，接第三方 SDK（如 Firebase Crashlytics）。無崩監而上架，猶閉目而御車也。

性能自察四條

1. 列表用 LazyVStack / List：長列表勿以常 VStack 硬排——懶載者，性能第一課也；
2. 圖勿硬塞原圖：用 AsyncImage（遠圖）與系統之緩存，5MB 原圖足以滯壞舊機；
3. 勿於主線程任重務：網絡、大檔皆行 async/await；
4. 以 Instruments 量之：Xcode 自帶性能剖析之器（Time Profiler、記憶體洩漏之檢）——先量而後優，勿憑感覺。

(1) 為 parseAmount 等邏輯補三單元測試（CmdU 行之而綠）；(2) 以斷點親捕自造之崩而修之；(3) Product Archive 打一包，由 TestFlight 邀友內測；(4) 有餘力則註冊開發者之賬，以記賬本真提而上架。前三程今夕可成。

本章小結

- 調試三斧：print 之誌、斷點、SwiftUI 預覽與視圖檢查器
- XCTest：單元測試（CmdU）+ UI 測試 + 手動測試，模板已預置
- 署名：免費賬號自動署名（七日一續）；付費賬號方能分發
- 打包：Generic iOS Device + Archive + Distribute；證書務必備份
- 上架六步：註冊素材上傳隱私問卷審核上架；TestFlight 先內測

第六十八章：進階之途——權限、通知與現代構架

能作 *App*，特其始耳。此章談自「能作」至「作善」之數關：何以雅而請權限、何以發通知而不厭於人、何以舉構架再上一層，及——*2026* 年，何以與 *AI* 相偶而書 *iOS* 之碼。凡此皆汝於真項中即將遇者。

一、權限：先叩門，後入門

相機、定位、通訊錄、通知……此等「敏感之域」，iOS 責 App 於用時向用戶彈窗而請。其規為「聲明 + 請求 + 雅而降級」三步：

```
// 1. Info.plist 中聲明用途之文（不聲明則立崩！）
// Privacy - Location When In Use Usage Description: "以查汝所於之城天氣"

// 2. 用時申請
import CoreLocation

@Observable
class LocationModel {
    private let manager = CLLocationManager()

    func request() {
        switch manager.authorizationStatus {
        case .notDetermined: manager.requestWhenInUseAuthorization()
        case .denied, .restricted: /* 引導至設定中開之 */
        default: /* 已授權，可行矣 */
        }
    }
}

// 3. 被拒則優雅降級：予提示 + 跳設定
// UIApplication.shared.open(URL(string: UIApplication.openSettingsURLString)!)
```

按：iOS 權限之「一次性」

iOS 權限之窗惟彈一過——用戶拒之則不復彈。故請之時，當擇「用戶方欲用之頃」（非一啟而彈一堆），文當明其用。見拒後界面之引（「往設置中啟定位」）乃標配，勿使用戶困於窮巷。

二、通知：於當時而言有用之言

通知者，App 達用戶之途也，亦最易為用戶所「拒」之能。iOS 用 UserNotifications 之框，發通知有三戒：惟發有用者、使戶能關、勿半夜擾之。其基法：

```
import UserNotifications

// 1. 申請通知之權（彈窗一次）
UNUserNotificationCenter.current()
    .requestAuthorization(options: [.alert, .sound, .badge]) { granted, _ in
        print(granted ? "已授權" : "被拒絕")
    }

// 2. 安排一條"每日 20:00 記賬之提醒"
func scheduleDailyReminder() {
    let content = UNMutableNotificationContent()
    content.title = "記賬提醒"
    content.body = "今日尚未記賬也"
    content.sound = .default

    var comps = DateComponents()
    comps.hour = 20
    let trigger = UNCalendarNotificationTrigger(dateMatching: comps, repeats: true)

    let request = UNNotificationRequest(
        identifier: "daily-reminder",
        content: content,
        trigger: trigger
    )
    UNUserNotificationCenter.current().add(request)
}
```

按：iOS 通知之權同「惟彈一過」，且用戶隨時可於系統設置中關之——故通知內容之價值甚要，濫發一過，或永失其戶。

三、構架進階：MVVM 之深與可測之性

第 65 章已用 MVVM，此補三進階之要：

- 協議之抽象：使 ViewModel 倚協議（protocol）而非具體之類——測時可注「假數據之

源」，此可測性之基石也；

- 一職之守：一 ViewModel 惟司一事（記賬者不問天氣）；能多則拆之；
- 勿過度設計：一人小項，「@Observable 類 + 協議抽象」足矣。VIPER、Clean Architecture 之重構，待團隊與繁難自來而後議——器者所以務其的，非以炫也。

小貼士：SwiftUI 之「生態紅利」

iOS 生態尚有二「獨此一家」之進階玩法：小組件（WidgetKit）使用戶不啟 App 而能觀信息（記賬本「本月支出」置於桌面），快捷之令（App Intents）使用戶以 Siri 或自動化調汝之能。二者皆有官方之框，入門之費不昂，而於用戶體驗之增益至巨——進階之途宜先解之。

四、2026 年之新法：AI 輔 iOS 之開發

2026 年，AI 已深嵌蘋果開發之程。汝欲提速，此組合之拳甚熟：

環節	AI 何以助	汝之職
書碼	Xcode 內置碼之補全 + 第三方 AI 插件以生之	明述所需，察其所產
查料	以官方之檔與報錯之訊擲 AI 而解之	關鍵之結回官方之檔核之
書測試	使 AI 按「常/邊/異」三類生 XCTest 之用例	核用例合於業之義
改 bug	貼崩潰之日誌 + 相關之碼，AI 定其位而修之	讀懂其改，行測試以驗

AI 輔 iOS 開發之四境

其具體之法——何以予 AI 指令、何以驗收、何以避坑——全十一章之文，在本書第七篇《AI 編程實戰篇》（第 37-47 章），力勸參而讀之：Swift 之語法付本篇，AI 協作之心法付第七篇。

來動手：為記賬本增「三件」

五、Swift 並發之要：@MainActor 與 Task

Swift 6 並發之制有二規，新手必曉：

- 界面惟可於主線程更之：SwiftUI 視圖默為 @MainActor，網絡回呼返主線程而後改態（await MainActor.run { ... } 或標 ViewModel 為 @MainActor）；
- 異步之務以 Task 管之：.task { } 修飾符隨視圖現而自始、沒而自罷——此 SwiftUI 啟異動之正姿也，勿於 body 中妄書 Task 而裸行。

```
struct WeatherScreen: View {
```

```
    @State private var vm = WeatherViewModel()

    var body: some View {
        Group {
            switch vm.state {
            case .loading: ProgressView()
            case .success(let d): WeatherContent(data: d)
            case .error(let m): Text(m)
            }
        }
        .task {                          // 現即始，隱即止
            await vm.search(city: "北京")
        }
    }
}
```

此二規之後，乃 Swift 6「數據競爭安全」之設——編譯器於汝書誤時即報，非待 App 崩而後覺。編譯器之報，非摘瑕也，乃代汝擋彈也。
(1) 為記賬本增「每日 20:00 記賬之提醒」（通知之權 + 本地通知）；(2) 易 ViewModel 數據之源為協議之抽象，且書一注假數據之單元測試；(3) 增「本月支出」桌面小組件。三件既成，汝之項乃有「正軍」之觀矣。

本章小結

- 權限三步：Info.plist 聲明（漏則必崩）、用時而請、見拒則引往設置
- iOS 權限之窗惟彈一過：時擇「方欲用之頃」，文明其用
- 通知三戒：惟發有用者、能關、不擾；UserNotifications 之框
- MVVM 進階：協議之抽象（可測）、一職之守、勿過度設計
- 生態紅利：WidgetKit 小組件 + App Intents 快捷之令
- AI 輔四境：書碼、查料、書測試、改 bug；法見第七篇（第 37-47 章）

第六十九章：速查之冊與資源

此章者，可獨用之速查冊也。汝記不清某寫法、或須速決之際，徑翻至此。宜視之為「囊中之卡」：常用碼之速查、常見之問、學習之資，一頁不遺。

一、常用碼速查之卡

欲作何事	關鍵之碼
頁面私有之態	@State private var x = 初值
跨頁之邏輯	`@Observable class VM { var x = 0 }`
豎列/橫列之佈	`VStack { }`/`HStack { }`
彈性之空	`Spacer()`
列表	List { ForEach(陣列) { ... } }
頁面之跳轉	`NavigationStack + NavigationLink`
底部彈窗	.sheet(isPresented: $show) { 內容}
闔當前頁	`@Environment(\.dismiss) var dismiss`
連網取數據	`try await URLSession.shared.data(from: url)`
JSON 之解析	`struct X: Codable`+`JSONDecoder().decode`
貯小配置	UserDefaults.standard.set(值, forKey: ”鍵”)
貯結構化數據	`@Model`+`modelContainer`+`@Query`
深色模式之配	用系統顏色（.primary / Color(.systemGray6)），勿寫死
行測試	CmdU

iOS 開發速查之卡（2026 年 8 月版）

二、常見之問 FAQ

Q1：吾無 Mac，可學 iOS 開發乎？

可學其語，難為全之開發。iPad/iPhone 之 Swift Playgrounds，足學第 61 章之全文；然 Xcode 為 macOS 所獨，終須有一 Mac。預算有限者，可先租雲 Mac 或購二手 Mac mini 以過

渡，學成其語再定所投。

Q2：學 iOS 開發，費幾何？

學習之期：0 元（Xcode 免費 + 免費 Apple ID 真機調試）。發布之期：開發者賬號 99 美元/年（約 700 元）。加一 Mac 硬件之投，此其全費也。較其所償——App Store 之上，雖一小具亦有付費之戶——此賬合算。

Q3：SwiftUI 與 UIKit，孰先學？

SwiftUI。2026 年新之項、新之教、新之職，皆以 SwiftUI 為主，本書終始用之。UIKit 為前代之框，猶多用於舊項——待汝 SwiftUI 嫻熟，按需補 UIKit（官方有互通之教），上手甚速。序勿倒置。

Q4：iOS 開發，覓職易乎？

2026 年之市情：徒「能繪界面」之初職，競爭烈；然「曉 Swift + SwiftUI + 有上架之作」之開發者，猶為所爭。iOS 職之俸中數，於諸技之列常居前。學習期之汝，至善之履歷，即商店中自作之 App。

Q5：App 為審核所拒，奈何？

勿惶，此常事也——首提見拒之率不低。蘋果拒之由，常書之甚具（何頁、何條之則），逐條改之，於 App Store Connect 復而說明，再提即可。常見拒之三大由：權限之用不明、界面有 bug、元數據（截圖/描述）與實物不符。先自察此三，可省一輪之候。

三、Xcode 高頻快捷鍵速查

操作	快捷鍵
運行 / 測試	CmdR / CmdU
啟畫布預覽	OptionCmd
速啟檔案	CmdO
全局搜索	CmdF
格式化縮進	CtrlI
註釋 / 去註釋	Cmd/
滌構建之緩存	CmdK
跳至其定義	Cmd 點按
易名（全局安改）	右鍵 Refactor Rename

Xcode 高頻快捷鍵速查

小貼士：編譯玄而莫測，先滌緩存

「昨日可行今日不可」之靈異，什六七為構建緩存之亂。CmdK 滌之而重構，可愈玄惑之大半。不驗則重啟 Xcode（並 Mac 同之），再不驗則「刪 DerivedData 之夾」——此 iOS 開發者口耳相傳之終極三連也。

四、薦學習之資

六、薦學習之資

類型	資源	薦之之由
官方	developer.apple.com	蘋果官方之檔，SwiftUI/Swift 之教悉在其中，權威居首
官方	Apple 之 SwiftUI Tutorials	官方親授之程，於新手至為友善
官方	Swift 官網 (swift.org)	語言官方之檔與在線演練之場
社群	Hacking with Swift	Paul Hudson 免費教程之庫，其質公認居首列
社群	Stack Overflow (swiftui 標籤)	報錯求救之首驛
視頻	Apple Developer 官方之頻道（WWDC 視頻）	歲歲新技之權威講解
設計	Apple 人機界面指南（HIG）	造「蘋果味」界面之官方聖經
配套	第七篇《AI 編程實戰篇》（第 37-47 章）	以 AI 輔助 iOS 開發之完整心法

iOS 開發學習資源清單（2026 年 8 月）

五、結語：從第一個 App 到第 N 個 App

回顧既往，爾自「不知 SwiftUI 為何物」，至裝好 Xcode、學會 Swift、以 SwiftUI 繪出界面、為 App 接上數據庫與網絡，再到測試、打包、上架——此路之上，爾已走完從零到一最難之一段。

技藝之世界變化極快：框架會升級，工具會換代，AI 會接手越來越多「敲鍵盤」之活。然有一物不會過時：爾把模糊之想法，拆解成清晰之步驟，再一步步實現之能力。工具者，放大器也；思路者，地基也。放大器可替換，地基不可動搖。

汝不復徒為" 玩手機之人"。汝能以胸中之念，化為數億 iPhone 上實運行之 App。自今日始，以鍵盤創物——此 iOS 開發者之浪漫也。

參考文獻

1. Apple Inc. (2026). SwiftUI Framework Documentation. developer.apple.com.

2. Apple Inc. (2026). Swift Programming Language Guide. swift.org.

3. Apple Inc. (2026). SwiftData Framework Documentation. developer.apple.com.

4. Apple Inc. (2026). App Store Review Guidelines. developer.apple.com/app-store/review.

5. Apple Inc. (2026). Human Interface Guidelines (HIG). developer.apple.com/design.

6. Apple Inc. (2026). Xcode 27 Release Notes and App Distribution Guide.

十、附錄

附錄 A：新手機開箱“保姆級”流程

一、開箱驗機“三部曲”：

- **觀外觀**：檢查機身、屏幕有無劃痕、磕碰。
- **對信息**：覈對盒上之 IMEI/序列號與發票上者為否一致。
- **試功能**：撥一電話試聽筒話筒，拍一張照檢查攝像頭。

二、激活與初始設置“重點提醒”：

- **網絡**：先連上穩定之 Wi-Fi。
- **賬號**：登錄君之蘋果 ID、華爲賬號、小米賬號等。
- **基礎設置**：將” 查尋我之手機” 打開，設一靠譜之鎖屏密碼。
- **eSIM 設置**：若君之手機支持 eSIM（如 iPhone、部分安卓旗艦），可於設置中添加 eSIM 套餐，無需插入實體 SIM 卡。出國時 eSIM 尤便，可徑購當地運營商之 eSIM 套餐。
- **數據遷移**：從舊手機遷移數據時，確保兩臺手機電量充足。iPhone 用” 快速開始”，安卓用官方換機 App。

三、裝 App“安全第一”：

- **首選官方商店**：蘋果用 App Store，安卓用手機自帶之應用商店。
- **權限勿亂給**：安裝時彈出之權限請求，想清楚” 此 App 真的需要嗎？”

附錄 B：動手練習小參考

- **基礎篇練習**：清理手機存儲，目標乃尋出並刪除彼”忘了是啥”之舊視頻、重複截圖與久不用之 App。
- **軟件篇練習**：設定微信之存儲清理，路徑通常爲【我】-【設定】-【通用】-【存儲空間】。
- **進階篇練習**：查看電池健康度，iPhone 於【設定】-【電池】-【電池健康】中。
- **通用技巧**：遇事先重啓！此乃解半數以上手機小恙之萬能鑰匙。

附錄 C：極簡術語“黑話”對照表

君或聽過之詞	實則即指……	一句話解釋
SoC / 芯片	手機之大腦與心臟	決定手機快否、遊戲卡否之核心部件。
RAM (運存)	手機之” 短期工作桌”	桌越大，能同時開之 App 越多，切換越流暢。
ROM (存儲)	手機之” 大倉庫”	存照片、視頻、App 之處。128G、256G 言之即它。
OLED / LCD	屏幕之兩種材質	OLED 色彩更豔、更省電；LCD 更護眼、成本低。
NFC	近距離無線通訊	令君手機能化身公交卡、門禁卡、銀行卡之” 貼貼” 功能。
OTA 更新	無線系統升級	於【設置】裏點之” 系統更新”，即它。
Root	安卓超級管理員權限	得之後可改系統文件，然亦有風險。
BL 鎖	Bootloader 鎖	廠商加之安全鎖，解鎖後方能刷機。
NPU	神經網絡處理器	專門處理 AI 任務之芯片，較 CPU 快數十倍、功耗更低。
端側 AI	手機本地運行之 AI	AI 於手機上徑行運行，不需聯網，速快、隱私好。
Qi2	新一代無線充電標準	帶磁吸定位之無線充電，類似 MagSafe，充電更便。
eSIM	嵌入式 SIM 卡	無實體卡之 SIM，徑寫入手機，換運營商不用換卡。
LTPO	自適應刷新率技術	屏幕刷新率依內容自動調節（1-120Hz），流暢又省電。

附錄 D：靠譜資源去哪裏找？

官方支持乃首選：

- **蘋果用戶**：apple.com/cn 與 iCloud.com。
- **安卓用戶**：手機品牌之官網皆有” 服務與支持” 板塊。
- **通用工具**：蘋果用 iTunes，華爲用華爲手機助手，小米用小米換機。

實用網站與博主：

- **查技巧**：於 B 站搜索”【君之手機型號】技巧”。
- **觀測評**：多觀幾個不同博主之測評視頻。
- **學拍照**：關注手機攝影教程博主。

安全自查一句話清單：

1. 鎖屏密碼設矣乎？
2. ” 查找我的手機” 開矣乎？
3. 重要賬號之雙重認證開矣乎？
4. 定期給照片、微信聊天記錄做備份矣乎？
5. 記防騙口訣：不輕信、不泄露、不轉賬、不貪心、不急躁。

附錄 E：感謝與說明

謝爾恆心讀畢此「手機課」！屬稿之際，吾儕參酌諸廠商官方說明書、用戶手冊暨公開之技術科普文章甚眾，以保信息之準確。

技術演進，日新月異，書中所述，據屬稿時主流之技術與產品。迨爾讀之，或又有新功能、新產品出焉。望此書所授之思路與方法，助爾從容應未來更多數碼新玩物也。

6 版更新要點

- 擴充「數字資產價值與管理理念」，新增「圖片與檔案系統管理」章節（時間 + 事件命名、分類整理、72 小時自動清理、EXIF 脫敏）。
- 完備「純淨/防護模式」與「電子寄生蟲」防護之體系。
- 新增 AI 時代常見詐騙套路（換臉擬聲/冒充權威/虛假投資）與破局防護之思。
- 重寫「開盒」與社工攻擊之防護。
- 擴充無障礙章節（旁白/TalkBack 完整操作、放大與色彩濾鏡、AI 圖像描述、實時字幕、RTT/TTY 通話雙通道）。
- 以「角色-背景-任務-限制」之思重寫提示詞框架，附 3 個完整場景案例。
- 新增三篇技術進階附錄：第一章附：安卓手機記憶體拓展解析（本質定義、zRAM/swap 原理、速度瓶頸、適用場景邊界、各廠商命名與開啟方式）。
- 第二十章附：Termux——Android 設備上之便攜式 Linux 終端環境（核心功能、插件生態、安裝配置、移動開發/伺服器運維/安全測試等典型場景）。
- 第二十三章附：GitHub 入門指南——從代碼託管到協作平臺（Git vs GitHub 區分、核心概念、註冊上手、Pull Request 協作流、安全提醒、國內訪問方案）。

7 版更新要點

一、新增 4 章內容

- 第 29 章：數字健康——螢幕使用時間分析、App 限額、勿擾模式、20-20-20 護眼法則、頸椎自救、睡眠管理、信息焦慮「選擇性失明」、一週數字健康挑戰。

- 第 30 章：投屏與車載——AirPlay / Miracast / DLNA 三陣營對照、有線投屏方案、Samsung DeX 桌面模式、CarPlay / Android Auto / HiCar / CarLife 四大車載方案、會議室投屏實戰。
- 第 31 章：短視頻創作——選題三問、拍攝 5 步設置、運鏡五招（推拉搖移跟）、剪映六步工作流、封面製作要點、「前 3 秒定生死」內容心法。
- 第 32 章：安全急救手冊——丟失黃金 30 分鐘 SOP、被騙 72 小時處理流程、中毒安全模式排查、帳號被盜奪回六步、勒索信息應對、日常預防五件事。

二、新增兩章內容

- 第 27 章：適老化與親情——簡易模式品牌對照表、防詐騙六大騙局與三道防線、遠程協助（暢連/向日葵）、微信關懷模式與語音輸入、SOS / 跌倒檢測 / 用藥提醒。
- 第 28 章：選購指南——三步選機法、「只看五項」參數表、五檔預算推薦、七大場景選機表、五大購機坑、購買時機與渠道分析。

三、新增朗讀與交互功能

- 浮動朗讀控制欄：支持播放/暫停、上一段/下一段、0.5x-2.5x 倍速調節、停止。
- 右側快捷耳機按鈕：一鍵啟動朗讀，自動從當前閱讀位置開始。
- 鍵盤快捷鍵：R 切換朗讀，ESC 停止。
- 朗讀體驗優化：自動連續翻章，打印時朗讀控件自動隱藏。

四、體系更新

目錄、學習路徑（四步五步）、章節地圖、版本號（1.61.7）、附錄編號（五六）、元數據全部同步刷新。

五、內容時效性更新

- 處理器標準：驍龍 8 Elite Gen5 / 天璣 9500 劃入頂級檔（2026 年基準）。
- 旗艦機型參考：iPhone 17 Pro、三星 S26 Ultra、小米 17 Ultra、華為 Pura 80 Ultra、vivo X300 Pro。
- AI 章節：更新到 DeepSeek R1、ChatGPT-5、端側 NPU 80 TOPS 算力標準。
- 摺疊屏：價格區間下探到 4000-5000 元檔。
- 選購指南：全部預算分檔和場景推薦刷新至 2026 年市場。
- 網絡與生態：Wi-Fi 7 / 5G-A / 鴻蒙生態 / LE Audio 全部更新到最新規範。

六、新增功能模組：學習儀表盤（位於 Hero 區域下方）

- 閱讀進度環形圖：實時顯示已讀章節百分比，配 SVG 圓環動畫。
- 手機健康自檢：8 項安全檢查清單（密碼、雙重認證、查找手機、雲備份、微信備份、支付安全、反詐 App、免密清理）。
- 狀態持久化：勾選狀態寫入 localStorage，健康分動態計算並以漸變進度條呈現。
- 分級提示語：根據得分顯示「滿分/不錯/及格/安全隱患」等不同反饋。

8 版更新要點

新增第七篇「AI 編程實戰篇」（第 37-47 章，共 11 章）：範式革命與 Agentic Engineering、AI 編程五大思維原則、上下文工程與規範驅動開發（SDD）、需求工程、2026 年 AI 編程工具全景、大模型全景與 API 實戰、對話工程、從 0 到 1 完整項目全流程、質量保障、進階之路（MCP/多 Agent 協作）與速查手冊。同步更新目錄、學習路徑（五步六步）、章節地圖與速查索引。內容源自《AI 編程思維：從零基礎到全棧實戰的方法論指南》2026 年 8 月版。

新增第八篇「安卓開發實戰篇」（第 48-58 章，共 11 章）與第九篇「iOS 開發實戰篇」（第 59-69 章，共 11 章）：兩大平臺皆覆蓋環境搭建（Android Studio / Xcode）、語言速成（Kotlin / Swift）、界面框架（Jetpack Compose / SwiftUI）、數據與網絡、兩個完整實戰項目（記賬本 + 天氣查詢）、測試與上架（Google Play / App Store）及速查手冊。同步更新目錄、學習路徑（六步八步）、章節地圖與速查索引。內容源自《從零開始的安卓開發》與《從零開始 iOS 開發》2026 年 8 月版。

內容增補：實用軟件與騷擾攔截

- 第十三章新增第六節「實用系統工具」：整理清濁等清理工具、系統自帶存儲清理、國家反詐中心、品牌手機管家、運營商防騷擾與 AdGuard，並立「系統優先 / 官方補充 / 第三方克制」之選型原則。
- 手機自帶騷擾攔截品牌對照表：覆蓋 iOS、小米/Redmi、華為/榮耀、OPPO/realme/一加、vivo/iQOO、三星之入口路徑與推薦開啟項。
- 第八章騷擾攔截能力擴充：補充來電識別、黑白名單、短信關鍵詞過濾、iOS 靜音未知來電、勿擾聯動等實操說明，並與第十三章互鏈。
- 三層防護建議：系統攔截 + 運營商防騷擾 + 國家反詐中心，強調白名單防誤殺。

內容增補：天氣預警 / 自然災害預警 / 地震預警

- 第十三章新增第七節：天氣預警（官方氣象渠道 + 系統天氣 App + 顏色等級）、自然災害預警（氣象/應急多渠道）、地震預警原理與誤區（預警 $\neq$ 預報）。
- 地震預警品牌開啟表：小米、華為/榮耀、OPPO 系、vivo、iPhone、三星等常見入口與權限注意事項。

- 第三十二章新增「場景零」：地震警報後避險之序、暴雨/颱風應對、與衛星 SOS / 無網通信之銜接；急救手冊與第十三章預警設置互鏈。
- 天氣小節聯動：旅行出行中之天氣 App 條目，補充預警通知與「打開通知方收得到」之提醒。

內容增補：跨設備協同與效率工具

- 第十三章第八節：手機遠控電腦（向日葵、ToDesk）及反詐紅線；AI 辦公（Trae / WorkBuddy / Dumate）與手機遙控電腦工作流；米哈遊人工桌面動態壁紙。
- 局域網快傳與剪貼板：LocalSend、飛鴿傳書、隔空投送；KDE Connect、通用剪貼板、品牌手機電腦互傳。
- 動態口令 2FA：Aegis、Microsoft Authenticator，備用碼與換機注意。
- 滴答清單與紙質掃描：日程/任務用法展開；掃描類工具與拍照技巧補全。

願爾玩轉手機，享受科技帶來之便利與樂趣，亦成為清醒、自信之數字公民。
——爾之數字生活嚮導

附錄 F：摺疊屏選購與使用專題

摺疊屏已自“未來科技”變爲“今能買”之成熟品類。然其與直板機於形態、使用習慣、保養方式上差異甚大，值不值得買、何以選、何以用，此篇爲君理清楚。

一、摺疊屏今已發展至何程度？

先言結論：**摺疊屏已非“噱鮮玩具”，然亦非“人人該買”之主流形態**。其介乎小衆極客產品與大衆消費品之間，適特定人羣。

目前主流分兩類：

- **橫向摺疊（書本式）**：合上乃普通手機大小，展開乃接近方形之小平板。代表：華爲 Mate X 系列、三星 Galaxy Z Fold 系列、小米 MIX Fold 系列、vivo X Fold 系列。主打”一部手機 = 手機 + 平板”。
- **縱向摺疊（翻蓋式）**：合上如粉餅盒，展開乃正常尺寸手機。代表：華爲 Pocket 系列、三星 Galaxy Z Flip 系列、OPPO Find N Flip、小米 MIX Flip。主打”小巧便攜 + 外屏速覽”。

2024-2026 年之關鍵進步：鉸鏈壽命普遍標稱 50 萬次摺疊（每天開合 100 次能用 13 年）、UTG 超薄玻璃取代早期塑料膜、摺痕肉眼可見但手感已甚輕微、價格自萬元檔下探至 4000-5000 元檔。

一句話判斷

若君今用直板機完全夠用、無“屏幕不夠大”之痛點，即勿買摺疊屏。其乃“錦上添花”之設備，非“雪中送炭”之設備。

二、何人適買摺疊屏？

依真實用戶反饋，此幾類人買摺疊屏“真香”概率高：

1. **重度閱讀/文檔處理者**：常於手機上觀 PDF、批註文檔、讀長文。展開後接近 7.5 英寸之內屏，觀 A4 PDF 不用縮放，體驗質變。

2. **經常出差/開會之商務用戶**：不願帶平板又嫌手機屏小。橫向摺疊屏展開觀 PPT、籤合同、雙屏對照郵件，比直板機效率高一截。

3. **喜歡”懸停態”創作之人**：折至 90 度放桌上，上半屏取景下半屏按快門（自拍/延時攝影）、上半屏視頻下半屏記筆記、上半屏直播下半屏觀評論——此乃直板機做不到之形態。

4. **追求便攜又欲大屏之女性用戶**：縱向摺疊屏合上能塞入小包，展開乃正常手機，外屏還能當化妝鏡、觀通知。

此幾類人勿買

- 手機只用來刷短視頻、聊微信、打電話者——摺疊屏對君毫無增益，反而更重更厚。
- 手小或手腕有腱鞘炎者——橫向摺疊屏展開後普遍 230g+，單手握持累。
- 預算緊張、一部手機要用 4 年以上者——摺疊屏二手保值率低於直板機，維修成本高。

三、選購要點：觀此 6 個指標

勿被“摺疊”二字晃花了眼，選購時重點觀：

- **鉸鏈類型**：水滴鉸鏈（摺痕淺、壽命長，今之主流）優於 U 型鉸鏈（摺痕深、早期方案）。問銷售或查評測確認爲何種。
- **內屏材質**：必須是 UTG 超薄玻璃，勿用早期那種塑料膜。UTG 手感接近玻璃，塑料膜指甲劃過會有凹陷感且易留痕。
- **摺痕表現**：去線下店親手展開觀。2026 年之合格水平乃：亮屏時正面幾觀不出，斜光下能看出一條淺溝，手指劃過有輕微凹陷但不割手。
- **外屏尺寸**：橫向摺疊屏之外屏越大越好（最好 6.4 英寸以上），合上能當正常手機用；縱向摺疊屏之外屏越大越實用（4 英寸以上能回消息、觀導航）。
- **重量與厚度**：橫向摺疊展開後 230-260g 乃常態，低於 230g 算優秀；合上厚度 12-14mm，低於 12mm 算薄。縱向摺疊合上後能輕鬆放進襯衫口袋方算合格。
- **售後與保修**：重點觀內屏碎裂之維修價格。有些品牌首年免費換一次內屏，有些自費要 2000-4000 元。此乃摺疊屏最大之隱性成本。

線下體驗清單

1. 親手開合 10 次，感受鉸鏈阻尼是否順滑、有無異響。
2. 展開後斜對燈光觀摺痕，再正面亮屏觀之。
3. 單手持展開之態三十秒，察其重可受與否。
4. 試用懸停態：折至 90 度放桌上，試試拍照與觀視頻。
5. 問清楚內屏單獨維修價格與保修政策。

四、摺疊屏獨有之使用技巧

買回來之後，此數個玩法乃直板機體驗不到者：

分屏與多任務

橫向摺疊屏既展，多數系統可上下/左右分屏。其常用者：上半屏觀網課視頻，下半屏錄筆記；左微信而右文檔；上地圖導航而下音樂。較直板機之分屏實用遠甚，蓋屏大，每半皆綽綽有餘。

懸停態（90 度摺疊）

這是摺疊屏的“殺手鐧”用法：

- **自拍/合影**：放桌上折 90 度，上半屏取景，下半屏按快門，不用尋人幫忙拍。
- **延時攝影/夜景**：同樣放桌上穩定拍攝，省三腳架。
- **視頻通話**：解放雙手，對方觀君之角度亦更自然。
- **看視頻**：放桌上當小電視，飛機/高鐵小桌板上特別實用。
- **外屏預覽 + 內屏拍攝**：用後置攝像頭拍自己時，外屏能實時顯示畫面，構圖更準。

外屏速覽

縱向摺疊屏合上後，外屏可以：觀時間/通知/天氣、回微信（語音轉文字或快捷回復）、刷公交卡、觀導航、當化妝鏡（調用前置攝像頭）。不必每次都展開，省電又省事。

電腦模式

部分橫向摺疊屏（華爲、三星）展開後可切換“電腦模式”，外接顯示器鍵鼠後像臺簡易 PC，臨時處理文檔、回郵件夠用。不是主力生產力工具，但應急很香。

五、保養要點：讓摺疊屏活得久一點

摺疊屏最嬌貴者，內屏與鉸鏈也。此數條習慣能顯著延長其壽命：

- **別在低溫下強行開合**：0℃ 以下 UTG 玻璃變脆，開合前先揣懷裏暖幾分鐘。此為官方說明書皆會寫但沒人看之重點。
- **避免指甲/硬物接觸內屏**：內屏雖有保護層但比外屏軟，用指腹操作，別用指甲戳。
- **合上時確認屏內無異物**：一粒沙子夾於中間合上，或徑壓壞內屏。合上前掃一眼。

- **勿貼普通鋼化膜**：內屏只能用官方專用膜或 UTG 保護膜，普通鋼化膜會影響摺疊與觸控。建議去官方售後貼膜。
- **鉸鏈勿進灰塵液體**：雖多數標稱 IPX8 防水，然防塵等級普遍較低。沙灘、工地等環境要小心。
- **開合力度均勻**：別單手暴力甩開，雙手自中間勻速展開最穩。

內屏損壞之高發場景

- 合上時夾住耳機線/充電線——直接壓裂內屏。
- 放包裏與鑰匙混裝——外屏沒事，內屏被鑰匙硌出裂痕。
- 冬天從冷處進暖房立刻開合——溫差導致玻璃應力變化。
- 用了一年後覺得”摺痕變深了”去撕原廠膜——撕膜會連內屏一起損壞。

六、摺疊屏的“坑”和心理預期

買之前心裏要有這幾本賬：

- **摺痕會一直在**：2026 年之技術做不到”完全無痕”，只能做到”亮屏正面幾觀不出”。若汝乃完美主義者，每天盯着摺痕難受，勿買。
- **更重更厚**：橫向摺疊屏展開後普遍 230g+，單手握持比直板機累；合上後厚度 12-14mm，褲兜裏更鼓。
- **續航普遍一般**：因爲要塞鉸鏈機構，電池空間被壓縮，同價位續航通常不如直板機。
- **保值率低**：一年後二手價通常為新機之 50-60%，比直板機跌得快。
- **維修貴**：內屏總成維修價 2000-4000 元，鉸鏈維修 500-1500 元。買之前查清楚保修政策。
- **App 適配參差**：主流 App 已適配大屏，但小衆 App 可能只為”拉伸放大”，體驗一般。

摺疊屏非“更好之手機”，乃“另一種形態之手機”。買它乃因君需其獨特之形態，非因其“更先進”。

——關於摺疊屏的選購哲學

七、主流品牌何以選（2026 年）

不寫具體型號（會過時），只寫品牌調性，助君快速定位：

- **華爲 Mate X 系列**：橫向摺疊標杆，鴻蒙生態協同強，外屏比例接近正常手機，合上能當主力機用。價格高然保值率相對最好。

- **三星 Galaxy Z Fold/Flip**：摺疊屏鼻祖，鉸鏈調校成熟，Z Flip 為縱向摺疊之顏值擔當。國內售後網點較少為短板。
- **小米 MIX Fold/Flip**：性價比路線，配置堆料足，價格相對親民。適合想嚐鮮又不想花太多之用戶。
- **vivo X Fold 系列**：橫向摺疊裏外屏比例做得最接近正常手機之，合上當直板機用體驗好。
- **OPPO Find N 系列**：橫向摺疊裏尺寸偏緊湊之，單手可握持，適合覺得大摺疊太重之用戶。
- **華爲 Pocket 系列**：縱向摺疊裏外屏最大最實用之，化妝鏡、防曬檢測等女性向功能做得細緻。

選購建議：預算充足追求生態選華爲 Mate X；欲最成熟鉸鏈選三星 Z Fold；性價比首選小米/vivo；縱向摺疊觀華爲 Pocket 與三星 Z Flip。

最後提醒

摺疊屏技術每年皆於迭代，本附錄寫於 2026 年。買之前務必查最新評測與用戶反饋，重點關注：鉸鏈壽命實測、摺痕表現、內屏維修政策這三項。線下親手摸永遠比觀評測重要。

結語：持好奇，持清醒

恭喜汝讀至了這里。

這本書自手機硬件拆至軟件生態，自日常設置挖至開發者模式，自安全防護聊至數字遺產。若它幫汝解決矣一個實際問題——哪怕只是學會矣關掉一個不該開之權限、養成矣一次備份之習慣——那此字遂沒白寫。

但比任何具體技巧皆重要之，乃兩件事：

- **持好奇**：手機每年皆於變，AI 於變，摺疊屏於變，衛星通信於變。今天學之操作步驟明天或遂過時矣，但”欲知道何故”之好奇心永遠未能過時。多問一句”此權限乃幹何之””此數據存在矣何處””此功能將未能泄露隱私”，比記住一百個操作步驟皆有用。
- **持清醒**：手機乃工具，非主人。它設計出來就是爲了吸引汝之注意力、佔據汝之時間、引導汝之消費。用它乃汝之選擇，爲它用乃汝之失控。定期抬頭觀屏幕時間，定期欲”吾乃於用手機作事，抑於爲手機消磨”，這非矯情，這乃數字時代之生存本能。

技術於變，但人之需求沒變——我們須安全感、須連接、須爲尊重。手機只是一個載體。真正之“手機課”不於書裏，於汝每天解鎖屏幕之那一瞬間。

終一個練習

合上這本書，拿起汝之手機，作一件事：**用 30 秒欲，汝之手機裏有哪三樣東西乃“丟矣將真正心疼”之？**而後，確保彼等爲備份矣。這本書遂值矣。

附錄 G：全書速查索引

按拼音排序，方便爾快速定位對應之章節。

A-B

- BL 鎖第二十四章
- ADB 第二十章
- App 安裝與管理第四章
- AI 助手第二十二章
- AI 手機第二十六章
- AI 換臉詐騙第三十二章
- AI 編程第三十七～四十七章
- AI 編程五大原則第三十八章
- API Key 第四十二章
- Agent 智能體第三十七章
- CLAUDE.md 第三十九章
- 安卓開發第四十八～五十八章
- Android Studio 第四十九章
- Activity 生命週期第五十二章
- App Store 上架第六十七章

C-D

- 車機互聯第三十四章
- 電池與續航第十五章

- 密碼管理第十四章
- Passkey 第十四章
- 開發者模式第二十章
- 兒童與手機第三十五章
- Claude Code / Cursor 第四十一章
- DeepSeek 第四十二章
- Compose 第五十一章
- DataStore 第五十三章

F-H

- Foldable 摺疊屏附錄 F
- 故障排查第十九章
- Google Play 上架第五十六章
- GaN 充電器第十五章
- iCloud 第十六章
- 智能家居第三十六章
- 計算攝影第十一章附

K-R

- 拍照技巧第十一章
- 權限管理第四章
- SafetyNet 認證第二十章
- SIM swap 攻擊第三十二章
- Root 權限第二十章 / 第二十四章
- Termux 第二十章附
- MCP 協議第四十六章
- PRD 需求文檔第四十章
- Prompt 工程第四十三章
- Kotlin 第五十章

- MVVM 架構第五十七章・第六十八章
- Room 數據庫第五十三章
- Retrofit 第五十三章

S-Z

- 數字遺產第二十五章
- 數據備份第十八章
- 騷擾攔截第八章・第十三章
- 天氣預警第十三章
- 自然災害預警第十三章・第三十二章
- 地震預警第十三章・第三十二章
- 實用軟件與清理工具第十三章
- 清濁第十三章
- UWB 超寬帶第十七章
- 網絡安全第二十一章
- 選購指南第二十八章
- 雲服務第十六章
- SDD 規範驅動開發第三十九章
- Token 第四十二章
- Vibe Coding 第三十七章
- Swift 第六十一章
- SwiftUI 第六十二章
- SwiftData 第六十四章
- URLSession 第六十四章
- Xcode 第六十章
- iOS 開發第五十九～六十九章

其他

- 檔案管理第二章
- 適老化第二十七章
- 工具類 App 推薦第十三章
- 防騷擾 / 國家反詐中心第十三章・第二十七章
- 遠控電腦（向日葵/ToDesk）第十三章
- AI 辦公（Trae 等）第十三章・第四十一章
- LocalSend / 局域網快傳第十三章
- KDE Connect / 剪貼板同步第十三章・第十七章
- 2FA 動態口令（Aegis 等）第十三章・第十四章
- 滴答清單第十三章
- 紙質檔案掃描第十三章
- 人工桌面（動態壁紙）第十三章
- 地震 / 災害手機應急第三十二章
- 投屏與車載第三十章
- 短視頻創作第三十一章
- 安全急救第三十二章
- Git/GitHub 第二十三章附
- Git 版本控制第四十五章
- 上下文工程第三十九章
- 需求工程第四十章
- 記賬本 App 第五十四章・第六十五章
- 天氣 App 第五十五章・第六十六章
- WidgetKit 小組件第六十八章

附錄 H：版本路線圖

此書猶續有更新。下陳未來版本規劃之方向，亦歡迎爾於反饋中告吾儕所最欲觀之內容。

v1.8 已落地：新增第七、八、九篇「AI 編程實戰篇」「安卓開發實戰篇」「iOS 開發實戰篇」（第 37-69 章，共 33 章）

自 AI 編程之範式革命、五大思維原則、上下文與需求工程、2026 工具與模型全景、API 實戰、對話工程、三個完整項目、質量保障與進階之路——攜爾自「會用 AI」升級至「指揮 AI 寫代碼」；至安卓與 iOS 兩大平臺完整開發之鏈路：環境搭建（Android Studio / Xcode）、語言速成（Kotlin / Swift）、界面框架（Jetpack Compose / SwiftUI）、數據與網絡、實戰項目（記賬本 + 天氣查詢）、測試與上架（Google Play / App Store）及速查手冊——攜爾自數碼用戶終成 App 開發者。

下一版（v1.9）計劃方向

- 手機與可穿戴設備：Apple Watch、華為手錶、智能戒指（Oura/Galaxy Ring）深度協同指南
- 旅行與手機：eSIM 實操、國際流量方案、離線地圖、海外支付、翻譯 App 對比
- 手機與健康管理：運動健康、睡眠監測、心率血氧、女性健康、心理健康 App
- AI 短視頻創作：剪映 AI、可靈 AI、數字人、AI 配音之實操教程
- 豎屏短劇入門：劇本結構、拍攝技巧、平臺政策

關於時效性之說明

手機技術迭代極快，本書內容據 2026 年中技術與市場之況而撰。AI 之能力、芯片型號、系統版本、價格區間，爾閱讀之時或已變矣。涉具體操作步驟，請以爾設備實際系統版本為準；涉購買決策，請查閱最新評測與用戶反饋。